

# Embedded Systems

## Indice:

- Sintesi dei circuiti digitali
  - sintesi
  - sintesi hardware vs sintesi software
  - Sintesi hardware
  - ottimizzazione della sintesi
  - metodologia della sintesi
  - Esempio sintesi
  - Scheduling
  - Binding
- Sintesi FPGA's
- Introduzione a VHDL
  - storia di vhd
  - Struttura vhd
  - VHDL
  - Esempio programma
  - Dichiarazioni
  - std\_logic
  - signal v variable
  - Sequential and concurrent statements
- Sintesi in VHDL
  - VHDL per sistemi di sintesi
  - Statement Concorrenziali
  - Operatori
  - Statement Sequenziali
  - Esempi di logiche sequenziali (D-latch)
  - Fallacies
  - Macchine a stati finiti + esempi
  - VHDL strutturato
  - Test bench
  - Esempio generale (shift comp)
- FPGA nel dettaglio
  - Logiche programmabili
  - Storia ed evoluzione tecnologica Circuiti integrati
  - I circuiti integrati
  - Circuiti non programmabili
  - circuiti programmabili
  - circuiti semi-custom
  - Macchina di Turing
  - Programmabilità fisica
  - Programmabilità logica

- Blocchi logici configurabili
- Interconnessioni
- Microprocessori e Microcontrollori
  - Teorema di Godel
  - La Macchina di Turing
  - I microprocessori
  - La complessità
  - Architetture dei Microprocessori
  - Strutture e Componenti
  - Memoria e Decodifica
  - Modalità di interfaccia Periferica
- DSP & SoC - MAC
  - DSP & SoC
  - Hardware
  - Addressing Modes
  - Miscellanea
  - Multifunctional SoC
- Multiply and Accumulate (MAC)
  - Digital Signal Processing Domain
  - Multiplication Blocks
  - Fixed - Point Notation
  - The Single Cycle MAC
  - The Pipelined MAC
- MIPS ARchitecture
  - The Micro Processor Quantitative Design
  - The Performance Equation
  - Micro Processor Architectural Types
  - The Sw/Hw Interface
  - The MIPS Single-Cycle Architecture
  - Esecuzione delle Istruzioni in un Datapath Multiciclo
- Cache Principles:
  - Introduzione
  - Cache Architecture
  - Reducing Misses
  - Coherence Strategies (when writing)

## **ES02**

# **Introduzione ai Sistemi Embedded**

L'evoluzione dei sistemi informatici ha portato allo sviluppo degli embedded system come risposta alla necessità di dispositivi più piccoli, economici, efficienti e specifici per

determinate applicazioni. La loro diffusione ha rivoluzionato numerosi settori, creando nuove opportunità per l'innovazione e l'automazione.

Linux è diventato il sistema operativo scelto per un gran numero di applicazioni integrate seguenti questo approccio, come router Internet, sistemi di navigazione satellitare GPS, dispositivi di archiviazione collegati in rete, ecc....

La prospettiva quindi di avere una realtà processor-centrica (tale per cui il software fosse alla base dell'elaborazione delle informazioni) e una miniaturizzazione dei circuiti integrati, ha portato alla nascita dei cosiddetti *sistemi embedded*.

Le ragioni che hanno condotto allo sviluppo di tali sistemi sono molteplici, tra cui troviamo:

- *La crescente complessità funzionale*: La necessità di aggiungere funzionalità avanzate ai dispositivi integrati ha richiesto l'integrazione di grandi quantità di software. Questo è stato reso possibile dall'aumento esponenziale della densità dei semiconduttori, descritto dalla legge di Moore, che afferma che il numero di transistor in un circuito integrato raddoppia approssimativamente ogni due anni. Assieme alla legge del ritorno accelerato, ciò ha permesso lo sviluppo di dispositivi sempre più potenti e complessi.
- *Efficienza e miniaturizzazione*: la combinazione HW/SW ottimizzata nei dispositivi embedded consente di ridurre le dimensioni fisiche e i costi di produzione. Questo è un passo fondamentale per la realizzazione di dispositivi compatti a basso consumo

## **Sistemi Embedded**

Si definisce *sistema embedded* (o sistema integrato) un sistema di elaborazione delle informazioni incorporato in un prodotto di maggiori dimensioni. Il problema tecnico legato ai processi fisici di un sistema integrato è quello della gestione di un tempo di concorrenza e di computazione di sistema.

## **CYBER PHYSICAL SYSTEMS**

Si introduce così il concetto di "cyber-physical systems" (sistemi informatico-fisici) o CPS che sono integrazioni di calcolo mediante processi fisici.

I sistemi CPS si riferiscono a sistemi ICT (Information and Communication Technologies) integrati di nuova generazione che sono interconnessi attraverso l'Internet of Things (IoT) che permette loro quindi di collaborare; si parla in questo contesto di "Industria 4.0".

Un CPS differisce da un tradizionale sistema di controllo digitale principalmente nella sua struttura: anziché avere un controllore che riceve un riferimento e confronta l'errore con il segnale di retroazione abbiamo un cyber che rappresenta la parte

computazionale che è più avanzata e include oltre al controllo funzioni di calcolo avanzate, comunicazione e analisi dati.

Contrariamente ai computer generici riprogrammabili, un sistema embedded ha dei compiti noti già durante lo sviluppo, che eseguirà dunque grazie ad una combinazione hardware/software studiata per la tale applicazione. Grazie a ciò l'hardware può essere ridotto ai minimi termini per contenere lo spazio occupato limitando così anche i consumi, i tempi di elaborazione (maggiore efficienza) ed il costo di fabbricazione. Inoltre l'esecuzione del software è spesso in tempo reale per permettere un controllo deterministico dei tempi di esecuzione.

In sostanza, i sistemi embedded sono sistemi di calcolo, comprendenti ogni tipo di calcolatore al di fuori di quelli progettati per essere di utilità generica. Più in generale, rispetto un processore general purpose:

- **Scopo e utilizzo:** un sistema embedded svolge compiti specifici all'interno di un sistema più grande, mentre un processore viene progettato per una vasta gamma di compiti.
- **Hardware:** un sistema embedded è progettato per essere altamente efficiente dal punto di vista energetico (spesso alimentato a batteria) ed ha risorse limitate in termini di memoria (RAM e ROM) e capacità di elaborazione. Un processore general purpose tende a consumare più energia e ha accesso a risorse più abbondanti.
- **Architettura e design:** un sistema embedded sfrutta architetture specifiche ottimizzate per un compito, e fa uso di interfacce come GPIO, ADC, DAC, I2C, UART, SPI. Un general purpose invece utilizza architetture più versatili (es: x86 e supporta una vasta gamma di periferiche come USB, HDMI, PCIe etc.
- **Software:** nei sistemi embedded si esegue software dedicato;

#### Applicazioni Sistemi Embedded

Le applicazioni che fanno uso di sistemi embedded sono molteplici e corrispondenti a diverse aree, ad esempio:

- **Automazione di fabbrica**

Al fine di ottimizzare ulteriormente le tecnologie di produzione, è possibile utilizzare la tecnologia CPS/IoT. La tecnologia CPS/IoT è la chiave per una produzione più flessibile favorendo il raggiungimento dell'obiettivo per l'industria 4.0.

- **Robotica**

La robotica è anche un'area tradizionale in cui sono stati utilizzati sistemi embedded CPS.

- *Trasporto e mobilità*

Elettronica in ambito automotive (le auto di oggi contengono una quantità significativa di elettronica);

- *Smart City (città intelligenti)*

Le *smart city* si riferiscono a strategie di pianificazione urbanistica che migliorano la qualità di vita in città, e cercano di soddisfare le esigenze ed i bisogni dei cittadini.

## **Tecnologia MOS e circuiti integrati**

La tecnologia MOS (Metal–Oxide–Semiconductor) costituisce il fondamento di tutta l'elettronica digitale moderna perché consente di realizzare su un singolo chip di silicio miliardi di transistor controllabili elettricamente, rendendo possibili microprocessori, memorie e sistemi embedded complessi. Tutto parte dal silicio ultrapuro, prodotto sotto forma di lingotto monocristallino mediante tecniche come il metodo Czochralski o il Float Zone, che garantiscono una struttura cristallina quasi perfetta; il lingotto viene poi tagliato in wafer sottili e lucidato, sui quali, attraverso processi estremamente precisi di fotolitografia, ossidazione, drogaggio ionico e deposizione di materiali, si costruiscono strati successivi di transistor e di interconnessioni metalliche. Il dispositivo base è il MOSFET, che funziona come un interruttore controllato in tensione: una tensione applicata al gate, separato dal canale da un sottilissimo strato di ossido di silicio, crea o distrugge un canale di conduzione tra source e drain, permettendo o impedendo il passaggio di corrente; la corrente dipende dalla geometria del transistor, in particolare dal rapporto W/L (larghezza su lunghezza del canale), dal tipo di portatori e dalla tecnologia. La riduzione delle dimensioni fisiche dei transistor consente di aumentare velocità, densità e integrazione, ma introduce anche problemi di perdite, campi elevati e controllo elettrostatico del canale, che vengono affrontati mediante tecnologie sempre più sofisticate. In questo contesto la tecnologia CMOS (Complementary MOS), che combina transistor nMOS e pMOS, è fondamentale perché permette di ridurre drasticamente il consumo statico, dato che idealmente uno dei due transistor è sempre spento. Per migliorare l'isolamento e prevenire interferenze e correnti parassite (come il fenomeno del latch-up), sono stati sviluppati processi come single-well, triple-well e trench isolation, nei quali i transistor vengono separati fisicamente tramite ossido scavato nel silicio. Sopra lo strato dei transistor viene poi costruita una rete sempre più complessa di interconnessioni metalliche multilivello, che oggi rappresenta spesso il vero limite di velocità dei chip più dei transistor stessi. Per continuare la scalabilità sono nate architetture avanzate come SOI (Silicon On Insulator), che riduce le capacità parassite, e strutture tridimensionali come FinFET e Gate-All-Around, che migliorano il controllo elettrostatico del canale avvolgendolo con il gate. Questa evoluzione tecnologica ha reso possibile il passaggio dai primi microprocessori con poche migliaia

di transistor ai chip moderni con miliardi di MOSFET, aumentando parallelismo, cache, larghezza di parola e prestazioni, e nei sistemi embedded tutto ciò che viene descritto in HDL o generato da strumenti di sintesi ad alto livello viene infine tradotto in queste strutture fisiche di transistor e metallo, motivo per cui comprendere la tecnologia MOS è essenziale per capire i limiti, i costi, i consumi e le prestazioni dell'hardware reale su cui girano i sistemi digitali.

## **Introduzione a Unix**

E' un Sistema Operativo: un software che astrae l'hardware. Esso è scritto in C, è "machine Independent" e non dipende dall'hardware in cui è installato.

*(Un sistema operativo è un software che gestisce le risorse hardware di un computer e fornisce un'interfaccia tra l'utente e la macchina. Coordina l'esecuzione dei programmi, gestisce la memoria, i file, i dispositivi di input/output e assicura che le diverse applicazioni possano funzionare correttamente. Inoltre, funge da intermediario tra l'hardware e i programmi applicativi, permettendo agli utenti di interagire con il computer in modo semplice ed efficiente.)*

Unix non è monolitico perché, pur avendo un kernel centrale, si basa su una filosofia modulare. In un sistema monolitico, tutte le funzionalità sono strettamente integrate nel kernel. Al contrario, Unix segue l'approccio "fai una cosa e falla bene", in cui diverse componenti (comandi, utility, shell) sono indipendenti e comunicano con il kernel, permettendo una maggiore flessibilità e semplicità nella manutenzione e sviluppo del sistema.

## **Struttura UNIX**

Kernel: Il nucleo (bios) è il primo strato di software che mette a disposizione le "handle" per parlare con l'hardware, e risiede in una memoria non volatile.

Shell: Racchiude i processi principali: La shell è un'interfaccia a riga di comando CLI che permette agli utenti di interagire con il sistema Unix. Interpreta i comandi dell'utente e li invia al kernel per l'esecuzione. Esistono diverse shell, come Bash, C shell, Korn shell, ecc. La shell permette anche l'esecuzione di script per automatizzare compiti ripetitivi.

Utility/Comandi: Strumenti e programmi preinstallati che svolgono operazioni specifiche (es. gestione file, editor di testo).

Servizi Esterni: Software Applicativi, File System La struttura in cui sono memorizzati e organizzati i file), DBMS (Database Management System) è un software che consente di creare, gestire e manipolare database)

Esecuzione dei comandi in primo piano e in background:

**Primo piano** Quando un comando viene eseguito semplicemente digitandolo nella shell (es. <comando>), questo viene eseguito in primo piano. Ciò significa che la shell è "bloccata" fino al completamento del comando, e l'utente non può eseguire altri comandi nel frattempo.

**Background** Se l'utente vuole continuare a usare la shell mentre un comando è in esecuzione, può lanciare il comando in background aggiungendo un simbolo "&" alla fine del comando (es. <comando>&). In questo modo, il comando continua a essere eseguito, ma la shell rimane libera per accettare nuovi comandi.

**Controllo dei processi in esecuzione:**

**Ctrl-Z** Se un comando in esecuzione in primo piano deve essere temporaneamente fermato, si può utilizzare la combinazione di tasti Ctrl-Z. Questo mette in pausa (sospende) il processo.

**bg:** Una volta sospeso, il processo può essere spostato in background utilizzando il comando bg, permettendo all'utente di continuare a interagire con la shell mentre il comando sospeso riprende l'esecuzione in background.

**fg:** Se si desidera riportare un processo in background nuovamente in primo piano, è possibile utilizzare il comando fg, che riporta il controllo alla shell e impedisce di eseguire altri comandi fino al completamento del processo.

## **Introduzione Circuiti Integrati**

Storia ed Evoluzione tecnologica dei Circuiti Integrati.

Dagli anni '80 le aziende iniziarono a progettare sistemi elettronici integrati custom, che condensavano tutte le funzioni in un unico circuito specifico per l'applicazione, i cosiddetti ASIC. Tali circuiti avevano un grosso problema:

*Erano costosi da realizzare e risolvevano un problema specifico per cui non erano riutilizzabili.*

I microprocessori sono quelli che consumano di più, mentre a parità di consumo i circuiti dedicati sono i più performanti, però hanno un costo importante. Le FPGA hanno i vantaggi di essere poco costosi e avere un rapporto consumo/performance molto buono.

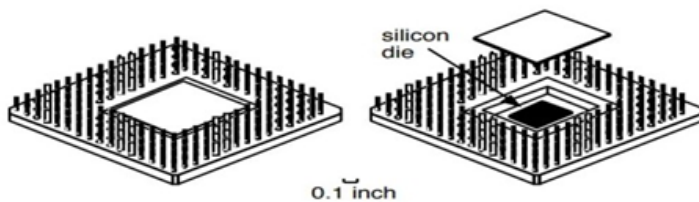
Costi presenti:

- Fixed Costs: Sono i costi che non dipendono dal numero di chip prodotti. Ad esempio, i costi di training, hardware and software tools, design costs, general costs, circuit test design, profit model
- Variable Costs: Materie prime (Silicon wafers, materials and disposables,), production costs, packaging, testing.

### I circuiti Integrati

I circuiti integrati IC, (Integrated Circuits) sono dispositivi elettronici che combinano un gran numero di componenti elettronici, come transistor, diodi, resistori e condensatori, su un singolo chip di materiale semiconduttore, solitamente il silicio. Essi costituiscono il cuore dei sistemi elettronici moderni e possono essere classificati in due categorie principali:

- Circuiti Non programmabili
- Circuiti Programmabili



### Circuiti non Programmabili

I circuiti non programmabili sono dispositivi la cui funzionalità è definita durante la fase di progettazione e produzione. Una volta fabbricati, il loro comportamento non può essere modificato. Questa categoria comprende:

#### Circuiti Custom:

Conosciuti anche come ASIC (Application-Specific Integrated Circuits), sono progettati su misura per una specifica applicazione o funzionalità.

Questi circuiti, tuttavia, richiedono alti costi di progettazione e realizzazione;

#### Circuiti Semi-custom:

Offrono un compromesso tra circuiti standard e custom, consentendo una certa personalizzazione senza i costi elevati degli ASIC. Fra di essi troviamo ad esempio gli MGA (Masked Gate Arrays), suddivisi in Channeled Gate Arrays, ovvero MGA

con canali predefiniti per le interconnessioni, e Channelless Gate Arrays, dove i canali sono eliminati;

### Circuiti Programmabili

I circuiti programmabili sono dispositivi la cui funzionalità può essere definita o modificata dall'utente dopo la produzione. Questa flessibilità li rende ideali per una vasta gamma di applicazioni. Essi comprendono:

- PLD (Programmable Logic Devices): Dispositivi che possono essere programmati per eseguire funzioni logiche semplici.
- FPGA (Field-Programmable Gate Arrays): circuiti integrati programmabili avanzati che possono essere implementati tramite funzioni complesse.

Per connessione programmabile, si intende, sia nei PLD, che negli FPGA, delle reti configurabili di interruttori elettronici che stabiliscono percorsi logici tra i blocchi del dispositivo. Il segnale digitale 1 o 0 è rappresentato da livelli di tensione, e queste connessioni possono essere programmate e riprogrammate per modificare il comportamento del circuito.

### Circuiti Semi-Custom Standard Cells o CBIC

Le Standard Cells sono celle logiche che rappresentano un blocco funzionale specifico come:

- Porte logiche AND, OR, NOT
- Latch e flip-flop
- Multiplexer

### Macchina di Turing

La macchina di Turing è un modello teorico di calcolo ideale che descrive un dispositivo a stati infiniti che manipola simboli su un nastro di lunghezza infinita secondo una serie di regole predefinite. Tale macchina è composta da un nastro, suddiviso in celle contenenti un simbolo di un alfabeto finito, da una testina di lettura/scrittura, da uno stato, e da una funzione di transizione che dato lo stato attuale e il simbolo letto il quale determina: il nuovo stato, la direzione della testina e il simbolo da scrivere nella cella corrente.

### MICROPROCESSORE COME TURING MACHINE

Un microprocessore può essere considerato una macchina di Turing grazie alla sua dimensione: sebbene abbia memoria finita, essa è sufficientemente grande da poter essere assimilabile al nastro infinito, la testina è assimilabile all'accesso alla RAM (in r/w) e l'unità di elaborazione corrisponde al ISA (Instruction Set Architecture).

Di conseguenza, con un microprocessore possiamo risolvere qualunque processo computazionale che ammette una soluzione algoritmica.

Quando un modello computazionale ha le capacità di soluzione di problemi pari ad una macchina di Turing diciamo che è “Turing complete”.

### ES03

# Sintesi dei Circuiti Digitali

## Due Tipologie di Design

1. Top Down: Parto dal problema e scendo fino a raggiungere il problema più semplice e realizzo la soluzione (più problema, meno dettagli)
2. Bottom up: Parto dal programmare la funzione e l'assemblo man mano.

## Sintesi

Nel **Top-Down Design Flow** applicato agli **embedded systems** la progettazione procede dal livello più astratto a quello più concreto: si parte dall'**idea**, cioè dalla definizione di cosa deve fare il sistema, si passa alla **formalization**, in cui l'idea viene tradotta in requisiti funzionali, temporali e di consumo, poi alla **block structure**, dove il sistema viene suddiviso in blocchi hardware e software (microcontrollore, sensori, attuatori, moduli software, eventuale RTOS). Successivamente si entra nel **detailed design**, in cui si progettano nel dettaglio i singoli blocchi (schemi elettrici, driver, task, interrupt, macchine a stati e gestione del timing), quindi nella **synthesis**, che consiste nella traduzione del progetto in una forma realizzabile (compilazione del codice, sintesi logica, mapping hardware/software), fino alla **realization**, ovvero l'implementazione finale del sistema su hardware reale. Durante tutte le fasi è fondamentale la **vérification**, che viene effettuata a ogni livello per controllare che le scelte progettuali rispettino le specifiche definite nei livelli superiori, individuando errori il prima possibile.

Il processo di sintesi è spesso diviso per livelli di astrazione (**idea** → **formalizzazione dell'idea** → **struttura a blocchi** → **progetto esecutivo**) in cui implementeremo una progettazione di tipo **top-down**, che partirà quindi dal livello di astrazione più alto fino ad arrivare a quello più basso.

Nei linguaggi di programmazione esiste una corrispondenza biunivoca tra costrutto sintattico e la sua semantica; cioè, la semantica espressa da un determinato costrutto non è interpretabile, ma ha un significato ben definito.

Il linguaggio hardware è la creazione di un algoritmo volto alla descrizione dell'hardware.

## Sintesi Hardware vs Sintesi Software

Nella **schematization of the design flow** di un sistema (tipicamente embedded o hardware-oriented) il processo è suddiviso in fasi consecutive: nella fase di **design** si parte dall'idea e si procede con il **modeling** del sistema, seguito dalla **synthesis & optimization**, in cui il modello viene trasformato in una soluzione implementabile ed efficiente, e dalla **validation**, che verifica la correttezza funzionale rispetto alle specifiche. Successivamente si passa al **testing**, dove il sistema viene sottoposto a test strutturati e a pattern di prova per individuare eventuali errori. Se il progetto riguarda componenti hardware, si entra poi nella fase di **fabrication**, che comprende la **mask fabrication** e la **wafer fabrication**, ovvero la realizzazione fisica del dispositivo, e infine nella fase di **packaging**, che include il **slicing** e l'incapsulamento finale del componente, rendendolo pronto per l'uso e l'integrazione nel sistema finale.

Nella sintesi software, l'obiettivo è mappare una descrizione comportamentale astratta (codice sorgente, es. C/C++) su una risorsa hardware fissa e generale (il processore). Poiché le risorse di calcolo sono limitate e condivise, il processo impone un **binding temporale**: le operazioni vengono serializzate nel tempo per essere eseguite sequenzialmente dall'automa.

*Il processo di automatizzazione dell'implementazione è esistente ma si perde controllo sul risultato finale (componente scritto nel silicio) a seconda del livello di astrazione che si raggiunge. Più specifiche di basso livello si implementano e più il componente sul silicio sarà simile a quello pensato. Ci sono dei livelli di astrazione che è sconsigliabile implementare per un uso generico (es Adder, non si specifica quale adder o come deve essere fatto ma solo la necessità di averne uno).*

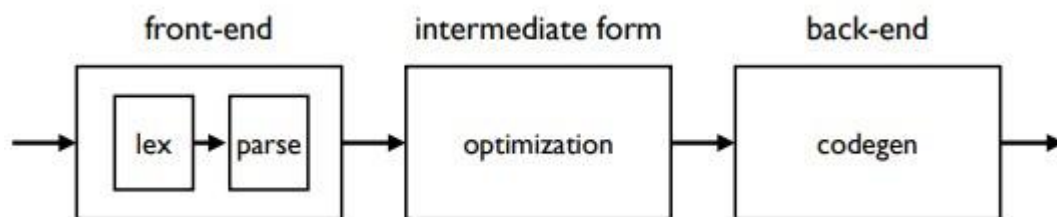
L'interfaccia critica è l'**Instruction Set Architecture (ISA)**, che funge da contratto tra HW e SW, definendo le operazioni primitive che la macchina può eseguire. Il processo di traduzione (Compilazione) avviene in tre fasi distinte:

1. **Front-End (Analisi e Astrazione):**
  - Esegue l'analisi **lessicale, sintattica e semantica** per verificare la correttezza grammaticale del codice.
  - Genera una prima rappresentazione intermedia, solitamente un **Abstract Syntax Tree (AST)**, che descrive la struttura logica del programma.
2. **Middle-End (Ottimizzazione Indipendente dall'Architettura):**

- Trasforma l'AST in una **Intermediate Representation (IR)** o in un *Control Data Flow Graph (CDFG)*.
  - Esegue ottimizzazioni matematiche e logiche (es. rimozione del codice morto, semplificazione dei cicli) senza preoccuparsi ancora di quale processore verrà usato.
3. **Back-End (Generazione del Codice e Mapping):**
- Traduce l'IR nell'Instruction Set specifico del target (es. ARM, x86).
  - Esegue la **Register Allocation**: mappa le variabili infinite del programma (binding logico) sul numero finito di registri fisici della CPU.
  - Esegue l'**Instruction Scheduling**: riordina le istruzioni per massimizzare l'efficienza della pipeline.

## DATAPATH

In quest'ottica definisco il Datapath come il cammino che i dati e le istruzioni devono seguire per essere lavorate;



Anche nel caso della **sintesi hardware** specifico un comportamento. Qui però l'hardware non è dato, ma devo costruirlo specificando cosa voglio realizzare. **La differenza rispetto alla sintesi software è dunque nel livello di astrazione più basso.**

**La sintesi hardware comporta, oltre al ricorso a dispositivi progettati ad hoc (eventualmente usando librerie di celle) anche la scelta di dispositivi a volte già esistenti (ad esempio dispositivi semi-custom) ed eventualmente quella dei particolari microprocessori da utilizzare.**

La selezione dei diversi dispositivi influenza a sua volta la generazione del software, che dunque dipende dalle decisioni prese nelle fasi iniziali della sintesi hardware.

Collegando il concetto di **datapath** al diagramma **front-end / intermediate form / back-end**, il processo può essere interpretato in modo chiaro anche per la **sintesi hardware**: nel **front-end** si analizza e si interpreta la specifica comportamentale del sistema (analogamente alle fasi di *lex* e *parse* nel software), definendo cosa il sistema

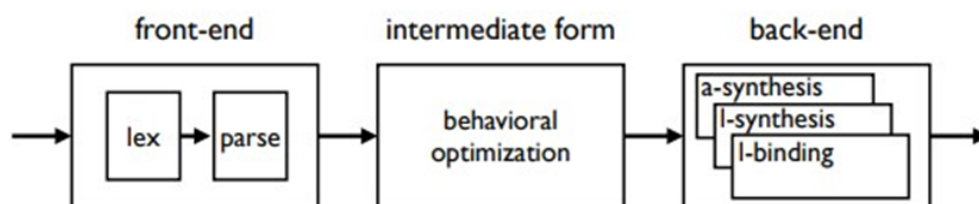
deve fare senza ancora stabilire come verrà realizzato. Nella **intermediate form** il comportamento viene riorganizzato e **ottimizzato**, individuando le operazioni fondamentali e i flussi di dati; è in questa fase che inizia a emergere la struttura del **datapath**, cioè il cammino seguito dai dati attraverso le unità funzionali. Nel **back-end**, infine, avviene la **code generation** o, nel caso hardware, la vera e propria **costruzione del datapath**, con la scelta dei componenti fisici (registri, ALU, bus, celle logiche o dispositivi semi-custom) e il loro collegamento. Le decisioni prese nel back-end della sintesi hardware determinano l'architettura finale del sistema e influenzano direttamente la generazione del software, che deve adattarsi al datapath e ai vincoli imposti dall'hardware progettato.

## PLATFORM

**Definizione di Platform:** A differenza del datapath, che descrive il cammino fisico del dato tra componenti, la piattaforma è l'astrazione hardware-software creata per interfacciarsi con i livelli applicativi superiori. Essa funge da infrastruttura di comunicazione e gestione delle risorse.

**Il Flusso di Sintesi: Front-end e IR** Il processo inizia nel **Front-end** con il *lexing* (tokenizzazione) e il *parsing* (struttura sintattica). Segue una fase di **Intermediate Form (IR)** dove avvengono ottimizzazioni comportamentali; qui il compilatore migliora l'efficienza logica senza ancora vincolarsi a una specifica tecnologia hardware.

**Back-end e Implementazione Fisica** Nella fase finale di **Back-end**, il design viene "calato" sull'hardware. Attraverso la sintesi architettonica e logica, le operazioni vengono mappate su componenti fisici tramite il **binding**, trasformando un comportamento descritto in codice in una struttura circuitale o in binario eseguibile.



## Modello Computazionale

Nel processo di progetto ci sono diverse fasi, le quali hanno un certo numero di funzioni che devono essere assolve. Il processo di progetto prevede che:

- *Deve essere possibile codificare il modello:* cioè devo avere un entry point in cui specifico questa funzionalità.
- *Valido e debuggo attraverso la simulazione al livello della codifica.*

- *Scelta delle opzioni architetturali*: scompongo la mia idea in blocchi funzionali;
- *Progetto esecutivo*: si parla di sintesi o co-sintesi: la prima genera l'hardware che realizza quel funzionamento mentre la seconda è data dal fatto che posso decidere di realizzare porzioni in hardware e porzioni in software su un microprocessore dedicato.

Il motivo per cui si è passati da un flusso di progetto hardware ad uno software è descritto dal cosiddetto *productivity gap*.

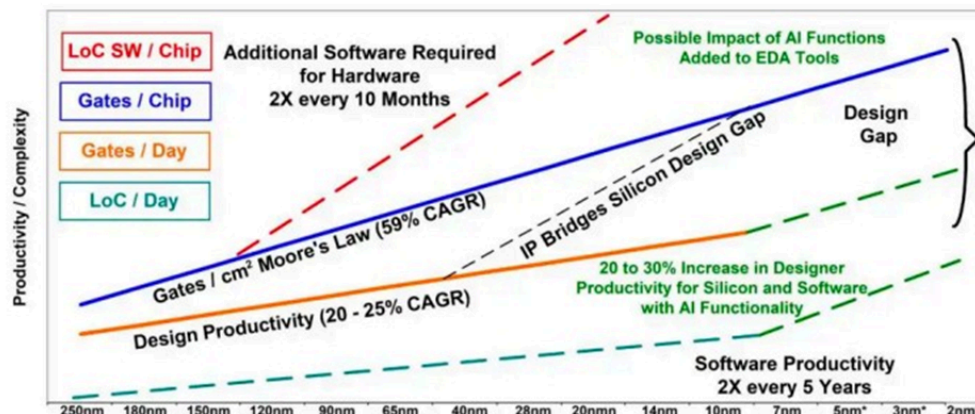


Figura: Productivity Gap

## Productivity Gap

Il numero di transistor resi disponibili dalla tecnologia è di gran lunga superiore, e cresce più rapidamente, del numero di transistori che VHDL mi permette di utilizzare dato il suo livello di astrazione.

Nascono così i software EDA (Electronic Design Automation) che consentono di catturare l'idea progettuale attraverso un modello HDL, di sintetizzare via via i livelli più bassi di astrazione e, infine, di ottimizzare il circuito e il parametro di progetto sulla base di una data metrica.

L'evoluzione tecnologica rende disponibili un numero di transistor per chip che cresce molto rapidamente (seguendo la legge di Moore), ma la capacità del progettista di sfruttarli cresce molto più lentamente. In particolare, il numero di transistor effettivamente utilizzabili è limitato dal livello di astrazione dei linguaggi HDL, come VHDL, e dalla produttività umana nel progettare sistemi complessi.

### Origine del divario.

Mentre la tecnologia consente di integrare sempre più logica sul silicio, la progettazione manuale a basso livello non scala allo stesso ritmo. Questo crea un divario tra la

complessità hardware disponibile e quella realmente gestibile dal progettista, noto come *productivity gap*.

### **Ruolo degli strumenti EDA.**

Per colmare questo divario nascono i software di EDA (Electronic Design Automation), che permettono di descrivere il sistema a un livello di astrazione più alto tramite un modello HDL, delegando agli strumenti automatici la traduzione verso livelli più bassi.

### **Sintesi e ottimizzazione.**

Gli strumenti EDA si occupano della sintesi logica, della generazione dell'hardware fisico e dell'ottimizzazione del progetto rispetto a metriche specifiche, come area, consumo di potenza, prestazioni o costo. In questo modo il progettista può concentrarsi sull'idea progettuale e sull'architettura del sistema, anziché sui dettagli implementativi.

### **Collegamento agli Embedded Systems.**

Negli embedded systems moderni, l'aumento della complessità hardware rende indispensabile l'uso di alti livelli di astrazione e di strumenti automatici, poiché software e hardware coesistono sullo stesso chip e devono essere progettati in modo coordinato.

## **Sintesi Hardware**

La sintesi hardware è il processo attraverso cui una specifica comportamentale di alto livello viene trasformata in hardware concreto. Questo processo prende una descrizione di cosa deve fare il circuito e la traduce in come deve essere realizzato fisicamente. La sintesi si caratterizza da due assi indipendenti, ma uno di questi è l'asse dell'astrazione. Si hanno 3 livelli di astrazione; dal basso verso l'alto:

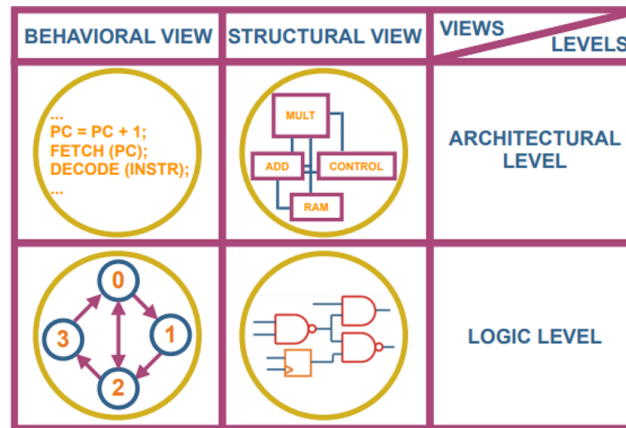
- Livello geometrico: (leggera astrazione del livello fisico)
- Livello logico (descrizione attraverso porte logiche): a questo livello di astrazione si minimizza l'area in presenza di vincoli sul ritardo di propagazione, inoltre si minimizza il ritardo di propagazione in presenza di vincoli sull'area.
- Livello architetturale: a questo livello di astrazione si determina il cycletime, si minimizza l'area in presenza di vincoli sulla latenza e infine si minimizza la latenza in presenza di vincoli sull'area; I livelli di astrazione possono essere visti in due modi:

Vista strutturale:

- Per il livello logico è la mappatura di porte logiche;
- Per il livello architetturale corrisponde ad uno schema a blocchi;

Vista comportamentale

- Per il livello logico è una macchina a stati finiti;
- Per il livello architetturale corrisponde al codice sorgente;



### Le diverse “view” di un sistema embedded.

Un sistema embedded può essere descritto secondo diverse *view* (punti di vista), che permettono di analizzare e progettare lo stesso sistema a diversi livelli di dettaglio e astrazione, mantenendo separati il *comportamento* e la *struttura*.

#### Behavioral view (vista comportamentale).

La vista comportamentale descrive *cosa fa* il sistema, senza specificare come è realizzato fisicamente. Si concentra sulla sequenza delle operazioni, sugli algoritmi e sul flusso di controllo (ad esempio: incremento del PC, fetch e decode delle istruzioni). È tipica delle descrizioni ad alto livello ed è indipendente dall’implementazione hardware.

#### Structural view (vista strutturale).

La vista strutturale descrive *come è fatto* il sistema, cioè i componenti che lo compongono e le loro interconnessioni. Include blocchi funzionali come ALU, unità di controllo, memoria e bus, mostrando l’organizzazione interna dell’hardware.

#### Livello architetturale (Architectural level).

A livello architetturale, la vista comportamentale esprime le funzioni del processore e del sistema (operazioni, istruzioni, flussi), mentre la vista strutturale mostra i macro-blocchi hardware e le loro relazioni. Questo livello è cruciale per il co-design hardware/software negli embedded systems.

#### Livello logico (Logic level).

A livello logico, la vista comportamentale può essere rappresentata da automi a stati finiti (FSM), che modellano il controllo del sistema, mentre la vista strutturale è descritta tramite porte logiche e reti combinatorie e sequenziali. Qui si entra nel dettaglio dell’implementazione digitale.

### Importanza della separazione tra view e livelli.

La separazione tra vista comportamentale e strutturale, applicata a diversi livelli di

astrazione, consente di gestire la complessità dei sistemi embedded, facilitare la progettazione modulare e supportare l'uso efficace dei linguaggi HDL e degli strumenti EDA.

#### **Livelli di astrazione e progetto.**

A ogni livello di astrazione corrisponde un livello di progetto. Scendendo da un livello di astrazione più alto a uno più basso si passa progressivamente da descrizioni più generali a descrizioni più dettagliate del sistema.

#### **Fasi di sintesi.**

Ogni passaggio da un livello di astrazione superiore a uno inferiore corrisponde a una **fase di sintesi**. La sintesi è una fase di ottimizzazione del progetto, finalizzata a soddisfare specifici vincoli richiesti (ad esempio prestazioni, area o consumo).

#### **Sintesi architetturale.**

La sintesi architetturale parte da una descrizione del **comportamento architetturale** e determina la **struttura macroscopica** del sistema, definendo i macro-blocchi principali e le loro interconnessioni.

#### **Sintesi logica.**

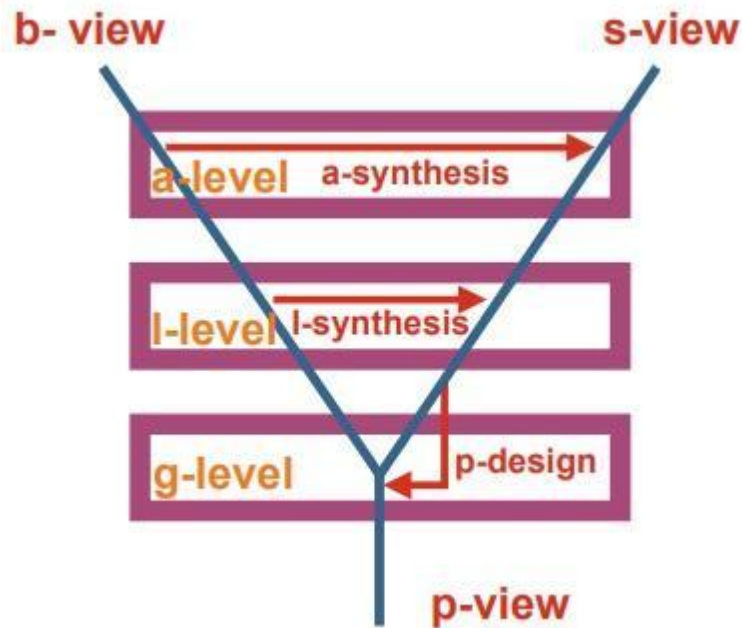
La sintesi logica parte da una descrizione del **comportamento logico** e produce la **struttura microscopica** del sistema, espressa in termini di porte logiche e delle relative interconnessioni.

#### **Sintesi geometrica.**

La sintesi geometrica riguarda la realizzazione fisica del circuito e determina il **layout**, cioè la definizione geometrica delle porte logiche, la loro posizione sul chip e le interconnessioni fisiche.

#### **Collegamento tra le viste.**

Durante queste fasi si passa progressivamente dalla **behavioral view** alle **structural e physical view**, mantenendo invariato il comportamento del sistema ma raffinando sempre di più la descrizione.



### HDL e flusso di modellazione.

I linguaggi HDL (come VHDL e SystemVerilog) sono utilizzati per descrivere il sistema lungo l'intero flusso di modellazione, passando da descrizioni più astratte a descrizioni sempre più concrete e vicine all'hardware.

### Behavioral view e modelli astratti.

Nella behavioral view, una descrizione HDL viene compilata per ottenere **modelli astratti** che rappresentano le **operazioni e le loro dipendenze**, tipicamente sotto forma di grafi di flusso dei dati e di sequenziamento. Questa rappresentazione è collocata al **livello architetturale**.

### Sintesi e ottimizzazione architetturale.

A partire dai modelli astratti di operazioni e dipendenze, si applicano processi di **sintesi architetturale e ottimizzazione**, che trasformano il comportamento in una struttura più definita.

### Modelli logici e FSM.

Sempre tramite compilazione HDL, il sistema può essere rappresentato tramite **FSM e funzioni logiche**, espresse come diagrammi di stato e reti logiche. Questo corrisponde al **livello logico** del progetto.

### Sintesi e ottimizzazione logica.

I modelli basati su FSM e funzioni logiche sono sottoposti a **sintesi logica e ottimizzazione**, che raffinano ulteriormente la struttura del sistema.

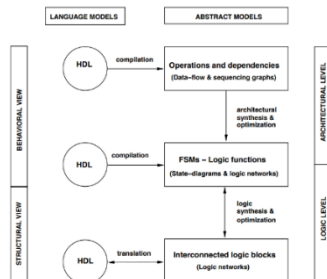
### Structural view e reti logiche.

Nella structural view, la descrizione HDL viene tradotta in **blocchi logici interconnessi**, cioè reti logiche che rappresentano direttamente la struttura dell'hardware.

## Relazione tra viste e livelli.

Il flusso mostra come, partendo da una descrizione HDL unica, sia possibile attraversare behavioral view e structural view, passando dal **livello architetturale** al **livello logico** attraverso fasi successive di compilazione, sintesi e ottimizzazione.

## HDLs (VHDL, SystemVerilog) and the Modeling Flow



## Ottimizzazione della sintesi

Nella sintesi software la macchina a stati è già definita, mentre nella sintesi hardware la macchina a stati viene generata durante il processo di sintesi. Questo processo di generazione dell'hardware comporta un incremento dell'informazione, poiché si passa da una descrizione astratta a una realizzazione sempre più dettagliata.

La sintesi è quindi un processo di ottimizzazione, cioè una ricerca funzionale tra diverse possibili configurazioni hardware, ciascuna caratterizzata da un certo "costo" o "energia" da minimizzare.

L'ottimizzazione avviene sulla base di vincoli, definiti tramite metriche di progetto, ovvero variabili funzionali di interesse che guidano la scelta dell'implementazione migliore.

## Metriche di ottimizzazione nei circuiti integrati

Le principali metriche considerate nella sintesi dei circuiti integrati sono:

Area occupata.

L'area è una proprietà estensiva: se un circuito svolge il doppio delle funzioni, occupa approssimativamente il doppio dell'area.

Performance.

La performance indica quanto velocemente il sistema può operare. Non è una proprietà estensiva e non ha una definizione univoca, poiché dipende dal tipo di circuito:

- nei circuiti combinatori è descritta dal ritardo di propagazione e dal cycle-time;

- nei circuiti sequenziali è descritta dalla latenza, cioè il tempo che intercorre tra il momento in cui i dati sono validi e quello in cui le uscite riflettono il cambiamento di stato;
- nei circuiti pipelined è descritta dal throughput, ovvero il numero di risultati prodotti per unità di tempo.

Nel caso delle pipeline, la latenza rimane costante, mentre aumenta il throughput; di conseguenza, la metrica rilevante non è più la latenza ma il throughput.

Testabilità.

Misura quanto facilmente il circuito può essere testato per individuare eventuali guasti.

Potenza dissipata.

Indica l'energia consumata dal circuito durante il funzionamento.

## **Metodologia della sintesi: elementi fondamentali**

Spazio di progettazione (S).

Lo spazio di progettazione include tutte le possibili implementazioni che soddisfano il comportamento desiderato. Ogni implementazione rappresenta un punto nello spazio S.

Spazio delle funzioni di valutazione (E).

Lo spazio E è ottenuto applicando funzioni di valutazione (metriche di progetto) a ogni punto di S. In pratica, E è l'immagine di S dopo la valutazione delle implementazioni.

Metriche di progetto (N).

Le metriche di progetto sono i criteri utilizzati per valutare le implementazioni (ad esempio area, prestazioni, consumo). Il numero di metriche determina la dimensione dello spazio E, che può essere multidimensionale.

## **Processo di ottimizzazione**

Dato lo spazio S e il corrispondente spazio E, la scelta dell'implementazione non è univoca. È quindi necessario introdurre ulteriori informazioni attraverso un processo di ricerca di un'implementazione ottima tra quelle funzionalmente corrette.

Un'implementazione ottima corrisponde a un minimo di una funzione di costo definita sulle metriche di progetto.

## Complessità del processo di sintesi

Dal punto di vista della complessità algoritmica, il processo di ottimizzazione è intrattabile. Il problema è multidimensionale e coinvolge un numero elevato di variabili, rendendo impossibile una soluzione ottima tramite algoritmi standard.

Per questo motivo, invece di cercare soluzioni perfette, si ricercano soluzioni sub-ottimali, scomponendo il problema in sotto-problemi di dimensione inferiore. In questo modo si adottano approcci euristici, che permettono di ottenere soluzioni efficaci e gestibili.

## Livelli di ottimizzazione e obiettivi

Livello architetturale.

A questo livello gli obiettivi principali sono:

- determinare il tempo di ciclo;
- minimizzare l'area rispettando i vincoli di latenza;
- minimizzare la latenza rispettando i vincoli di area.

Livello logico.

A questo livello gli obiettivi sono:

- minimizzare l'area rispettando i vincoli di ritardo di propagazione;
- minimizzare il ritardo di propagazione rispettando i vincoli di area.

Gli strumenti **EDA (Electronic Design Automation)** permettono di **catturare l'idea progettuale** attraverso modelli HDL, che descrivono il comportamento e la struttura del sistema.

A partire dai modelli HDL, gli strumenti EDA consentono di **sintetizzare progressivamente il progetto**, passando da livelli di astrazione più elevati a livelli più bassi.

Durante questo processo, gli strumenti EDA **ottimizzano il circuito e i parametri di progetto** sulla base di **metriche di progetto scelte** (come area, prestazioni, potenza e testabilità).

A livello di sistema, gli strumenti EDA permettono di **introdurre il supporto hardware/software (hw/sw)** e di **sintetizzare automaticamente hardware, software e interfacce**.

### Esempio di Sintesi

#### ESEMPIO EQUAZIONE DIFFERENZIALE

L'esempio mostra come una **equazione differenziale del secondo ordine** possa essere trasformata in un **algoritmo iterativo** implementabile come **hardware (o software embedded)**.

L'equazione da risolvere è:

$$y'' + 3xy' + 3y = 0$$

con condizioni iniziali:

$$x(0) = 0, \quad y(0) = y_0, \quad y'(0) = u_0, \quad x \in [0, a].$$

#### Introduzione della variabile ausiliaria

Per semplificare il problema, si introduce:

$$u = y'$$

In questo modo l'equazione del secondo ordine viene trasformata in un **sistema di equazioni del primo ordine**:

- $\frac{dy}{dx} = u$
- $\frac{du}{dx} + 3xu + 3y = 0$

#### Discretizzazione (derivation 1)

Si passa dal continuo al discreto usando un **passo di integrazione  $dx$** .

Le derivate vengono approssimate tramite incrementi:

- $u \approx u_0 - (3xu + 3y) dx$
- $y \approx y_0 + u dx$

Queste equazioni descrivono **come aggiornare i valori di  $u$  e  $y$**  a ogni passo.

#### Forma iterativa (derivation 2)

Le equazioni discrete vengono usate in modo **iterativo**:

- $x_l = x + dx$
- $u_l = u - (3 \cdot x \cdot u \cdot dx) - (3 \cdot y \cdot dx)$
- $y_l = y + u \cdot dx$

Dopo ogni iterazione, i nuovi valori diventano quelli correnti:

$$x \leftarrow x_l, u \leftarrow u_l, y \leftarrow y_l$$

### Struttura del codice *diffeq*

Il blocco *diffeq* rappresenta l'**implementazione algoritmica** del metodo numerico:

- `read(x, y, u, dx, a)`  
legge le condizioni iniziali e i parametri.
- `repeat { ... } until (x < a)`  
implementa il **ciclo iterativo** sull'intervallo  $[a]$ .
- Gli aggiornamenti  $x_l$ ,  $u_l$ ,  $y_l$   
corrispondono alle **equazioni discretizzate**.
- `write(x, y)`  
produce il risultato finale.

### Significato nel contesto Embedded / Sintesi

Questo esempio mostra che:

- un **problema matematico continuo** può essere riscritto come **sequenza di operazioni discrete**;
- tale sequenza può essere descritta come **comportamento** (HDL);
- il comportamento può poi essere **sintetizzato** in hardware (FSM + datapath).

È un esempio tipico di **behavioral description** che può essere trasformata, tramite sintesi, in una struttura hardware.

```

diffeq
{
  read (x, y, u, dx, a);
  repeat
  {
    x1 = x + dx;
    u1 = u - (3 * x * u * dx) - (3 * y * dx);
    y1 = y + u * dx;
    c = x < a;
    x = x1; u = u1; y = y1;
  } until (c);
  write (x, y);
}

```

## COMPONENTI

**Si può osservare che ci serviranno almeno 1 moltiplicatore, ed 1 ALU (perché ci servono sia sommatore che sottrattori) e 1 unità di controllo/memoria (per salvare i risultati parziali);**

Congruentemente a quello che abbiamo visto essere il flusso di sintesi, **dobbiamo passare da una rappresentazione testuale (sequenziale) ad una "funzionale"**.

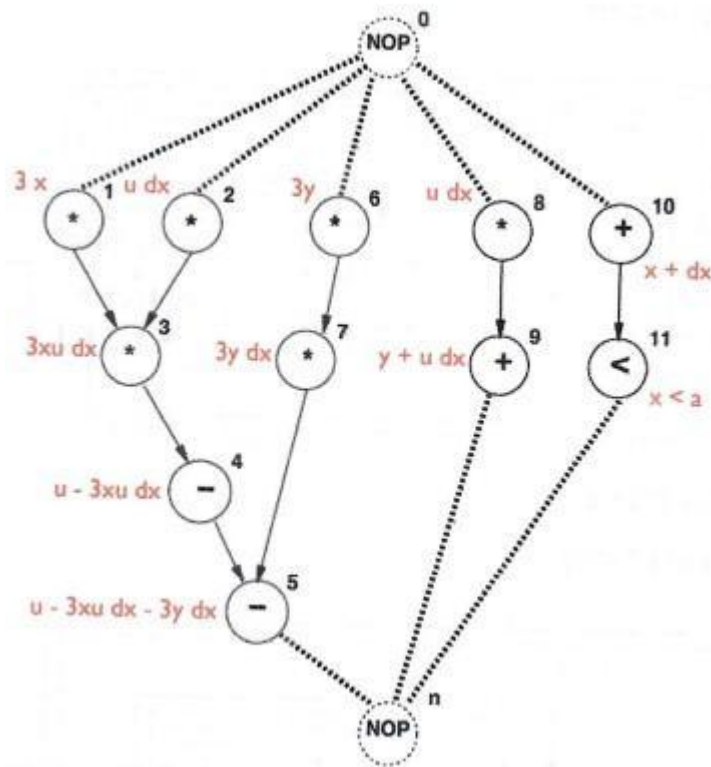
Inizieremo da una forma intermedia che di fatto è un grafo di esecuzione.

Da questa rappresentazione funzionale si vede il parallelismo e quindi anche il numero di componenti che mi servono

## DATA FLOW GRAPH

Questa serie di istruzioni che ciclerò N volte, mi definisce il cosiddetto direct acyclic graph (DAG) che ha un punto di partenza che è un "no istruzione" (NOP).

**Negli alberi, per passare al prossimo ramo, devo aver concluso tutte le operazioni su quel livello.**



### QUAL E' IL NUMERO DI COMPONENTI OTTIMALI?

Per scegliere il numero "ottimale" di componenti (es. moltiplicatori e ALU) bisogna valutare il **costo** della soluzione sia in **area** sia in **latenza**. In pratica: più risorse metto, più posso fare operazioni in parallelo (potenziale riduzione della latenza), ma aumento l'area.

### Definizione di latenza

La **latenza** è l'intervallo di tempo che passa da quando gli **ingressi** di un blocco sono validi a quando sono valide le **uscite** corrispondenti. Per calcolarla devo capire l'**ordine delle operazioni** e quante operazioni posso eseguire **in parallelo** a ogni step temporale.

### Assunzioni sui costi (figure di merito)

Si assume che le metriche principali siano **area** e **performance**, e che i costi dei blocchi siano:

- **Moltiplicatore:** area = 5, latenza = 1 (unità di tempo)
- **ALU:** area = 1, latenza = 1
- **Unità di controllo + memoria:** area = 1, latenza = 0 (non “consuma tempo”) *(nota: nella realtà questi valori possono essere diversi, ma qui sono semplificati).*

### Caso studiato: soluzione (1,1)

Si considera il caso con:

- **1 moltiplicatore**
- **1 ALU**
- **1 unità di controllo/memoria**

Questa configurazione viene indicata come soluzione **(1,1)**.

### Calcolo dell'area (proprietà estensiva)

L'area è una **proprietà estensiva**: se raddoppio il numero di componenti, raddoppia il costo in area.

Nel caso (1,1) il costo totale in area è:

- Moltiplicatore: 1·5
- ALU: 1·1

- Controllo/memoria: 1·1

Quindi:

$$Area = 5 + 1 + 1 = 7$$

### Calcolo della latenza: scheduling delle operazioni

Per calcolare la latenza devo definire uno **scheduling**: in quali **step temporali** (clock) eseguo le operazioni.

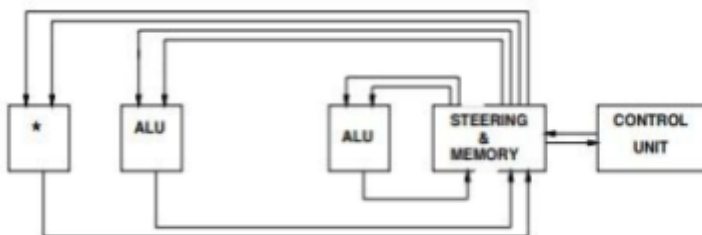
Con **1 moltiplicatore** e **1 ALU**, a ogni colpo di clock posso fare:

- **al massimo 1 moltiplicazione** (perché ho un solo moltiplicatore)
- **al massimo 1 operazione ALU** (somma/sottrazione/confronto) per volta

Guardando l'ordine/parallelismo possibile (come nello schema con gli step numerati), si ottiene che per completare tutte le operazioni servono **7 step**.

Quindi la **latenza totale** della soluzione (1,1) è:

LATENZA = 7

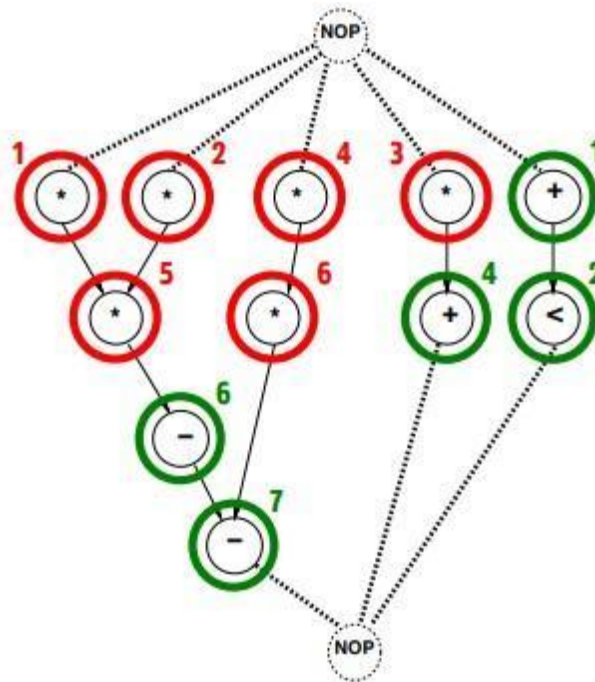


| 1 MUL; 1 ALU (1, 1) |       |         |
|---------------------|-------|---------|
| Resource            | Area  | Latency |
| Multiplier          | 1 x 5 | 1       |
| ALU                 | 1 x 1 | 1       |
| Steering & Control  | 1     | 0       |
| Total               | 7     | 7       |

### LATENZA

Per capire la latenza, devo determinare **come posso eseguire le operazioni nei diversi casi**. Indico gli step temporali con i quali faccio fare una specifica operazione;

ad ogni colpo di clock scandito dalla macchina, gli stati possono fare una moltiplicazione ed una somma/sottrazione/confronto per volta;



Si ottiene quindi una latenza pari a 7, per concludere tutte le operazioni.

### Soluzione (2,1)

In questa configurazione si dispone di **due moltiplicatori**, **una sola ALU** e un'**unità di controllo con memoria**. L'aumento del numero di moltiplicatori consente di **eseguire più moltiplicazioni in parallelo**, riducendo la latenza complessiva rispetto al caso (1,1).

### AREA (proprietà estensiva)

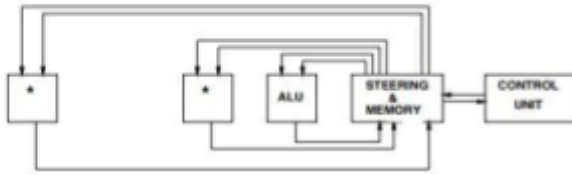
L'area è una **proprietà estensiva**, quindi cresce linearmente con il numero di risorse:

- Moltiplicatori:  $2 \times 5 = 10$
- ALU:  $1 \times 1 = 1$
- Steering & Control: 1

Il costo totale in area è:

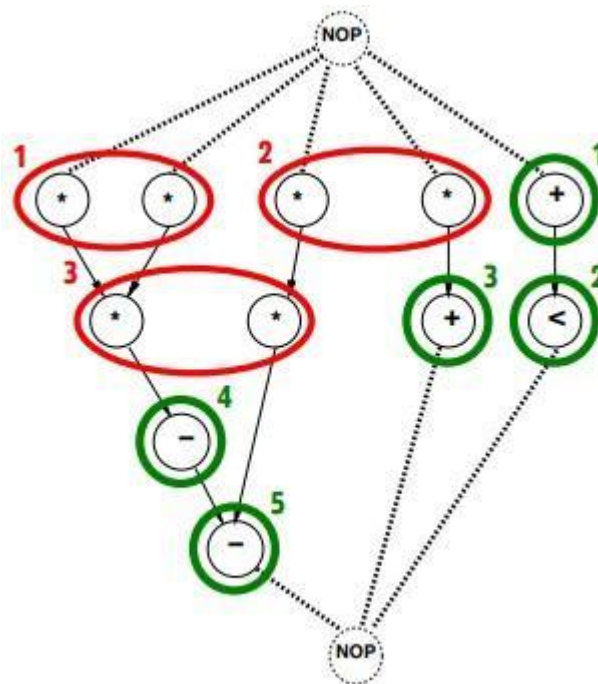
$$Area = 10 + 1 + 1 = 12$$

- Cost o =  $2 * 5 + 1 * 1 + 1 * 1 = 12$



| 2 MUL; 1 ALU (2, 1) |       |         |
|---------------------|-------|---------|
| Resource            | Area  | Latency |
| Multiplier          | 2 x 5 | 1       |
| ALU                 | 1 x 1 | 1       |
| Steering & Control  | 1     | 0       |
| Total               | 12    | 5       |

## LATENZA



Con 2 moltiplicatori, a ogni colpo di clock è possibile eseguire fino a due moltiplicazioni in parallelo. Tuttavia, avendo una sola ALU, le operazioni di somma/sottrazione/confronto restano serializzate.

Nel diagramma temporale:

- le operazioni cerchiare in rosso (moltiplicazioni) vengono accoppiate nello stesso step quando possibile;
- le operazioni cerchiare in verde (ALU) sono eseguite una per step.

### Calcolo della latenza

Grazie al parallelismo sulle moltiplicazioni, il numero totale di step necessari per completare tutte le operazioni si riduce. Dallo scheduling mostrato si ottiene una latenza totale pari a 5.

$$Latenza = 5$$

### Soluzione (1,2)

In questa configurazione sono presenti **un solo moltiplicatore**, **due ALU** e un'unità di **controllo con memoria**. L'aumento del numero di ALU permette di **parallelizzare le operazioni aritmetico-logiche**, mentre le moltiplicazioni restano **serializzate** perché c'è un solo moltiplicatore.

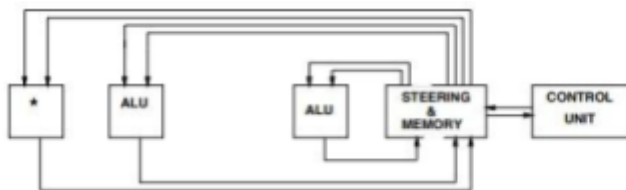
### AREA (proprietà estensiva)

L'area è una **proprietà estensiva**:

- Moltiplicatore:  $1 \times 5 = 5$
- ALU:  $2 \times 1 = 2$
- Steering & Control: 1

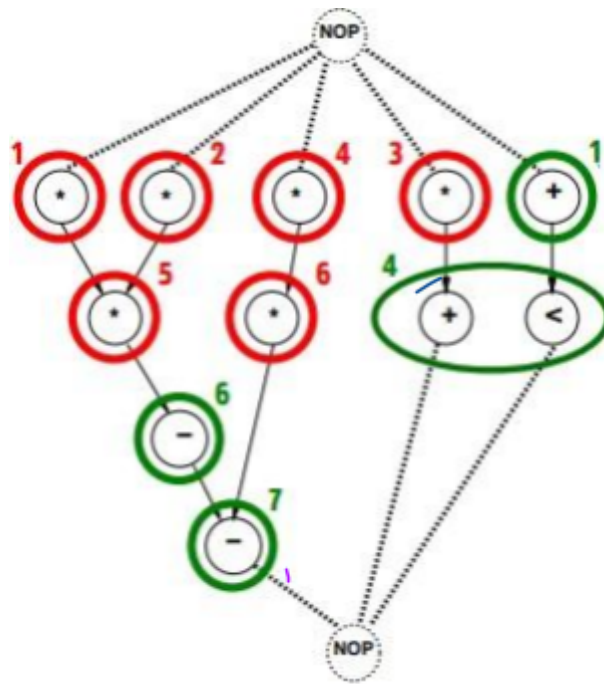
Il costo totale in area è:

$$Area = 5 + 2 + 1 = 8$$



| 1 MUL; 2 ALU (1, 2) |       |         |
|---------------------|-------|---------|
| Resource            | Area  | Latency |
| Multiplier          | 1 x 5 | 1       |
| ALU                 | 2 x 1 | 1       |
| Steering & Control  | 1     | 0       |
| Total               | 8     | 7       |

### LATENZA



Con 1 moltiplicatore, a ogni colpo di clock è possibile eseguire una sola moltiplicazione; per questo motivo le moltiplicazioni (cerchi rossi) devono essere distribuite su più step temporali.

Con 2 ALU, invece, è possibile eseguire due operazioni ALU in parallelo (somma/sottrazione/confronto), come evidenziato dai cerchi verdi accoppiati nello stesso step.

Calcolo della latenza

Nonostante il parallelismo sulle ALU, la latenza complessiva resta vincolata dal numero di moltiplicazioni, che non possono essere accelerate avendo un solo moltiplicatore. Dallo scheduling mostrato si ottiene quindi una latenza totale pari a 7.

*Latenza = 7*

### Soluzione (2,2)

In questa configurazione sono presenti **due moltiplicatori, due ALU e un'unità di controllo con memoria**. È la soluzione con **massimo parallelismo** tra quelle considerate, perché consente di eseguire **in parallelo sia le moltiplicazioni sia le operazioni ALU**.

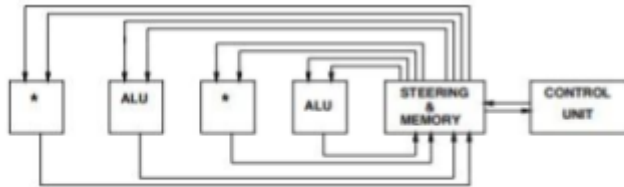
### Calcolo dell'area

L'area è una **proprietà estensiva**:

- Moltiplicatori:  $2 \times 5 = 10$
- ALU:  $2 \times 1 = 2$
- Steering & Control: 1

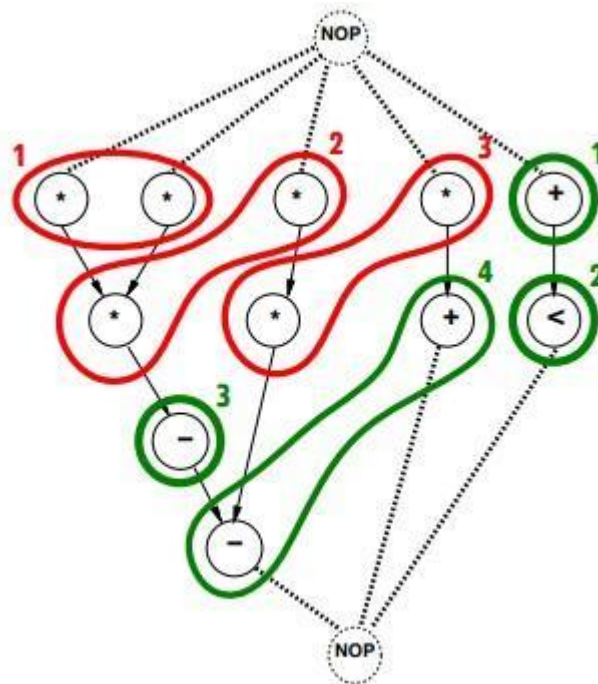
Il costo totale in area è:

$$\text{Area} = 10 + 2 + 1 = 13$$



| 2 MUL; 2 ALU (2, 2) |       |         |
|---------------------|-------|---------|
| Resource            | Area  | Latency |
| Multiplier          | 2 x 5 | 1       |
| ALU                 | 2 x 1 | 1       |
| Steering & Control  | 1     | 0       |
| Total               | 13    | 4       |

**LATENZA**



Con 2 moltiplicatori, a ogni colpo di clock è possibile eseguire due moltiplicazioni in parallelo (operazioni cerchiare in rosso).

Con 2 ALU, è possibile eseguire due operazioni aritmetico-logiche in parallelo (cerchi verdi).

Lo scheduling mostrato sfrutta completamente il parallelismo disponibile, riducendo al minimo il numero di step temporali necessari.

#### Calcolo della latenza

Grazie al parallelismo sia sulle moltiplicazioni sia sulle operazioni ALU, tutte le operazioni vengono completate in 4 step temporali.

*Latenza* = 4

#### Confronto qualitativo

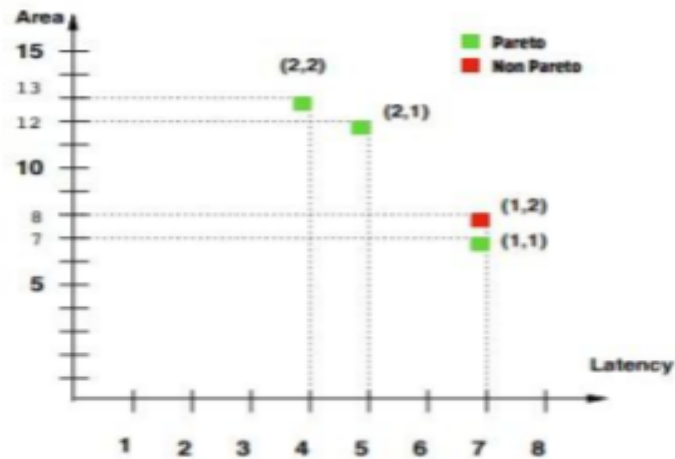
Rispetto alle altre soluzioni:

- la latenza è minima;
- l'area è massima.

Questa configurazione rappresenta l'estremo del trade-off area–latenza: massime risorse hardware per ottenere il minimo tempo di esecuzione.

#### Analisi Soluzioni

Rappresentando le diverse soluzioni in un **grafico area–latenza**, è possibile confrontarle in modo oggettivo. Dal confronto grafico emergono soluzioni che sono **oggettivamente peggiori** di altre, cioè soluzioni per cui esiste almeno un'altra configurazione che ha **area minore e latenza minore**. Questi punti vengono detti **dominati** e possono essere eliminati dal processo di scelta. Nel grafico, la soluzione **(1,2)** è un punto **non Pareto**, perché ha la stessa latenza di (1,1) ma un'area maggiore. Una volta eliminati i punti dominati, si ottiene la **curva di Pareto** (o frontiera di Pareto). Essa è l'insieme dei **punti di Pareto**, cioè delle soluzioni per cui **non è possibile migliorare una metrica senza peggiorarne un'altra**. La curva di Pareto rappresenta l'insieme delle **soluzioni ammissibili** nel processo di ottimizzazione. La scelta finale tra i punti di Pareto dipende dai **vincoli di progetto**: ad esempio, se il vincolo principale è la latenza si sceglierà un punto, se invece è l'area se ne sceglierà un altro.



## S

### Non Scheduled Execution Graph

Il **Non Scheduled Execution Graph** rappresenta l'insieme delle **operazioni da eseguire** e delle loro **dipendenze** logiche, **senza indicare quando** (a quale istante di clock) esse verranno eseguite.

Il grafo mostra solo l'**ordine parziale imposto dalle dipendenze dei dati**, non uno scheduling temporale.

### Concetto di Scheduling

Con **scheduling** si intende il processo di determinare **quando**, cioè **a quale istante di clock**, deve essere eseguita una determinata operazione.

Lo scheduling assegna quindi a ogni nodo del grafo un **tempo di esecuzione**, rispettando le dipendenze.

Lo scheduling può essere effettuato:

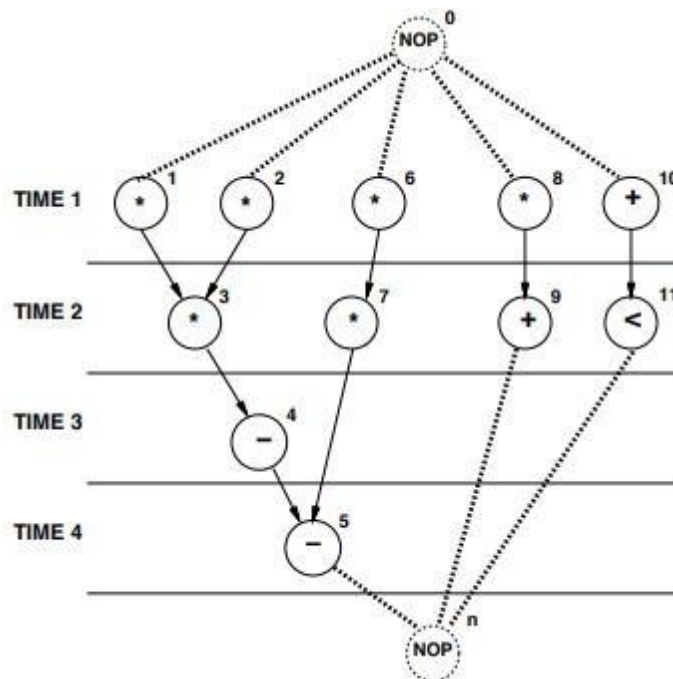
- **senza vincoli sulle risorse** (risorse infinite);
- **con vincoli sulle risorse** (numero limitato di moltiplicatori, ALU, ecc.).

Esistono tre tipi di Scheduling:

### ASAP Scheduling (As Soon As Possible)

Le operazioni sono eseguite il prima possibile, schedulando i vertici partendo dal primo, assegnato al vertice  $vi$  il tempo di esecuzione come il **massimo tra quelli già schedulati + il suo ritardo di propagazione**;

Lo **ASAP scheduling** è una tecnica di scheduling che assegna a ogni operazione il **primo istante di clock possibile**, compatibilmente con le **dipendenze** tra le operazioni. Non tiene conto di vincoli sul numero di risorse disponibili.

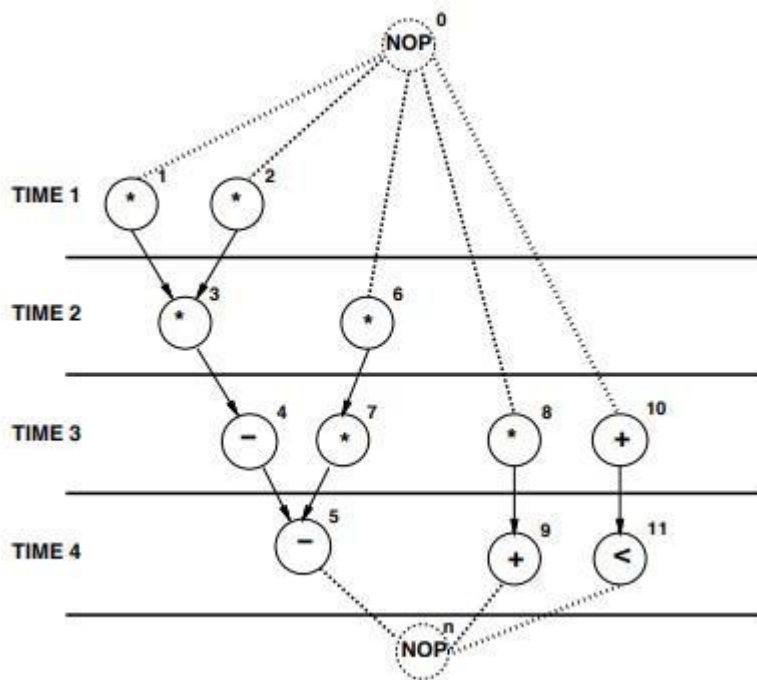


### ALAP Scheduling (As Late As Possible)

Lo **ALAP scheduling** assegna a ogni operazione l'**istante di clock più tardivo possibile**, senza violare le dipendenze del grafo e **fissata una latenza finale**. A differenza dell'ASAP, procede **all'indietro nel tempo**. Nel metodo ALAP si parte **dall'ultimo vertice** del grafo (nodo finale) e si assegna a questo nodo l'ultimo istante di clock consentito dalla latenza complessiva. Per ogni operazione precedente, il tempo di esecuzione viene calcolato **sottraendo i ritardi di propagazione** delle operazioni successive.

In pratica, un'operazione viene schedulata **il più tardi possibile**, purché consenta alle

operazioni dipendenti di iniziare in tempo.



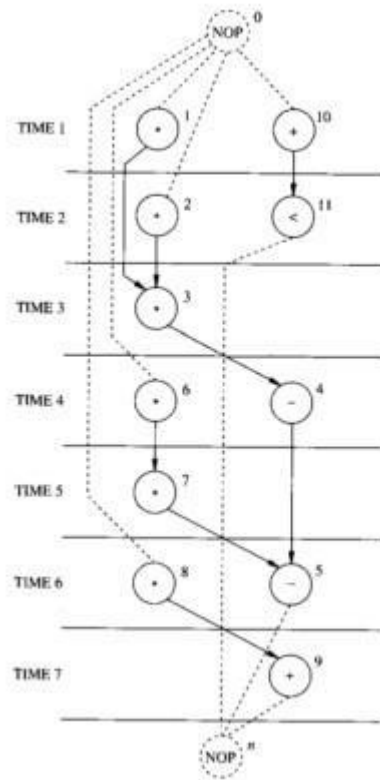
### Resource Constrain Scheduling

Il **resource-constrained scheduling** viene eseguito **dopo** aver determinato uno scheduling **ASAP o ALAP**. Questi due forniscono i limiti temporali entro cui le operazioni possono essere collocate. Dai risultati di ASAP e ALAP si ottiene, per ciascuna operazione, un **intervallo temporale ammesso**. All'interno di questi limiti, il grafo viene **riallocato nel tempo** per rispettare i vincoli sulle risorse disponibili. Nel resource-constrained scheduling si assume un **numero limitato di risorse hardware** (ad esempio un certo numero di ALU o moltiplicatori).

Di conseguenza, **non è possibile eseguire simultaneamente due operazioni dello stesso tipo** se esiste una sola risorsa disponibile per quel tipo. Per evitare la **sovrapposizione nell'uso delle risorse**, alcune operazioni vengono **posticipate** rispetto allo scheduling ASAP.

In ogni intervallo di tempo, quindi, **non esiste la copresenza di due operazioni che richiedono la stessa risorsa**. Questo tipo di scheduling produce uno scheduling **realizzabile in hardware**, perché tiene conto delle reali risorse disponibili.

È il passaggio che rende lo scheduling compatibile con la successiva fase di **resource binding**.



## Binding

Il binding è la fase del progetto in cui si decide come associare gli elementi del comportamento (operazioni) agli elementi strutturali (risorse hardware). Dopo scheduling, il binding stabilisce chi fa cosa nel circuito.

### Resource Binding

Con il resource binding si assegnano le operazioni del grafo schedulato alle risorse hardware disponibili (ad esempio moltiplicatori, ALU, comparatori).

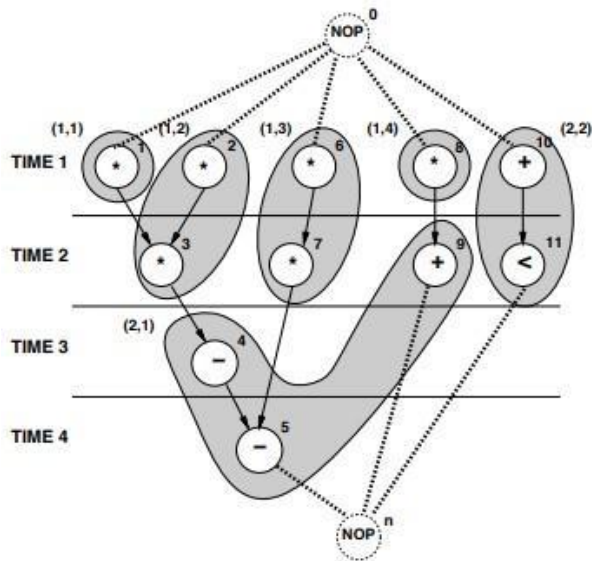
Ogni risorsa esegue, nel tempo, più operazioni diverse, secondo quanto stabilito dallo scheduling.

### Ruolo della macchina a stati

L'assegnazione delle operazioni alle risorse è gestita tramite una macchina a stati.

Ogni stato della FSM corrisponde a uno o più istanti di clock e specifica:

- quale operazione deve essere eseguita,
- su quale risorsa,
- con quali ingressi e uscite.



## Sintesi FPGA'S

### Hardware Synthesis

La **sintesi hardware** è il processo che trasforma una **specifica comportamentale** nell'hardware **che la implementa**.

La specifica in ingresso **deve indicare cosa il circuito deve fare**, ma **non come** il circuito finale deve essere realizzato fisicamente.

### Definizioni

#### Abstract Behavior

Descrive il comportamento del circuito in termini di:

- variabili lette e scritte,
- condizioni di lettura e scrittura,
- valori temporanei,
- valori finali delle uscite,
- relazioni temporali.

La specifica **non contiene informazioni sulla struttura del circuito**.

#### Control-Flow Behavior

Descrive il comportamento del circuito in termini di:

- registri,
- logica combinatoria,
- reazioni del sistema,
- ordine di esecuzione delle operazioni.

## Datapath

Il **datapath** è la **catena di risorse hardware** (ALU, moltiplicatori, registri, ecc.) necessarie per eseguire le operazioni richieste dal comportamento.

## Livelli di Astrazione

Ci sono diversi livelli di astrazione nel descrivere un circuito:

**Gate Level:** Descrizione a livello di porte logiche;

**Logic Level:** Simile al gate level, ma in termini di funzioni booleane;

**Register- Transfer Level:** Descrive i trasferimenti di dati tra registri e le operazioni combinatorie. Questo livello include una parte comportamentale (come i dati si spostano) e una parte strutturale (come i registri sono connessi tra loro):

**Parte Comportamentale (register-transfer behavior):** Descrive il comportamento del circuito a livello RTL. Vengono rappresentate solamente le transazioni visibili a questo livello. I segnali di controllo o sono dati per impliciti o espressi a un livello astratto;

**Parte Strutturale (register-transfer structure):** Descrive il circuito attraverso una descrizione strutturale a livello RT nella quale sono specificati i registri, gli operatori funzionali e le loro interconnessioni. Il controllo si suppone ricompreso implicitamente nella descrizione o specificato a parte;

**Nei circuiti Custom, devo mappare una descrizione dell'hardware che voglio implementare sulle risorse disponibili.** Definisco quindi:

## Obiettivi della sintesi

Idealmente, la sintesi mira a:

- massimizzare la **velocità**;
- minimizzare l'**area** e l'occupazione di risorse;
- minimizzare i **consumi di potenza**;
- ridurre il **tempo di progettazione**;
- massimizzare **affidabilità** e **testabilità** del circuito.

## Vincoli della sintesi

Il processo di sintesi è soggetto a diversi vincoli, tra cui:

- limiti tecnologici (es. assenza di tristati, memoria integrata);
- ritardi temporali tra eventi;
- limiti dell'area;
- numero di pin disponibili;
- limiti sul tempo di esecuzione;
- vincoli di affidabilità;
- vincoli di testabilità.

La sintesi si compone dei seguenti passi:

- **Sintesi dai livelli di astrazione superiori a quelli più bassi:** Può avvenire manualmente o automaticamente.
- **Allocare le risorse** in cui si implementano delle ottimizzazioni;
- **Design Transformation**, cioè trasformazioni per soddisfare i vincoli;
- **Composizione/Decomposizione**, cioè il raggruppamento/suddivisione dei blocchi funzionali per corrispondere ai blocchi tecnologici disponibili
- **Scheduling**, per assegnare gli istanti di tempo ai quali svolgere le diverse operazioni. Questa esecuzione schedulata sarà poi mappata sui vari registri e sulle risorse e verranno instradati i vari percorsi in modo tale da realizzare una determinata operazione.
- **Binding:** cioè, l'assegnazione delle operazioni alle risorse disponibili. Avrò quindi una ALU (se non la ho me la realizzo).

## TIPI DI SINTESI

**Sintesi Comportamentale:** Traduce il comportamento astratto e algoritmico in una rappresentazione a flusso di dati.

**Sintesi RTL (Register-Transfer Level):** Converte la rappresentazione a flusso di dati in una a livello di trasferimento tra registri.

**Sintesi Logica:** Converte la rappresentazione RTL in una logica basata su porte.

### 1) From Behavioral to Scheduled Behaviors

La descrizione **behavioral** specifica **quali operazioni** devono essere eseguite, ma **non quando**.

Con lo **scheduling** (ASAP, ALAP o vincolato dalle risorse) si assegna a ogni operazione un **istante di clock**, ottenendo un **comportamento schedulato**.

Il risultato è un comportamento temporizzato che rispetta le dipendenze e (se richiesto) i vincoli sulle risorse.

### 2) From Scheduled to Datapath Behaviors

A partire dal comportamento schedulato si costruisce il **datapath behavior**.

In questa fase:

- si introducono i **registri** per memorizzare dati e risultati intermedi;
  - si definiscono gli **operatori** (ALU, moltiplicatori) attivi a ciascun clock;
  - si esplicita il **flusso dei dati** tra registri e operatori nel tempo.
- Il **control flow** (non completamente mostrato) stabilisce selezioni, carichi e operazioni a ogni ciclo.

### 3) From Datapath Behaviors to RTL

Il **datapath behavior** viene trasformato in una descrizione **RTL (Register-Transfer Level)**.

A livello RTL:

- registri e blocchi funzionali sono **esplicitamente definiti**;
- i **trasferimenti di dati** avvengono sui fronti di clock;
- il **controllo** è espresso tramite segnali che abilitano selezioni e load.

Questa è la rappresentazione tipica in **VHDL/Verilog RTL**.

#### **4) From RTL to Logic Structure**

La descrizione RTL viene convertita in **struttura logica**.

In questa fase:

- i registri diventano **flip-flop**;
- gli operatori (ALU, adders, ecc.) diventano **reti di porte logiche**;
- il controllo è implementato come **logica combinatoria e sequenziale**.

Il risultato è una **descrizione a livello logico**, pronta per la sintesi fisica.

## Introduzione a VHDL

### Storia di VHDL

#### VHDL: SIGNIFICATO

VHDL, acronimo di "Very High Speed Integrated Circuits HDL", è un linguaggio di descrizione hardware concepito attorno al 1980 per rispondere a specifiche esigenze del settore tecnologico. Questo linguaggio è nato con l'obiettivo di standardizzare i metodi di progettazione e unificare i vari dialetti HDL esistenti in un unico linguaggio, migliorando la portabilità dei progetti tra diversi strumenti di progettazione assistita da computer □EDA□. Grazie a VHDL, il tempo di progettazione dei circuiti digitali è stato ridotto notevolmente: un processo che richiedeva da 6 a 18 mesi è stato compresso attraverso un approccio più efficiente.

La necessità degli **HDL (Hardware Description Language)** è emersa a causa del cosiddetto "Productivity Gap", ossia un divario tra la crescente complessità dei circuiti digitali e la capacità produttiva disponibile. Per superare questa limitazione, il settore ha adottato una nuova prospettiva, passando dalla progettazione a livello di porte logiche a livello di astrazione più elevati. L'approccio VHDL permette così di descrivere circuiti in termini di linguaggio software, supportato da strumenti EDA e facilitando la transizione dalla progettazione manuale alla sintesi automatica.

#### VHDL: NASCITA

Nel giugno del 1981, durante un workshop tenutosi a Woods Hole, Massachusetts, esponenti del governo statunitense e della comunità accademica definirono le caratteristiche dei Very High Speed Integrated Circuits (VHSIC). Nel luglio del 1983, DARPA □Defense Advanced Research Projects Agency), in collaborazione con Intermetrics, IBM e Texas Instruments, firmò un contratto per lo sviluppo di VHDL, con l'obiettivo di creare uno standard per la progettazione di circuiti ad alta velocità.

#### VHDL: EVOLUZIONE

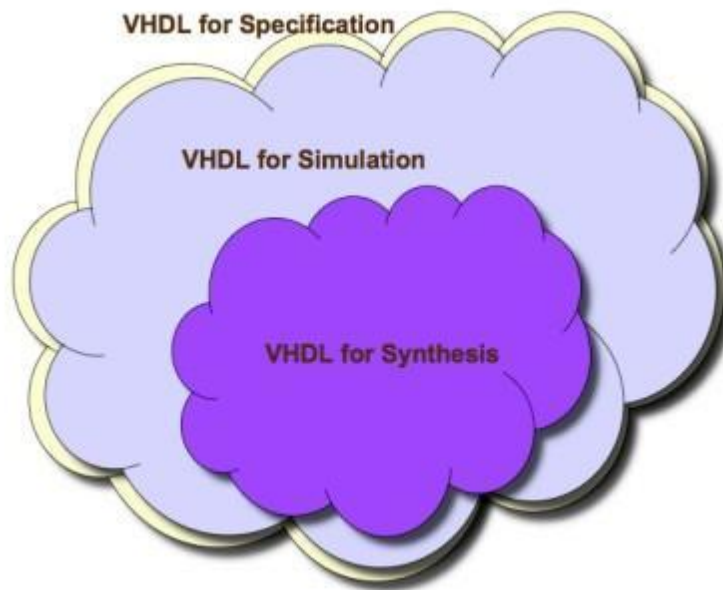
Nell'agosto del 1985 venne rilasciata la versione 7.2 di VHDL, e nel dicembre 1987 il linguaggio fu ufficialmente riconosciuto come standard IEEE. Da allora, sono state rilasciate diverse versioni successive: quella del 1987 fu seguita da aggiornamenti rilevanti come le versioni del 1993 e del 2008, mentre altre, come quelle del 2000 e del 2002, hanno avuto un impatto minore.

### La Struttura di VHDL

#### I livelli di VHDL

La struttura di VHDL si articola in **tre livelli principali**, ognuno dei quali risponde a esigenze specifiche nel processo di progettazione e verifica dei circuiti digitali:

- **VHDL for Specification:** Questo livello di VHDL è dedicato alla descrizione generale del design per verificare il comportamento funzionale del circuito hardware. A questo livello, il codice permette di testare se il progetto risponde correttamente alle specifiche iniziali, concentrandosi quindi sugli aspetti logici e comportamentali del design.
- **VHDL for Simulation:** in questo livello, il codice è scritto con l'obiettivo di consentire una simulazione accurata del circuito. VHDL for Simulation permette di testare e verificare come il circuito risponderà in diverse condizioni, attraverso una simulazione dettagliata che imita l'effettiva operatività del dispositivo. Questo livello consente di individuare eventuali errori prima della realizzazione fisica del circuito.
- **VHDL for Synthesis:** Questo livello è pensato per la generazione del circuito fisico e il codice è ottimizzato per essere interpretato e convertito in hardware reale, tipicamente FPGA o ASIC. A differenza dei livelli precedenti, VHDL for Synthesis include solo istruzioni che possono essere tradotte in componenti hardware effettivi, eliminando elementi puramente comportamentali o di simulazione.



## ENTITY & ARCHITECTURE

In VHDL, la struttura di un design è effettivamente composta da due componenti fondamentali: **Entity** e **Architecture**, che insieme definiscono completamente il comportamento del circuito.

- **Entity.** L'entity rappresenta l'interfaccia del blocco hardware, stabilendo le connessioni con l'esterno, cioè gli input e output del circuito. Qui vengono definite le porte (input e output) e i tipi di segnale che il circuito può

ricevere o inviare. In altre parole, l'entity specifica *cosa* il circuito può fare, descrivendo la sua interfaccia, ma non *come* deve realizzarlo.

- **Architecture:** L'architecture contiene la descrizione funzionale e strutturale di *come* il circuito implementa il comportamento definito nell'entity. È qui che vengono specificate le operazioni logiche, i processi e le relazioni tra i segnali interni, determinando in dettaglio le operazioni che il circuito deve eseguire. L'architecture può essere descritta a diversi livelli di astrazione, come logico o gate-level, oppure a un livello di astrazione più alto, come il livello RTL (Register Transfer Level).

Quindi, schematicamente, un design VHDL contiene:

- **Input e Output** definiti nell'entity.
- **Blocco hardware**, descritto attraverso l'architecture, che specifica come il circuito processa i segnali in ingresso per generare quelli in uscita.

Questo schema generale consente di separare chiaramente l'interfaccia del circuito (entity) dalla sua logica interna (architecture), agevolando la comprensione e la manutenzione del design.

## SYSTEM VERILOG

Accanto a VHDL, l'altro grande linguaggio di descrizione hardware è

**Verilog/SystemVerilog**. Verilog nasce nel 1984 come linguaggio di simulazione dei circuiti logici e diventa standard IEEE nel 1995; nel 2005 viene esteso in

**SystemVerilog (IEEE 1800)**, che introduce costrutti più moderni, una sintassi più compatta e funzionalità avanzate per la verifica e la modellazione ad alto livello. Oggi SystemVerilog è lo standard dominante nell'industria commerciale dei semiconduttori, mentre VHDL rimane molto diffuso in ambito europeo, militare e universitario. Dal punto di vista concettuale, entrambi i linguaggi descrivono lo stesso tipo di oggetti fisici: blocchi hardware con ingressi e uscite. In SystemVerilog questi blocchi sono chiamati **module**, mentre in VHDL sono chiamati **entity**. In entrambi i casi è possibile descrivere un circuito sia in modo **comportamentale (behavioral)**, specificando cosa deve fare, sia in modo **strutturale (structural)**, specificando come è costruito a partire da blocchi più semplici. La differenza principale tra i due linguaggi non è quindi nel tipo di hardware che possono descrivere, ma nella filosofia: VHDL è più rigoroso, fortemente tipizzato e vicino a una descrizione formale dell'hardware, mentre SystemVerilog è più compatto, flessibile e orientato alla produttività e alla verifica.

## VHDL

### Generalità

#### Case sensitivity

VHDL è un linguaggio case-insensitive, cioè non distingue tra maiuscole e minuscole. Ad esempio:

databus = DataBus = DATABUS

tutti fanno riferimento allo stesso identificatore.

### NOMI ED ETICHETTE

Per essere validi, i nomi e le etichette in VHDL devono rispettare alcune regole:

- Devono iniziare con una lettera (A–Z o a–z)
- Possono contenere lettere, cifre (0–9) e underscore \_
- Non sono ammessi due underscore consecutivi
- Non possono contenere simboli di punteggiatura (! ? . , + & ecc.)
- Devono essere **univoci** all'interno della stessa entity o architecture

### FORMATTAZIONE DEL CODICE

In VHDL, non ci sono regole convenzionali obbligatorie per la formattazione del codice, ma è buona pratica essere ordinati e, idealmente, mantenere un file separato per ogni entity. La leggibilità è migliorata rispettando spaziature e indentazioni coerenti.

### COMMENTI

I commenti in VHDL iniziano con “ - - ” e si estendono fino alla fine della riga. Non esistono commenti a blocco; i commenti devono essere brevi e utilizzati solo quando strettamente necessari per chiarire il codice. Alcuni esempi: “-- commento “

### Dichiarazioni

#### LIBRARY DECLARATION

**Library Declaration** importa librerie esterne necessarie per il codice VHDL, fornendo accesso a tipi di dati e pacchetti standard. È posizionata all'inizio del file, prima della definizione dell'entity e dell'architecture.

```
LIBRARY ieee;  
USE ieee.std_logic_1164.ALL;
```

## ENTITY DECLARATION

L'**Entity Declaration** definisce l'interfaccia del componente hardware, specificando i segnali di ingresso e uscita.

Ogni entity è seguita da un'**Architecture** che ne descrive il comportamento.

```
ENTITY example_entity IS
    PORT (
        a: IN BIT;           -- Ingressi
        b: IN BIT;           -- Ingressi
        z: OUT BIT           -- Uscita
    );
END example_entity;
```

## OUTPUT MODE OUT con un segnale extra

Se è necessario leggere il valore di un'uscita, si può utilizzare un segnale interno per memorizzare il valore che sarà poi assegnato all'output

```
signal temp_z: BIT;
z <= temp_z;  -- Assegnazione dell'uscita
```

## INPUT BUFFER

L'output dichiarato come Buffer può essere letto all'interno dell'entity e può anche essere utilizzato come un segnale intermedio per assegnare valori. Si differenzia da OUT perché permette la lettura.

```
z: BUFFER BIT;
```

## BIDIRECTIONAL MODE: INOUT

I Segnali dichiarati come INOUT permettono di leggere e scrivere un valore. Questa modalità è usata quando si desidera che il segnale possa operare sia come ingresso che come uscita.

È utile, ad esempio, per segnali di dati che possono essere inviati e ricevuti.

```
data_bus: INOUT BIT;
```

## ARCHITECTURE DECLARATION

L'**Architecture Declaration** fornisce la descrizione del comportamento e della struttura interna di un'entità definita in VHDL. Può contenere dettagli riguardanti la logica, il flusso di dati e come i segnali sono gestiti all'interno dell'entità.

interconnessioni tra le entità.

```
ARCHITECTURE behavior OF example_entity IS
BEGIN
    -- Comportamento del circuito
END behavior;
```

In VHDL, ci sono due tipi principali di architettura:

- **Comportamentale:** Descrive cosa fa il circuito, senza dettagli sulla sua implementazione fisica.
- **Strutturale:** Descrive come è composto il circuito, includendo le

### std\_logic

Il tipo BIT è limitato a due valori logici: '0' (basso) e '1' (alto). Tuttavia, nel contesto della progettazione digitale, ci sono situazioni in cui è necessario rappresentare condizioni più complesse che non possono essere catturate semplicemente usando questi due valori. Per questo motivo, si raccomanda di utilizzare std\_logic per le porte delle entità in VHDL. Questo tipo di dato non solo permette di rappresentare i valori logici '0' e '1' (**Definiti segnali forti, ossia forniti da un componente attivo**), ma offre anche una varietà di stati aggiuntivi, come indeterminatezza e alta impedenza, che sono fondamentali per descrivere comportamenti più complessi nei circuiti digitali.

### DIFFERENZA std\_logic & std\_ulogic

#### std\_logic:

È un tipo di dato che rappresenta un singolo bit e può assumere 9 stati distinti. È progettato per essere utilizzato in situazioni in cui più stati devono essere rappresentati, come in circuiti complessi.

#### std\_ulogic:

È simile a std\_logic, ma rappresenta un singolo bit con una restrizione in più: può assumere solo uno stato alla volta. Non supporta la condizione di alta impedenza ('Z') e

le condizioni indeterminate ('X'). È generalmente utilizzato in contesti dove è necessario un controllo più rigoroso sugli stati del segnale.

VALORI SPECIALI DI std\_logic:

- "X" Value (Indeterminato): Rappresenta uno stato non definito [dato da un conflitto tra due segnali](#)
- "Z" Value (Alta impedenza): Indica che la linea non è pilotata (tri-state).
- H (High): Alta resistenza.
- L (Low): Bassa resistenza.
- ["U" \(Uninitialized\): segnale non inizializzato, solo per la simulazione](#)
- ["W": Analogo di X per conflitti puramente resistivi](#)
- "-": Don't Care: Il valore 'don't care' ('-') viene utilizzato nella sintesi logica per assegnare un'etichetta di irrilevanza al valore logico che la funzione da sintetizzare può assumere in corrispondenza a specifici valori di input. Questo valore è utile per ottimizzare il costo della sintesi in termini di utilizzo delle risorse, come ad esempio nella copertura delle mappe di Karnaugh.

## WIRES AND BUS

Nel contesto di VHDL, i **wires** (filamenti) e le **buses** (bus) rappresentano strutture di interconnessione fondamentali per la comunicazione tra vari elementi all'interno di un circuito.

- **Wires:** I fili sono utilizzati per trasmettere segnali singoli. Possono essere utilizzati per connettere porte o segnali all'interno di un'entità, facilitando il passaggio di un singolo valore logico, come un std\_logic o un std\_logic\_vector.
- **Buses:** I bus, invece, sono utilizzati per trasmettere più segnali contemporaneamente. Rappresentano un gruppo di fili che possono trasmettere un insieme di valori logici attraverso un'unica connessione.

## COSTANTI NUMERICHE

Quando si assegnano valori costanti ai segnali in VHDL, è importante distinguere tra la notazione per i fili e quella per i bus:

**Costanti Singole:** Per assegnare una costante a un wire di tipo std\_logic, si utilizzano le **virgolette singole**. Ad esempio:

```
my_wire <= '1'; -- Assegna il valore logico alto al wire
```

**Costanti Multiple:** Per assegnare costanti a un bus di tipo `std_logic_vector`, si utilizzano le **virgolette doppie**. Ad esempio:

```
my_bus <= "11001010"; -- Assegna un valore binario al bus std_logic_vector
```

Il tipo `std_logic_vector` è un array di elementi di tipo `std_logic` e permette di rappresentare un gruppo di bit, consentendo una gestione più efficiente e strutturata dei dati. Può essere utilizzato per rappresentare bus di dati, indirizzi o altri segnali che richiedono più di un singolo bit. Ad esempio, un `std_logic_vector (7 downto 0)` rappresenta un bus di 8 bit.

## DOWN vs TO

Questa distinzione si riferisce all'orientamento del vettore, ovvero quale bit è considerato il più significativo (MSB, Most Significant Bit) e quale è il meno significativo (LSB, Least Significant Bit).

**Down to:** La sintassi `downto` viene utilizzata per definire un vettore in cui il bit più significativo si trova all'indice più alto. Ad esempio

```
signal my_vector : std_logic_vector (7 downto 0); -- a <= "00000001"; con 1 in  
posizione meno significativa, ovv ero a = 1;
```

**To:** La sintassi `to` è l'opposto, dove il bit più significativo si trova all'indice più basso. Ad esempio:

```
signal my_vector: std_logic_vector(0 to 7); --a <= "00 000001"; con 1 in  
posizione più significativa, ovvero a = 128;
```

## OPERATORE CONCATENATO

Il **concatenation operator** in VHDL è rappresentato dall'operatore `&`. Viene utilizzato per unire due o più segnali o vettori in un unico vettore. Questo è particolarmente utile quando si desidera combinare dati provenienti da diverse fonti in un'unica rappresentazione.

Ecco un esempio di utilizzo dell'operatore di concatenazione:

```
signal a : std_logic_vector(3 downto 0); -- 4 bit
signal b : std_logic_vector(3 downto 0); -- 4 bit
signal c : std_logic_vector(7 downto 0); -- 8 bit

c <= a & b; -- Unisce i vettori a e b in c
```

## Signal vs Variable

### VARIABLE

Le variabili in VHDL hanno una semantica simile a quella dei linguaggi di programmazione come il C e servono a memorizzare valori utilizzati per l'elaborazione. **È importante notare che non producono hardware;** le variabili mantengono informazioni su cui è possibile scrivere e leggere valori, ma non generano un circuito fisico.

### SIGNAL

I segnali in VHDL, al contrario, trasmettono informazioni circuitali e producono hardware. **I segnali creano un registro fisico che conserva informazioni e generano circuiti reali.** Questa distinzione è fondamentale nella progettazione VHDL, poiché le variabili e i segnali vengono utilizzati in modi diversi per rappresentare e gestire le informazioni nel design hardware

## Sequential and Concurrent Statements

### SEQUENTIAL STATEMENTS

Le istruzioni sequenziali specificano l'ordine in cui devono essere eseguiti i passaggi dell'algoritmo per ottenere un'elaborazione specifica. Queste istruzioni funzionano come in linguaggi di programmazione come il C, dove l'ordine delle operazioni è cruciale. Le istruzioni sequenziali possono trovarsi solo all'interno di un processo, che è un blocco di codice che definisce un algoritmo sequenziale. Le operazioni all'interno di un processo vengono eseguite una dopo l'altra, producendo un risultato finale.

### CONCURRENT STATEMENTS

Le istruzioni concorrenti, invece, descrivono la struttura di una porzione di circuito e specificano elaborazioni hardware che evolvono simultaneamente. Queste operazioni vengono descritte in modo tale da non richiedere un ordine specifico di esecuzione, a differenza delle istruzioni sequenziali. In altre parole, i segnali e le connessioni tra i vari componenti vengono aggiornati in modo concorrente

## Confronto tra SV e VHDL: programmi

### A Behavior Example

La funzione implementata è:

$$Y = A^{\sim}B^{\sim}C^{\sim} + AB^{\sim}C^{\sim} + AB^{\sim}C$$

cioè, una **somma di prodotti**: tre AND collegati a una OR.

Fisicamente questo è:

- 3 NOT
- 3 AND a 3 ingressi
- 1 OR a 3 ingressi

Un circuito combinatorio puro.

Per comprendere come i linguaggi HDL descrivano l'hardware, è utile analizzare un esempio concreto in cui lo stesso circuito viene scritto sia in **SystemVerilog** sia in **VHDL**. In entrambi i casi si descrive un circuito combinatorio che calcola una funzione booleana di tre ingressi *A*, *B* e *C*. La funzione è una somma di prodotti e rappresenta una rete di porte logiche composta da NOT, AND e OR. L'obiettivo non è "eseguire" un algoritmo, ma **costruire un circuito fisico** che produca l'uscita *Y* a partire dagli ingressi.

### SystemVerilog

```
module aFunction (input logic a, b, c,  
                  output logic y);
```

```
    assign y = ~a & ~b & ~c |  
              a & ~b & ~c |  
              a & ~b & c;
```

```
endmodule
```

In SystemVerilog il circuito è descritto tramite un **module**, che rappresenta un blocco hardware con ingressi e uscite. All'inizio del codice vengono dichiarati i segnali di ingresso e di uscita, specificando il tipo logic, che rappresenta un segnale digitale fisico capace di assumere i valori 0, 1, alta impedenza o stato indeterminato.

Il cuore del modulo è l'istruzione assign, che definisce una **connessione combinatoria** tra gli ingressi e l'uscita. In pratica, assign indica che l'uscita è continuamente calcolata in funzione degli ingressi, come avviene in un circuito reale. Gli operatori ~, & e | corrispondono rispettivamente alle porte NOT, AND e OR. La formula booleana specificata con questi operatori descrive una rete di porte che implementa fisicamente la funzione logica richiesta.

Non esiste alcun ordine di esecuzione né memoria né clock: ogni variazione di  $A$ ,  $B$  o  $C$  provoca immediatamente una variazione di  $Y$ , proprio come in un circuito combinatorio reale.

## VHDL

```
library IEEE;  
use IEEE.STD_LOGIC_1164.all;
```

```
entity aFunction is  
  port (a, b, c : in  STD_LOGIC;  
        y      : out STD_LOGIC);  
end;
```

```
architecture behavior of aFunction is  
begin  
  y <= ((not a) and (not b) and (not c)) or  
      ( a and (not b) and (not c)) or  
      ( a and (not b) and c);  
end;
```

In VHDL lo stesso circuito viene descritto in modo più strutturato. Il codice è suddiviso in tre parti. La prima parte importa la libreria IEEE STD\_LOGIC\_1164, che definisce il tipo STD\_LOGIC, necessario per rappresentare segnali digitali reali con più stati possibili.

Segue la entity, che definisce l'interfaccia del circuito, cioè quali sono gli ingressi e le uscite. In questo caso,  $A$ ,  $B$  e  $C$  sono ingressi di tipo STD\_LOGIC, mentre  $Y$  è un'uscita dello stesso tipo. La entity specifica quindi che cosa è il circuito dal punto di vista esterno.

La architecture descrive invece come funziona il circuito. Al suo interno compare un'assegnazione concorrente del tipo  $y <= \dots$ , che definisce l'espressione booleana che lega l'uscita agli ingressi. Gli operatori not, and e or rappresentano le stesse porte logiche viste in SystemVerilog. Anche qui il risultato è una rete combinatoria che implementa fisicamente la funzione richiesta. Le parentesi sono necessarie perché in VHDL gli operatori logici non hanno precedenze implicite.

## A Behavior Example

In questo esempio viene descritto un **addizionatore a 32 bit**, cioè un circuito che somma due numeri binari a 32 bit e produce un risultato a 32 bit. Questo è un blocco

fondamentale di qualsiasi processore, microcontrollore o unità aritmetico-logica (ALU).

#### SystemVerilog

```
module adder ( input logic [31:0] a,
               input logic [31:0] b,
               output logic [31:0] y);
    assign y = a + b;
endmodule
```

Note that the inputs and outputs are 32-bit busses.

#### VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;
use IEEE.STD_LOGIC_UNSIGNED.all;
entity adder is
    port (a, b: in STD_LOGIC_VECTOR (31 downto 0);
          y: out STD_LOGIC_VECTOR (31 downto 0));
end;
architecture behavior of adder is
begin
    y <= a + b;
end;
```

Observe that the inputs and outputs are 32-bit vectors. They must be declared as `STD_LOGIC_VECTOR`.

### Descrizione in SystemVerilog

In SystemVerilog il circuito è definito tramite un module chiamato `adder`.

Gli ingressi `a` e `b` sono dichiarati come `logic [31:0]`, cioè come bus di 32 bit, mentre l'uscita `y` è anch'essa un bus di 32 bit.

La riga:

```
assign y = a + b;
```

non rappresenta un'operazione eseguita da un programma, ma la descrizione di un **circuito combinatorio** che implementa un addizionatore binario a 32 bit. Il sintetizzatore trasformerà questa espressione in una rete di **full adder**, collegati in cascata o secondo un'architettura più efficiente (carry-lookahead, carry-save, ecc.), ma questo dettaglio è nascosto al progettista.

Ogni volta che uno dei bit di `a` o `b` cambia, il valore di `y` viene ricalcolato istantaneamente, proprio come in un circuito fisico.

### Descrizione in VHDL

In VHDL lo stesso addizionatore è descritto tramite una entity chiamata `adder`, che definisce gli ingressi e l'uscita come `STD_LOGIC_VECTOR (31 downto 0)`, cioè vettori di 32 bit. Questo tipo è l'equivalente del bus a 32 bit di SystemVerilog.

Nella architettura viene specificata l'assegnazione:

```
y <= a + b;
```

che descrive esattamente lo stesso circuito combinatorio dell'esempio in SystemVerilog. Anche qui il simbolo `+` non indica un'operazione eseguita nel tempo, ma una **rete hardware** che realizza la somma binaria dei due vettori.

L'uso delle librerie `STD_LOGIC_1164` e `STD_LOGIC_UNSIGNED` è necessario in VHDL per permettere di trattare i vettori di bit come numeri su cui applicare l'operazione di addizione.

Una volta scritto un circuito in SystemVerilog o in VHDL, il primo passo non è costruirlo fisicamente, ma simularlo. La simulazione serve a verificare che la descrizione hardware produca esattamente il comportamento logico previsto dalle specifiche.

## Sintesi

Dopo che un circuito descritto in SystemVerilog o VHDL è stato verificato tramite simulazione, il passo successivo è la **sintesi**. La sintesi è il processo che trasforma il codice HDL in una rappresentazione concreta dell'hardware che verrà fisicamente realizzato su FPGA o ASIC.

Esistono due livelli di sintesi.

La **architectural synthesis** traduce una descrizione comportamentale o strutturale in una rete di blocchi logici, determinando quali componenti devono essere utilizzati e come devono essere organizzati.

La **logic synthesis**, invece, va più in profondità e converte il codice HDL in una **netlist**, cioè una descrizione esplicita di tutte le porte logiche (AND, OR, NOT, flip-flop, adders, ecc.) e delle connessioni tra di esse.

Durante questa fase il sintetizzatore non si limita a tradurre il codice in modo diretto, ma applica anche **ottimizzazioni** per ridurre:

- il numero di porte,
- il consumo di area,
- il consumo di potenza,
- o il ritardo di propagazione.

## VHDL per sistemi di Sintesi

In VHDL, non tutte le parti di un progetto ammettono sintesi, ovvero la conversione del codice VHDL in hardware reale. La sintesi è limitata a un sottoinsieme specifico dei tipi e delle strutture del linguaggio, che permette la creazione di circuiti digitali fisici basati sul design descritto.

## Tipologie di Sintesi

### TIPI CHE AMMETTONO SINTESI

I tipi seguenti sono compatibili con la sintesi e possono essere utilizzati nella progettazione di circuiti hardware:

**Tipi Enumerati:** **bit**: rappresenta due valori logici, '0' e '1'. **boolean**: rappresenta valori logici come true e false.

- **std\_logic** e **std\_ulogic**: rappresentano più valori logici e stati come '0', '1', 'Z' (alta impedenza) e 'X' (non definito).
- **character**: per rappresentare caratteri ASCII (usato raramente in sintesi, ma supportato).

### Tipi Numerici:

- **integer**: supporta numeri interi. **natural**: rappresenta numeri interi non negativi.
- **positive**: include solo valori interi positivi.
- **Array**:
  - Gli array ammettono sintesi se hanno confini statici definiti, ovvero lunghezza fissa. Ad esempio, un vettore definito come `std_logic_vector(7 downto 0)`.

### Sottotipi:

- I sottotipi sono ammessi se il loro range è definito come un sottoinsieme di valori di tipo enumerato, come `bit` o `std_logic`. Ad esempio, `subtype my_bit is std_logic range '0' to '1'`.

Non tutti i tipi in VHDL possono essere tradotti in hardware. Ad esempio, Access Type, ovvero puntatori a memoria, e File, adatti solo alla simulazione.

## Logica combinatoria e operatori bitwise

Nei sistemi digitali, la **logica combinatoria** è costituita da circuiti in cui l'uscita dipende esclusivamente dai valori degli ingressi nello stesso istante di tempo. Non esiste memoria né stato interno: se un ingresso cambia, l'uscita cambia immediatamente di conseguenza. Questo comportamento è esattamente quello delle porte logiche fisiche (AND, OR, NOT...).

Per questo motivo, la descrizione HDL della logica combinatoria utilizza costrutti che esprimono direttamente connessioni hardware (connessioni dirette e non sequenze di istruzioni). Gli operatori bitwise agiscono su singoli bit o su interi bus di bit, generando reti di porte logiche che operano in parallelo. Se `a` è un bus a 4 bit `a[3:0]`, ogni operazione viene eseguita in parallelo sui 4 bit.

### Operatore NOT (~, not)

L'operatore NOT è il più semplice esempio di operatore bitwise. Esso inverte ogni bit di un segnale o di un vettore di bit.

In **SystemVerilog**, se `a` è un bus di 4 bit, l'istruzione

```
assign y = ~ a;
```

indica che ogni bit di y è la negazione del bit corrispondente di a.  
Se a è logic [3:0], questa riga crea **4 porte NOT**, una per ogni bit.

In **VHDL**, lo stesso comportamento è ottenuto con:

```
y <= not a;
```

dove a e y sono dichiarati come STD\_LOGIC\_VECTOR (3 downto 0). Anche qui il risultato fisico è una banca di 4 invertitori hardware che lavorano in parallelo.

### Altri operatori bitwise

Oltre al NOT, i principali operatori bitwise sono:

| Operatore | SystemVerilog | VHDL     | Significato     |
|-----------|---------------|----------|-----------------|
| AND       | a & b         | a and b  | AND bit per bit |
| OR        | a   b         | a or b   | OR bit per bit  |
| XOR       | a ^ b         | a xor b  | XOR bit per bit |
| NAND      | ~(a & b)      | a nand b | NOT - AND       |
| NOR       | ~(a   b)      | a nor b  | NOT - OR        |

Se a e b sono bus di 4 bit, ogni operatore genera **4 porte logiche in parallelo**, una per ogni coppia di bit.

Per esempio, l'istruzione

```
assign y1 = a & b;
```

genera quattro porte AND che operano simultaneamente sui bit di a e b.

### Assegnamenti continui e segnali concorrenti

In **SystemVerilog**, le istruzioni del tipo:

```
assign y = a & b;
```

sono chiamate **continuous assignments**.

Questo significa che il valore di y viene ricalcolato automaticamente ogni volta che a o b cambiano. Non esiste un “momento di esecuzione”: il circuito è sempre attivo, come nell’hardware reale.

In **VHDL**, l’equivalente è l’assegnamento concorrente:

y <= a and b;

Ogni variazione degli ingressi provoca immediatamente l’aggiornamento dell’uscita. tutte le assegnazioni sono valutate **in parallelo**, come le porte reali. Al posto di and

Questi costrutti descrivono **logica combinatoria pura**.

N.B. al posto di “&” e “and” possono esserci altri operatori.

### **Reduction operators (operatori di riduzione)**

Gli **operatori di riduzione** servono a trasformare un intero **bus di bit** in un **singolo bit**, applicando una porta logica a **tutti i bit del vettore**.

In altre parole, prendono un segnale come a[7:0] e producono un unico valore logico y.

Dal punto di vista hardware, questo significa costruire una **porta a più ingressi** (per esempio una AND a 8 ingressi) che combina tutti i bit del bus.

### **SystemVerilog**

In SystemVerilog, se a è un bus di 8 bit, la scrittura:

assign y = &a;

è un **reduction AND** e significa:

y = a[7] & a[6] & a[5] & a[4] & a[3] & a[2] & a[1] & a[0]

Fisicamente, questa singola riga genera una **porta AND a 8 ingressi**.

### **VHDL**

VHDL **non ha operatori di riduzione**.

Per ottenere lo stesso effetto bisogna:

- scrivere manualmente tutte le AND  
oppure
- usare il costrutto generate

Nell’esempio della slide, il reduction AND viene scritto esplicitamente come:

y <= a(7) and a(6) and a(5) and a(4) and  
a(3) and a(2) and a(1) and a(0);

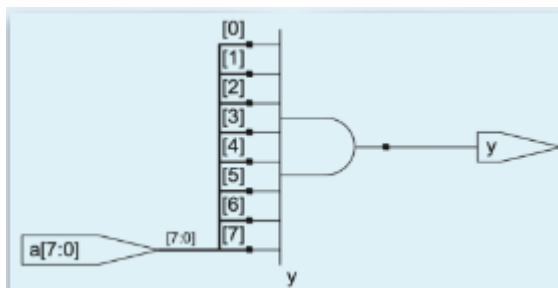
Dal punto di vista hardware, anche questo crea una **porta AND a 8 ingressi**, ma la sintassi è più lunga e meno compatta rispetto a SystemVerilog.

La figura in basso mostra esattamente questo:  
otto fili che entrano in una grande porta AND e producono un unico segnale di uscita y.

Questo tipo di operazione è fondamentale nei sistemi embedded per:

- verificare se **tutti i bit** sono a 1
- controllare se **tutto un registro** è zero
- generare flag di stato
- controllare condizioni di validità

Gli operatori di riduzione permettono di scrivere queste operazioni in modo semplice e direttamente traducibile in hardware.



## Conditional Assignment e multiplexer

Negli HDL, il **conditional assignment** è il modo principale per descrivere un **multiplexer**, cioè un circuito che seleziona uno tra più ingressi in base a un segnale di controllo. In un sistema digitale, i multiplexer servono a decidere quale dato deve essere inoltrato, quale registro deve essere letto o quale risultato deve essere usato: sono quindi fondamentali nei sistemi embedded.

## SystemVerilog

```
SystemVerilog
module mux2 ( input logic [3:0] d0, d1,
              input logic s,
              output logic [3:0] y );
    assign y = s ? d1 : d0;
endmodule
```

In SystemVerilog, il conditional assignment si realizza tramite l'operatore ternario `?:`, che ha la forma:

`condition ? value_if_true : value_if_false`

Nel codice della slide, il modulo mux2 ha due ingressi a 4 bit (d0 e d1), un segnale di selezione s e un'uscita a 4 bit y.

L'istruzione:

assign y = s ? d1 : d0;

significa:

- se s = 1, allora y = d1
- se s = 0, allora y = d0

Dal punto di vista hardware, questa singola riga genera un **multiplexer 2→1 a 4 bit**, cioè quattro multiplexer 2→1 in parallelo, uno per ogni bit del bus.

## VHDL

```
VHDL
library IEEE; use IEEE.STD_LOGIC_1164.all;
entity mux2 is
    port (d0, d1,
          in STD_LOGIC_VECTOR (3 downto 0);
          s: in STD_LOGIC;
          y: out STD_LOGIC_VECTOR (3 downto 0));
end;
architecture behavior of mux2 is
begin
    y <= d0 when s = '0' else d1;
end;
```

In VHDL lo stesso comportamento è ottenuto con un **conditional signal assignment**, scritto come:

y <= d0 when s = '0' else d1;

Questo comando dice che:

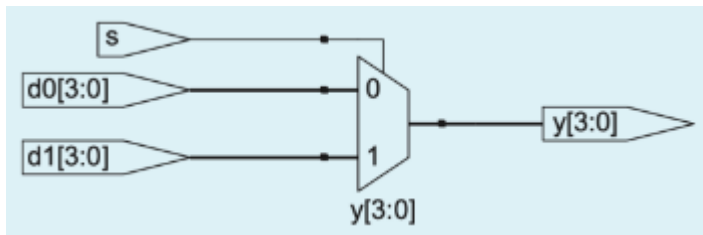
- se s = '0', l'uscita y assume il valore di d0
- altrimenti assume il valore di d1

Anche qui il risultato fisico è un **multiplexer 2→1 a 4 bit**. La differenza è solo sintattica: SystemVerilog usa l'operatore "?:", mentre VHDL usa la forma "when ... else".

## Significato hardware

L'immagine in basso mostra proprio questo: due bus di dati (d0[3:0] e d1[3:0]) entrano in un multiplexer controllato dal segnale s. A seconda del valore di s, uno dei due bus viene collegato all'uscita y [3:0].

Questo esempio chiarisce che le istruzioni di assegnamento condizionale non descrivono un "if" software, ma un **vero circuito di selezione**, che è una delle componenti più importanti dell'hardware digitale.



## MULTIPLEXER

Un multiplexer è un circuito combinatorio che seleziona uno tra più ingressi e lo porta in uscita in base a un segnale di selezione.

Un MUX  $4 \rightarrow 1$  ha:

- 4 ingressi (d0, d1, d2, d3)
- 2 bit di selezione (s[1:0])
- 1 uscita (y)

SystemVerilog

```
module mux4 (
    input logic [3:0] d0, d1, d2, d3, // quattro ingressi a 4 bit
    input logic [1:0] s,           // segnale di selezione a 2 bit
    output logic [3:0] y           // uscita a 4 bit
);
    assign y = s[1] ? (s[0] ? d3 : d2)
               : (s[0] ? d1 : d0);
endmodule
```

```
// Operatore condizionale annidato:
// Se s[1] = 1 → seleziona il gruppo (d3, d2)
// Se s[1] = 0 → seleziona il gruppo (d1, d0)
// Poi s[0] sceglie dentro al gruppo
// s = 00 → d0
// s = 01 → d1
// s = 10 → d2
// s = 11 → d3
```

Nel codice SystemVerilog il multiplexer  $4 \rightarrow 1$  viene descritto utilizzando **operatori condizionali ternari annidati** (?:). Il segnale di selezione s è composto da due bit, s[1] e s[0], che insieme determinano quale dei quattro ingressi deve essere inoltrato verso l'uscita y. Il primo bit, s[1], sceglie quale coppia di ingressi considerare: se vale 1, viene selezionato il gruppo d3–d2, mentre se vale 0 viene selezionato il gruppo d1–d0. Successivamente il bit s[0] decide quale dei due segnali all'interno del gruppo scelto deve essere mandato in uscita. In questo modo l'istruzione `assign y = s[1] ? (s[0] ? d3 : d2) : (s[0] ? d1 : d0);` realizza esattamente la funzione di un multiplexer  $4 \rightarrow 1$ , ma

costruito come una gerarchia di multiplexer 2→1. Dal punto di vista hardware, questa riga non è un “if” eseguito in sequenza, ma descrive una rete combinatoria di selettori che lavorano in parallelo sui 4 bit del bus.

VHDL

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity mux4 is
  port (
    d0, d1, d2, d3 : in  STD_LOGIC_VECTOR(3 downto 0); -- ingressi
    s              : in  STD_LOGIC_VECTOR(1 downto 0); -- selezione
    y              : out STD_LOGIC_VECTOR(3 downto 0) -- uscita
  );
end;
```

```
architecture behavior1 of mux4 is
begin
  -- Conditional signal assignment:
  -- seleziona l'ingresso in base al valore di s
  y <= d0 when s = "00" else
    d1 when s = "01" else
    d2 when s = "10" else
    d3;
end;
```

Nel codice VHDL lo stesso multiplexer è espresso in modo più dichiarativo tramite un **conditional signal assignment** oppure tramite una forma simile a un case. L'assegnazione `y <= d0 when s = "00" else d1 when s = "01" else d2 when s = "10" else d3;` specifica esplicitamente quale ingresso deve essere collegato all'uscita per ciascun valore possibile del segnale di selezione `s`. In alternativa, la versione con `with s select` fa la stessa cosa in modo ancora più leggibile, associando ogni combinazione di `s` a un ingresso. In entrambi i casi, VHDL descrive una **logica combinatoria pura**, che il sintetizzatore trasformerà in un vero multiplexer a 4 ingressi. Anche qui non c'è nessuna esecuzione temporale: il valore di `y` cambia immediatamente ogni volta che cambiano `s` o uno degli ingressi, esattamente come in un circuito fisico.

**Nel caso non si specifichi una condizione perché non verificabile, verrà sintetizzato un latch per memorizzare il valore precedente nel caso arrivi la combinazione non specificata. È fondamentale, quindi, specificarle tutte pure se inutilizzate per risparmiare area ed evitare latch.**

L'immagine sotto mostra cosa succede dopo la **sintesi**.

Il tool trasforma il codice HDL in una rete fisica di porte e multiplexer elementari.

Il diagramma mostra:

- i segnali di selezione `s[1]` e `s[0]`
- una rete di porte AND e NOT che genera i segnali di abilitazione

- un grande multiplexer che convoglia d0, d1, d2, d3 verso y

In pratica:

- s[1] e s[0] vengono decodificati
- ogni ingresso viene abilitato solo quando la sua combinazione di s è attiva
- solo uno dei quattro ingressi raggiunge l'uscita

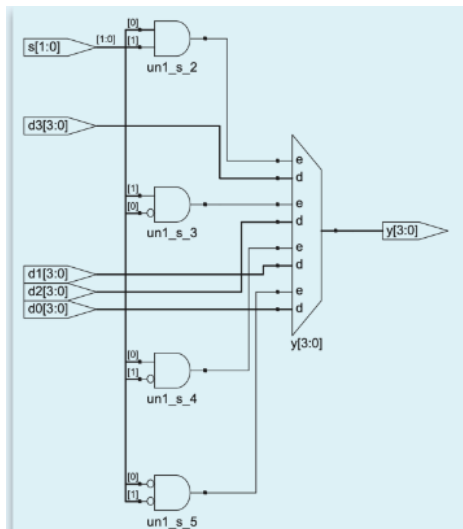
Questo dimostra una cosa fondamentale del corso:

Una singola riga HDL che usa `? : o when ... else` genera una vera rete fisica di porte e multiplexer.

Ed è proprio così che vengono costruiti:

- datapath
- ALU
- selettori di registri
- pipeline nei microprocessori

HDL non descrive un programma, ma descrive un circuito.



## Internal Signals e Full Adder

Nei linguaggi HDL, per costruire circuiti complessi è spesso necessario introdurre **segnali interni**, cioè fili che non fanno parte dell'interfaccia esterna del modulo, ma servono per collegare tra loro le varie parti della logica interna. Questi segnali rappresentano esattamente i fili che esisterebbero dentro un circuito fisico.

Nell'esempio della slide viene descritto un **full adder**, cioè il blocco fondamentale che somma due bit (a e b) e un riporto in ingresso (cin), producendo una somma (s) e un riporto in uscita (cout). La realizzazione hardware di un full adder usa due segnali intermedi molto noti:

- p (propagate), che vale a XOR b
- g (generate), che vale a AND b

Questi segnali non sono visibili all'esterno, ma sono fondamentali per costruire l'uscita.

## SystemVerilog

Nel codice SystemVerilog, p e g sono dichiarati come logic, cioè segnali interni del modulo. Le istruzioni assign definiscono una rete di connessioni hardware:

- $p = a \oplus b$  crea una porta XOR
- $g = a \& b$  crea una porta AND
- $s = p \oplus cin$  crea un'altra XOR
- $cout = g \mid (p \& cin)$  crea una rete di AND e OR

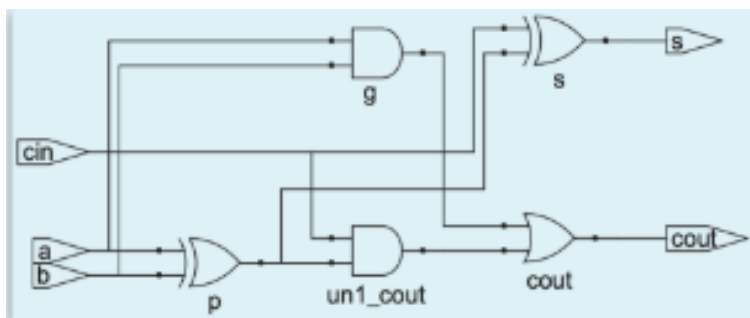
Tutte queste assegnazioni sono **concorrenti**: non c'è un ordine di esecuzione. Ogni volta che uno dei segnali a, b o cin cambia, l'hardware ricalcola automaticamente p, g, s e cout come avviene in un vero circuito.

## VHDL

In VHDL lo stesso circuito viene descritto usando segnali interni p e g dichiarati nell'architecture. Le assegnazioni  $p \leq a \text{ xor } b$  e  $g \leq a \text{ and } b$  generano gli stessi segnali intermedi del full adder. L'uscita s e il riporto cout sono poi calcolati usando questi segnali interni. Anche qui le assegnazioni sono **concorrenti**, quindi l'ordine delle righe non ha alcuna importanza: l'hardware è sempre "attivo" e ricalcola i segnali ogni volta che cambia un ingresso.

Questo esempio mostra una cosa fondamentale degli HDL:

non si scrive un algoritmo che calcola la somma, ma si descrive una **rete di fili e porte** che la realizza fisicamente.



## Precedenza degli operatori negli HDL

Quando si scrive un'espressione HDL che combina più operatori (come AND, OR, XOR, somma, moltiplicazione, ecc.), è fondamentale sapere in che ordine vengono valutati. Questo ordine si chiama operator precedence. A differenza del software, negli

HDL la precedenza non influisce solo sul risultato di un calcolo, ma sull'architettura del circuito che verrà sintetizzato.

| SystemVerilog                   |              |                               | VHDL |                                    |                                               |
|---------------------------------|--------------|-------------------------------|------|------------------------------------|-----------------------------------------------|
|                                 | Op           | Meaning                       |      | Op                                 | Meaning                                       |
| H<br>i<br>g<br>h<br>e<br>s<br>t | ~            | NOT                           | H    | not                                | NOT                                           |
|                                 | *, /, %      | MUL, DIV, MOD                 | i    | *, /,<br>mod, rem                  | MUL, DIV,<br>MOD, REM                         |
|                                 | +, -         | PLUS, MINUS                   | h    | +, -,<br>&                         | PLUS, MINUS,<br>CONCATENATE                   |
|                                 | <<, >>       | Logical Left / Right Shift    | e    | rol, ror,<br>srl, sll,<br>sra, sla | Rotate,<br>Shift logical,<br>Shift arithmetic |
|                                 | <<<, >>>     | Arithmetic Left / Right Shift | t    | =, /=,<br><, <=,<br>>, >=          | Comparison                                    |
|                                 | <, <=, >, >= | Relative Comparison           |      | and, or,<br>nand, nor,<br>xor      | Logical<br>Operations                         |
| L<br>o<br>w<br>e<br>s<br>t      | ==, !=       | Equality Comparison           |      |                                    |                                               |
|                                 | &, ~&        | AND, NAND                     |      |                                    |                                               |
|                                 | ^, ~^        | XOR, XNOR                     |      |                                    |                                               |
|                                 | , ~          | OR, NOR                       |      |                                    |                                               |
|                                 | ?:           | Conditional                   |      |                                    |                                               |

## SystemVerilog

In SystemVerilog la precedenza degli operatori segue regole simili a quelle dei linguaggi di programmazione tradizionali come C. In particolare, l'operatore AND (&) ha precedenza maggiore dell'operatore OR (|).

Per esempio, l'espressione:

```
assign cout = g | p & cin;
```

viene interpretata automaticamente come:

```
cout = g | (p & cin)
```

cioè, prima viene calcolato  $p \& cin$  e poi il risultato viene messo in OR con  $g$ . Questo è esattamente ciò che si vuole nella formula del full adder. In questo caso le parentesi non sono necessarie perché la precedenza degli operatori è già quella giusta.

## VHDL

In VHDL, invece, tutti gli operatori logici (and, or, xor, ecc.) hanno la stessa precedenza. Questo significa che un'espressione come:

```
cout <= g or p and cin;
```

viene interpretata da sinistra a destra come:

```
cout <= (g or p) and cin;
```

che è un circuito diverso da quello desiderato. Per ottenere il comportamento corretto del full adder bisogna scrivere esplicitamente:

```
cout <= g or (p and cin);
```

In VHDL quindi le parentesi sono essenziali quando si combinano operatori logici, altrimenti il sintetizzatore costruisce una rete di porte sbagliata.

## Numeri in SystemVerilog

In **SystemVerilog**, i numeri possono specificare sia la **dimensione in bit** sia la **base**. La forma generale è

**N'base valore**, dove N è il numero di bit e base indica la base (b binario, o ottale, d decimale, h esadecimale).

Per esempio 9'h25 indica un numero a 9 bit in esadecimale che corrisponde al valore decimale 37 e viene quindi rappresentato in binario come 000100101. Se la dimensione **N** non è specificata, il numero viene automaticamente **esteso** al numero di bit richiesto dal segnale a cui è assegnato, aggiungendo zeri a sinistra. Questo comportamento è chiamato *zero extension* ed è utile ma può essere pericoloso, per cui è buona pratica indicare sempre la dimensione dei numeri.

SystemVerilog fornisce anche due scorciatoie molto utili: '0 e '1, che riempiono automaticamente un bus con tutti 0 o tutti 1, indipendentemente dalla sua lunghezza.

In **VHDL**, i singoli bit (STD\_LOGIC) si scrivono tra apici singoli ('0', '1'), mentre i vettori (STD\_LOGIC\_VECTOR) si scrivono tra virgolette doppie, in binario o esadecimale, ad esempio "1010" oppure x"F3". La base predefinita è binaria, ma può essere indicata esplicitamente con b o x.

## Numbers Tables

SystemVerilog

| Numbers      | Bits | Base | Val | Stored       |
|--------------|------|------|-----|--------------|
| 3'b101       | 3    | 2    | 5   | 101          |
| 'b11         | ?    | 2    | 3   | 000...0011   |
| 8'b11        | 8    | 2    | 3   | 00000011     |
| 8'b1010_1011 | 8    | 2    | 171 | 10101011     |
| 3'd6         | 3    | 10   | 6   | 110          |
| 6'o42        | 6    | 8    | 34  | 100010       |
| 8'hAB        | 8    | 16   | 171 | 10101011     |
| 42           | ?    | 10   | 42  | 00...0101010 |
| '1           | ?    | n/a  |     | 11...111     |

VHDL

| Numbers | Bits | Base | Val | Stored   |
|---------|------|------|-----|----------|
| "101"   | 3    | 2    | 5   | 101      |
| B"101"  | 3    | 2    | 5   | 101      |
| X"AB"   | 8    | 16   | 161 | 10101011 |

### Statement Concorrenziali

I *concurrent statements* in VHDL descrivono operazioni che avvengono simultaneamente, anziché seguire una sequenza temporale come accade negli statement sequenziali. Questa caratteristica rende i statement concorrentiali particolarmente adatti per descrivere comportamenti hardware in cui più segnali possono cambiare contemporaneamente.

### Conditional Statement

I *conditional statements* vengono utilizzati per assegnare valori ai segnali in base a determinate condizioni. Si basano su espressioni logiche che controllano il flusso delle

assegnazioni e consentono di implementare, ad esempio, multiplexer e altre funzioni condizionali. I conditional statements vengono spesso scritti con la sintassi when...else.

ESEMPIO MUX4 -> 1

### Selection Statements

I *selection statements* sono simili ai conditional statements, ma si usano quando si devono gestire più condizioni (o "casi") per un singolo segnale di controllo.

La sintassi più comune è with...select, che funziona come un multiplexer a più vie. Qui ogni condizione è associata a un valore specifico, come accade per una selezione di input in un circuito multiplexe

### ZS and TRISTATE

Quando si progettano sistemi digitali reali, soprattutto microprocessori, memorie e bus di comunicazione, non basta poter rappresentare solo i valori 0 e 1. Serve anche un terzo stato che indichi che un filo **non sta guidando nulla**, cioè è elettricamente scollegato. Nei linguaggi HDL questo stato è chiamato **Z**, che significa **alta impedenza** (*high impedance*). Un segnale a Z non forza né 0 né 1: è come se il filo fosse lasciato libero.

Questo concetto è fondamentale perché in molti circuiti lo stesso bus fisico è condiviso da più dispositivi. Se due dispositivi provassero a scrivere contemporaneamente uno 0 e l'altro un 1 sullo stesso filo, si avrebbe un cortocircuito. La soluzione è usare **tri-state buffer**, cioè buffer che possono essere attivi o disconnessi. Un tri-state buffer ha un ingresso dati, un segnale di abilitazione e un'uscita. Quando il segnale di abilitazione è attivo, l'uscita copia l'ingresso; quando non è attivo, l'uscita va in stato Z, quindi si scollega dal bus.

In SystemVerilog questo comportamento si descrive con una riga come:

```
assign y = en ? a : 'bz;
```

Questa espressione significa: se en vale 1, allora y prende il valore di a; se en vale 0, allora y viene messo in alta impedenza ('bz). Il segnale y deve essere dichiarato come tri, perché può avere più driver collegati allo stesso bus, ad esempio:

```
output tri [3:0] y;
```

Questo indica che il bus y può essere guidato da più moduli, ma solo uno alla volta deve essere abilitato.

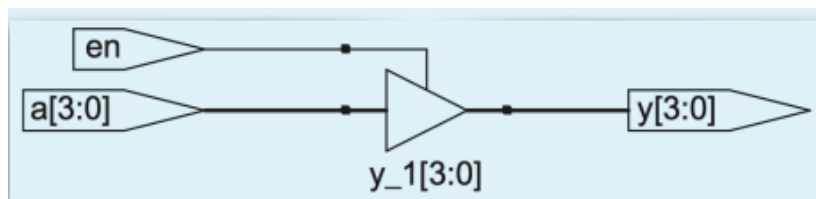
In VHDL lo stesso concetto si esprime così:

```
y <= "ZZZZ" when en = '0' else a;
```

Anche qui, se en è zero l'uscita viene posta a Z, mentre se en è uno l'uscita segue l'ingresso a. In VHDL il valore Z fa parte del tipo STD\_LOGIC, che rappresenta segnali reali con stati come 0, 1, Z e X.

Il punto chiave è che **Z non è un valore logico**, ma rappresenta uno stato fisico del circuito: il filo è elettricamente scollegato. Questo permette a più moduli di condividere lo stesso bus senza interferire tra loro. In pratica, su un bus tri-state, tutti i dispositivi sono collegati in parallelo, ma solo quello abilitato in quel momento guida effettivamente le linee.

Questo meccanismo è alla base di moltissimi sistemi hardware reali: bus di dati delle memorie, linee di comunicazione tra CPU e periferiche, GPIO condivisi e architetture a bus nei microcontrollori. Quando in HDL si usa Z e i tri-state, non si sta facendo una “scorciatoia software”, ma si sta descrivendo esattamente **come vengono collegati i fili nel circuito fisico**.



## UNDEFINED AND FLOATING INPUTS AKA Xs

Quando si lavora con HDL non esistono solo i valori logici 0 e 1. Per poter modellare correttamente quello che succede nei circuiti reali, esistono anche valori speciali che rappresentano situazioni fisicamente problematiche o non determinate. Uno di questi è **Z**, che indica alta impedenza, cioè un filo che non è guidato da nessuno. Un altro è **X**, che indica un valore logico **non valido o indefinito**.

La X viene usata quando un segnale si trova in una situazione in cui il simulatore non può stabilire se il valore corretto sia 0 o 1. Un caso tipico è quando due dispositivi tri-state cercano di guidare contemporaneamente lo stesso bus, uno verso 0 e l'altro verso 1: questo è un cortocircuito logico, chiamato **contenzione**, e il simulatore lo segnala con X. Un altro caso è quando un ingresso di una porta logica è in stato Z, cioè flottante: in quel caso la porta potrebbe interpretarlo come 0 o come 1, e quindi l'uscita diventa indefinita e viene marcata come X.

All'inizio della simulazione c'è un'altra sorgente di incertezza: i flip-flop non sono ancora stati inizializzati. Per questo in SystemVerilog i loro output partono in stato X, mentre in VHDL partono nello stato U, che significa **uninitialized**. Questi valori servono a individuare bug, perché se un segnale viene usato prima di essere correttamente inizializzato, l'indeterminazione si propaga e compare nell'uscita.

In SystemVerilog un segnale può assumere quattro valori: 0, 1, Z e X.

| & |   | A |   |   |   |
|---|---|---|---|---|---|
|   |   | 0 | 1 | z | x |
| B | 0 | 0 | 0 | 0 | 0 |
|   | 1 | 0 | 1 | x | x |
|   | z | 0 | x | x | x |
|   | x | 0 | x | x | x |

In VHDL, usando STD\_LOGIC, i valori sono più ricchi: '0', '1', 'Z', 'X' e 'U'. La differenza è che 'U' indica esplicitamente un segnale mai inizializzato, mentre 'X' indica un valore logicamente inconsistente o invalido.

| AND |   | A |   |   |   |   |
|-----|---|---|---|---|---|---|
|     |   | 0 | 1 | z | x | u |
| B   | 0 | 0 | 0 | 0 | 0 | 0 |
|     | 1 | 0 | 1 | x | x | u |
|     | z | 0 | x | x | x | u |
|     | x | 0 | x | x | x | u |
|     | u | 0 | u | u | u | u |

Le porte logiche, come AND e OR, hanno delle **tabelle di verità estese** che tengono conto anche di questi valori speciali. Per esempio, una porta AND restituisce sempre 0 se uno degli ingressi è 0, anche se l'altro è Z o X. Questo perché fisicamente l'uscita di una AND è forzata a 0 se uno degli ingressi è 0. In questi casi il simulatore può determinare l'uscita senza ambiguità. In tutte le altre combinazioni che coinvolgono Z, X o U, l'uscita diventa X oppure U, perché il comportamento reale sarebbe imprevedibile.

Il punto fondamentale è che, quando in simulazione compaiono X o U, quasi sempre significa che c'è un **errore nel progetto**: un segnale non inizializzato, un bus lasciato flottante, o più dispositivi che guidano lo stesso filo. Nell'hardware reale queste situazioni portano a comportamenti casuali, perché il circuito può interpretare il valore come 0 o come 1 in modo imprevedibile.

Per questo motivo i valori X e U sono uno strumento potentissimo di debug: non rappresentano un valore reale, ma ti dicono che **c'è qualcosa che non va nel modo in cui stai usando il circuito**.

## Bit coalescing

Quando si progettano circuiti digitali spesso è necessario **prendere singoli bit o sottoparti di bus** e combinarli in un bus più grande. Questa operazione si chiama **bit coalescing** (o anche *bit swizzling*): significa letteralmente "mettere insieme bit e sotto-bus in un'unica parola".

Dal punto di vista hardware questo non crea logica: non sono porte, non sono operazioni aritmetiche. È solo **come i fili vengono collegati**.

## SystemVerilog

Nel codice della slide c'è:

```
assign y = {c[2:1], {3{d[0]}}, c[0], 3'b101};
```

Le parentesi graffe { } in SystemVerilog servono per **concatenare** bus e bit. Questa riga costruisce y unendo più pezzi, nell'ordine in cui sono scritti:

1. `c[2:1]` → prende i bit 2 e 1 del bus c
2. `{3{d[0]}}` → ripete tre volte il bit d[0]
3. `c[0]` → prende il bit 0 di c
4. `3'b101` → una costante binaria a 3 bit

Tutti questi pezzi vengono messi uno dopo l'altro per formare un bus più grande. Se li conti: 2 bit + 3 bit + 1 bit + 3 bit = 9 bit → quindi y è un bus a 9 bit.

Il costrutto `{3{d[0]}}` è particolarmente importante: significa **replica**. Se d[0] vale 1, diventa 111; se vale 0, diventa 000.

## VHDL

In VHDL la stessa cosa si scrive così:

```
y <= c(2 downto 1) &  
      d(0) & d(0) & d(0) &  
      c(0) &  
      "101";
```

Qui l'operatore & non è AND: significa **concatenazione**. Serve solo a "incollare" bit e bus uno dopo l'altro.

Il risultato è lo stesso: un bus a 9 bit costruito prendendo pezzi di c, di d e una costante.

### Perché è importante specificare la dimensione?

Nella costante `3'b101` il numero 3 è fondamentale: dice che quella costante è larga **esattamente 3 bit**.

Se non fosse specificato, il tool potrebbe inserire zeri in modo implicito e creare un bus di dimensione sbagliata, spostando i bit nel posto sbagliato.

Quindi il bit coalescing non è solo "mettere insieme cose": è **definire esattamente quali fili vanno dove**.

Questa riga di codice non genera porte, né logica. Genera solo **collegamenti**.

Questo meccanismo è usato ovunque:

- nei datapath
- nei registri
- negli indirizzi
- nei formati delle istruzioni
- nei pacchetti di comunicazione

Il bit coalescing è il modo con cui HDL permette di **costruire parole di dati** combinando singoli bit e sotto-bus.

Non è un'operazione logica, ma un'operazione di **cablaggio**: descrive come i fili di un circuito vengono fisicamente organizzati per formare bus più grandi.

## Output splitting

Se moltiplichiamo due numeri a 8 bit ( $a[7:0]$  e  $b[7:0]$ ), il prodotto può richiedere fino a **16 bit**. Quindi il circuito di moltiplicazione produce un bus a 16 bit, che possiamo chiamare  $prod[15:0]$ . A quel punto spesso ti interessa separare:

- la **parte alta** (most significant byte) → bit **[15:8]**
- la **parte bassa** (least significant byte) → bit **[7:0]**

Questo è utile, ad esempio, quando vuoi gestire overflow, fare DSP, o salvare metà risultato per volta.

## SystemVerilog

Nella slide in SystemVerilog c'è:

```
module mul ( input logic [7:0] a, b,
             output logic [7:0] upper, lower);
    assign {upper, lower} = a * b;
endmodule
```

Qui succedono due cose importanti:

1.  $a * b$  genera un risultato a **16 bit** (in pratica  $prod[15:0]$ ).
2.  $\{upper, lower\}$  è una **concatenazione a sinistra**: significa che il risultato a 16 bit viene "spaccato" così:
  - i bit più significativi vanno in upper
  - i bit meno significativi vanno in lower

In altre parole, è equivalente a scrivere:

```
assign lower = (a*b) [7:0];
```

```
assign upper = (a*b) [15:8];
```

## VHDL

In VHDL la slide fa i passaggi in modo più dichiarativo e leggibile:

```
signal prod : STD_LOGIC_VECTOR(15 downto 0);
```

```
prod <= a * b;
```

```
upper <= prod(15 downto 8);
```

```
lower <= prod(7 downto 0);
```

Qui l'idea è la stessa:

- calcolo prima il prodotto completo prod
- poi estraggo “a fette” i bit alti e bassi con lo slicing (15 downto 8) e (7 downto 0).

La moltiplicazione  $a * b$  sintetizza un **moltiplicatore combinatorio**

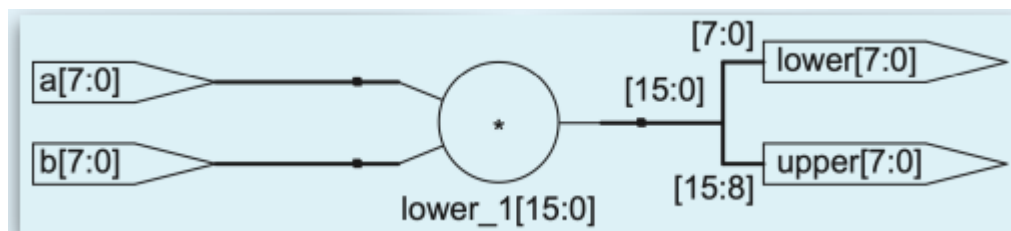
Lo splitting invece non crea logica complessa: è principalmente **cablaggio** (selezione di linee) che prende i bit [15:8] e li porta a upper, e i bit [7:0] a lower.

In HDL puoi:

- generare un bus più grande con un'operazione (qui una moltiplicazione)
- e poi **smistare** i bit del risultato su più uscite in modo pulito e controllato.

Output splitting è l'opposto di bit coalescing:

- **coalescing**: costruisci un bus grande mettendo insieme pezzi
- **splitting**: prendi un bus grande e lo dividi in più segnali



Sing extension

Quando un numero con segno viene rappresentato in binario (in complemento a due), il bit più significativo non è un normale bit di valore: è il **bit di segno**. Per esempio, in un numero a 16 bit, il bit `a[15]` indica se il numero è positivo (0) o negativo (1). Quando questo numero deve essere portato in un bus più grande, per esempio da 16 a 32 bit, non basta aggiungere zeri a sinistra, perché così si cambierebbe il valore dei numeri negativi. Serve invece copiare il bit di segno nelle nuove posizioni più significative: questa operazione si chiama **sign extension**.

In SystemVerilog questo si esprime in modo molto compatto con:

```
assign y = {{16{a[15]}}, a[15:0]};
```

Qui `a[15]` è il bit di segno. L'espressione `{16{a[15]}}` significa "ripeti 16 volte il bit `a[15]`". Se `a[15]` è 0, i 16 bit più alti di `y` diventano tutti 0; se `a[15]` è 1, diventano tutti 1. Questi 16 bit vengono poi concatenati con i 16 bit originali `a[15:0]`, producendo un numero a 32 bit che rappresenta **lo stesso valore con segno** del numero originale a 16 bit.

In VHDL la stessa operazione viene descritta in modo più esplicito:

```
y <= X"0000" & a when a(15) = '0' else  
      X"FFFF" & a;
```

Qui si controlla il bit di segno `a(15)`. Se è 0, si concatenano 16 zeri (`X"0000"`) davanti ad `a`, facendo una zero-extension. Se è 1, si concatenano 16 uno (`X"FFFF"`) davanti ad `a`, facendo una sign-extension vera e propria. Il risultato è un bus a 32 bit che mantiene il valore numerico originale.

Dal punto di vista hardware, la sign extension non richiede calcoli: è solo una rete di fili che copia il bit di segno su tutti i bit più significativi. Nell'immagine della slide si vede infatti che `a[15]` viene semplicemente collegato a tutte le linee alte di `y[31:16]`.

Questo meccanismo è essenziale nei processori. Per esempio, quando un'istruzione carica un valore a 16 bit da memoria e deve usarlo in un'ALU a 32 bit, è fondamentale che un numero negativo rimanga negativo dopo l'estensione. La sign extension è ciò che rende possibile questo comportamento corretto nei datapath.

## Delays

Quando si progettano circuiti digitali reali, ogni porta logica ha un certo ritardo di propagazione: se un ingresso cambia, l'uscita non cambia istantaneamente, ma dopo

un piccolo intervallo di tempo. I linguaggi HDL permettono di modellare questi ritardi usando il concetto di delay, cioè un tempo associato a una assegnazione.

In SystemVerilog, i delay si indicano con il simbolo #. Per esempio:

```
assign #1 bb = a;
```

```
assign #2 n1 = a & b;
```

significa che il segnale bb cambierà 1 unità di tempo dopo che a cambia, mentre n1 cambierà 2 unità di tempo dopo che a o b cambiano. Queste unità di tempo non sono fissate di per sé: dipendono dalla direttiva timescale, per esempio:

```
`timescale 1ns / 1ps
```

che indica che un'unità di tempo è 1 nanosecondo e la risoluzione della simulazione è 1 picosecondo.

In VHDL lo stesso concetto si esprime con la clausola after:

```
bb <= not a after 1 ns;
```

```
n1 <= a and b after 2 ns;
```

Anche qui il valore dell'uscita viene aggiornato dopo il tempo specificato.

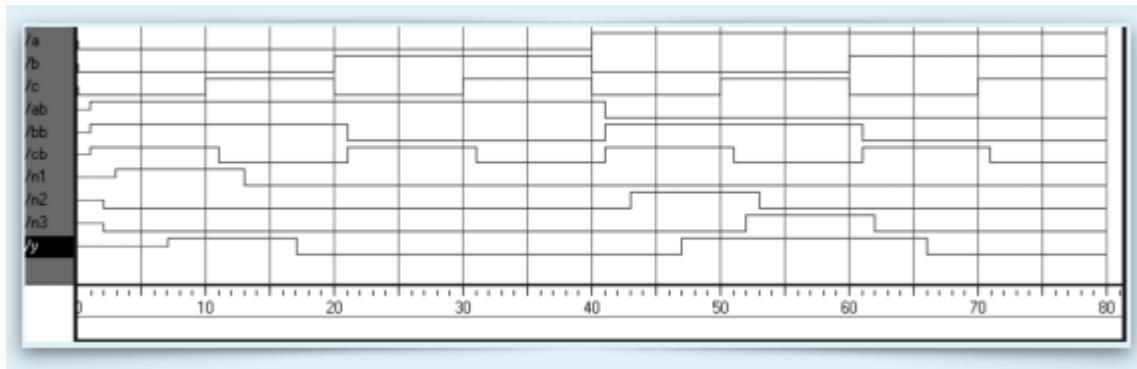
Il punto fondamentale, però, è che questi delay non vengono sintetizzati. Servono solo in simulazione per:

- capire come i segnali si propagano
- individuare glitch
- studiare problemi di temporizzazione
- trovare errori di progettazione

Quando il circuito viene sintetizzato, il tool ignora i delay e costruisce l'hardware in base alle porte e ai collegamenti, non ai numeri scritti dopo # o after.

Il diagramma temporale mostra proprio questo: come un cambiamento sugli ingressi a, b, c si propaga prima su segnali intermedi (ab, bb, n1, n2, n3) e solo dopo, con vari ritardi, arriva all'uscita y. Questo permette di osservare:

- ritardi cumulativi
- sovrapposizioni
- eventuali glitch temporanei



i delay sono strumenti di simulazione, non di progetto hardware.

Servono per capire come si comporterà nel tempo un circuito reale, ma l'hardware finale dipende solo dalla logica, non dai valori dei delay scritti nel codice.

### **Caveat: packed vs. unpacked Arrays (solo SV)**

Quando si usano i vettori in SystemVerilog, è fondamentale distinguere tra packed arrays e unpacked arrays, perché rappresentano due modi completamente diversi di organizzare i bit in hardware.

Un packed array è un unico vettore di bit contiguo che viene suddiviso logicamente in più parti.

Per esempio:

```
logic [2:0][5:0] a1;
```

Qui a1 è un array di 3 elementi, ciascuno largo 6 bit. Ma la cosa importante è che questi 18 bit sono tutti contigui in memoria, uno dopo l'altro, come se fosse un unico bus a 18 bit. La notazione [2:0][5:0] significa che:

- il primo indice [2:0] seleziona quale “sotto-vettore”
- il secondo indice [5:0] seleziona il bit dentro quel sotto-vettore

Quindi a1[0], a1[1] e a1[2] sono tre campi da 6 bit ricavati dallo stesso bus fisico.

Dal punto di vista hardware questo è perfetto per descrivere:

- registri suddivisi in campi
- istruzioni di una CPU
- pacchetti di dati

È un unico blocco di fili che viene semplicemente “tagliato” in pezzi.

Un unpacked array, invece, è un vero array di elementi separati.  
Per esempio:

```
logic [5:0] a2 [2:0];
```

Qui ogni elemento `a2[0]`, `a2[1]`, `a2[2]` è un vettore indipendente da 6 bit. Non è garantito che questi 18 bit siano contigui: il sintetizzatore può trattarli come tre registri separati.

Dal punto di vista concettuale:

- packed array → un bus grande suddiviso in campi
- unpacked array → più bus separati messi in un array

Questa differenza è cruciale perché solo i packed arrays sono garantiti essere contigui in bit. Questo significa che solo i packed arrays possono essere facilmente:

- concatenati
- divisi
- reinterpretati come un unico valore

Gli unpacked arrays, invece, sono più simili a array di variabili, come in un linguaggio software.

Un packed array viene visto come:

```
[ a1[2] ][ a1[1] ][ a1[0] ]
```

 tutti su un'unica striscia di bit,

mentre un unpacked array è:

```
a2[0] a2[1] a2[2]
```

 ognuno con il suo bus indipendente.

Un packed array è un vettore di bit suddiviso in campi.

Un unpacked array è un array di vettori separati.

Se stai modellando un datapath o un formato di istruzione, usi un packed array. Se stai modellando banchi di registri o memorie, usi un unpacked array.

## STRUCTURAL STYLE

### MULTIPLEXER

Quando si parla di **structural modeling**, si intende descrivere un circuito non come un'espressione o un algoritmo, ma come un **insieme di moduli più piccoli collegati tra loro**, esattamente come uno schema elettrico. In questo stile, un modulo è visto

come un blocco con porte di ingresso e uscita, e il circuito completo è costruito **istanziando** più blocchi e collegandoli con fili.

La slide mostra come costruire un **multiplexer 4→1** utilizzando **tre multiplexer 2→1**. Questo è un esempio di **riuso strutturale**: invece di riscrivere da zero la logica del 4→1, si riutilizza più volte un blocco più semplice.

## SystemVerilog

*Nel codice SystemVerilog troviamo:*

```
module mux4 ( input logic [3:0] d0, d1, d2, d3,  
             input logic [1:0] s,  
             output logic [3:0] y);
```

Questo è il modulo del multiplexer 4→1.  
Subito dopo vengono dichiarati due segnali interni:

```
logic [3:0] low, high;
```

Questi sono i fili che collegheranno i moduli interni.

Poi vengono istanziati tre mux2:

```
mux2 lowmux (d0, d1, s[0], low);  
mux2 highmux(d2, d3, s[0], high);  
mux2 finalmux(low, high, s[1], y);
```

Qui:

- lowmux sceglie tra d0 e d1 usando s[0]
- highmux sceglie tra d2 e d3 usando s[0]
- finalmux sceglie tra low e high usando s[1]

Il risultato è un multiplexer 4→1 costruito come una **gerarchia di mux 2→1**. Ogni riga è un **blocco fisico** che viene collegato con fili (low, high).

## VHDL

In VHDL la stessa cosa è più verbosa ma concettualmente identica.

Prima si dichiara che esiste un componente mux2:

```
component mux2  
port ( d0, d1 : in STD_LOGIC_VECTOR(3 downto 0);
```

```

s    : in STD_LOGIC;

y    : out STD_LOGIC_VECTOR(3 downto 0);

```

*end component;*

Questo serve solo a dire al compilatore:

“Userò un modulo chiamato mux2 con queste porte”.

Poi si dichiarano i segnali interni:

```

signal low, high : STD_LOGIC_VECTOR(3 downto 0);

```

Infine si istanziano i tre mux:

```

lowmux : mux2 port map (d0, d1, s(0), low);

```

```

highmux : mux2 port map (d2, d3, s(0), high);

```

```

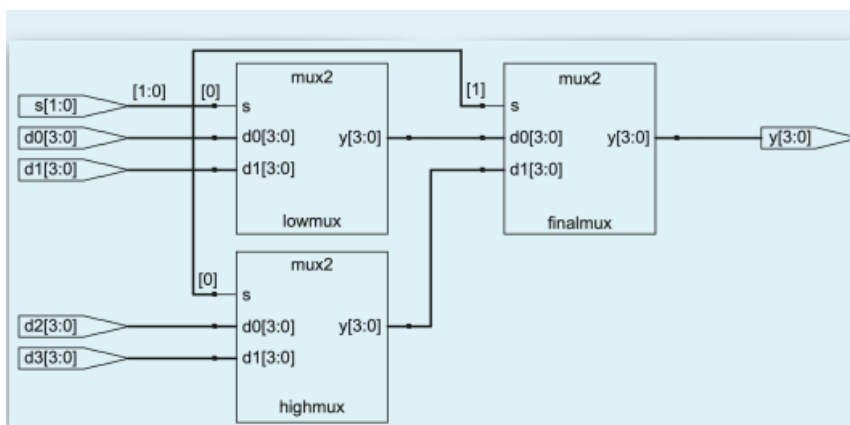
finalmux: mux2 port map (low, high, s(1), y);

```

È esattamente lo stesso schema del SystemVerilog, solo scritto in stile VHDL.

L'architettura deve prima dichiarare le porte del mux2 usando l'istruzione di dichiarazione component. Questo permette agli strumenti VHDL di controllare che il componente che si desidera utilizzare abbia le stesse porte dell'istruzione entity, che è stata dichiarata altrove in un'altra entità. Tuttavia, la dichiarazione dei componenti rende il codice VHDL piuttosto macchinoso. Si noti che l'architettura di mux4 è stata chiamata structure, mentre le architetture dei moduli con descrizione comportamentale nelle sezioni precedenti erano chiamate behavior. In VHDL, i nomi delle architetture servono solo a distinguerle tra loro e non hanno alcun significato per gli strumenti CAD. Tuttavia, il codice VHDL sintetizzabile contiene generalmente una sola architettura per ogni entity, quindi non viene trattata la sintassi VHDL che permette di scegliere quale architettura usare quando ne esistono più di una tramite le configuration.

Il disegno mostra tre blocchi mux2 collegati a cascata. I due mux del primo stadio selezionano coppie di ingressi, l'altro seleziona tra i loro risultati. Questa è la realizzazione strutturale del multiplexer 4→1.



## Tristate buffer

Quando si costruisce un multiplexer in stile strutturale, uno dei modi più semplici è usare **due tri-state buffer** che guidano lo stesso bus di uscita. L'idea è che solo uno dei due è attivo alla volta: uno quando  $s = 0$ , l'altro quando  $s = 1$ . In questo modo l'uscita viene collegata o a  $d0$  o a  $d1$ .

## SystemVerilog

Nel codice SystemVerilog abbiamo:

```
module mux2 ( input logic [3:0] d0, d1,  
             input logic s,  
             output tri [3:0] y );
```

```
    tristate t0 (d0, ~s, y);
```

```
    tristate t1 (d1, s, y);
```

```
endmodule
```

Qui  $y$  è dichiarata come tri perché ha **due driver**: i due buffer tri-state  $t0$  e  $t1$ .  
 $t0$  collega  $d0$  a  $y$  quando  $\sim s$  è vero, quindi quando  $s = 0$ .  
 $t1$  collega  $d1$  a  $y$  quando  $s = 1$ .

Quando uno dei due è abilitato, l'altro è in stato Z, quindi non interferisce.  
Fisicamente questo è un vero multiplexer  $2 \rightarrow 1$  costruito con tri-state buffer.

In SystemVerilog è possibile scrivere direttamente  $\sim s$  nella lista delle porte dell'istanza, anche se questo è considerato poco leggibile.

## VHDL

In VHDL la stessa cosa viene scritta così:

```
sneg <= not s;
```

```
t0: tristate port map (d0, sneg, y);
```

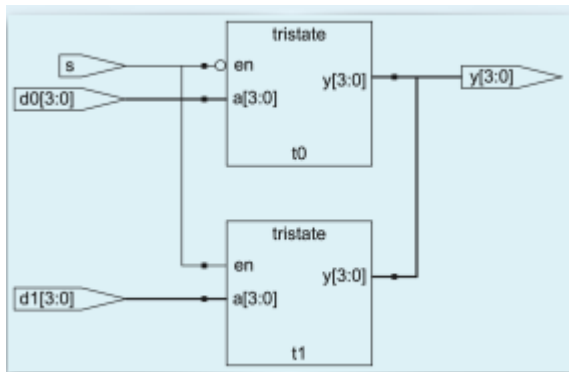
```
t1: tristate port map (d1, s, y);
```

Qui non è permesso usare  $\text{not } s$  direttamente nella port map; quindi, si crea un segnale intermedio  $\text{sneg}$ .

Anche qui:

- $t0$  guida  $y$  quando  $s = 0$
- $t1$  guida  $y$  quando  $s = 1$

Il disegno mostra due blocchi tri-state collegati allo stesso bus y. Solo uno è attivo alla volta, per cui il bus viene guidato o da d0 o da d1.



Questo è un esempio perfetto di **structural style**: il multiplexer non è descritto con un'espressione (? : o when), ma come connessione fisica di due blocchi tri-state.

È così che si costruiscono davvero i bus nei microprocessori e nei SoC.

## Accessing Parts of Busses

Quando si lavora con bus larghi, spesso non serve elaborare tutto il bus con un unico blocco grande. È molto più naturale e modulare **dividere il bus in parti** e usare moduli più piccoli già esistenti. Questa slide mostra proprio questo: come costruire un **multiplexer 2→1 a 8 bit** utilizzando **due multiplexer 2→1 a 4 bit**, uno per il nibble basso e uno per il nibble alto.

In SystemVerilog il modulo è:

```
module mux2_8 ( input logic [7:0] d0, d1,
               input logic s,
               output logic [7:0] y );
```

```
    mux2 lsbmux(d0[3:0], d1[3:0], s, y[3:0]);
```

```
    mux2 msbmux(d0[7:4], d1[7:4], s, y[7:4]);
```

```
endmodule
```

Qui succede una cosa molto importante:

d0[3:0] e d1[3:0] sono i 4 bit meno significativi, mentre d0[7:4] e d1[7:4] sono i 4 bit più significativi. Il primo mux2 seleziona quale nibble basso mandare in y[3:0], il secondo mux2 seleziona quale nibble alto mandare in y[7:4]. In questo modo i due mux lavorano in parallelo, uno sulla parte bassa e uno sulla parte alta del byte.

In VHDL la stessa struttura è espressa con:

lsbmux: mux2 port map

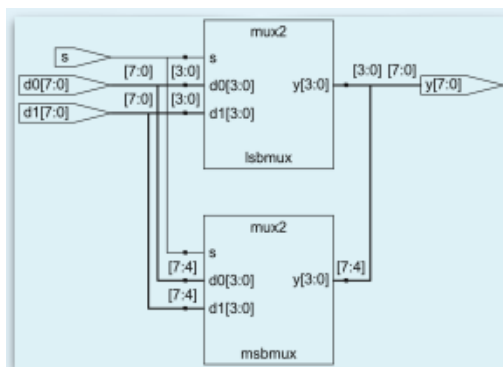
```
(d0(3 downto 0), d1(3 downto 0), s, y(3 downto 0));
```

msbmux: mux2 port map

```
(d0(7 downto 4), d1(7 downto 4), s, y(7 downto 4));
```

Anche qui si vede chiaramente lo slicing dei bus e il collegamento ai due blocchi mux2.

Il disegno di sintesi sotto mostra esattamente questo: due blocchi mux2 che ricevono solo metà dei bit ciascuno, e poi le loro uscite vengono ricombinate per formare il bus a 8 bit.



I sistemi complessi vengono progettati in modo gerarchico. Si costruiscono blocchi grandi collegando blocchi più piccoli, e questi a loro volta possono essere descritti strutturalmente o comportamentalmente. È buona pratica tenere separati questi stili: un modulo o è strutturale o è comportamentale, per mantenere il progetto chiaro e ben organizzato.

Questo è esattamente il modo in cui si costruiscono datapath, pipeline e intere CPU: per livelli, usando pezzi sempre più semplici.

## SEQUENTIAL LOGICS

Dobbiamo ricordare che l'obiettivo ultimo della codifica HDL è la **sintesi**, automatica o semi-automatica, di un **circuito digitale corretto e funzionante**. A questo scopo, i sintetizzatori HDL riconoscono determinati **schemi di codice (idioms)** e li trasformano in **specifici circuiti sequenziali**. Altri stili di codifica possono simulare correttamente, ma sintetizzarsi in circuiti con **errori funzionali evidenti o sottili**. Questa sezione presenta gli **idiomi corretti** per descrivere i circuiti sequenziali, cioè quelli che includono **registri e latch** che, come è noto, rappresentano lo **stato** del sistema dinamico a tempo discreto che è qualsiasi circuito digitale.

### Registri

Nei sistemi digitali moderni, quasi tutta la memoria è realizzata usando **registri costruiti con flip-flop D a fronte di salita** (*positive edge-triggered D flip-flops*). Un registro è semplicemente un insieme di flip-flop che memorizzano un vettore di bit e lo aggiornano tutti insieme sul fronte di salita del clock.

In SystemVerilog questo tipo di registro si descrive con:

```
module flipflop ( input logic clk,  
                input logic [3:0] d,  
                output logic [3:0] q );
```

```
always_ff @(posedge clk)
```

```
    q <= d;
```

```
endmodule
```

Questa riga significa:

ogni volta che il clock clk ha un **fronte di salita**, il valore presente su d viene copiato dentro q. Tra un fronte di clock e il successivo, q rimane costante: questo è esattamente il comportamento di un registro hardware.

L'operatore <= è un **non-blocking assignment** ed è fondamentale nei circuiti sequenziali, perché modella il fatto che tutti i flip-flop di un registro si aggiornano **simultaneamente** al fronte di clock.

In VHDL lo stesso flip-flop si scrive così:

```
process (clk)
```

```
begin
```

```
    if clk'event and clk = '1' then
```

```
        q <= d;
```

```
    end if;
```

```
end process;
```

Anche qui si dice che quando si verifica un **evento sul clock** e il clock è 1, cioè quando avviene il fronte di salita, q prende il valore di d. In alternativa si può usare la forma equivalente:

```
if rising_edge(clk) then
```

```
    q <= d;
```

```
end if;
```

La slide sottolinea che sia in Verilog che in VHDL esiste un costrutto generale chiamato `always` o `process`, che ha una **sensitivity list**. Il codice all'interno viene eseguito quando uno dei segnali nella lista cambia. A seconda di quali segnali si mettono nella sensitivity list e di come si scrive il corpo del processo, lo stesso costrutto può descrivere:

- logica combinatoria
- latch
- flip-flop

Questo è molto potente, ma anche pericoloso, perché è facile descrivere **hardware sbagliato** senza accorgersene. Per questo SystemVerilog introduce costrutti specifici come:

- `always_ff` per i flip-flop
- `always_latch` per i latch
- `always_comb` per la logica combinatoria

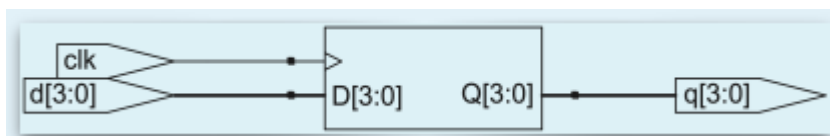
Usando `always_ff @(posedge clk)` il tool sa che stai descrivendo un **registro sincronizzato dal clock** e può anche segnalare errori se il codice non è coerente con un flip-flop.

Un registro non è una variabile che cambia quando vuoi:

è un insieme di flip-flop che catturano i dati solo sul fronte del clock.

Infatti `q` è un registro perché viene aggiornato solo sul fronte di clock; in tutti gli altri istanti mantiene il valore precedente, quindi il circuito deve avere memoria, cioè flip-flop

Negli `always` di SystemVerilog e nei `process` di VHDL, i segnali mantengono il loro valore fino a quando non avviene esplicitamente un cambiamento. Pertanto, questo tipo di codice, con una lista di sensibilità appropriata, può essere usato per descrivere circuiti sequenziali che hanno memoria. Per esempio, il flip-flop include solo il clock nella lista di sensibilità. Esso ricorda il valore precedente di `q` fino al fronte di salita del clock successivo, anche se `d` cambia nel frattempo. Al contrario, le assegnazioni continue di SystemVerilog e le assegnazioni concorrenti di VHDL vengono ricalcolate ogni volta che uno degli ingressi sul lato destro cambia. Pertanto, questo tipo di codice descrive necessariamente logica combinatoria.



## Resettable Registers

Quando la simulazione inizia o quando l'alimentazione viene applicata per la prima volta a un circuito (e quindi la simulazione inizia), l'uscita dei registri è sconosciuta.

Questo è indicato con x in SystemVerilog e con u in VHDL. In generale è buona pratica usare registri con reset, così che all'accensione si possa portare il sistema in uno stato noto. Il reset può essere implementato in modo asincrono o sincrono. I reset sincroni vengono forzati sui fronti di salita del clock, mentre i reset asincroni sono forzati immediatamente. I reset asincroni richiedono flip-flop speciali con pin di reset asincrono. Distinguere i reset sincroni da quelli asincroni in uno schema può essere difficile. Alcuni strumenti di disegno pongono i reset sincroni sul lato sinistro di un flip-flop e quelli asincroni in basso. Questo rende più facile distinguere quali reset usano meno transistor e riduce il rischio di problemi di temporizzazione sul fronte di discesa del reset. Tuttavia, se si usa il clock gating, bisogna fare attenzione che tutti i flip-flop vengano correttamente resettati all'avvio.

Un **reset sincrono** è un reset che viene applicato **solo in corrispondenza del fronte di clock**. Questo significa che anche se il segnale reset cambia valore in un istante qualsiasi, **non succede nulla** finché non arriva il **prossimo fronte di salita del clock**. Solo in quel momento il registro controlla il valore di reset e decide se azzerarsi oppure caricare il nuovo dato.

Questo comportamento è evidente nel codice. In SystemVerilog:

```
always_ff @(posedge clk)
```

```
    if (reset)
```

```
        q <= 4'b0;
```

```
    else
```

```
        q <= d;
```

La sensitivity list contiene **solo posedge clk**. Quindi il codice dentro viene eseguito **solo** quando il clock fa un fronte di salita. Il segnale reset viene letto solo in quell'istante: se vale 1 in quel momento, il registro viene azzerato; se vale 0, il registro carica d. Se reset cambia tra due clock, non ha alcun effetto.

In VHDL succede la stessa cosa:

```
process(clk)
```

```
    begin
```

```
        if clk'event and clk = '1' then
```

```
            if reset = '1' then
```

```
                q <= (others => '0');
```

```
            else
```

```
                q <= d;
```

```
            end if;
```

*end if;*

*end process;*

Anche qui il processo si attiva **solo quando cambia clk**, e il reset viene valutato solo sul **rising edge**.

Il reset sincrono viene campionato solo sul fronte di salita del clock, esattamente come i dati; quindi, il registro si azzerava solo quando arriva un clock, anche se il segnale di reset cambia prima.

Un **reset asincrono** è un reset che **non aspetta il clock**.

Nel momento in cui reset diventa attivo, il registro si azzerava **subito**, anche se il clock non sta facendo un fronte.

Quindi:

reset asincrono = reset **non legato al clock**

### SystemVerilog

```
always_ff @(posedge clk, posedge reset)
```

```
if (reset)
```

```
    q <= 4'b0;
```

```
else
```

```
    q <= d;
```

Qui la sensitivity list contiene **due eventi**:

- posedge clk
- posedge reset

Questo significa:

- se arriva un fronte di clock → il registro può aggiornarsi
- se arriva un fronte di reset → il registro si azzerava **immediatamente**

Se reset passa da 0 a 1 tra due clock, q viene azzerato subito, senza aspettare il prossimo clock.

### VHDL

```
process (clk, reset)
```

```
begin
```

```
    if reset = '1' then
```

```

q <= (others => '0');
elsif clk'event and clk = '1' then
    q <= d;
end if;
end process;

```

Qui reset è nella sensitivity list, quindi:

- quando reset cambia → il processo si attiva
- se reset = 1 → q viene azzerato subito
- altrimenti, se c'è un fronte di clock → q prende d

### Differenza chiave rispetto al reset sincrono

#### Reset sincrono

Agisce solo sul clock

Reset letto sul fronte di clk

@(posedge clk)

#### Reset asincrono

Agisce subito

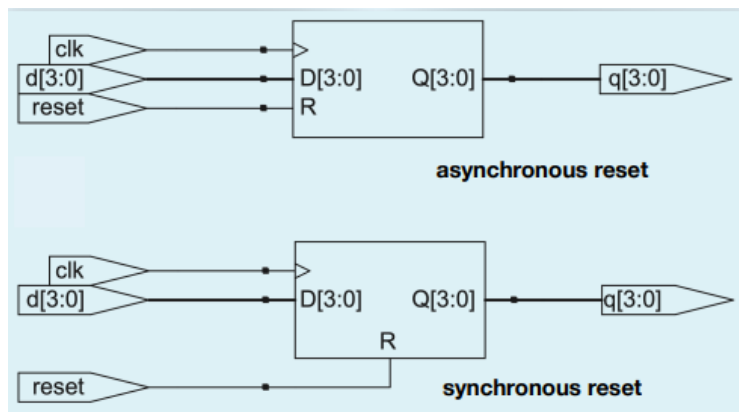
Reset letto appena cambia

@(posedge clk, posedge  
reset)

In VHDL, senza reset asincrono, i flip-flop partono in stato U (unknown).

Con reset asincrono, appena reset viene portato a 1, tutti i registri vengono inizializzati a 0.

Un reset asincrono forza il flip-flop indipendentemente dal clock: quando il reset si attiva, il registro si azzerava immediatamente, senza aspettare il fronte di clock.



## Resettable enabled registers

Qui non stiamo più parlando di un semplice flip-flop, ma di un **registro controllato**:

un registro che:

- può essere **azzerato** (reset)
- può essere **aggiornato** solo quando en = 1
- altrimenti **mantiene il valore**

## SystemVerilog

```
always_ff @(posedge clk)
```

```
    if (reset)
```

```
        q <= 4'b0;
```

```
    else if (en)
```

```
        q <= d;
```

Qui succedono tre casi sul fronte di clock:

| reset | en | cosa fa q               |
|-------|----|-------------------------|
| 1     | X  | q = 0                   |
| 0     | 1  | q = d                   |
| 0     | 0  | q mantiene il<br>valore |

Notare la cosa fondamentale:

se reset = 0 e en = 0, q non viene assegnato → quindi mantiene il valore

Ed è questo che lo rende un registro.

## VHDL

```
if clk'event and clk = '1' then
```

```
    if reset = '1' then
```

```
        q <= (others => '0');
```

```
    elsif en = '1' then
```

```
        q <= d;
```

```
    end if;
```

end if;

Identico comportamento:

- reset vince su tutto
- en decide se caricare
- se nessuna condizione è vera  $\rightarrow$  q resta uguale

Nel disegno vedi che:

- il flip-flop riceve:
  - o D
  - o clk
  - o reset
  - o enable

Ma **enable non è un pin del flip-flop**, è implementato con un **mux davanti a D**:

se  $en = 1 \rightarrow$  passa D

se  $en = 0 \rightarrow$  rimanda indietro Q

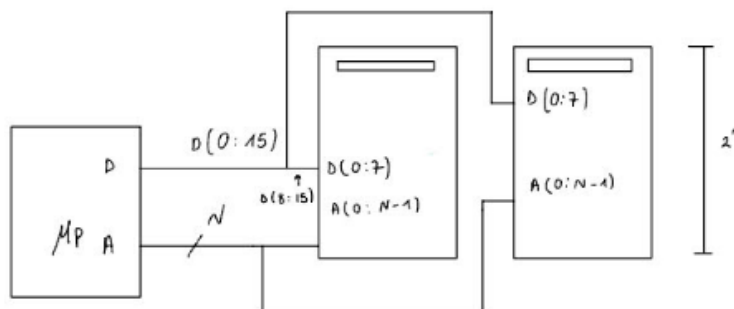
Questo è un **registro abilitato**.

Quasi tutti i registri reali hanno: **clock + reset + enable**

Un registro con enable viene aggiornato solo sul fronte di clock quando  $en = 1$ ; se  $en = 0$  mantiene il valore. Il reset ha priorità e forza il registro a zero.

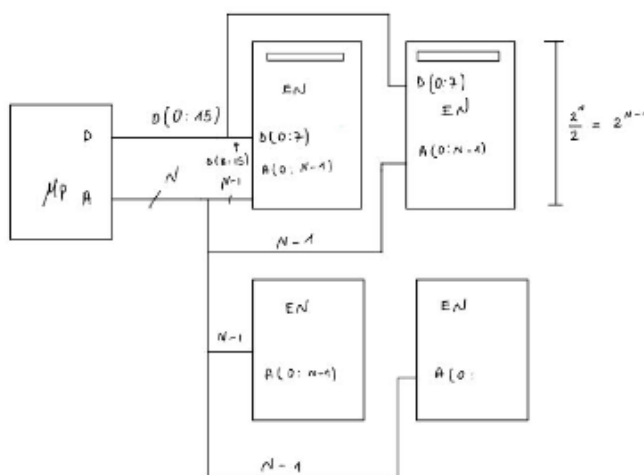
## Organizzazione della memoria e decodifica degli indirizzi

Il professore sta mostrando come una memoria logica di dimensione  $2^N$  può essere costruita utilizzando più memorie fisiche più piccole, e come gli indirizzi vengono usati per selezionare quale parte della memoria deve rispondere.



Nel modello ideale, un microprocessore utilizza un bus di indirizzi  $A[0: N - 1]$  per accedere a una memoria unica contenente  $2^N$  locazioni. Ogni combinazione dei  $N$  bit di indirizzo identifica una cella di memoria. Il disegno mostra come una CPU con bus dati a 16 bit e bus indirizzi a  $N$  bit usa due memorie da 8 bit per parola per realizzare una memoria da  $2^N$  parole a 16 bit.

In pratica, però, una memoria così grande non viene realizzata come un unico chip, ma come un insieme di moduli più piccoli. Per esempio, una memoria da  $2^N$  locazioni può essere costruita usando due memorie da  $2^{N-1}$  locazioni ciascuna. In questo caso, ogni chip contiene metà dello spazio totale di memoria.



Il bus degli indirizzi viene allora diviso in due parti:

- i bit meno significativi  $A[0: N - 2]$  vengono inviati a entrambi i chip e selezionano la cella interna al singolo banco;
- il bit più significativo  $A[N - 1]$  non seleziona una cella, ma viene usato per scegliere quale dei due chip deve essere attivo.

Il bit  $A[N - 1]$  viene quindi portato a un circuito di **decodifica**, che genera due segnali di **enable** (EN), uno per ciascun banco di memoria. Quando  $A[N - 1] = 0$ , viene abilitato il primo banco; quando  $A[N - 1] = 1$ , viene abilitato il secondo. Solo il banco abilitato può leggere o scrivere sul bus dati, mentre l'altro rimane disattivato.

Questo meccanismo può essere esteso a più banchi: usando più bit dell'indirizzo è possibile suddividere lo spazio di memoria in  $2^k$  banchi più piccoli. In questo caso è necessario un decoder più grande per generare i segnali di enable dei vari chip.

Per questo motivo, **la logica di decodifica diventa tanto più complessa quanto più piccolo è ciascun banco di memoria**: più banchi servono per coprire lo spazio totale, più combinazioni devono essere riconosciute dal decoder.

Un solo processo HDL può descrivere più registri fisici.

In particolare, viene mostrato un synchronizer, cioè una catena di due flip-flop in serie, usata per portare un segnale asincrono dentro un dominio di clock in modo sicuro.

SystemVerilog:

```
always_ff @(posedge clk) begin  
    n1 <= d;  
    q <= n1;  
end
```

Anche in VHDL:

$\leq$  è un **signal assignment**

A prima vista sembra che:

- prima n1 prende d
- poi q prende n1

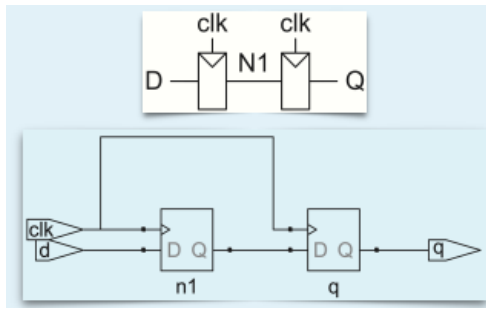
Ma questa è **una falsa intuizione da software**. In realtà  $\leq$  è un **non-blocking assignment**. Questo significa:

Tutti gli assegnamenti vengono calcolati con i **valori vecchi** e poi aggiornati **tutti insieme** sul fronte di clock.

Quindi sul fronte di clock:

- n1 prende il valore di d
- q prende il valore **precedente** di n1

Questo crea **due flip-flop in cascata**.



## Perché si chiama synchronizer?

Se d arriva da un altro clock o dall'esterno, può essere **asincrono**.

Il primo flip-flop può diventare metastabile.

Il secondo flip-flop "ripulisce" il segnale.

Per questo motivo in tutti i chip reali gli ingressi asincroni entrano sempre attraverso **due flip-flop in serie**.

Un singolo `always_ff` può descrivere più registri: usando non-blocking assignment, gli aggiornamenti avvengono in parallelo, quindi due assegnamenti in cascata realizzano due flip-flop in serie, come in un synchronizer.

## Transparent Latches

Un **D-latch** è detto **trasparente** quando, mentre il clock è alto ( $\text{clk} = 1$ ), il dato passa liberamente dall'ingresso d all'uscita q.

Quando invece  $\text{clk} = 0$ , il latch diventa **opaco**: smette di seguire l'ingresso e **mantiene l'ultimo valore memorizzato**.

Quindi:

- $\text{clk} = 1 \rightarrow q = d$  (passaggio diretto)
- $\text{clk} = 0 \rightarrow q$  resta costante

Questo comportamento è molto diverso da un flip-flop:

il flip-flop campiona solo sul fronte di clock, il latch invece è "aperto" per tutto il tempo in cui il clock è alto.

## SystemVerilog

*`always_latch`*

*`if (clk) q <= d;`*

Questo codice dice:

Se clk è 1, q prende d.

Se clk è 0, non succede nulla → q mantiene il valore precedente.

Ed è proprio questo “non succede nulla” che crea la memoria → **un latch**.

In SystemVerilog:

- always\_latch è equivalente a always @(clk, d)
- quindi il blocco viene rieseguito ogni volta che cambia clk o d
- se clk è alto, l'ingresso passa all'uscita
- se clk è basso, il valore resta bloccato

Il compilatore può anche dare un warning se il blocco non genera davvero un latch.

## VHDL

```
process(clk, d)
```

```
begin
```

```
  if clk = '1' then
```

```
    q <= d;
```

```
  end if;
```

```
end process;
```

Qui succede esattamente la stessa cosa.

La sensitivity list contiene sia clk che d, quindi:

- ogni variazione di clk o d fa rieseguire il processo
- se clk = 1, q segue d
- se clk = 0, non c'è assegnamento → q mantiene il valore

Anche questo descrive un **latch trasparente attivo alto**.



Se non necessario, usa flip-flop edge-triggered invece dei latch.

Perché?

- i latch creano **cammini temporali difficili da controllare**
- possono introdurre **race condition**

- sono spesso creati per errore da if incompleti

Infatti in SystemVerilog e VHDL:

Un if senza else in logica sequenziale → genera un latch

Un latch è trasparente quando il clock è alto: in quel periodo l'ingresso passa direttamente all'uscita; quando il clock è basso il valore viene mantenuto. Questo comportamento è descritto da un if senza else ed è molto diverso da un flip-flop che campiona solo sul fronte.

### Contatore – Behavioral vs Structural

Un contatore è un **registro** che, a ogni fronte di salita del clock, aggiorna il proprio valore secondo la legge:

- se reset = 1 → va a 0
- altrimenti → incrementa di 1

Nel **behavioral style** questo comportamento si scrive direttamente, ad esempio  $q \leq q + 1$ , lasciando al sintetizzatore il compito di costruire l'hardware.

In VHDL si usa spesso un segnale interno ( $q\_int$ ) perché l'uscita  $q$  non viene usata come stato.

Nel **structural style**, invece, lo stesso contatore viene costruito collegando blocchi:

- un **adder** che calcola  $q + 1$
- un **flip-flop** che memorizza il valore successivo

Il segnale nextq è l'uscita dell'adder ed entra nel registro, che produce  $q$ .

### Conclusione:

behavioral e structural descrivono lo **stesso circuito fisico** (registro + sommatore), ma

- il behavioral descrive *cosa fa*
- lo structural descrive *come è fatto*.

### Shift Register con Parallel Load

Questo circuito è un **registro a scorrimento a 4 bit** che può funzionare in due modi:

1. **Parallel load**  
Se load = 1, il registro carica in un colpo solo i 4 bit  $d[3:0]$
2. **Shift**  
Se load = 0, a ogni clock il registro:
  - sposta i bit verso sinistra

- inserisce sin come nuovo bit meno significativo

In entrambi i casi il comportamento avviene **sul fronte di salita del clock**.

In **SystemVerilog**

```
always_ff @(posedge clk)
  if (reset)    q <= 0;
  else if (load) q <= d;
  else         q <= {q[2:0], sin};
```

```
assign sout = q[3];
```

Significato:

- Se reset = 1 → il registro viene azzerato
- Se load = 1 → carica tutti i bit di d
- Altrimenti → fa lo shift:
- $q \leftarrow \{q[2], q[1], q[0], \text{sin}\}$

cioè:

```
q3 ← q2
```

```
q2 ← q1
```

```
q1 ← q0
```

```
q0 ← sin
```

sout è semplicemente il **bit più significativo** (quello che esce dal registro quando scorre).

In VHDL si fa la stessa cosa, ma usando un segnale interno qInt:

```
if reset = '1' then
  qInt <= "0000";
elsif load = '1' then
  qInt <= d;
else
  qInt <= qInt(2 downto 0) & sin;
end if;
```

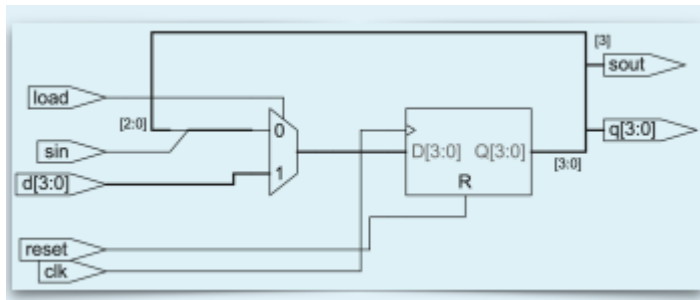
e poi:

```
q <= qInt;
```

```
sout <= qInt(3);
```

& è la concatenazione (come { } in SystemVerilog).

Il disegno mostra esattamente cosa succede:



- Un **multiplexer** sceglie:
  - o o d[3:0] (se load=1)
  - o o {q[2:0], sin} (se load=0)
- Il risultato entra in un **registro**
- L'uscita più alta (q[3]) diventa sout

Questo circuito è un registro a scorrimento con caricamento parallelo: può caricare 4 bit in un solo colpo oppure scorrere i dati inserendo sin e facendo uscire sout. È realizzato con un multiplexer davanti a un registro.

## ALWAYS/PROCESS

Finora abbiamo visto che gli assegnamenti (assign) possono essere usati per descrivere logica combinatoria.

Tuttavia, è possibile descrivere comportamenti combinatori anche usando always (SystemVerilog) o process (VHDL), se le loro sensitivity list rispondono a tutti gli ingressi responsabili dei cambiamenti dell'uscita.

Gli HDL supportano assegnamenti **blocking** (=) e **non-blocking** (<=) all'interno di un blocco always.

Un gruppo di assegnamenti blocking viene valutato nell'ordine in cui appare nel codice, come in un normale linguaggio di programmazione.

Un gruppo di assegnamenti non-blocking invece viene valutato in modo concorrente: tutte le espressioni a destra vengono calcolate prima che qualunque valore a sinistra venga aggiornato.

Per ora possiamo assumere che sia più efficiente usare gli assegnamenti **blocking** per la logica combinatoria e quelli **non-blocking** per la logica sequenziale.

SystemVerilog

```
module inv ( input logic [3:0] a,  
            output logic [3:0] y);  
  
    always_comb  
        y = ~a;  
  
endmodule
```

Questo modulo è un **invertitore a 4 bit**.

- a è un bus di 4 bit
- y è un bus di 4 bit
- ~a significa “nega ogni bit di a”

La riga `always_comb` dice al compilatore che questo è un blocco di **logica combinatoria**. E che deve rieseguire ogni volta che cambia un segnale usato dentro.

Quindi:

- se cambia anche solo un bit di a
- y viene ricalcolato automaticamente

È **identico** dal punto di vista hardware a `assign y = ~a;` Solo che qui lo scrivi usando un blocco `always`.

VHDL

```
process(a)  
begin  
    y <= not a;  
end process;
```

Anche qui stiamo descrivendo un invertitore a 4 bit.

`process(a)` significa: riesegui questo processo ogni volta che cambia a  
`y <= not a` significa: assegna a y la negazione di a.

Quindi, ogni volta che a cambia, y cambia immediatamente. Anche questo è logica combinatoria pura. Non solo `assign`, ma anche `always` e `process` possono descrivere porte logiche e reti combinatorie, se la sensitivity list è completa.

In altre parole:

always\_comb in SystemVerilog e process(a, b, c, ...) in VHDL sono un modo per descrivere circuiti di porte, non sequenze di istruzioni.

N.B.

La sincronia non dipende da <= o =, ma dalla presenza del clock nella sensitivity list.

In SystemVerilog si usano = per la logica combinatoria e <= per i registri.

In VHDL := è per le variabili (blocking) e <= per i segnali (non-blocking).

## Full Adder

Il full adder è un circuito combinatorio che somma tre bit (a, b e cin) producendo un bit di somma s e un riporto cout.

È un blocco fondamentale dell'ALU e viene usato per costruire addizionatori multi-bit.

Nel full adder si introducono i segnali propagate  $p = a \text{ xor } b$  e generate  $g = a \text{ and } b$ . p indica se il carry in ingresso viene propagato, mentre g indica se la coppia di bit genera direttamente un carry.

SystemVerilog

```
module fulladder ( input logic a, b, cin,
                  output logic s, cout);
```

```
    logic p, g;
    always_comb begin
        p  = a ^ b;      // blocking
        g  = a & b;      // blocking
        s  = p ^ cin;    // blocking
        cout = g | (p & cin); // blocking
    end
endmodule
```

### Spiegazione:

- In questo caso, scrivere always @(a, b, cin) oppure always @(\*) sarebbe stato equivalente a always\_comb: tutti e tre rieseguono il blocco ogni volta che cambiano a, b o cin.
- Tuttavia, always\_comb è preferito perché è più compatto e permette ai tool di generare un warning se il blocco **per errore** descrive logica sequenziale.
- La struttura begin / end è necessaria perché dentro l'always ci sono più istruzioni (analogo alle parentesi graffe {} in C/Java).
- Questo esempio usa assegnamenti blocking per descrivere logica combinatoria: le istruzioni vengono valutate in ordine, calcolando prima p, poi g, poi s, e infine cout.

Vhdl

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.all;
```

```
entity fulladder is
```

```
  port(a, b, cin : in  STD_LOGIC;
```

```
        s, cout  : out STD_LOGIC);
```

```
end;
```

```
architecture behavioral of fulladder is
```

```
begin
```

```
  process(a, b, cin)
```

```
    variable p, g : STD_LOGIC;
```

```
  begin
```

```
    p := a xor b;           -- blocking
```

```
    g := a and b;           -- blocking
```

```
    s <= p xor cin;         -- nonblocking
```

```
    cout <= g or (p and cin); -- nonblocking
```

```
  end process;
```

```
end;
```

### Spiegazione

- La sensitivity list deve includere a, b e cin perché la logica combinatoria deve rispondere ai cambiamenti di qualunque ingresso. Se ne mancasse uno, il codice potrebbe sintetizzare logica sequenziale oppure comportarsi diversamente tra simulazione e sintesi.
- L'esempio usa assegnamenti blocking per p e g così che ottengano i nuovi valori prima di essere usati per calcolare s e cout che dipendono da loro.
- p e g compaiono a sinistra di := dentro un process, quindi devono essere dichiarate come variable.

### 7-Segments Digit Decoder

I due esempi precedenti erano applicazioni piuttosto semplici degli statement always / process. In entrambi i casi, si sarebbero potuti usare anche assegnamenti assign. Inoltre, gli statement always non erano strettamente necessari per descrivere i segnali di uscita, perché appartenevano alla logica combinatoria. Tuttavia, quando si usano always / process, è possibile descrivere logiche più complesse rispetto a semplici strutture di porte.

L'esempio seguente descrive un decoder per display a 7 segmenti. L'interesse di questa descrizione è che il livello di astrazione è più alto: qui non si ragiona in termini di porte, ma in termini di valori booleani. Il decoder prende in ingresso un numero a 4 bit e visualizza la cifra decimale corrispondente su un display a 7 segmenti.

SystemVerilog

```
always_comb
```

```
  case (data)
```

```
    0: segments = 7'b1111110;
```

```

1: segments = 7'b0110000;
2: segments = 7'b1101101;
...
9: segments = 7'b1111011;
default: segments = 7'b0000000;
endcase

```

Dove data è un numero binario a 4 bit (0–9), mentre segments[6:0] controlla i 7 LED del display (a, b, c, d, e, f, g).

La clausola default è un modo comodo per definire l'uscita nei casi non esplicitamente elencati, garantendo logica combinatoria.

In SystemVerilog:

- Un case dentro always deve coprire tutti i casi
- Se mancano dei casi, l'uscita mantiene il valore precedente → latch

Per questo il default è obbligatorio se vogliamo vera logica combinatoria. Inoltre, senza default, se data è tra 10 e 15 (esadecimale A–F), l'uscita manterrebbe il valore precedente: questo non è comportamento combinatorio ed è pericoloso in hardware.

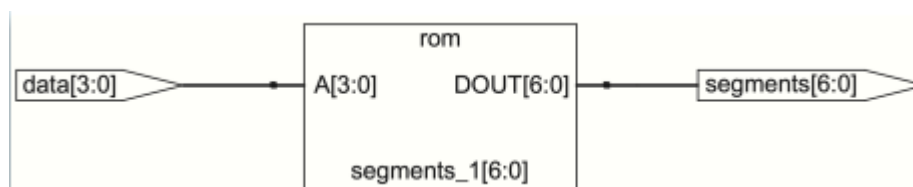
VHDL

```

process(data)
begin
  case data is
    when X"0" => segments <= "1111110";
    when X"1" => segments <= "0110000";
    ...
    when X"9" => segments <= "1111011";
    when others => segments <= "0000000";
  end case;
end process;

```

Qui data è a 4 bit, segments è un bus a 7 bit e when others è l'equivalente di default. Il case controlla il valore di data e assegna segments di conseguenza. In VHDL il case deve coprire tutti i valori. Se non lo fa, la logica non è combinatoria. VHDL supporta anche assegnamenti concorrenti con select, ma qui viene usato process per descrivere logica combinatoria.



La figura mostra che questo decoder è in realtà una ROM. Ogni valore di data seleziona una parola di memoria da 7 bit che accende i segmenti giusti.

Un case completo (con default / when others) dentro always\_comb o process descrive una vera rete combinatoria, equivalente a una ROM o a una tabella di verità.

## Generic Decoder

I decoder generici sono spesso scritti usando gli statement case.

Questo esempio descrive un decoder  $3 \rightarrow 8$ .

### SystemVerilog

```
module decoder3_8 ( input logic [2:0] a,
                   output logic [7:0] y);
    always_comb
    case (a)
        3'b000: y = 8'b00000001;
        3'b001: y = 8'b00000010;
        3'b010: y = 8'b00000100;
        3'b011: y = 8'b00001000;
        3'b100: y = 8'b00010000;
        3'b101: y = 8'b00100000;
        3'b110: y = 8'b01000000;
        3'b111: y = 8'b10000000;
    endcase
endmodule
```

No default statement is needed in this case because all cases are covered.

### VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;
entity decoder3_8 is
    port (a: in STD_LOGIC_VECTOR(2 downto 0);
          y: out STD_LOGIC_VECTOR(7 downto 0));
end;
architecture behave of decoder3_8 is
begin
    process(a) begin
        case a is
            when "000" => y <= "00000001";
            when "001" => y <= "00000010";
            when "010" => y <= "00000100";
            when "011" => y <= "00001000";
            when "100" => y <= "00010000";
            when "101" => y <= "00100000";
            when "110" => y <= "01000000";
            when "111" => y <= "10000000";
            when others => y <= (OTHERS => 'X');
        end case;
    end process;
end;
```

Some VHDL tools require an others clause because combinations such as "1zx" of the nine-value logic are not covered.

L'ingresso è  $a[2:0] \rightarrow 3$  bit

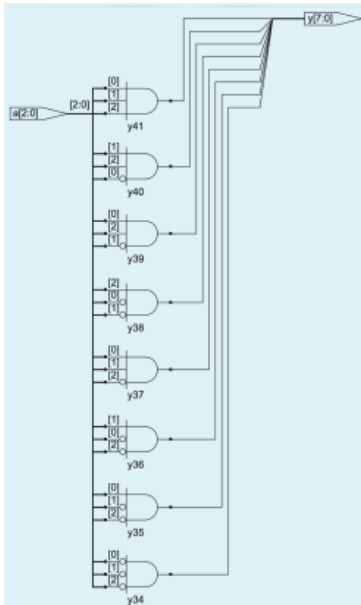
L'uscita è  $y[7:0] \rightarrow 8$  bit

Il decoder è one-hot:

- per ogni valore di a, esattamente un bit di y vale 1
- tutti gli altri valgono 0

Esempio:

- $a = 000 \rightarrow y = 00000001$
- $a = 001 \rightarrow y = 00000010$
- ...
- $a = 111 \rightarrow y = 10000000$



## FLOW CONTROL

### If statements

Gli statement always / process possono contenere anche istruzioni if.

L'if può essere seguito da un else.

Quando tutte le possibili combinazioni degli ingressi sono gestite, l'istruzione implica logica combinatoria; altrimenti produce logica sequenziale.

L'esempio seguente descrive un circuito di priorità che imposta l'uscita uguale all'ingresso più significativo che vale 1.

SystemVerilog

*always\_comb*

```
if (a[3])    y = 4'b1000;
else if (a[2]) y = 4'b0100;
else if (a[1]) y = 4'b0010;
else if (a[0]) y = 4'b0001;
else        y = 4'b0000;
```

Questo codice:

- controlla i bit di a partendo dal più significativo (a[3])
- restituisce un vettore y con un solo bit a 1
- se più bit sono 1, **vince quello con indice più alto**

Quindi è un **priority encoder**.

In SystemVerilog, gli if devono apparire dentro un always.

VHDL

*process(a)*

*begin*

*if a(3) = '1' then*

```

    y <= "1000";
  elsif a(2) = '1' then
    y <= "0100";
  elsif a(1) = '1' then
    y <= "0010";
  elsif a(0) = '1' then
    y <= "0001";
  else
    y <= "0000";
  end if;
end process;

```

A differenza di Verilog, VHDL supporta assegnamenti condizionali di segnale che funzionano come if, ma possono apparire anche fuori da un process. Per questo c'è meno motivo di usare i process per descrivere logica combinatoria.

### casez Statements

SystemVerilog fornisce anche lo statement casez per descrivere tabelle di verità con "don't care" (indicati con ?).

L'esempio seguente mostra come descrivere lo stesso circuito di priorità usando casez.

SystemVerilog – casez

```

always_comb
  casez(a)
    4'b1???: y = 4'b1000;
    4'b01???: y = 4'b0100;
    4'b001?: y = 4'b0010;
    4'b0001: y = 4'b0001;
    default: y = 4'b0000;
  endcase

```

Qui:

? significa don't care:

1??? significa "a[3]=1, gli altri non contano"

01?? significa "a[3]=0, a[2]=1"

Alcuni sintetizzatori possono generare realizzazioni di circuito leggermente diverse rispetto alla versione precedente, ma i due circuiti sono logicamente equivalenti.

If / else e case / casez sono selettori hardware e un if a priorità è un circuito di priorità fisico, non una decisione software.

### BLOCKING AND NON – BLOCKING

HDL non è un linguaggio di programmazione, infatti serve a **descrivere** hardware reale. Se usiamo = o := o <= nel modo sbagliato, la simulazione può sembrare corretta ma l'hardware sintetizzato è sbagliato. Per questo ci sono regole precise.

### Regola 1 - Logica sequenziale (registri, flip-flop)

Usa `always_ff @(posedge clk)` e non-blocking assignments (`<=`) per modellare logica sequenziale sincrona (cioè registri e flip-flop). Con `<=` tutte le assegnazioni vengono valutate insieme sul fronte di clock e il circuito si comporta come un vero banco di flip-flop.

### Regola 2 - Logica combinatoria semplice

Usa continuous assignments (`assign` in SystemVerilog e `<=` in VHDL) per descrivere la logica combinatoria semplice. Qui l'uscita cambia immediatamente quando cambia un ingresso.

### Regola 3 - Logica combinatoria più complessa

Usa `always_comb` e assegnamenti blocking (`=`) per descrivere logica combinatoria più complessa, quando un blocco `always` è utile.

`=` significa calcolare usare subito il valore appena assegnato; serve quando un segnale dipende da risultati intermedi. In VHDL questo equivale a usare:

- `:=` per le variabili
- `<=` per i segnali

### Regola 4 - Un solo driver per segnale

Non assegnare lo stesso segnale in più di un blocco `always` o in più di un'assegnazione continua. Eccezione: i bus tri-state. Un segnale deve avere un solo "chi lo guida". Se più blocchi lo assegnano nasce un conflitto e l'hardware è sbagliato.

### Il problema: full adder e tipo di assegnamenti

Il full adder è un circuito puramente **combinatorio** che calcola la somma *s* e il riporto *cout* a partire da *a*, *b* e *cin*.

Per semplificare il circuito si introducono due segnali intermedi:

- **p (propagate)** =  $a \oplus b$
- **g (generate)** =  $a \wedge b$

Da questi si ricavano:

$$s = p \oplus cin \qquad \qquad \qquad cout = g \vee (p \wedge cin)$$

In HDL questo significa che *s* e *cout* dipendono da *p* e *g*; quindi, *p* e *g* devono essere calcolati prima di *s* e *cout*.

### La versione corretta del full adder ha il blocking per logica combinatoria

Il full adder è scritto con **blocking assignments** (`=` in SystemVerilog, `:=` per variabili in VHDL).

Questo fa sì che le istruzioni vengano eseguite **in ordine**, come in un programma.

Nel codice in SystemVerilog:

```
p = a ^ b;  
g = a & b;  
s = p ^ cin;  
cout = g | (p & cin);
```

il simulatore calcola prima *p*, poi *g*, poi *s* usando il nuovo valore di *p*, e infine *cout* usando i nuovi valori di *p* e *g*.

Questo rispecchia esattamente la dipendenza logica del circuito e produce sempre il valore corretto.

In VHDL lo stesso effetto si ottiene usando **variabili** (p, g) con :=, perché le variabili si aggiornano subito e possono essere usate immediatamente nelle espressioni successive.

### **Cosa succede se uso non-blocking nella logica combinatoria ?**

Il full adder viene riscritto usando non-blocking (<=) per tutti i segnali, incluso p e g.

Il problema è che con <= tutte le assegnazioni vengono valutate in parallelo usando i valori vecchi.

Se ad esempio a passa da 0 a 1 mentre b e cin sono 0, allora:

- p dovrebbe diventare 1
- s dovrebbe diventare 1

Ma quando si esegue:

$s \leq p \wedge cin$ ; il p usato è ancora quello vecchio (0), perché l'aggiornamento di  $p \leq a \wedge b$  non è ancora avvenuto.

Quindi s viene temporaneamente calcolato come 0, che è sbagliato.

Questo non succede con i blocking perché lì p viene aggiornato prima di essere usato.

### **Perché alla fine funziona comunque ("Lucky") ?**

Anche usando solo non-blocking, il risultato alla fine diventa corretto.

Questo accade perché, quando p cambia (da 0 a 1), il blocco always\_comb viene eseguito una **seconda volta**.

Alla seconda esecuzione p è ormai aggiornato, quindi s viene finalmente calcolato come 1.

Quindi:

- il circuito finale è giusto,
- ma il simulatore ha dovuto fare **due passaggi** per arrivarci.

Questo rende la simulazione più lenta e meno pulita, ed è un segnale che il codice è scritto male.

### **Il vero pericolo: la sensitivity list**

Se al posto di always\_comb si usa una sensitivity list esplicita come:

always @(a, b, cin) allora il processo non si riattiva quando cambiano p o g.

Ma siccome con i non-blocking s dipende dal vecchio p, il valore sbagliato non viene più corretto.

Il risultato è che:

- la simulazione dà un valore sbagliato,
- ma alcuni sintetizzatori costruiscono comunque l'hardware giusto.

Non bisogna usare mai non-blocking (<=) per logica combinatoria che usa segnali intermedi.

Per la logica combinatoria:

- in SystemVerilog si usa always\_comb + =
- in VHDL si usano process + := per variabili

I non-blocking ( $\leq$ ) vanno usati solo per registri e flip-flop, cioè per la logica sequenziale.

### Synchronizer

Un synchronizer è una piccola catena di flip-flop usata per rendere “sicuro” un segnale che arriva da un altro dominio di clock o dall'esterno del sistema.

Serve a ridurre il rischio di metastabilità: il primo flip-flop può entrare in uno stato incerto, ma il secondo lo “ripulisce”.

```
In systemVerilog
always_ff @(posedge clk)
begin
    n1 <= d;
    q <= n1;
end
```

Questa è la forma canonica di un sincronizzatore. Alla salita del clock: n1 prende il valore di d nello stesso istante q prende il valore precedente di n1. Questo è esattamente quello che fa una catena di due flip-flop fisici. Il motivo per cui funziona è che  $\leq$  è non-blocking: tutti i valori a destra vengono letti prima e poi tutti i registri vengono aggiornati insieme. Quindi q non vede il nuovo n1, ma quello vecchio, cioè l'uscita del primo flip-flop del ciclo precedente.

```
vhdl
process(clk)
begin
    if clk'event and clk='1' then
        n1 <= d;
        q <= n1;
    end if;
end process;
```

È identico concettualmente.

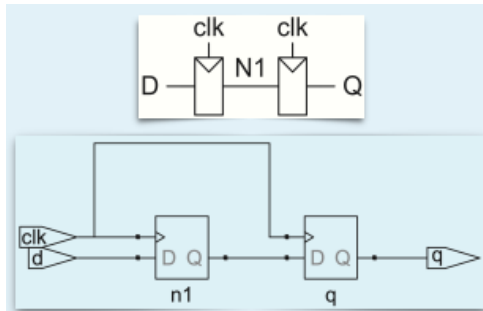
$\leq$  in VHDL è non-blocking, quindi produce esattamente due flip-flop in cascata. Il non-blocking è perfetto per la logica sequenziale, ed è sbagliato per la logica combinatoria con dipendenze.

Nel synchronizer vogliamo che q prenda il valore vecchio di n1 e non quello appena aggiornato. Questo perché stiamo modellando due registri fisici distinti. Dunque, è corretto usare il non-blocking.

Se usassimo blocking (=):

```
n1 = d;
q = n1;
allora q diventerebbe uguale a d nello stesso ciclo e avremmo un solo flip-flop, non due.
```

Il synchronizer è l'esempio "puro" che mostra perché  $\leq$  esiste: serve a descrivere più registri che si aggiornano in parallelo sul clock.



Nella logica sequenziale bisogna usare **esclusivamente non-blocking assignments**. Anche se con trucchi e riordinando le istruzioni potresti far funzionare i blocking in alcuni casi, non danno alcun vantaggio e introducono solo rischi. Alcuni circuiti sequenziali **non funzioneranno mai correttamente** se scritti con blocking, qualunque sia l'ordine delle istruzioni.

### Macchine a Stati Finiti (FSM)

Una **Finite State Machine (FSM)** è un modello di circuito digitale che combina **logica sequenziale** e **logica combinatoria** per descrivere un sistema che può trovarsi solo in un numero **finito di stati**.

Lo stato rappresenta la "memoria" del sistema: indica in quale fase del comportamento ci troviamo.

In pratica, una FSM è un circuito che:

- ricorda il proprio stato tramite **registri (flip-flop)**,
- decide il prossimo stato tramite **logica combinatoria**,
- genera le uscite tramite **logica combinatoria**.

Gli HDL (SystemVerilog e VHDL) permettono di descrivere una FSM in modo naturale, perché separano esattamente queste tre parti.

### Struttura generale di una FSM

Ogni FSM è composta da tre blocchi fondamentali:

1. **State Register (stato presente)**  
È un registro che memorizza lo stato attuale della macchina. Cambia solo sul fronte di clock.
2. **Next-State Logic (logica di transizione)**  
È una rete combinatoria che calcola il **prossimo stato** in funzione dello stato presente e degli ingressi.
3. **Output Logic (logica di uscita)**  
È una rete combinatoria che genera le **uscite** della FSM.

Questo è esattamente ciò che mostra la slide:

gli ingressi entrano nella logica di next-state, che calcola il nuovo stato; il registro lo memorizza al clock; e la logica di uscita produce gli output.

### Modelli di FSM: Moore e Mealy

Esistono due grandi famiglie di FSM:

## Moore FSM

In una **Moore FSM** le uscite dipendono **solo dallo stato presente**.

Formalmente:

$$output = f(stato)$$

Questo significa che:

- gli output cambiano **solo quando cambia lo stato**,
- e quindi **solo sul fronte di clock**.

Vantaggi:

- uscite molto **stabili**,
- comportamento **sincronizzato** con il clock,
- meno rischio di glitch.

È il modello più usato nei sistemi sincroni.

## Mealy FSM

In una **Mealy FSM** le uscite dipendono sia dallo **stato presente** sia dagli **ingressi**.

Formalmente:

$$output = f(stato, input)$$

Questo significa che:

- se cambia un ingresso, può cambiare subito anche l'uscita,
- senza aspettare il prossimo clock.

Vantaggi:

- la macchina è **più reattiva**.

Svantaggi:

- gli output possono avere **glitch** o variazioni rapide se gli ingressi oscillano.

## Differenza strutturale

Nella figura:

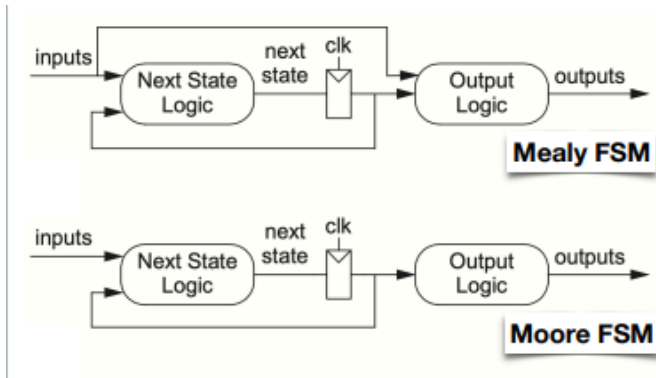
- **Mealy FSM:**  
gli ingressi vanno sia nella logica di next-state sia nella logica di uscita → l'uscita dipende anche dagli input.
- **Moore FSM:**  
gli ingressi vanno solo nella logica di next-state → l'uscita dipende solo dallo stato.

## Collegamento con HDL

Quando si scrive una FSM in HDL:

- il **registro di stato** è descritto con un `always_ff / process(clk)` e non-blocking (`<=`),
- la **next-state logic** è descritta con `always_comb` o `process(...)` combinatorio,
- la **output logic** è descritta come logica combinatoria (Moore: solo stato, Mealy: stato + input).

Questa separazione è esattamente quella mostrata nella slide.



### Divide-by-3 FSM

Questa FSM produce in uscita un segnale  $y$  che vale **1 ogni tre cicli di clock**.

È quindi un **divisore di frequenza per 3**: se il clock è  $f$ ,  $y$  è  $f/3$ .

Lo fa usando **3 stati** che rappresentano il resto della divisione per 3:

- $S0$  = resto 0
- $S1$  = resto 1
- $S2$  = resto 2

Ad ogni clock la FSM avanza:

$S0 \rightarrow S1 \rightarrow S2 \rightarrow S0 \rightarrow S1 \rightarrow S2 \rightarrow \dots$

### 1 – Registro di stato (logica sequenziale)

SystemVerilog:

```
always_ff @(posedge clk)
  if (reset) state <= 2'b00;
  else      state <= nextstate;
```

VHDL:

```
process(clk)
  if clk'event and clk='1' then
    if reset='1' then state <= "00";
    else state <= nextstate;
  end if;
end process;
```

Questo è il **registro di stato** della FSM:

- memorizza lo stato corrente
- cambia solo sul fronte di clock
- usa  $\leq$  (non-blocking) perché è **sequenziale**

### Parte 2 – Logica di Next-State (combinatoria)

SystemVerilog:

```
always_comb
  case (state)
    2'b00: nextstate = 2'b01;
    2'b01: nextstate = 2'b10;
```

```
2'b10: nextstate = 2'b00;
default: nextstate = 2'b00;
endcase
```

VHDL:

```
nextstate <= "01" when state="00" else
    "10" when state="01" else
    "00";
```

Questa parte implementa il **diagramma degli stati**:

00 → 01 → 10 → 00

È **logica combinatoria pura**:

- dipende solo da state
- non usa clock
- dice quale sarà lo stato dopo

Il default serve per sicurezza: se lo stato è illegale, la FSM torna a 00.

### Parte 3 – Logica di uscita (Moore FSM)

SystemVerilog:

```
assign y = (state == 2'b00);
```

VHDL:

```
y <= '1' when state="00" else '0';
```

Questa è una **Moore FSM**, perché:

l'uscita dipende solo dallo stato

y vale 1 **solo nello stato S0**.

Dato che S0 arriva ogni 3 clock → y è alto 1 volta ogni 3 cicli.

Perché divide per 3 ?

Se il clock è:

```
clk:  ↑  ↑  ↑  ↑  ↑  ↑
state: 00 01 10 00 01 10
y:      1  0  0  1  0  0
```

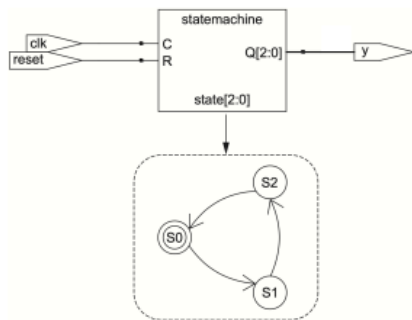
Ogni 3 fronti di clock y torna a 1 → frequenza divisa per 3.

Il tool di sintesi può mostrare:

- il **diagramma degli stati**
- oppure i **flip-flop + logica**

Entrambe rappresentano lo stesso hardware:

- un registro che memorizza lo stato
- una rete combinatoria che calcola nextstate
- una rete combinatoria che genera y



## State Enumeration

Normalmente gli stati di una FSM sono numeri binari:

00, 01, 10, 11...

Ma questi non hanno significato per chi legge il codice.

Con le enumerazioni puoi scrivere:

S0, S1, S2 e lasciare al tool di sintesi il compito di scegliere che numeri binari usare. Il circuito non cambia, cambia solo come lo descrivi.

È esattamente come dare nomi simbolici agli stati invece di numeri.

## SystemVerilog

```
typedef enum logic [1:0] {S0, S1, S2} statetype;
```

```
statetype state, nextstate;
```

Qui stai dicendo:

Definisco un nuovo tipo chiamato statetype che può assumere i valori S0, S1, S2.

*logic[1:0]* vuol dire che sotto sotto verranno usati **2 bit** (perché servono almeno 2 bit per 3 stati).

Poi scrivi:

```
statetype state, nextstate;
```

cioè:

- state e nextstate non sono numeri
- sono etichette di stato

## Registro di stato

```
always_ff @(posedge clk)
```

```
if (reset) state <= S0;
```

```
else state <= nextstate;
```

“se reset VAI nello stato S0”

## Logica di next-state

```
always_comb
```

```
case (state)
```

```
  S0: nextstate = S1;
```

```
  S1: nextstate = S2;
```

```
  S2: nextstate = S0;
```

```
  default: nextstate = S0;
```

```
endcase
```

Questo è **esattamente** il diagramma:

S0 → S1 → S2 → S0

Non c'è più bisogno di ricordare che:

- S0 = 00
- S1 = 01
- S2 = 10

Il codice **racconta la FSM**.

### Logica di uscita

```
assign y = (state == S0);
```

Vuol dire:

L'uscita vale 1 quando siamo nello stato S0.

In binario prima era:

```
assign y = (state == 2'b00);
```

Ma ora è semanticamente chiaro.

Il tool di sintesi sceglie l'encoding binario degli stati.

Puoi forzare l'encoding (se vuoi)

```
typedef enum logic [2:0] { S0=3'b000, S1=3'b001, S2=3'b010 } statetype;
```

Lo fai solo se vuoi:

- ottimizzare
- debuggare
- interfacciarti con hardware esterno

### VHDL è identico concettualmente

```
type statetype is (S0, S1, S2);
```

```
signal state, nextstate : statetype;
```

Esattamente la stessa cosa:

stati simbolici, non numeri.

Questo è il modo giusto di scrivere FSM perché:

- il codice diventa leggibile
- è facile aggiungere stati
- eviti errori sui bit
- la FSM è descritta a livello logico, non fisico

È programmare una macchina a stati, non cablare flip-flop a mano.

La FSM precedente aveva un'uscita e nessun ingresso, a parte clock e reset.

L'esempio seguente descrive una macchina a stati finiti con un ingresso a e due uscite.

L'uscita y vale 1 quando l'ingresso è uguale adesso a quello che era nei due cicli di clock precedenti.

Il diagramma degli stati qui sotto indica una macchina di tipo **Mealy**, perché l'uscita dipende sia dagli ingressi correnti sia dallo stato.

Le uscite sono etichettate su ogni transizione dopo l'ingresso.

In questa FSM l'uscita vale 1 se l'ingresso attuale è uguale a quello di due cicli di clock fa.

In altre parole, la FSM deve ricordare:

- il valore di a adesso

- quello di a un ciclo fa
- quello di a due cicli fa

e confrontarli.

Questa è una FSM perché non basta un registro: serve memoria strutturata cioè stati.

Si tratta di una Mealy FSM perché l'uscita y dipende da:

- lo stato (cioè cosa è successo nei cicli passati)
- l'ingresso a di ora

Nel codice SystemVerilog:

```
typedef enum logic [2:0] {S0, S1, S2, S3, S4} statetype;
statetype state, nextstate;
```

Ci sono 5 stati. Questi stati non sono “numeri”, sono situazioni logiche che rappresentano la storia recente di a.

Il diagramma degli stati che vedi nella slide rappresenta tutte le combinazioni possibili degli ultimi bit osservati.

### Registro di stato

```
always_ff @(posedge clk)
  if (reset) state <= S0;
  else      state <= nextstate;
```

Questo è sempre lo stesso schema:

- stato = memoria
- cambia solo al clock

### Logica di next-state

```
always_comb
  case (state)
    S0: if (a) nextstate = S3; else nextstate = S1;
    S1: if (a) nextstate = S3; else nextstate = S2;
    S2: if (a) nextstate = S4; else nextstate = S2;
    S3: if (a) nextstate = S4; else nextstate = S1;
    S4: if (a) nextstate = S4; else nextstate = S2;
    default: nextstate = S0;
  endcase
```

Questa è la **tabella di transizione**.

Ogni riga dice: “Se sono nello stato X e l'ingresso a vale Y, vado nello stato Z”. Questa logica codifica il diagramma degli stati.

### Logica di uscita (Mealy)

```
assign y = ((state == S1 | state == S2) & ~a) |
  ((state == S3 | state == S4) & a);
```

Questa è la cosa chiave.

L'uscita vale 1 se:

- siamo in certi stati e
- l'ingresso a ha un certo valore

Quindi:

y dipende sia dallo stato che dall'input infatti è una FSM di Mealy

Gli stati rappresentano la sequenza degli ultimi due valori di a.

La FSM sta facendo una cosa equivalente a uno shift register + confronto, ma in modo logicamente strutturato.

Ogni stato codifica qualcosa come:

- “gli ultimi due bit erano 00”
- “gli ultimi due bit erano 01”
- ecc.

Perché serve una FSM e non solo registri?

Perché:

- vuoi ricordare la storia
- ma anche reagire subito all'ingresso

Quindi serve:

1. memoria (stato)
2. logica che guarda stato + input
3. uscita che dipende da entrambi

Esattamente una Mealy FSM.

Ragionamento logicamente identico viene fatto per il VHDL.

## MORE IN DEPTH

### Type Idiosyncrasies – systemverilog

Lo **standard Verilog** utilizza principalmente due tipi: **reg** e **wire**.

Nonostante il nome, un segnale reg **può essere associato oppure no a un registro**.

Questo è stato una grande fonte di confusione per chi stava imparando il linguaggio. La sintassi è stata poi parzialmente rilassata con l'introduzione del tipo **logic** in

SystemVerilog, che ha allentato alcuni vincoli.

Qui spieghiamo i tipi reg e wire in modo più dettagliato, principalmente per compatibilità con il codice Verilog legacy.

In **Verilog**, se un segnale appare **sul lato sinistro di un <= o = in un blocco always**, allora deve essere dichiarato come **reg**.

Altrimenti, se non appare sul lato sinistro, un reg può rappresentare un **flip-flop**, un **latch** oppure **logica combinatoria**, a seconda della sensitivity list e del contenuto dell'**always**.

Gli input e gli output delle porte di un modulo non possono essere dichiarati reg a meno che non siano assegnati dentro un **always**.

Per questo motivo, un registro come un flip-flop è descritto tipicamente come reg e di default un'uscita è wire.

Si noti che **clk** ed **a** nel seguente esempio sono wire e **q** è esplicitamente dichiarato reg perché appare sul lato sinistro di **<=** in un **always**.

```
module flop (input clk, input [3:0] d, output reg [3:0] q);  
  always @(posedge clk)  
    q <= d;  
endmodule
```

Come già detto, **SystemVerilog introduce il tipo logic**.

**logic** è sinonimo di **reg** e rimuove l'interpretazione fuorviante del nome.

Inoltre, SystemVerilog allenta le regole sugli assegnamenti ai segnali. I segnali di tipo **logic** possono essere assegnati in **blocchi always**, in **assign** continui e anche in **istanziamenti gerarchici di moduli**.

Pertanto, quasi tutti i segnali vengono dichiarati come **logic**.

L'unica limitazione di **logic** è che **non può avere più di un driver**.

Questo è accettabile e desiderabile perché normalmente un segnale ha un solo driver.

SystemVerilog può generare un errore se un segnale **logic** è pilotato da più sorgenti.

Quando più driver sono necessari, come nell'esempio del **bus tri-state**, si devono usare i tipi **net**.

In questi casi **wire** e **tri** sono usati in modo intercambiabile quando c'è un solo driver, mentre **tri** è usato quando più driver sono presenti.

In pratica, in codice SystemVerilog **logic** è il tipo preferito per segnali con un solo driver.

Quando è pilotato da una sola sorgente assume quel valore.

Quando non è pilotato assume **z**.

Se è pilotato da più driver assume **x** (contenzione).

Esistono anche altri tipi di **net** che risolvono diversamente i driver multipli.

Non sono usati spesso, ma **tri** può essere sostituito ovunque con **trior** o **triand** per segnali con più driver.

Ognuno è descritto nella tabella

**Net Resolution**

| Net Type | No Driver      | Conflicting Drivers |
|----------|----------------|---------------------|
| tri      | z              | x                   |
| triand   | z              | 0 if any are 0      |
| trior    | z              | 1 if any are 1      |
| triereg  | previous value | x                   |
| tri0     | 0              | x                   |
| tri1     | 1              | x                   |

## Type Idiosyncrasies – VHDL

A differenza di SystemVerilog, VHDL impone un sistema di tipi molto rigido, che può proteggere l'utente da alcuni errori ma rende anche il linguaggio più verboso.

In VHDL esistono sei sistemi di tipi principali. Abbiamo già visto STD\_LOGIC e STD\_LOGIC\_VECTOR nelle librerie STD\_LOGIC\_1164. Inoltre, STD\_LOGIC\_1164 non ha operazioni aritmetiche come addizione, confronto, shift e conversione a interi per i segnali di tipo STD\_LOGIC\_VECTOR.

Queste operazioni sono invece definite nelle librerie IEEE.NUMERIC\_STD e IEEE.STD\_LOGIC\_SIGNED.

VHDL ha anche un tipo BOOLEAN e due valori: true e false.

Le assegnazioni vengono fatte usando operatori condizionali when ... else.

È facile confondersi e pensare che il valore BOOLEAN true sia equivalente a STD\_LOGIC = '1' e false a STD\_LOGIC = '0', ma questi tipi non sono intercambiabili. Per esempio, il seguente codice è illegale:

```
y <= d1 when s else d0;  
q <= '1' when state = s2 else '0';
```

Bisogna invece scrivere:

```
y <= d1 when s = '1' else d0;  
q <= '1' when state = s2 else '0';
```

Anche se non possiamo assegnare direttamente segnali a variabili BOOLEAN, essi sono automaticamente implicati dalle comparazioni e dalle istruzioni condizionali.

VHDL ha inoltre un tipo INTEGER, che rappresenta i numeri interi con segno da  $-2^{31}$  a  $2^{31}-1$ . I valori INTEGER sono usati come indici dei bus.

Per esempio, nell'istruzione:

```
y <= a(3) and a(2) and a(1) and a(0);
```

i numeri 0, 1, 2 e 3 sono INTEGER che selezionano i bit del segnale.

Non possiamo però indicizzare direttamente un bus con uno STD\_LOGIC o STD\_LOGIC\_VECTOR. Invece, dobbiamo convertire il segnale in INTEGER. Questa conversione è fatta tramite la funzione CONV\_INTEGER, che è definita nella libreria STD\_LOGIC\_UNSIGNED e converte uno STD\_LOGIC\_VECTOR positivo (unsigned) in un INTEGER.

Quando si progetta un circuito digitale in HDL bisogna sempre ricordare che un'uscita non è un normale segnale interno, ma rappresenta fisicamente un pin del chip pilotato da un buffer di uscita. Questo significa che un'uscita non è un nodo logico "libero" su cui si possono fare calcoli: è il punto finale della catena.

Per questo motivo, se un valore deve essere usato sia per il calcolo interno sia come uscita, non si deve mai usare direttamente l'uscita come variabile intermedia. Il modo corretto è introdurre un segnale interno che rappresenta il risultato logico e poi collegare quell'informazione all'uscita. In pratica si separa:

- la logica interna del circuito
- dal buffer che pilota il pin di uscita

Questo riflette esattamente l'hardware reale: prima si calcola il segnale dentro il chip, poi lo si invia all'esterno.

Se si cerca invece di usare un'uscita come ingresso di altre operazioni logiche, si crea una situazione innaturale: si sta tentando di leggere il valore di un pin fisico come se fosse un nodo interno del circuito. Questo può portare a errori di sintesi, conflitti di driver o strutture hardware non volute.

Il principio corretto è quindi sempre questo: le uscite vanno pilotate, non lette.

Se serve riutilizzare un valore, si usa un segnale interno e poi lo si copia sull'uscita.

### **Parameterized Modules (SV) e Generic (VHDL): stessa logica, larghezze diverse**

Finora molti moduli erano "a larghezza fissa" (es. mux 2:1 da 4 bit, da 8 bit, ecc.). L'idea dei **moduli parametrizzati** è: *scrivo una volta il modulo, e poi scelgo la larghezza (N) quando lo istanzio*. È fondamentale perché in progetti reali vuoi riusare lo stesso blocco su bus di dimensioni diverse (datapath, registri, indirizzi...).

### **SystemVerilog (parametro)**

In SV puoi dichiarare un parametro direttamente nella definizione del modulo, tipo:

- #(parameter N = 8) → se non dici nulla, N vale 8.
- Poi usi N per definire le ampiezze: logic [N-1:0] d0, d1, y.
- L'assegnazione del mux resta la stessa: assign y = s ? d1 : d0;  
Quindi la struttura logica è identica, cambiano solo le ampiezze.

### **VHDL (generic)**

In VHDL lo stesso concetto si chiama **generic**:

- generic (N : integer := 8);
  - e poi le porte: STD\_LOGIC\_VECTOR(N-1 downto 0)
- Anche qui: stesso mux, stessa funzione, ma con ampiezza N decisa "da fuori".

### **Parameterized Modules – Instantiation: come creo mux "8-bit" o "12-bit" dallo stesso mux2**

Si introduce il concetto di *riusabilità*.

### **In SystemVerilog: override del parametro quando istanzi**

Se fai un 4:1 usando tre mux 2:1, puoi istanziare:

- mux "low" e "high" con la stessa larghezza,
- poi un mux finale.

Quando vuoi un **4:1 a 12 bit**, invece di riscrivere tutto:

- istanzi mux2 #(12) lowmux (...)
- istanzi mux2 #(12) himux (...)
- istanzi mux2 #(12) outmux (...)

Quindi *stesso modulo*, solo parametro diverso.

### **VHDL: generic map**

In VHDL fai la stessa cosa con:

- generic map (N => 12)  
e poi port map (...).

I parametri e i generics servono a rendere i moduli "scalabili" e riusabili: stessa architettura, taglie diverse.

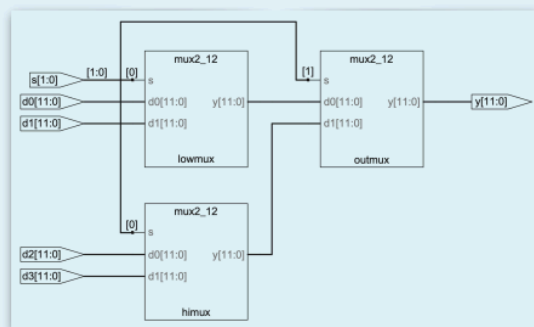
## Parameterized Modules – Mux Synthesis: cosa diventa dopo sintesi

La sintesi ti fa vedere che:

- anche se tu scrivi “mux2\_12”, in realtà è lo stesso mux 2:1 ma su 12 linee,
- e il 4:1 viene costruito gerarchicamente (due mux al primo livello + uno al secondo).

Questa è una dimostrazione visiva del fatto che **parametrizzare non cambia il tipo di hardware**, cambia solo quante “copie” di bit-slice ci sono.

### Parametrized Modules - Mux Synthesis



Parameterized Decoder: decoder  $N \rightarrow 2^N$  con codice compatto

Qui l'idea è: un decoder grande (es.  $8 \rightarrow 256$ ) con case diventa lunghissimo. Invece con codice parametrizzato fai:

1. Prima metti tutte le uscite a 0.
2. Poi metti a 1 solo la linea corrispondente all'indice a.

Se l'ingresso ha N bit, può rappresentare  $2^N$  valori diversi.

Un decoder deve quindi avere  $2^N$  uscite, una per ciascun valore possibile.

SystemVerilog

```
module decoder #(parameter N = 3)
```

```
(  
    input logic [N-1:0] a,  
    output logic [2**N-1:0] y  
);
```

```
always_comb begin
```

```
    y = 0;
```

```
    y[a] = 1;
```

```
end
```

```
endmodule
```

Come funziona?

1.  $y = 0$ ;  
Tutte le  $2^N$  uscite vengono azzerate.
2.  $y[a] = 1$ ;  
L'indice  $a$  (che è un numero binario su  $N$  bit) viene usato come indice del vettore  $y$ .

Se per esempio:

- $N = 3$
- $a = 3'b101 = 5$

allora:

- $y[5] = 1$
- tutte le altre uscite restano a 0.

Questo è esattamente il comportamento di un decoder.

Non importa se  $N$  vale 3, 8 o 16: la sintesi costruisce  $2^N$  linee di uscita e una rete che seleziona quella giusta.

Tu scrivi 3 righe di codice → l'hardware cresce automaticamente.

## VHDL

In VHDL il concetto è identico, ma serve più lavoro perché in VHDL gli indici di un vettore devono essere integer, non `STD_LOGIC_VECTOR`.

Ecco perché si usano conversioni.

```
variable tmp : STD_LOGIC_VECTOR(2**N - 1 downto 0);  
tmp := CONV_STD_LOGIC_VECTOR(0, 2**N);  
tmp(CONV_INTEGER(a)) := '1';  
y <= tmp;
```

Perché servono le conversioni:

- $a$  è un `STD_LOGIC_VECTOR`
- ma  $tmp(a)$  non è legale
- quindi serve: `CONV_INTEGER(a)` che converte il vettore binario  $a$  in un numero intero e `CONV_STD_LOGIC_VECTOR(0, 2**N)` che crea un vettore di lunghezza  $2^N$  inizializzato a zero.

## Generate statement

Il `generate` serve quando vuoi che il tool crei un numero variabile di istanze/porte in base a un parametro.

Esempio: catena di AND a 2 ingressi che “propaga”:

- $x[1] = a[0] \& a[1]$
- $x[2] = x[1] \& a[2]$
- ...
- $y = x[N-1]$

Con un `for-generate`:

- non scrivi  $N$  righe,
- ma dici al compilatore “ripetilo per  $i = 1..N-1$ ”.

Concetto fondamentale: `generate` non è “runtime”, è elaborazione a compile/synthesis time. Cioè, decide *quanta circuiteria* esiste davvero.

### Memory – RAM separated r/w buses: RAM sincrona con bus separati

Qui la memoria ha:

- un bus **din** (write data)
- un bus **dout** (read data)
- un indirizzo addr
- un write enable we
- un clock clk

Comportamento tipico

- Scrittura sincrona: avviene sul fronte di clock quando we=1.
- Lettura: spesso mostrata come combinatoria (dipende dall'indirizzo) oppure come uscita registrata, ma nella slide si vede che dout viene collegato al contenuto della memoria all'indirizzo.

Le RAM reali in FPGA/ASIC hanno primitive dedicate; il codice HDL “modella” la RAM ma poi il tool cerca di inferire una macro RAM efficiente, non una rete di flip-flop.

### Memory – RAM multiplexed r/w bus: un solo bus bidirezionale (inout) + tri-state

Qui la differenza è che invece di avere din e dout separati, hai **un solo bus dati** bidirezionale, tipo data.

- Quando **scrivi**: la CPU guida il bus e la RAM legge.
- Quando **leggi**: la RAM guida il bus e la CPU legge.
- Per evitare conflitti: quando non guida nessuno, il bus va in **Z** (alta impedenza).

Per questo nel codice compare:

- inout in SV/VHDL,
- e assegnamenti che mettono data = 'z quando non si deve guidare.

In VLSI moderno e su FPGA spesso si preferiscono connessioni point-to-point e mux interni, perché i tri-state “globali” non sempre sono supportati come una volta; però come concetto didattico è fondamentale per capire la risoluzione dei net e i bus condivisi.

### Multiport Register Files: più porte di lettura/scrittura

Un **register file multiport** è una “memoria di registri” dove:

- puoi leggere da più indirizzi nello stesso ciclo (es. due read port per un datapath tipo RISC: rs1 e rs2),
- e scrivere su un indirizzo (write-back) nello stesso ciclo.

La slide mostra:

- indirizzi separati per letture (es. a1, a2)
- un indirizzo per scrittura (es. a3)
- dati di uscita separati (d1, d2)
- un segnale di write enable.

È la base dei datapath dei microprocessori: due letture per alimentare ALU, una scrittura per il risultato.

### Memory – ROM: read-only memory via case

Una ROM piccola è spesso descritta così:

- `case(addr) →` per ogni indirizzo assegni un valore costante dout.  
Questo viene sintetizzato in:
- logica combinatoria (reti di mux/porte),
- o in una ROM/lookup-table a seconda della tecnologia e della dimensione.

ROM = tabella di verità memorizzata. Per ROM piccole conviene logica; per ROM grandi conviene macro.

## TESTING THE DESIGN

### Testbenches – Visual Verification: testbench “manuale”

Un **testbench** è un modulo HDL che:

- istanzia il DUT (device under test),
- genera stimoli (input),
- osserva le uscite.

Qui la slide mostra un testbench che applica combinazioni (pattern) una dopo l'altra con ritardi temporali (#10, wait for 10 ns, ecc.).

L'idea è: guardi le waveform e verifichi che l'uscita sia corretta.

È utile per esempi piccoli, ma non scala bene: diventa noioso e soggetto a errori umani.

### Testbenches – Automatic Verification: self-check con assert

Qui il salto di qualità è che il testbench:

- conosce l'output atteso,
- e usa assert/controlli per dire “pass/fail”.

In SV puoi fare `assert(...) else $error("...")`.

In VHDL puoi fare `assert condition report "..."` severity error;

L'obiettivo è avere testbench che “si giudicano da soli”, così se rompi qualcosa nel design te ne accorgi immediatamente.

### Testbenches – Reading Test Vectors from File (1): perché leggere da file

Quando i vettori diventano tanti, scriverli a mano nel testbench è improponibile. Allora li metti in un file (es. `example.tv`) con righe tipo:

- input + expected output.

Il flusso:

1. all'inizio della simulazione, il TB apre il file;
2. legge una riga alla volta;
3. applica input al DUT;
4. aspetta un po' (o sincronizza su clock);
5. confronta output reale con output atteso;
6. conta errori e alla fine stampa un report.

### Testbenches – Reading Test Vectors from File (2): struttura pratica del TB

Qui si vede tipicamente:

- un array in cui memorizzi i vettori letti,
- un ciclo che scorre i vettori,
- sincronizzazione col clock (applico su fronte di salita, controllo su fronte di discesa oppure dopo un delay fissato).

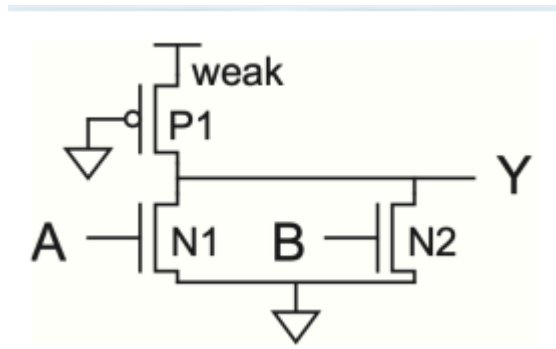
Il TB diventa “data-driven”: non cambi il codice quando cambi i test, cambi solo il file.

## NETLISTS / PRIMITIVES (SV)

### SystemVerilog Netlists – NOR Gate (pseudo-nMOS) e primitive a transistor

Qui il corso entra nel livello “molto basso”: SV/Verilog permette di descrivere circuiti usando primitive tipo:

- porte logiche (and, or, nor, ...),
- oppure addirittura primitive “transistor-like” (tranif1, tranif0, rtranif1, ...).



Il caso mostrato è un **pseudo-nMOS NOR**:

- pull-up “debole” verso VDD,
- rete di nMOS verso GND comandata dagli ingressi.  
Se un ingresso è 1, conduce verso massa e porta l’uscita bassa. Se entrambi sono 0, la pull-up la tira alta.

Perché compare tri sull’uscita y?

- Perché più dispositivi possono “guidare” lo stesso nodo in modo non esclusivo: il nodo è un **net risolto**, non un semplice logic a singolo driver.  
Quindi qui il linguaggio sta davvero modellando un nodo elettrico con possibili contese/forzature.

### SystemVerilog Netlists – Latch: feedback + rischio di contesa e uso di “trireg”

Un latch a livello transistor può essere delicato:

- Un latch ha **feedback**: l’uscita torna all’ingresso interno per mantenere lo stato.
- Se contemporaneamente hai anche un “feedforward path” (il dato che entra), puoi creare condizioni in cui un nodo interno può:
  - o “flottare” (nessuno lo guida),
  - o oppure andare in contention (due driver opposti).

Per evitare che un nodo “flotti”, viene usato un tipo di net che mantiene l’ultimo valore: trireg.

- trireg è come dire: se nessuno guida, il nodo conserva la carica (modello semplificato).

Un concetto importante è:

- molte primitive a transistor sono soprattutto per simulazione, perché la sintesi vera spesso non lavora “a livello transistor” con questi costrutti.

- Inoltre, alcune primitive (tipo tranif) sono bidirezionali in simulazione (source/drain simmetrici), mentre nella fisica reale spesso il comportamento effettivo è più complesso. Questo può introdurre mismatch se usate male.

Queste primitive sono utili per capire fenomeni di bus tri-state, contese, nodi flottanti, e per descrivere netlist o modelli “quasi elettrici”, ma nella progettazione RTL moderna si preferisce descrivere comportamento e lasciare alla tecnologia l’implementazione fisica.

## FPGA nel Dettaglio

### Logiche Programmabili

In questo ambito, con logiche programmabili, intendiamo non macchine di Turing, bensì logiche la cui configurazione circuitale è programmabile, infatti, queste logiche non possono eseguire dei programmi, ma la loro struttura circuitale è completamente programmabile.

I microprocessori sono quelli che consumano di più, mentre a parità di consumo i circuiti dedicati sono i più performanti, però hanno un costo importante. Le FPGA hanno i vantaggi di essere poco costosi e avere un rapporto consumo/performance molto buono.

Costi presenti:

- Fixed Costs: Sono i costi che **non** dipendono dal numero di chip prodotti. Ad esempio, i costi di training, hardware and software tools, design costs, general costs, circuit test design, profit model
- Variable Costs: Materie prime (Silicon wafers, materials and disposables,), production costs, packaging, testing.

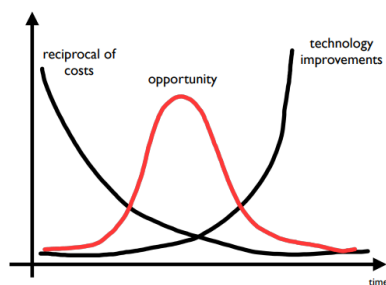
Se  $N$  è il numero di dispositivi prodotti:

$$\text{Costo Totale} = FC + VC \cdot N$$

Se produci pochi pezzi, domina il **FC**.

Se produci tantissimi pezzi, domina il **VC**.

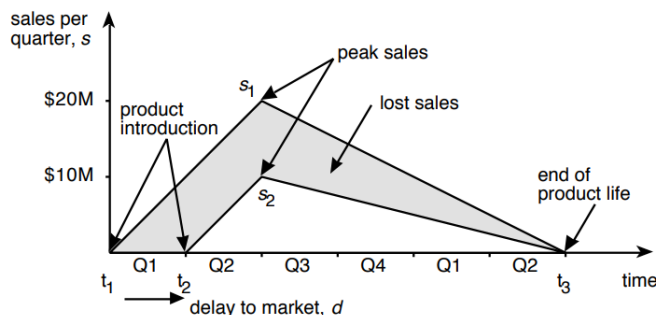
### The Custom / Semi-custom Circuits Paradox



L'immagine mostra come nel tempo diminuiscano i costi di progettazione dell'hardware mentre aumentano le capacità tecnologiche, creando una finestra centrale di massima convenienza per i circuiti custom o semi-custom.

Questa "finestra di opportunità" indica il momento in cui realizzare hardware dedicato è economicamente più vantaggioso rispetto a soluzioni programmabili.

## Profit Model



Questa figura mostra come il ritardo nel portare un prodotto sul mercato (time-to-market) riduca il profitto totale.

La curva  $s_1$  rappresenta le vendite ideali se il prodotto esce subito; la curva  $s_2$  mostra cosa succede se l'uscita è ritardata di un tempo  $d$ : il picco delle vendite avviene nello stesso istante temporale ma la curva  $s_2$ , essendo partita in ritardo, ha raggiunto un picco minore e una parte delle vendite viene persa per sempre (area grigia "lost sales"), perché il mercato ha una finestra temporale limitata prima della fine del ciclo di vita del prodotto.

Con l'evoluzione della tecnologia dei circuiti integrati, le prestazioni sono cresciute in modo esponenziale, ma purtroppo anche i costi, in particolare i costi fissi di progetto e produzione (maschere, progettazione, verifica, test).

Questo ha fatto sì che la finestra temporale ed economica in cui è conveniente realizzare circuiti integrati completamente custom si sia progressivamente ristretta. Inoltre, la crescente complessità dei circuiti ha ulteriormente aumentato questi costi iniziali.

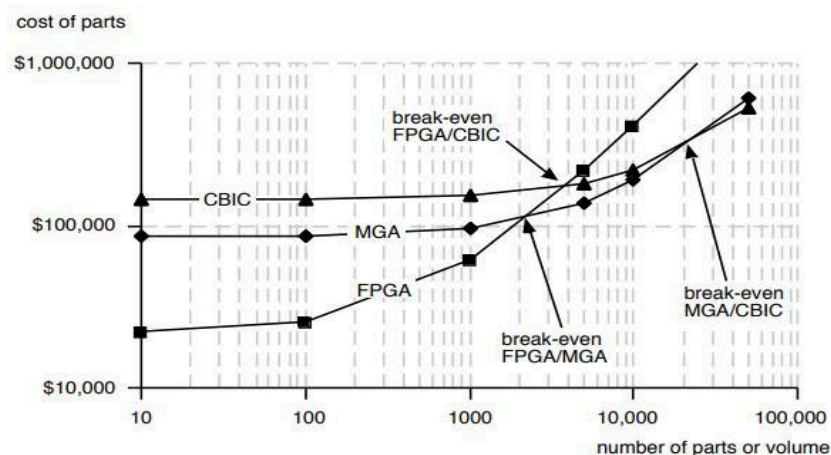
Di conseguenza, l'andamento tipico di un prodotto è il seguente: inizialmente il costo è basso ma la tecnologia non è ancora sufficiente; con il miglioramento tecnologico il prodotto entra in una fase di forte crescita e raggiunge un picco di vendite; successivamente, però, l'aumento dei costi di produzione e di sviluppo fa diminuire la convenienza economica e quindi anche le vendite.

Per questo motivo la strategia più efficace è diventata quella di ridurre i costi fissi, anche a scapito di un leggero aumento dei costi per singolo pezzo.

In questo contesto si sono sviluppate delle tecnologie alternative ai circuiti completamente custom:

- i circuiti semi-custom (MGA – Masked Gate Array e CBIC – Cell-Based Integrated Circuit),
- i circuiti programmabili (PLD e FPGA).

È importante sottolineare che, in questo ambito, programmabilità non significa esecuzione di software, ma la possibilità di trasformare un supporto hardware generico in uno specifico circuito elettronico, configurandone fisicamente la struttura logica. In altre parole, un FPGA non esegue un programma come una CPU, ma viene riconfigurato per diventare un circuito dedicato.



Le tre curve mostrano l'andamento del costo complessivo al variare del volume:

- **FPGA**  
Parte con un costo molto basso per pochi pezzi, perché non ha costi di maschere né di fabbricazione dedicata: compri il chip già fatto e lo programmi. Però ogni singolo chip FPGA è molto costoso → il costo cresce rapidamente con il volume.
- **MGA (Gate Array)**  
Ha costi fissi medi (serve una maschera parziale) e costi unitari più bassi degli FPGA.  
È una soluzione intermedia.
- **CBIC (ASIC full custom o standard-cell)**  
Ha costi fissi enormi (maschere, layout, verifiche, test...), ma costo per pezzo molto basso.  
Convieni solo se produci tantissimi pezzi.

## I circuiti Integrati

I **circuiti integrati** (IC, *Integrated Circuits*) sono dispositivi elettronici che integrano un grande numero di componenti — come **transistor, diodi, resistori e condensatori** — all'interno di un unico chip di materiale semiconduttore, tipicamente **silicio**.

Essi costituiscono il cuore dei sistemi elettronici moderni, perché permettono di realizzare circuiti **compatti, veloci, affidabili ed efficienti** dal punto di vista energetico e dei costi.

Dal punto di vista della progettazione e dell'utilizzo, i circuiti integrati si dividono in due grandi categorie:

- **circuiti non programmabili**, la cui funzione è fissata in fase di fabbricazione;
- **circuiti programmabili**, la cui struttura interna può essere configurata dopo la produzione per svolgere una funzione specifica.

### **Circuiti non Programmabili**

I circuiti non programmabili sono dispositivi la cui funzionalità è definita durante la fase di progettazione e produzione. Una volta fabbricati, il loro comportamento non può essere modificato. Questa categoria comprende:

- **Circuiti Custom: Conosciuti** anche come **ASIC** (Application-Specific Integrated Circuits), sono progettati su misura per una specifica applicazione o funzionalità. Questi circuiti, tuttavia, richiedono alti costi di progettazione e realizzazione;
- **Circuiti Semicustom:** Offrono un compromesso tra circuiti standard e custom, consentendo una certa personalizzazione senza i costi elevati degli ASIC. Fra di essi troviamo ad esempio gli **MGA (Masked Gate Arrays)**, suddivisi in **Channeled Gate Arrays**, ovvero MGA con canali predefiniti per le interconnessioni, e **Channelless Gate Arrays**, dove i canali sono eliminati;

### **Gate Array**

I **Gate Array** sono circuiti integrati semi-custom in cui la struttura di base dei transistor è già predisposta, ma le interconnessioni vengono personalizzate durante la fase di progettazione. Esistono diverse varianti di **Gate Array**, ciascuna con differenze strutturali e funzionali:

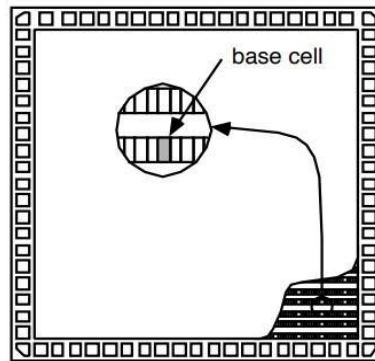
#### **Channelled Gate Array:**

Struttura a canali predefiniti per le interconnessioni tra i blocchi di logica (transistor e porte logiche). Questi canali sono spazi vuoti nel layout che vengono utilizzati per realizzare le connessioni personalizzate durante il processo di fabbricazione.

In questi chip in concetto di standard-cell è esteso a tutto il circuito:

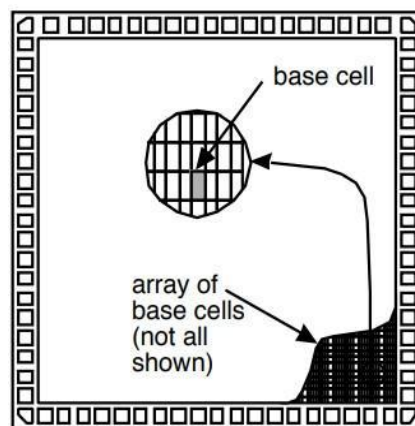
possiamo customizzare le interconnessioni

sfruttando lo spazio predefinito tra le righe di base cells. Una differenza è che l'altezza è predefinita mentre in un CBIC è definito dal progettista



### Channelless Gate Array (o Sea of Gates):

Non ci sono canali predefiniti. L'intera superficie del chip è coperta da un'alta densità di transistor (definita come un "mare" di transistor o "sea of gates"). Le interconnessioni vengono inserite **sopra** i transistor usando strati metallici.



### Structured Gate Array

È un compromesso tra i **Gate Array** tradizionali e i circuiti completamente custom. Comprende blocchi di funzionalità predefinite (come memorie, processori, o blocchi analogici), integrati insieme a blocchi programmabili di transistor.

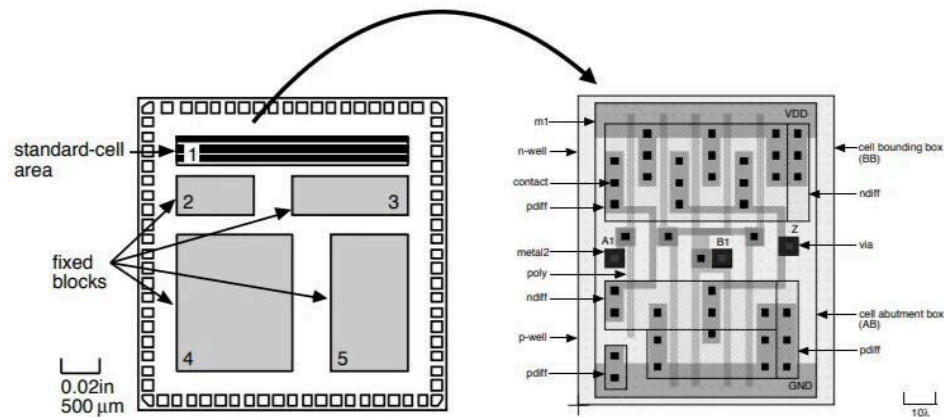
### Circuiti Semi-Custom Standard Cells o CBIC

Nei **Cell-Based IC (CBIC)** il chip non viene progettato transistor per transistor come nei circuiti custom, ma usando una **libreria di celle standard** pre-progettate e ottimizzate. Le **standard cells** sono blocchi logici elementari che implementano funzioni tipiche, come:

- porte logiche (AND, OR, NOT, NAND, NOR...)
- latch e flip-flop
- multiplexer, adders, ecc.

Queste celle sono già caratterizzate dal costruttore (ritardi, consumo, area) e vengono poi posizionate e collegate automaticamente per realizzare la funzione desiderata

Essi sono pre-progettati e ottimizzati che vengono utilizzati nella progettazione di circuiti integrati, come nei circuiti integrati su misura (Custom ICs) e nei circuiti basati su celle (Cell-Based IC o **CBIC**).



## Proprietà

Il costo fisso delle standard-cells si abbatte molto rispetto ad altre tipologie di circuiti integrati come per esempio i circuiti custom, e allo stesso tempo si può mantenere la variabilità e la personalizzazione del circuito secondo il nostro progetto prestabilito. Realizzando le interconnessioni tra i vari blocchi fissi riusciamo, infatti a personalizzare il circuito e di conseguenza a realizzare ciò che ci serve per un progetto specifico.

## FIXED BLOCK

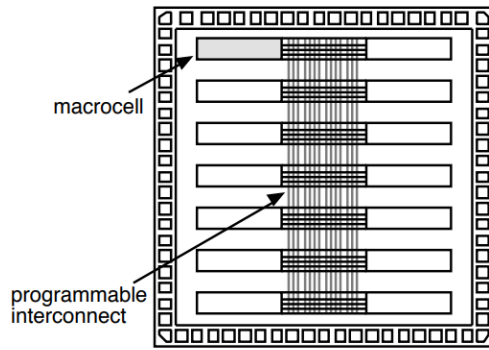
I **Fixed Block** (o blocchi fissi) sono parti di un circuito integrato che rappresentano funzionalità predefinite e complesse, come memoria (RAM, ROM), processori, o blocchi analogici. A differenza delle **standard cells**, i **Fixed Block** hanno dimensioni e posizioni predeterminate all'interno del layout del chip e non possono essere ridimensionati o riposizionati liberamente.

Otteniamo quindi un mix tra i componenti che sono di utilizzo comune per realizzare un circuito integrato e celle standard da interconnettere come vogliamo per realizzare una logica specifica che può servirci in un determinato progetto.

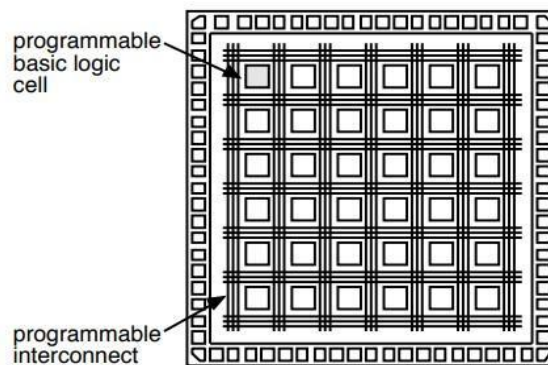
## Circuiti Programmabili

I circuiti programmabili sono dispositivi la cui funzionalità può essere definita o modificata dall'utente dopo la produzione. Questa flessibilità li rende ideali per una vasta gamma di applicazioni. Essi comprendono:

**PLD (Programmable Logic Devices):** Dispositivi che possono essere programmati per eseguire funzioni logiche semplici.



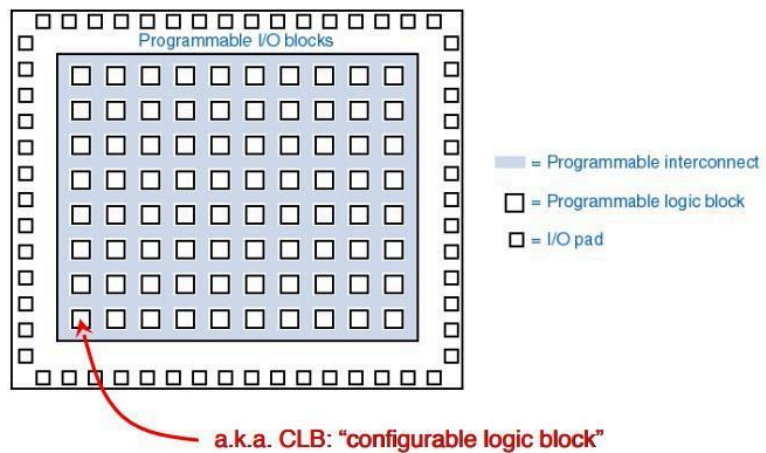
**FPGA (Field-Programmable Gate Arrays):** Circuiti integrati programmabili avanzati che possono implementare funzioni logiche complesse. Essi sono composti da blocchi logici configurabili, interconnessi tra loro con connessioni programmabili



Per **connessione programmabile**, si intende, sia nei PLD, che negli FPGA, delle **reti configurabili di interruttori elettronici** che stabiliscono percorsi logici tra i blocchi del dispositivo. Il segnale digitale  $\square 1 \text{ o } 0 \square$  è rappresentato da livelli di tensione, e queste connessioni possono essere programmate e riprogrammate per modificare il comportamento del circuito.

### Architettura Sistema

L'Architettura del sistema che viene utilizzata è sempre la stessa che abbiamo visto fin ora. **Alla periferia del chip ci sono dei BLOCCHI di I/O (input/output)** che acquisiscono un segnale analogico che influenza il segnale finale. Questi blocchi possono fare sia da ingresso che da uscita. La parte centrale del chip è invece realizzata tramite un array regolare di logiche programmabili che quando vengono interconnesse generano un certo andamento funzionale complessivo.



**NOTA:** i segnali logici sono una semplice astrazione, gli unici segnali che realmente esistono sono le forme d'onda analogiche che poi opportunamente interpretate "diventano" segnali logici digitali.

Gli **elementi costitutivi** sono come abbiamo già detto gli I/O BLOCKS e i BLOCCHI LOGICI che sono interconnettibili attraverso degli switch di interconnessione. Creando così una matrice di blocchi programmabili interconnessi, su cui posso fare ciò che voglio, programmando ciascun blocco per fare la funzione che ti serve e dato che sono sulla RAM puoi riprogrammare i blocchi a piacere.

## Programmabilità Fisica

### Evoluzione delle memorie a stato solido

Le **ROM (read-only-memory)** sono memorie a sola lettura, e fanno da contraltare alle **(RAM random-access memory)**, che sono nate come evoluzione della memoria magnetica: un tempo le memorie erano seriali, poi con le memorie a semiconduttore l'**accesso è diventato random** ovvero, **possiamo andare a selezionare il bit che ci interessa senza dover per forza scorrere sequenzialmente tutta la catena. Le ROM sono più o meno la stessa cosa però non sono riprogrammabili né scrivibili come le RAM.**

In origine, infatti, le interconnessioni non erano riprogrammabili, perché create con processi irreversibili (es. bruciare un fusibile) (**PROM-programmable read only memory**) significa che una volta scritto il dato con questo processo "distruttivo" non si poteva più tornare indietro. In seguito, venne considerata un'alternativa reversibile e non volatile, realizzata con un mosfet a doppio gate (**EPROM erasable programmable read only memory**), in questo caso abbiamo delle memorie cancellabili (attraverso una esposizione a luce ultravioletta). Se nel gate intermedio non c'è carica, il mosfet funziona normalmente; viceversa se il gate intermedio ha carica sufficiente per

bilanciare quella del gate superiore, non si crea il canale d' inversione e perciò il mosfet non funziona mai.

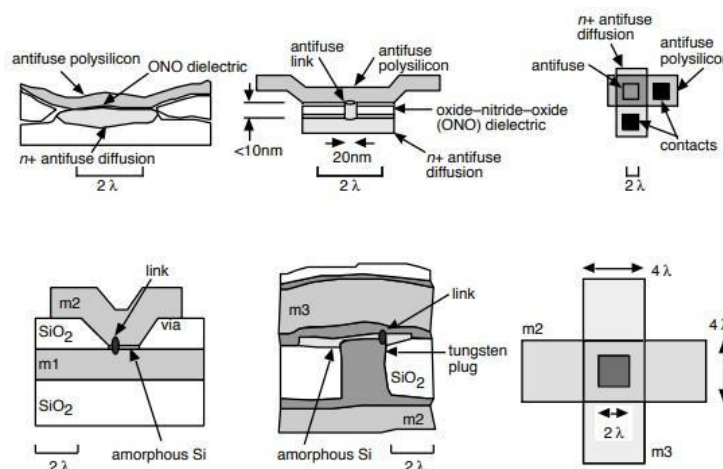
NOTA □ ci sono anche le EEPROM (**electric erasable programmable read only memory**): sono come le EPROM ma in questo caso possono essere cancellate elettricamente senza dover far uso dei raggi UV.

Un PLD contiene componenti sia di logica che di memoria (contenenti le informazioni di configurazione); quest'ultime possono essere di tipo:

- Antifusibili al silicio
- SRAM
- Flash
- Celle EPROM

### Antifusibili al Silicio (Normalmente OFF)

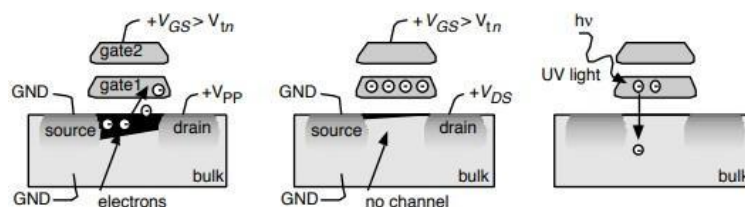
Originariamente la programmazione dei dispositivi una volta realizzata non era più riconfigurabile (ovvero non era un evento reversibile). La procedura più utilizzata era l'Antifuse, dove era presente un plug resistivo (di norma un fusibile che veniva bruciato) che poteva essere fuso attraverso l'applicazione di una data tensione, che in tal modo, metteva definitivamente in contatto due piste, realizzando così, una connessione permanente. Con questo processo non si può più tornare indietro perché è appunto un processo distruttivo. I circuiti integrati che utilizzano la tecnologia "antifuse", quindi impiegano una barriera sottile di materiale dielettrico di silicio amorfo (circa 9 nanometri di nitruro di silicio) tra due conduttori metallici. Quando viene applicata una tensione sufficientemente alta (mediante un breve impulso di circa 1 millisecondo dell'ampiezza di circa 16 volt), tutto il silicio amorfo si trasforma in una lega policristallina silicio-metallo con una bassa resistenza, che è conduttiva .



## Non Volatile Control EPROM ed EEPROM

Si tratta in questo caso di una **programmazione non distruttiva ma riprogrammabile e recuperabile**. Fa uso di un transistor MOS che possiamo paragonare come ben sappiamo a un "rubinetto". Applicando una tensione di soglia sul gate si crea un canale di elettroni che si muovono dal drain al source o viceversa. In questo caso prendiamo il gate e lo isoliamo con un ossido (gate isolato) il MOS diventa un condensatore con un'armatura isolata completamente.

**Sopra il gate viene posto il gate 2 (che è flottante), applicando una tensione che è maggiore a quella di soglia sul gate 2 (di controllo) "la carica si sposta dal canale di elettroni al gate 1 (non c'è più canale di elettroni), è quindi in questo modo che si crea/elimina la connessione programmando di conseguenza il circuito.** La programmazione in questo caso è reversibile perché il processo non è distruttivo, per ripristinare le condizioni iniziali basterà esporre il circuito a luce UV.

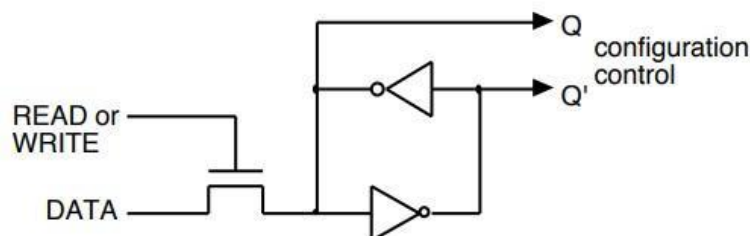


## Static RAM Control

La programmazione la faccio attraverso la RAM. Utilizzando questa tecnologia nel momento in cui si spegne il circuito (non si fa passare più corrente) la memoria "scompare" (si tratta di un flip-flop) ovvero una cella di memoria RAM statica.

**Read/Write:** comanda un pass transistor che abilita la lettura, i due inverter retroazionati costituiscono il latch di memoria.

I due inverter retroazionati fungono da **Flip-Flop che consiste in una memoria statica**.



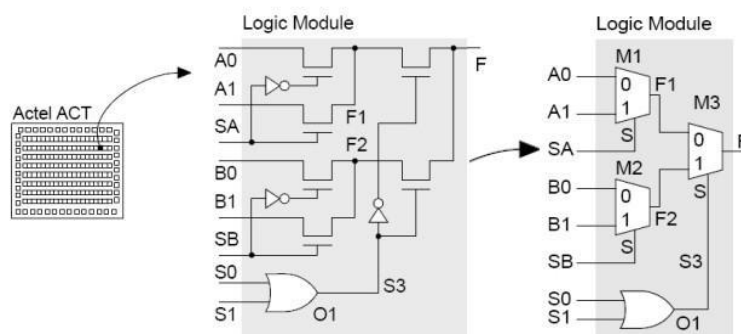
## Programmabilità Logica

La **programmabilità logica** si riferisce alla capacità di configurare dinamicamente il comportamento di un circuito logico, modificando le connessioni tra i blocchi logici interni per implementare diverse funzioni. Questa funzionalità rende possibile la

realizzazione di dispositivi versatili e adattabili, utili in applicazioni che richiedono variazioni rapide senza la necessità di riprogettare il circuito fisico.

### Esempio Actel ACT 1

L'**Actel ACT 1** rappresenta un caso prototipico di dispositivo a logica programmabile, una delle prime implementazioni che ha sfruttato blocchi logici configurabili e un'architettura a matrice regolare. Questa struttura ha introdotto una versatilità notevole, rendendo possibile la configurazione dinamica delle connessioni per creare una gamma di funzioni logiche senza bisogno di ridefinire il circuito fisico. I multiplexer, utilizzati come blocchi di configurazione fondamentali, offrono la possibilità di selezionare diverse combinazioni di input e simulare comportamenti di memoria ROM, rendendo il sistema adattabile a varie applicazioni logiche.



### The Motivation of The Cell Structure

La progettazione delle celle logiche programmabili mira a realizzare funzioni complesse attraverso una struttura modulare. Questa struttura permette di suddividere funzioni logiche avanzate in blocchi più semplici, rendendo le configurazioni più flessibili e risparmiando risorse. Ogni cella è costruita per essere riprogrammabile e utilizzabile in diverse combinazioni, garantendo al dispositivo la capacità di adattarsi a nuove configurazioni in modo efficiente.

### Mappatura di una Singola Cella

Consideriamo, ad esempio, la funzione logica:

$$F = AB + B'C + D$$

Questa funzione può essere riformulata utilizzando la logica booleana per semplificarne la configurazione, come segue:

$$F = AB + B'C + D = AB + B'C + (B+B')D = B(A+D) + B'(C+D) = B \cdot F2 + B'F1;$$

Dove:

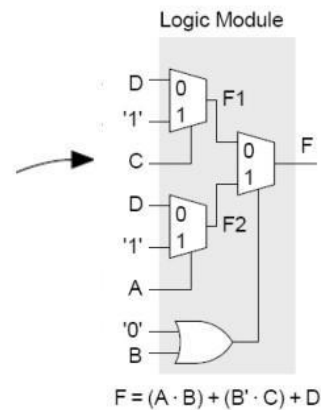
- $F_1 = C + D = C \cdot 1 + C' \cdot D$
- $F_2 = A + D = A \cdot 1 + A' \cdot D$

Questa formulazione consente di implementare F suddividendo la funzione in due sotto funzioni, F1 e F2 che possono essere configurate su blocchi logici più semplici e poi interconnesse.

$$F = AB + B'C + D = B'F_1 + BF_2$$

$$F_1 = C + D = C \cdot 1 + C' \cdot D$$

$$F_2 = A + D = A \cdot 1 + A' \cdot D$$



### Quante funzioni di due variabili binarie esistono?

La versatilità della programmabilità logica può essere compresa esaminando il numero di funzioni che è possibile creare con due variabili binarie. Con due ingressi, ci sono  $2^2 = 4$  configurazioni di input possibili, il che implica la possibilità di creare  $2^4 = 16$  funzioni logiche diverse. Tra queste troviamo:

- Una funzione sempre zero
- Quattro funzioni con un solo "1" e tre "0"
- Quattro funzioni con un solo "0" e tre "1"
- Sei funzioni con due "1" e due "0"
- Una funzione sempre uno

Questa varietà di combinazioni consente ai dispositivi programmabili di adattarsi a numerose esigenze circuitali, rappresentando una soluzione flessibile per le applicazioni logiche. Questa gamma completa di possibilità rende le logiche programmabili strumenti particolarmente potenti per implementare qualsiasi operazione logica di base.

For  $n = 2$ , the functions are 16:

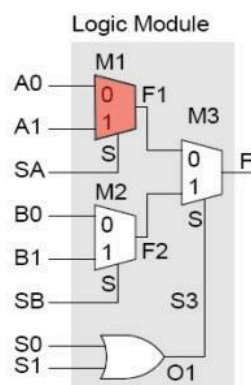
|                                |
|--------------------------------|
| 1 x tutti "zero"               |
| 4 x un solo "uno" e tre "zero" |
| 4 x un solo "zero" e tre "uno" |
| 6 x due "uno" e due "zero"     |
| 1 x tutti "uno"                |

|                |                        |
|----------------|------------------------|
| $F_0 = "0"$    | $F_8 = A' + B'$        |
| $F_1 = A' B'$  | $F_9 = A'$             |
| $F_2 = A' B$   | $F_{10} = A$           |
| $F_3 = A B'$   | $F_{11} = B'$          |
| $F_4 = A B$    | $F_{12} = B$           |
| $F_5 = A + B$  | $F_{13} = A' B' + A B$ |
| $F_6 = A + B'$ | $F_{14} = A B' + A' B$ |
| $F_7 = A' + B$ | $F_{15} = "1"$         |

## Funzioni svolte dal MUX

Un singolo multiplexer (MUX) può essere utilizzato per realizzare molte delle funzioni logiche fondamentali. Grazie alla sua capacità di selezionare tra vari ingressi, un MUX può essere configurato come una sorta di generatore di funzioni logiche. In una configurazione programmabile, il MUX seleziona le combinazioni di input desiderate per ottenere l'uscita logica richiesta.

Ad esempio, collegando opportunamente i dati in ingresso di un MUX, è possibile rappresentare tutte le combinazioni di funzione logica basate su due variabili. La configurazione del MUX rappresenta una forma semplificata di programmabilità logica, che offre un modo versatile per creare numerose operazioni a partire da un singolo componente logico.

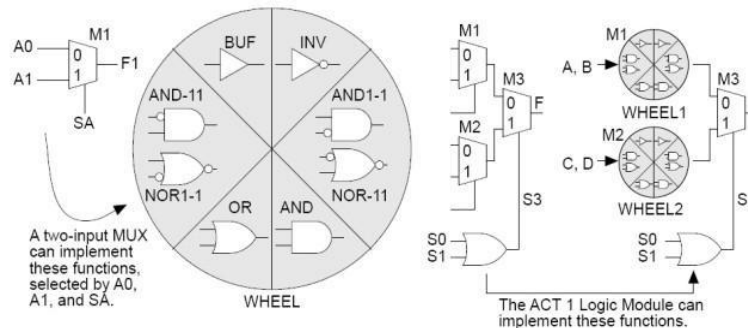


| A0 | A1 | SA | F     | F <sub>n</sub>  |
|----|----|----|-------|-----------------|
| 0  | —  | 0  | "0"   | F <sub>0</sub>  |
| —  | 1  | 1  | "1"   | F <sub>15</sub> |
| 0  | A  | B  | A B   | F <sub>4</sub>  |
| A  | 0  | B  | A B'  | F <sub>3</sub>  |
| B  | 0  | A  | A' B  | F <sub>2</sub>  |
| —  | A  | 1  | A     | F <sub>10</sub> |
| —  | B  | 1  | B     | F <sub>12</sub> |
| 1  | 0  | B  | B'    | F <sub>11</sub> |
| 1  | 0  | A  | A'    | F <sub>9</sub>  |
| A  | 1  | B  | A + B | F <sub>5</sub>  |

## Wheel of Fortune

Nella programmabilità logica, il concetto di "Wheel of Fortune" rappresenta un modo per visualizzare le diverse funzioni ottenibili con i blocchi logici programmabili. Come una ruota che può ruotare per selezionare una diversa funzione a seconda della posizione, i dispositivi programmabili consentono di passare da una configurazione logica all'altra in modo dinamico.

Questa capacità è fondamentale per applicazioni che richiedono aggiornamenti o cambiamenti frequenti delle funzioni logiche implementate. La "ruota" delle funzioni rappresenta dunque la flessibilità intrinseca dei circuiti programmabili, che possono essere configurati per implementare funzioni diverse in modo semplice e rapido.



## Blocchi Logici Configurabili

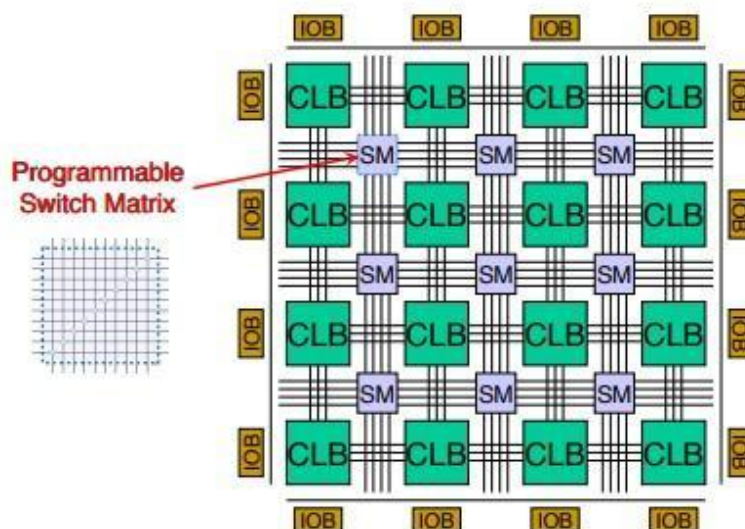
Il **Configurable Logic Block (CLB)** rappresenta il componente fondamentale nelle FPGA (Field-Programmable Gate Array). Ogni CLB contiene elementi logici programmabili e risorse di memoria, che possono essere configurati per implementare funzioni combinatorie o sequenziali. I CLB sono organizzati in una matrice regolare e interconnessi da una rete di commutazione programmabile, che permette la configurazione personalizzata del comportamento logico complessivo.

All'interno di ogni CLB sono tipicamente presenti:

**Blocchi logici combinatori**, spesso realizzati con Look-Up Table (LUT), per implementare funzioni logiche.

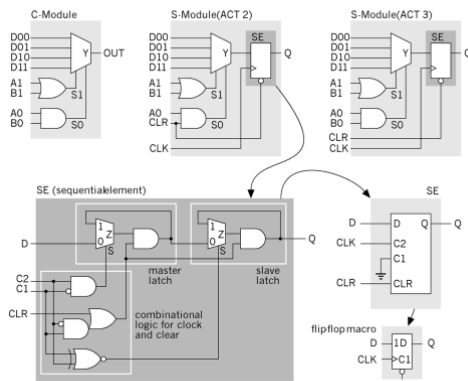
**Flip-flop** per la memorizzazione di valori logici e la gestione di operazioni sequenziali.

**Multiplexer** per selezionare gli input o combinare le uscite.



## ACTEL ACT 1

Le **celle logiche ACTEL ACT1** utilizzano un'architettura basata su antifusibili per interconnessioni permanenti. Questo approccio garantisce alte prestazioni e un basso consumo energetico, ma rende la configurazione irreversibile.



Ogni cella ACTEL ACT1 include:

- **Elementi combinatori** per eseguire funzioni logiche di base.
- **Flip-flop** per supportare operazioni sequenziali.
- Una struttura di interconnessione semplice e stabile, ottimizzata per applicazioni a bassa latenza.

## MODELLO DEI RITARDI DELLA LOGICA

modello di temporizzazione per le celle logiche ACTEL ACT1 è definito dai seguenti parametri:

- **Critical Path:**  $t_{PD} + t_{SUD} + t_{CO}$
- $t_{PD}$ : Tempo di propagazione, dipendente dalla funzione combinatoria implementata.
- $t_{SUD}$ : Tempo di setup.
- $t_{CO}$ : Tempo di clock-to-output, influenzato dal fan-out.
- $t_H$ : Tempo di hold del flip-flop.

La temporizzazione reale dipende dalla logica implementata e dalle interconnessioni del blocco.

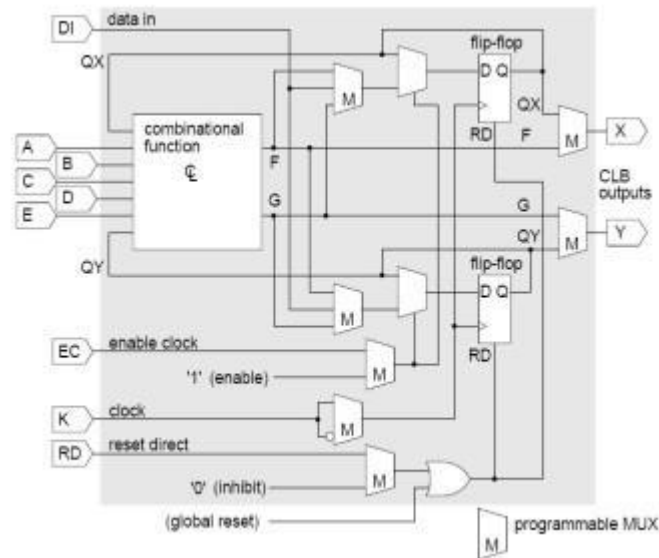
## Xilinx XC3000

Le celle logiche Xilinx XC3000 utilizzano un'architettura basata su Look-Up Table

□LUT□ per implementare funzioni logiche combinatorie. Ogni cella logica comprende:

- Una **LUT a 3 ingressi** per realizzare funzioni logiche di base.
- Un **flip-flop** per operazioni sequenziali.
- Un'unità di controllo configurabile per gestire il comportamento della cella.

Questo design consente una maggiore flessibilità rispetto agli antifusibili, rendendo le celle riprogrammabili.

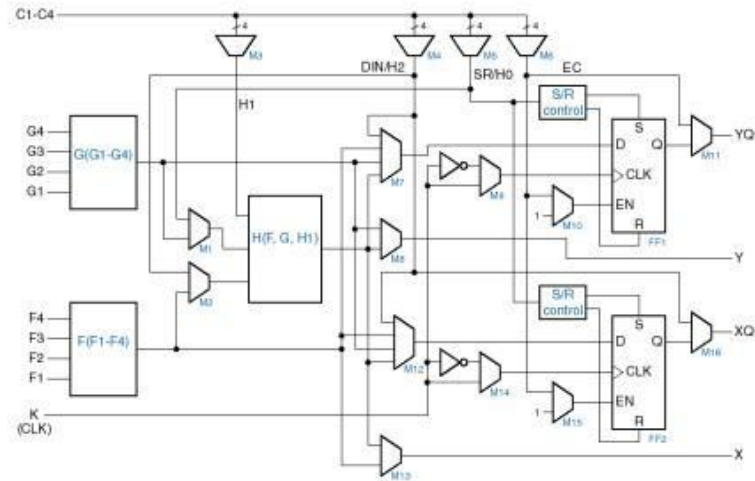


### Xilinx XC4000

Le celle logiche Xilinx XC4000 rappresentano un'evoluzione rispetto alle XC3000, offrendo una maggiore capacità e flessibilità. Le caratteristiche principali includono:

- Una **LUT a 4 ingressi**, che consente di implementare funzioni logiche più complesse.
- Una struttura di **interconnessione avanzata** per supportare applicazioni ad alta densità.
- **Flip-flop integrati** per il supporto delle operazioni sequenziali.

Questa architettura è particolarmente adatta per applicazioni che richiedono alte prestazioni e scalabilità.



## Look-up Table

Le **Look-Up Table** ☐ **LUT** ☐ sono il cuore dell'implementazione logica nelle FPGA. Una LUT è essenzialmente una piccola memoria che memorizza i valori di output per tutte le combinazioni possibili di input.

## DIFFERENZA CON L'IMPLEMENTAZIONE LOGICA

Le LUT permettono di implementare qualsiasi funzione logica combinatoria in modo efficiente. Rispetto ai circuiti tradizionali:

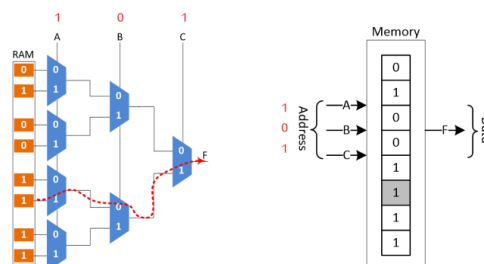
- Le LUT richiedono meno risorse hardware per funzioni logiche complesse.
- Sono altamente configurabili e ottimizzate per operazioni parallele.

## STRUTTURA

La struttura di una LUT prevede:

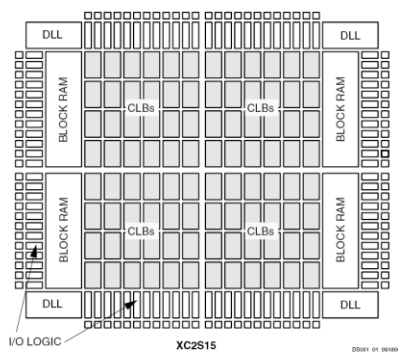
- **Ingressi logici** che selezionano l'indirizzo nella memoria.
- **Uscite logiche** che corrispondono ai valori memorizzati per l'indirizzo selezionato.
- Una configurazione programmabile per personalizzare le funzioni logiche.

## Look-up Tables



## Xilinx Spartan II Architecture

L'FPGA Xilinx Spartan-II è organizzato come una grande matrice di blocchi logici programmabili (CLB) al centro del chip, collegati da una rete di interconnessioni configurabili, e affiancati da blocchi specializzati che svolgono funzioni dedicate: ai lati sono presenti le Block RAM, che forniscono vere memorie hardware efficienti per dati e buffer, mentre lungo il perimetro si trovano i blocchi di I/O che gestiscono l'interfacciamento elettrico con l'esterno; agli angoli sono invece collocati i DLL (Delay Locked Loop), che servono a distribuire e sincronizzare correttamente il clock riducendo lo skew. Insieme, questi elementi permettono all'FPGA di implementare sistemi digitali completi, in cui la logica combinatoria e sequenziale viene realizzata nei CLB, la memoria nei blocchi RAM dedicati, il timing nei DLL e la comunicazione con l'esterno nei moduli di I/O, rendendo l'FPGA una piattaforma hardware completamente riconfigurabile.

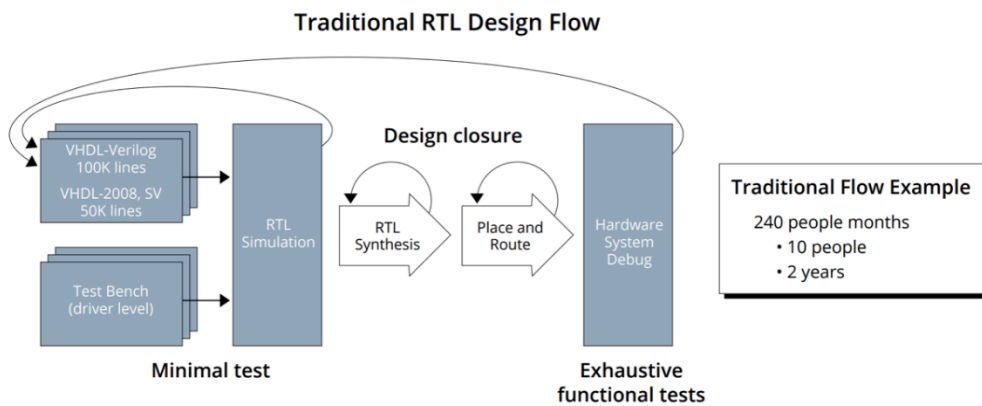


## Conventional RTL Synthesis

Nel flusso di **RTL synthesis tradizionale** il progetto hardware viene descritto direttamente in **VHDL o Verilog** a livello di registri e segnali (RTL). Il codice viene prima verificato tramite **simulazione RTL**, poi sintetizzato, mappato sull'hardware (place & route) e infine testato sul sistema reale.

Questo processo è fortemente **iterativo**: se dopo il place & route non si rispettano i vincoli di timing, area o potenza, si deve tornare indietro a modificare il codice RTL. Questo ciclo di **design closure** richiede personale altamente specializzato e tempi molto lunghi. Nei progetti industriali può arrivare facilmente a **decine o centinaia di**

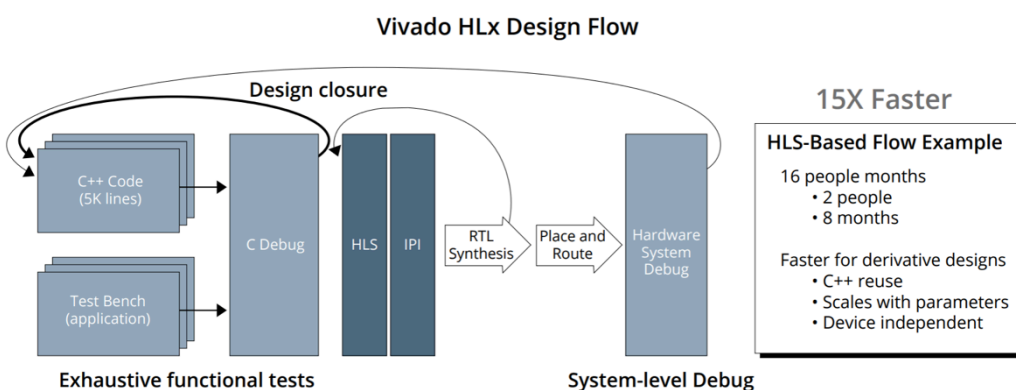
**persone-mese** (es. 20–200 people months).



Con la **High-Level Synthesis** l'approccio cambia: invece di descrivere direttamente i registri e la logica in VHDL/Verilog, il progettista scrive l'algoritmo in **C**, **C++** o **SystemC**, cioè in un linguaggio ad alto livello molto più compatto ed espressivo.

Il tool di HLS (ad esempio **Xilinx Vivado HLS**) traduce automaticamente questo codice in **RTL (VHDL/Verilog)**, generando un **blocco hardware IP** (Intellectual Property). Questi IP possono essere inseriti in una **libreria di IP**, cioè blocchi hardware già pronti, che si riutilizzano e si collegano come moduli: è l'analogo hardware di usare una funzione di libreria in C (come usare printf()) invece di scrivere da zero il codice per stampare caratteri).

Anche qui esiste una retroazione (debug e iterazione), ma avviene a livello di **C e di sistema**, molto più velocemente rispetto all'RTL. Per questo il flusso HLS può ridurre drasticamente i tempi di sviluppo, arrivando anche a essere **10–15 volte più veloce** rispetto al flusso RTL tradizionale.



**DIFFERENZA CON L'IMPLEMENTAZIONE LOGICA**  
**Differenza con l'implementazione logica: le LUT**

Negli FPGA la logica non è costruita con porte fisse (AND, OR, NOT) come nei circuiti ASIC, ma tramite **LUT (Look-Up Table)**.

Una LUT è in pratica una **piccola memoria** che implementa una funzione logica: per ogni combinazione degli ingressi, nella memoria è memorizzato il valore dell'uscita. Questo rende le LUT:

- estremamente **flessibili** (possono implementare qualsiasi funzione booleana),
- **efficienti** per funzioni complesse,
- naturalmente adatte al **parallelismo**.

Struttura di una LUT

Una LUT è composta da:

- ingressi logici, che funzionano come indirizzo della memoria,
- una memoria interna che contiene i valori della funzione,
- un'uscita che restituisce il valore memorizzato all'indirizzo selezionato.

Configurando i bit della memoria interna, la LUT può essere programmata per realizzare qualsiasi funzione logica combinatoria.

Questo è il meccanismo fondamentale che rende le FPGA riconfigurabili e così potenti.

### Configurable Logic Block (CLB)

Il Configurable Logic Block (CLB) è il blocco fondamentale di calcolo di una FPGA. È l'unità base con cui vengono implementate sia la logica combinatoria sia la logica sequenziale.

Ogni CLB è costituito principalmente da:

- LUT (Look-Up Table), che implementano le funzioni logiche combinatorie,
- flip-flop, che permettono di memorizzare stati e realizzare logica sequenziale,
- reti di routing locali, che collegano tra loro LUT, flip-flop e altri CLB.

Dal punto di vista funzionale, un CLB può quindi:

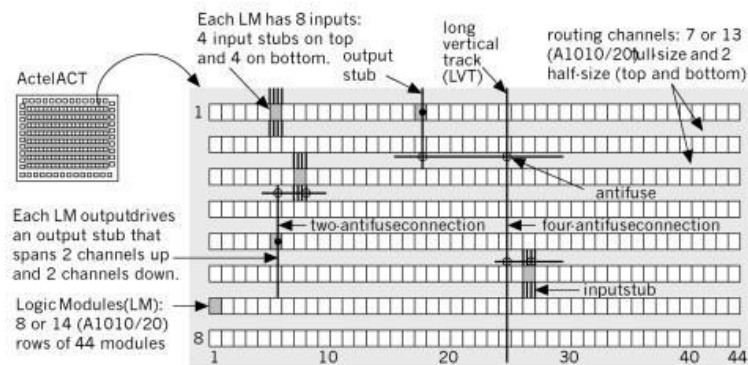
- realizzare qualsiasi funzione booleana tramite le LUT,
- memorizzare i risultati e sincronizzarli con il clock grazie ai flip-flop,
- essere collegato agli altri CLB per costruire circuiti complessi come FSM, datapath, pipeline e controllori.

Grazie alla configurabilità delle LUT e delle connessioni interne, ogni CLB può essere adattato esattamente alla logica richiesta dal progetto. In questo modo, l'FPGA non è un insieme di porte fisse, ma una rete di CLB programmabili che viene modellata dal tool di sintesi per implementare l'hardware desiderato.

### Interconnessioni

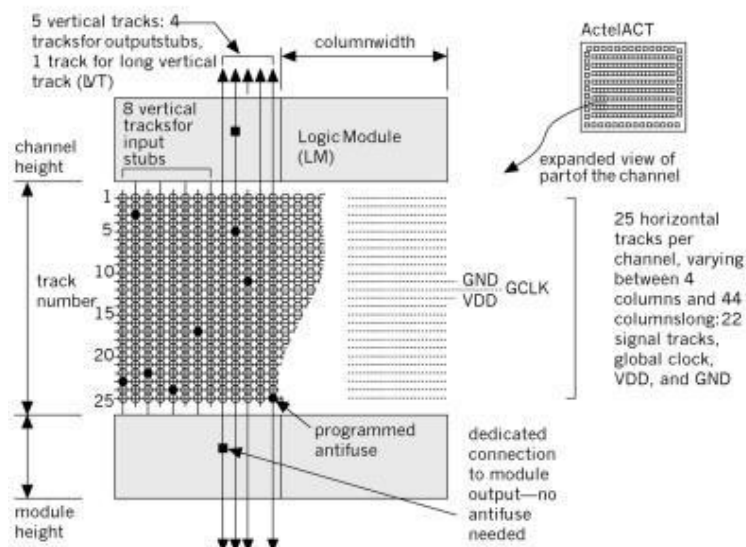
Le **interconnessioni** rappresentano una componente cruciale nelle FPGA, poiché determinano la capacità di collegare tra loro i blocchi logici configurabili (CLB) e di definire il comportamento complessivo del circuito. Tuttavia, richiedono un elevato utilizzo di risorse, in particolare moduli RAM, per gestirne la programmazione. Inoltre, costituiscono la principale causa di ritardo nel circuito, influenzando in modo significativo le prestazioni generali.

## ACTEL ACT 1 Architettura

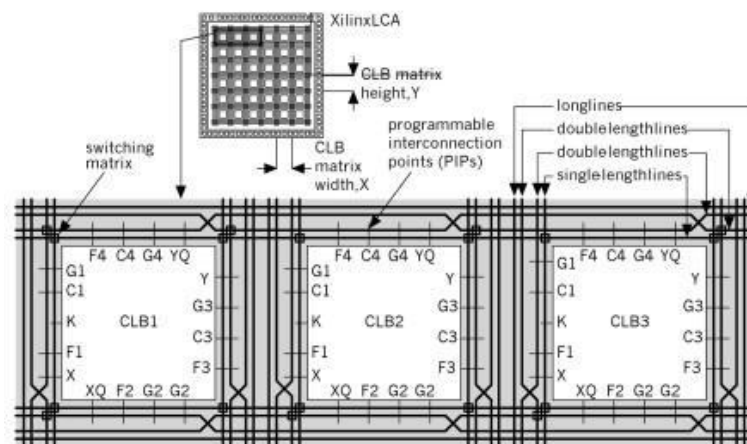


### Zooming the Channel

Il **channel** rappresenta l'area dedicata alle linee di interconnessione tra i CLB. Ogni channel è composto da più piste e punti di intersezione che permettono la programmazione delle connessioni. L'ottimizzazione di questa rete è essenziale per minimizzare i ritardi e migliorare le prestazioni. Sono gli svincoli che permettono ai segnali che scendono di poter svoltare e uscire a destra:

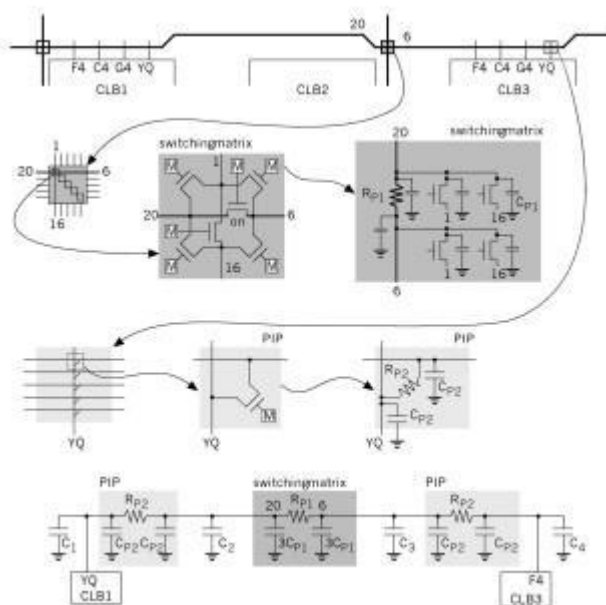


## Xilinx Architettura



### Interconnessione a Matrice

La matrice di interconnessione utilizza un **pass-transistor** tra ogni possibile coppia di linee. Attivando selettivamente i pass-transistor, è possibile stabilire quali collegamenti sono attivi. Questo modello può essere tradotto in un modello elettrico, dal quale si calcola facilmente il **ritardo dovuto all'interconnessione**.



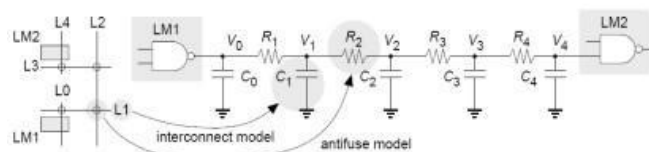
### Stima del Critical Path - Il ritardo di Elmore

La **Critical Path Estimation** è un'analisi fondamentale per verificare che i ritardi di propagazione siano compatibili con i requisiti temporali dell'applicazione. Ogni applicazione ha requisiti caratteristici, e il percorso critico dipende sia dalla funzione implementata sia dalla sua disposizione fisica (layout). Pertanto, è necessaria una verifica **post-layout (post place & route)** per confermare la compatibilità temporale.

Il **ritardo di Elmore** è un modello utilizzato per stimare i tempi di propagazione nelle interconnessioni. Questo modello è basato sull'analisi della resistenza e della capacità delle linee di connessione, fornendo un'approssimazione del ritardo introdotto dalle interconnessioni all'interno del circuito.

### ESEMPIO NELL'ACTEL

Nel caso delle FPGA **Actel**, il ritardo di propagazione è fortemente influenzato dalla semplicità dell'architettura basata su antifusibili. La struttura delle interconnessioni permanenti riduce il carico parassita e, di conseguenza, i ritardi complessivi. Questo rende le Actel particolarmente adatte per applicazioni a bassa latenza.



$$V_4 = - \sum_k R_{k4} C_k (dV_4/dt)$$

$$\tau_{D4} = \sum_k R_{k4} C_k = R_{14}C_1 + R_{24}C_2 + R_{34}C_3 + R_{44}C_4 =$$

$$= R_1C_1 + (R_1+R_2)C_2 + (R_1+R_2+R_3)C_3 +$$

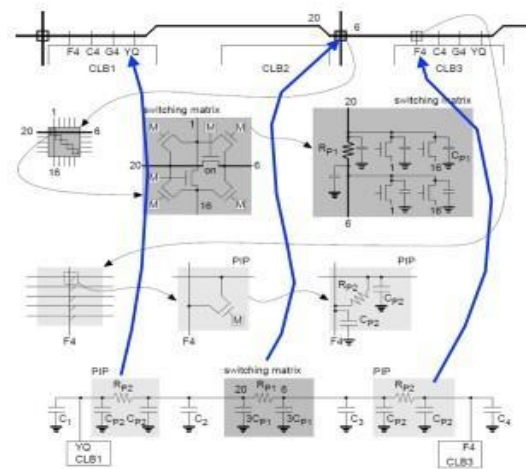
$$+ (R_1+R_2+R_3+R_4)C_4$$

if  $R_i = R_j = R$  and  $C_i = C_j = C$

$$\tau_{D4} = 10 RC$$

### ESEMPIO NELL'XILINX

L'architettura **Xilinx** si basa su una **LUT a 5 ingressi**, riconfigurabile come due LUT a 4 ingressi (purché non si utilizzino più di 5 segnali distinti). Questa configurazione consente una migliore ottimizzazione delle risorse quando la funzione combinatoria ha una complessità ridotta. Tuttavia, la maggiore flessibilità dell'architettura LUT comporta un aumento del ritardo di propagazione rispetto alle interconnessioni basate su antifusibili.



## **Implementazione**

Per comprendere il funzionamento delle FPGA, si può immaginare un esempio pratico in cui una funzione logica è suddivisa tra diversi CLB. Gli ingressi vengono instradati attraverso la rete di interconnessione, i blocchi eseguono le funzioni logiche assegnate e i risultati vengono combinati per produrre l'output desiderato.

### **Vantaggi delle FPGA:**

- **Prestazioni superiori** rispetto ai processori tradizionali in applicazioni che richiedono operazioni logiche intensive.
- **Consumi energetici ottimizzati** per unità di area rispetto ad altre tecnologie.

### **Svantaggi delle FPGA:**

- **Complessità di programmazione**, che richiede competenze specifiche per sfruttarne appieno il potenziale.

## **Condizionamento di Segnale**

Il **condizionamento del segnale** è un processo fondamentale nel trattamento dei segnali analogici per consentire la loro corretta interpretazione come segnali digitali. Poiché i segnali analogici possono avere andamenti variabili e non sempre interpretabili univocamente, è necessario normalizzarli e filtrarli. Questo garantisce che possano essere rappresentati come simboli digitali "0" o "1" senza ambiguità.

Nel campo dell'acquisizione dati, il condizionamento del segnale è cruciale: i segnali provenienti dai sensori devono essere adattati per rientrare nei parametri di funzionamento dei circuiti interni di un dispositivo. I blocchi di input/output □I/O□ svolgono questa funzione, introducendo e normalizzando i segnali esterni all'interno del chip.

## **BLOCCHI INPUT/OUTPUT**

I blocchi di I/O gestiscono l'interazione tra l'esterno del chip e i circuiti interni, svolgendo diverse funzioni essenziali:

- **Condizionamento dei segnali esterni.**
- **Protezione contro scariche elettrostatiche.**
- **Fornitura di alimentazione e riferimenti di tensione.**

## **REQUISITI FUNZIONALI DEI BLOCCHI I/O**

I blocchi I/O possono gestire diverse tipologie di ingressi e uscite, come:

- **Ingressi di potenza:** per alimentare il dispositivo.
- **Segnali di clock:** utilizzati per sincronizzare i circuiti digitali.

- **Ingressi/Uscite in corrente continua** □DC□□ per pilotare LED, relè o altri piccoli carichi resistivi.
- **Ingressi/Uscite in corrente alternata** □AC□□ per segnali ad alta frequenza, logiche veloci, linee seriali e bus dati.

Dal punto di vista funzionale, ogni dispositivo che pilota una linea esterna è considerato un buffer.

### BLOCCHI OUTPUT (50-200mA)

Il **buffer di uscita** consente di pilotare carichi capacitivi significativi. Questo avviene caricando o scaricando capacità esterne con tempi di propagazione adeguati. I buffer sono spesso configurabili come ingressi o uscite.

### ESEMPIO: CONTROLLO MOTORI

Nel caso di un controllo motore mediante FPGA□

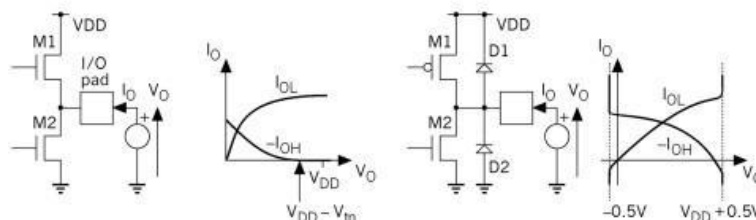
- I buffer di uscita non pilotano direttamente il motore.
- È necessario interporre uno stadio di potenza, ad esempio un ponte di transistori, per gestire le correnti richieste.

### TOTEM-POLE OUTPUT

La configurazione **totem-pole** è uno stadio attivo in cui due transistor lavorano in opposizione di fase (uno acceso, l'altro spento). Questa configurazione include diodi di protezione (o diodi di clamping) che proteggono da sovratensioni o sottotensioni dovute a carichi induttivi.

### PROTEZIONE CONTRO CARICHI INDUTTIVI

Se una bobina, per esempio, accumula energia induttiva e il generatore viene spento, la bobina tende a richiamare corrente, causando un aumento di tensione pericoloso. I diodi di protezione evitano che il dielettrico tra drain e gate dei transistor venga danneggiato.

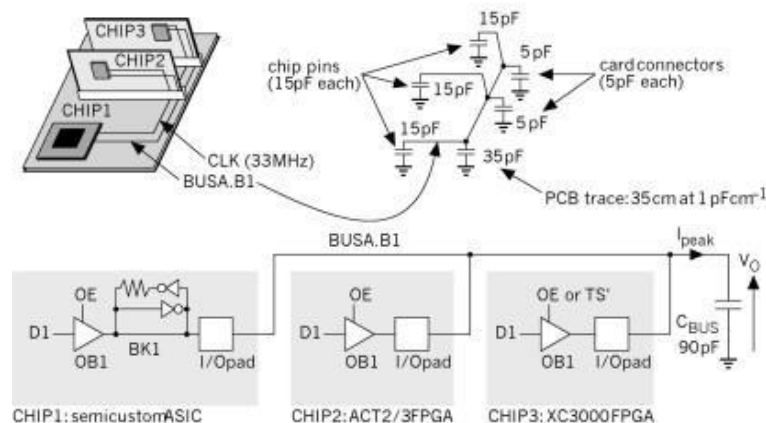


### TRI-STATE

Un **buffer tri-state** consente tre stati distinti:

- Stato "0": transistor PD acceso, PU spento.
- Stato "1": transistor PU acceso, PD spento.
- Stato ad alta impedenza: entrambi i transistor spenti, senza conduzione.

Questa configurazione è essenziale per condividere bus tra più dispositivi, garantendo che solo uno alla volta possa trasmettere. Per evitare conflitti, il buffer viene disabilitato (ad alta impedenza) quando non è necessario.



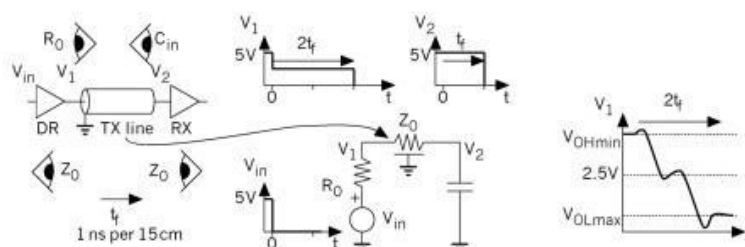
## LINEE DI TRASMISSIONE

In presenza di **commutazioni rapide rispetto alle impedenze coinvolte**, si deve considerare il **tempo di propagazione**, noto come **tempo di volo** ( $t_f$ ). Tipicamente, è dell'ordine di **1 ns ogni 30 cm di linea di trasmissione** (circa metà della velocità della luce nel vuoto).

L'onda si propaga fino alla fine della linea e viene riflessa.

Gli effetti di propagazione diventano significativi quando la **commutazione all'uscita** avviene in un tempo minore di **2 volte il tempo di volo**.

Questi fenomeni possono generare **fluttuazioni indesiderate** nel segnale, causando **malfunzionamenti** nei circuiti logici.



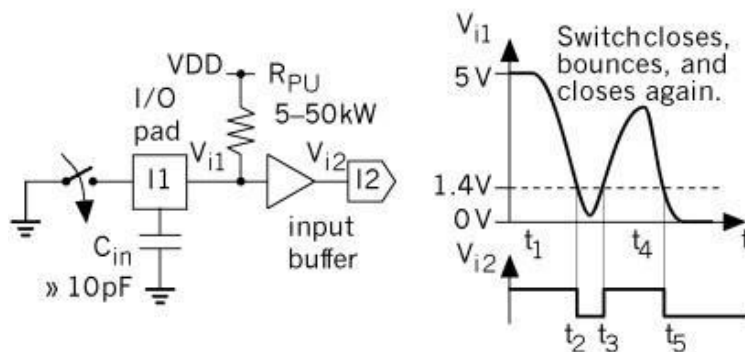
Per mitigare tali problemi, si utilizzano le **terminazioni di adattamento**, che includono:

- **Circuito aperto**: sfrutta l'impedenza d'ingresso del ricevitore.

- **Resistenza in parallelo:** riduce il rischio di riflessi, ma aumenta il consumo di potenza in corrente continua.
- **Terminazione di Thévenin:** offre un consumo di potenza in corrente continua ridotto.
- **Adattamento alla sorgente:** assicura la corrispondenza di impedenza tra sorgente e linea.
- **Adattamento in parallelo con condensatore in serie:** combina vantaggi di resistenza e capacità per migliorare il bilanciamento del segnale.

## INPUT BOUNCING

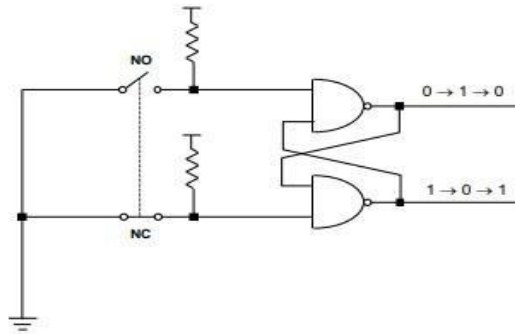
Quando un segnale digitale presenta **rimbalzi** all'ingresso, possono verificarsi **interpretazioni errate** di stati logici ("0" o "1"). Per prevenire tali errori, si ricorre a tecniche di **debouncing**.



### 1. Debouncing con Flip-Flop SR

Un circuito anti-rimbalzo può essere implementato tramite **porte logiche**, come NAND o NOR, per creare un **flip-flop Set-Reset (SR)**.

- **Funzionamento:**
  - Il flip-flop **memorizza lo stato dell'uscita**, ignorando gli impulsi di disturbo sull'ingresso.
  - Per cambiare lo stato dell'uscita, è necessario applicare un impulso su **due ingressi distinti** (Set e Reset).
- **Vantaggi:** elevata immunità al disturbo.



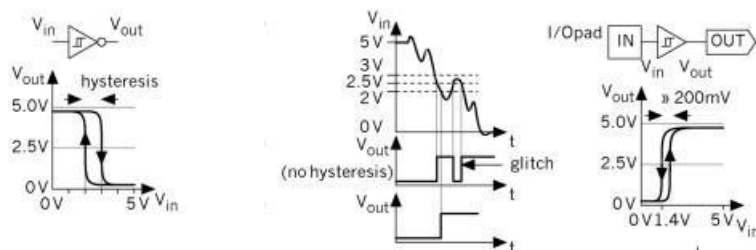
## 2. Debouncing con Trigger di Schmitt

I dispositivi di **Trigger di Schmitt** vengono utilizzati per il **condizionamento del segnale**, rimuovendo **rumore** e **rimbalzi** nei contatti degli interruttori.

- **Funzionamento:**

- Il Trigger di Schmitt introduce una **isteresi**, che trasforma un segnale analogico in uno digitale.
- L'uscita varia tra due valori di tensione predefiniti, a seconda che l'ingresso superi una soglia superiore o scenda sotto una soglia inferiore.

- **Vantaggi:** consente uno **squadrimento** del segnale, garantendo una maggiore stabilità e precisione.



## CLOCK INPUTS

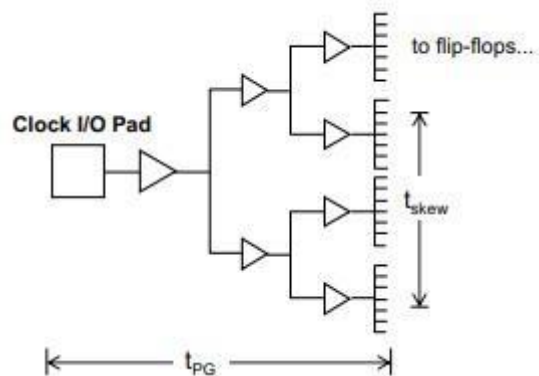
Alcuni ingressi nelle FPGA sono dedicati esclusivamente ai segnali di **clock**, che costituiscono il riferimento per l'evoluzione temporale dell'intera rete sincrona. Il clock deve essere distribuito a tutti i dispositivi in modo sincronizzato e con **bassa latenza e basso skew**.

La rete di distribuzione del clock adotta una struttura ad **albero bilanciato**, che garantisce una propagazione uniforme del segnale. Questa configurazione riduce al minimo lo skew, definito come la **differenza temporale tra i fronti del clock** ricevuti dai vari dispositivi.

**Schema del Ritardo:** Tra l'oscillatore al quarzo, che genera il segnale di clock, e i singoli flipflop, esistono cammini di propagazione identici. Questo assicura che il **ritardo di propagazione** sia presente, ma lo skew sia pressoché nullo.

## Vantaggi e Considerazioni

- Le FPGA sincrone sfruttano la rete di clock per garantire il funzionamento coordinato di tutti i dispositivi.
- Sebbene le **logiche asincrone** consumino meno potenza rispetto a quelle sincrone, le FPGA sincrone permettono di gestire complessità circuitali che sarebbero difficilmente raggiungibili con logiche asincrone.



## POWER INPUTS

Tutti i dispositivi richiedono ingressi di alimentazione dedicati, tipicamente:

- **VDD** e **GND** per il funzionamento normale.
- **VPP** per la fase di programmazione (se necessaria).

Nei dispositivi di grandi dimensioni, è necessario prevedere **più pin di alimentazione e massa** per mantenere i riferimenti di tensione:

- **Stabili** e insensibili ai transienti di corrente.
- In grado di gestire elevate richieste di corrente senza compromettere la stabilità del sistema.

L'aumento del numero di pin dedicati all'alimentazione riduce la disponibilità di **pin digitali I/O**, limitando il numero di connessioni utilizzabili per scopi di elaborazione o comunicazione.

## ESEMPIO CON XILINX 4000: I/O E CLOCK

Nel caso in cui sia necessario utilizzare **più segnali di clock**, si può impiegare un pin generico come sorgente del clock. Tuttavia, è fondamentale considerare alcune accortezze per garantire la sincronizzazione:

### Ritardi Programmabili:

- Quando il clock non utilizza la rete di distribuzione dedicata, è necessario introdurre ritardi programmabili.

- Questi ritardi permettono di adattare il segnale in modo tale che arrivi a più celle in maniera **sincrona**, eliminando eventuali disallineamenti temporali.

### Reti Dedicate per il Clock:

Le FPGA Xilinx 4000 dispongono di logiche dedicate per la gestione del clock, che permettono di:

- Ridurre lo **skew** (de-skew), ovvero la dispersione tra i fronti del segnale.
- Eseguire operazioni avanzate sul clock, come:
  - **Shift di fase** per sincronizzare circuiti con esigenze temporali differenti.
  - **Divisione o moltiplicazione** della frequenza del clock.
  - **Sintesi di frequenze**, utile per applicazioni che richiedono segnali di clock personalizzati.

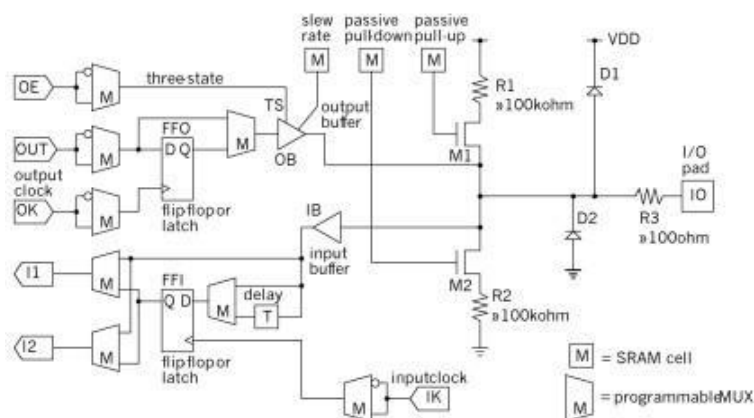
Negli ingressi/uscite, la gestione dei segnali richiede particolare attenzione per evitare conflitti elettrici:

### Conflitti Elettrici:

- Se un PU (Pull-Up) e un PD (Pull-Down) fossero attivi contemporaneamente, potrebbe verificarsi un cortocircuito o un sovraccarico.
- Per evitare ciò, si utilizza un **PU resistivo**, il quale limita la corrente e riduce la dissipazione di potenza, mantenendo il sistema operativo stabile.

### Configurazioni Open-Drain e Open-Source:

- Queste configurazioni evitano conflitti utilizzando: **PU attivo** combinato con **PD passivo**, oppure viceversa.
- La resistenza collegata a VDD garantisce livelli di corrente e tensione compatibili con le caratteristiche del MOSFET, evitando sovraccarichi.



## ES6

# Micro-processors/controllers

## Teoremi Principali

All'inizio del 1900, i matematici, guidati da Hilbert, cercarono di formalizzare la matematica in un sistema assiomatico rigoroso, nel quale tutte le branche derivassero da un insieme di assiomi fondamentali. Tuttavia, Kurt Gödel, con il suo **Teorema di Incompletezza**, dimostrò che:

1. **Esistenza di proposizioni indecidibili:** In ogni sistema matematico assiomatico sufficientemente potente da contenere l'aritmetica elementare, esistono proposizioni che non possono essere né dimostrate né confutate all'interno del sistema stesso, pur essendo vere o false indipendentemente.
2. **Limiti della consistenza:** La consistenza di un sistema matematico  $F$  non può essere dimostrata all'interno dello stesso sistema  $F$ . Ciò implica che non è possibile garantire la totale affidabilità del sistema basandosi esclusivamente sugli assiomi e le regole interne.

Questo risultato ha avuto un profondo impatto sulla logica, la matematica e la filosofia, dimostrando che la matematica non può essere completamente ridotta a un insieme di regole meccaniche.

## La macchina di Turing

Alan Turing affrontò il problema della formalizzazione della computabilità, elaborando la **Macchina di Turing**, un modello teorico capace di rappresentare qualsiasi processo algoritmico.

### Concetti fondamentali:

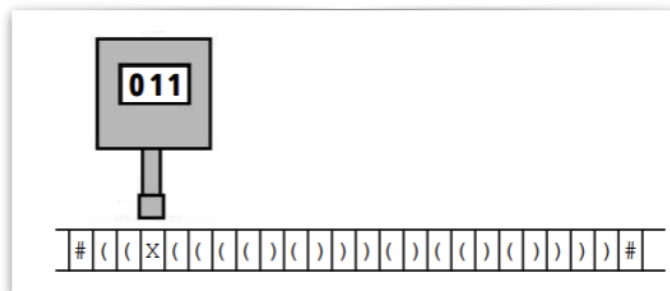
- **Congettura di Church-Turing:** Qualunque problema computazionale che ammette una soluzione algoritmica può essere risolto da una macchina automatica, ossia una macchina di Turing.
- **Effettiva calcolabilità:** Una funzione è definita "effettivamente calcolabile" se i suoi valori possono essere determinati attraverso un processo puramente meccanico, come quello implementato da una macchina di Turing.

**Struttura della macchina di Turing:** Formata da 3 componenti: il nastro (Memoria), una sequenza di celle, considerata infinita, ciascuna delle quali può contenere un simbolo appartenente a un alfabeto finito; la testina di lettura/scrittura (TLS), cioè l'interfaccia, il meccanismo che legge il simbolo corrente, sovrascriverlo o spostarsi di una cella a destra o sinistra; la Control Unit (FSM), il "cervello", definita da una quintupla di elementi ( $s$ : lo stato attuale;  $i$ : il simbolo letto dal nastro,  $S(s,i)$ : lo stato successivo;  $I(s,i)$ : il simbolo che verrà scritto sul nastro;  $V(s,i)$ : la direzione di movimento della testina).

**Funzionamento:** La macchina opera su intervalli discreti di tempo: ad ogni istante, il suo stato attuale e le azioni future dipendono dallo stato precedente e dal simbolo letto. Questo modello, sebbene teorico, è sufficiente a risolvere qualsiasi problema computazionale che possa essere espresso in termini algoritmici.

Esempio: Verifica di una sequenza di parentesi

Prendiamo una sequenza di parentesi (scritte su un nastro) delimitata da due caratteri speciali. La macchina di Turing funziona che la testina si muove a destra finché non trova la prima parentesi chiusa e la sostituisce con un simbolo, poi si muove verso sinistra finché non trova una parentesi aperta e la sostituisce anch'essa con il simbolo. Questo ciclo si ripete finché non si sostituiscono tutte le parentesi oppure se rimangono parentesi aperte.



## I Microprocessori

I microprocessori possono essere considerati come versioni altamente evolute della macchina di Turing, ma con il numero di stati sia limitato, anche se la quantità di stati disponibili nei microprocessori moderni è così grande da poterli considerare **praticamente infiniti**. Di conseguenza con un microprocessore e il programma opportuno si può risolvere qualunque problema di tipo algoritmico. Questa proprietà li rende **Turing-Completi**, che vuol dire che può simulare una macchina di Turing

universale, cioè supporta istruzioni condizionali (Branch), salti (Loop) e memoria arbitraria (lettura/scrittura).

Esempio: Un modello computazionale basato su **porte logiche NAND** è Turing-Completo. Poiché la porta NAND è funzionalmente completa, quindi può rappresentare qualsiasi funzione logica booleana, combinando porte NAND si costruiscono componenti complessi (come ALU, Registri, MUX) e questi componenti formano un processore Turing-Completo.

## La Complessità

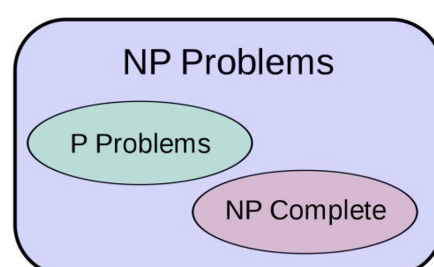
### La Complessità Computazionale

La **complessità computazionale** si occupa di analizzare come crescono il tempo e lo spazio, necessari per risolvere un problema mediante un algoritmo, all'aumentare dell'input  $N$ .

La complessità di un algoritmo è indicata tramite la **notazione asintotica** (Big-O) che esprime come il tempo cresce al variare di  $N$ :  $O(\log N)$  logaritmica,  $O(N)$  lineare,  $O(N^2)$  polinomiale,  $(2^N)$  esponenziale.

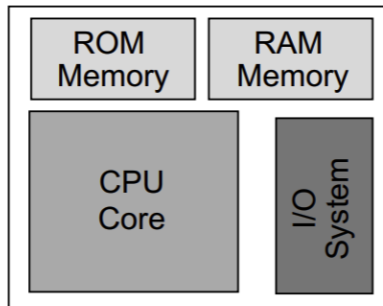
Classi di Problemi:

1. Problemi Polinomiali (P): Algoritmi efficienti, risolvibili in tempi ragionevoli con risorse limitate. La complessità cresce con una funzione polinomiale (ad esempio:  $O(N)$ ,  $O(N^2)$ ,  $O(N \times \log N)$ ) (esempio pratico: Merge Sort)
2. Problemi Nondeterministici Polinomiali (NP): Diventano rapidamente impraticabili, con il calcolo che aumenta drasticamente. Richiedono una crescita esponenziale  $O(n)$ ,  $O(n!)$
3. Problemi NP-Completi: Sono i problemi più difficili della classe NP. Se si trovasse che un problema NP-completo è risolvibile in tempo polinomiale, tutti i problemi della classe NP diverrebbero polinomiali.



# Strutture e Componenti

## Architettura Generale Semplificata



Studiando i Microcontrollori possiamo partire da uno schema ad alto livello della sua architettura generale tipica, che comprende CPU, memorie e sistemi di I/O:

## Componenti Generali

**CPU Core** (Central Processing Unit): È il cuore del microcontrollore del sistema. Esegue le istruzioni, fa i calcoli aritmetici/logici e gestisce il flusso dei dati. Il set di istruzioni ottimizzato per gestire direttamente l'Input/Output.

**ROM Memory** (Read-Only Memory / NVRAM): È la memoria **non volatile** (On-chip). Contiene il codice del programma e il Sistema Operativo, che devono rimanere salvati anche quando spegni il dispositivo. La CPU esegue il codice direttamente da qui.

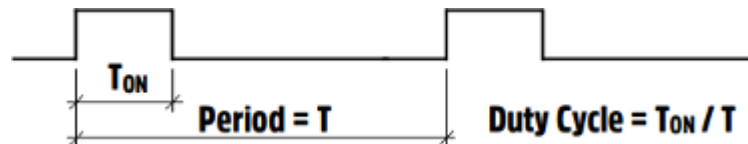
**RAM Memory** (Random Access Memory): È la memoria **volatile** (On-chip). Contiene lo stack e i **dati temporanei**, su cui il programma sta lavorando in quel momento. Si cancella quando togli corrente ed è veloce quanto la CPU.

**Memoria Off-Chip:** Sebbene l'obiettivo sia l'integrazione, i sistemi complessi possono estendere la memoria tramite bus esterni. Tuttavia, l'accesso *Off-chip* introduce latenza e consuma più energia, motivo per cui si preferisce massimizzare l'uso delle risorse interne.

**I/O System:** Composto da...

- **Input/Output pins:** Utilizzati per la comunicazione con l'esterno, con direzione programmabile, possono leggere e scrivere valori alti ( $V_{DD}$ ) o bassi (GND). Poiché sono l'interfaccia programmabile e di pin c'è un numero limitato spesso si utilizza il **Pin Multiplexing** dove uno stesso pin è configurato via software per svolgere funzioni diverse. (tipica corrente 20-60 mA)
- **Timers e Counters:** Sono registri interni configurati come contatori, solo che il Counter viene incrementato da un segnale esterno (fronti di salita/discesa su un pin), mentre il Timer è pilotato dal Clock del sistema. Quest'ultimo quando raggiunge il massimo valore genera un interrupt e tramite l'Auto-Reload si azzerà.

- **PWM (Pulse Width Modulation):** Tecnica di modulazione digitale che genera una tensione media variabile tramite impulsi rettangolari, facendo così sembrare il segnale digitale uno analogico. Fa ciò, usando i timer interni, inviando impulsi rettangolari di durata variabile così da creare una tensione media (con l'alimentazione a 5V: un Duty Cycle del 10% ottengo una tensione media percepita di 0,5V, se 50% → 2,5V, se 90% → 4,5V)



- **Capture Inputs:** È un counter, assegnato ad ogni pin, con il compito di contare gli eventi esterni che riceve in input (senza dover fare polling per controllare i pin). Normalmente, il sistema è progettato per generare un interrupt quando il contatore raggiunge un valore prestabilito (permettendo al sistema di reagire solo quando è necessario (es: buffer pieno))
- **A/D e D/A Converters:** Gli A/D convertono, via software, i segnali analogici dell'esterno in segnali digitali leggibili dal microcontrollore, con una accuratezza di conversione nel range degli 8-12-16 bit. Mentre i D/A fanno il contrario con una precisione di 1-2 word del processore.
- **UART (Universal Asynchronous Receiver-Transmitter):** È una periferica programmabile per la comunicazione seriale digitale a bassa velocità in banda base. Un tempo era esterno al processore, ma ora è integrato direttamente come periferica on-chip. Supporta principalmente modalità asincrone utilizzando standard come l'RS232.

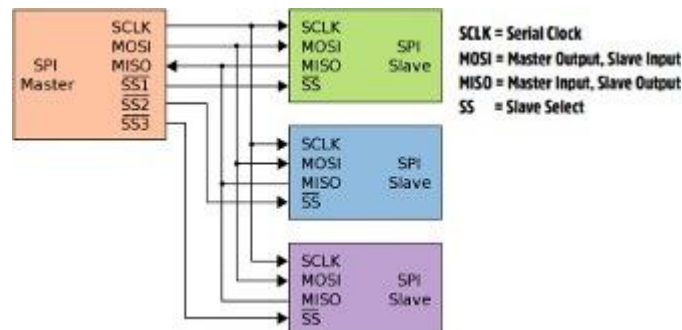


## Protocolli dei Micro

Lo **SPI** (Serial Peripheral Interface) è un protocollo sincrono, quindi abbiamo il clock che dice quando leggere e scrivere i dati rendendo la comunicazione molto più veloce e affidabile (rispetto alla UART o I2C). La configurazione base prevede una topologia con un Master (che controlla il timing) e uno o più Slave, con 4 linee principali del Bus SPI per la connessione fisica:

- **SCLK (Serial Clock):** Segnale generato dal master per sincronizzare la trasmissione. Determina la velocità della trasmissione, nessun handshake, lo slave deve stare al passo.
- **MOSI (Master Output, Slave Input):** Linea dati in uscita dal Master e in ingresso allo Slave.

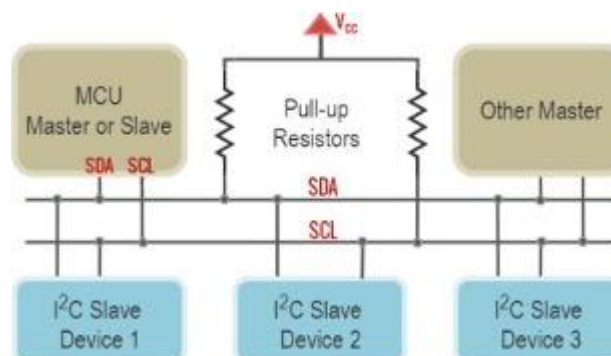
- MISO (Master Input, Slave Output): Linea dati in uscita dallo Slave e in ingresso al Master.
- SS (Slave Select): Linea dedicata per ogni Slave. Il Master porta a livello basso (Low) la linea SS dello Slave desiderato.



## The I<sup>2</sup>C (Inter-Integrated Circuit) Protocol

Sviluppato da Philips (ora NXP), è un bus seriale sincrono a due fili progettato per collegare periferiche a bassa velocità (sensori, RTC, EEPROM) minimizzando il numero di pin del microcontrollore. Con le seguenti caratteristiche:

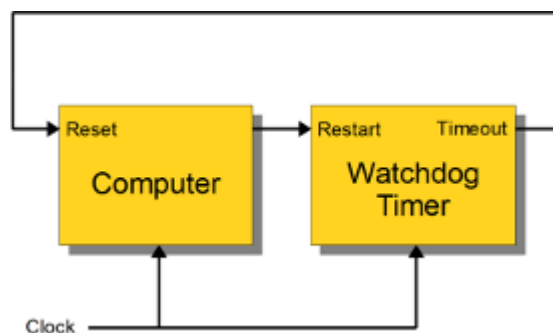
1. A differenza dello SPI, l'I<sup>2</sup>C utilizza un'architettura **Open-Drain** con resistenze di **Pull-Up** esterne. Dove i dispositivi possono solo forzare le linee a livello basso (Logic 0). Il livello alto (Logic 1) è ripristinato passivamente dalle resistenze. Questo evita cortocircuiti in configurazione Multi-Master e permette di collegare dispositivi con tensioni diverse.
2. La selezione dello Slave avviene via software: il Master segnala l'inizio tirando giù SDA (Serial Data), mentre SCL (Serial Clock) è alto. Poi invia 7 bit di indirizzo + 1 bit di R/W (Lettura/Scrittura). Dopo ogni byte, il ricevitore deve "tirare giù" la linea SDA per confermare la ricezione (Acknowledge).



## Watchdog Timer (WDT)

È un timer hardware autonomo progettato per rilevare anomalie software (loop infiniti, deadlock, crash) e hardware (glitch di alimentazione). Per garantire la massima sicurezza, il WDT è solitamente alimentato da una sorgente di clock interna indipendente separata dal clock principale del sistema. In questo modo, il WDT può resettare il sistema anche in caso di guasto totale.

Funzionamento: Il WDT conta alla rovescia partendo da un valore preimpostato. Il software, durante il suo normale ciclo di funzionamento, deve periodicamente scrivere un valore specifico in un registro del WDT per riportare il contatore all'inizio. Se ciò accade il sistema continua, mentre se non succede scatta l'azione di sicurezza del WDT, che consiste in un interrupt non-mascherabile che mette la macchina in sicurezza.



## Memoria e Decodifica

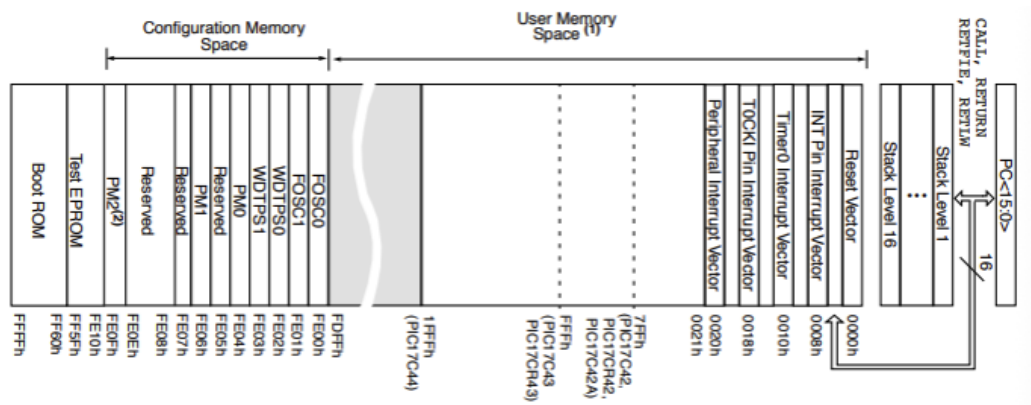
### Organizzazione della Memoria

Ogni microprocessore o microcontrollore possiede una mappa predefinita del proprio spazio di memoria (sia interna che esterna). Questo comporta che:

- Abbiamo una **Mappatura delle periferiche**, le periferiche vengono assegnate a specifici indirizzi nello spazio di memoria. Ogni dispositivo deve essere correttamente abilitato quando il processore accede al suo indirizzo assegnato. Questo processo è noto come **decodifica degli indirizzi**.
- Otteniamo un **Partizionamento della memoria** dove lo spazio di memoria è suddiviso in sezioni predefinite:
  - Memoria di sistema: Area protetta riservata al firmware e alle operazioni vitali del microcontrollore.
  - Memoria utente: Area libera destinata all'esecuzione dei programmi scritti dall'utente (applicativi).

- Stack: Area organizzata a "pila" (LIFO), essenziale per gestire le chiamate a funzione, i salti e i ritorni (salvataggio del contesto).

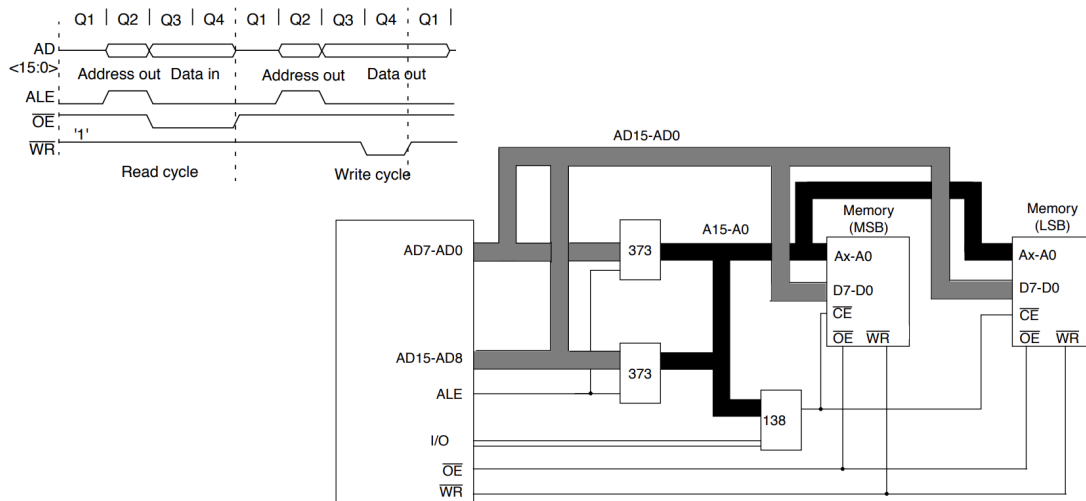
Esempio pratico di Mappa di Memoria:



## Memory Interface

Il microprocessore comunica con la memoria esterna tramite bus di indirizzi e dati, questo diventa costoso, poiché un processore a 16 bit con 16 bit di indirizzi avrebbe bisogno di 32 pin solo per il bus (16 dati + 16 indirizzi). Allora per ridurre il numero di pin del package, si utilizza un bus multiplexato, dove gli stessi 16 pin portano prima l'indirizzo e poi il dato.

Andando a comunicare, nella fase Q1, del primo ciclo, la CPU pone l'indirizzo sulle linee del bus (**AD15-AD0**) e attiva il segnale di **ALE** (Address Latch Enable) che va alto, dicendo che sul bus c'è un indirizzo. A questo punto i blocchi **373**, che sono dei latch, diventano trasparenti, lasciando passare il segnale. Quando il segnale ALE va basso, i chip 373 si chiudono, memorizzando l'ultimo valore visto e lo inviano alla memoria sulle linee nere (**A15-A0**). Questo permette alle memorie di sapere dove lavorare grazie alle linee nere stabili, mentre le linee grigie sono libere di fare altro. Il blocco **138** funge da decoder, legge alcuni bit dell'indirizzo e decide quale chip di memoria attivare, tramite il segnale **CE**. Ora che l'indirizzo è stato salvato nei latch, le linee grigie (AD) cambiano funzione diventando Data Bus (finendo nelle memorie agli indirizzi **D7-D0**)



Lo schema mostra due chip di memoria (Memory MSB e Memory LSB). Poiché ogni chip è tipicamente a 8 bit, ne mettiamo due in parallelo per raggiungere i 16 bit richiesti, dando ad uno i bit da D0 a D7 e all'altro quelli da D8 a D15. Questo ci permette di leggere una Word intera in un solo ciclo.

Infine, abbiamo i segnali di controllo: **OE** (Output Enable) il quale abilita la lettura dalla memoria, **WR** (Write) che abilita la scrittura, **CE** (Chip Enable) che attiva il chip di memoria selezionato, generato tramite un decodificatore (138).

## Decodifica

La decodifica degli indirizzi consente al microcontrollore di selezionare specifiche sezioni di memoria periferiche. La quantità di memoria indirizzabile dipende dal numero di bit del bus di indirizzi.

- **Spazio di indirizzamento:** Con un bus di indirizzi a 16 bit, è possibile indirizzare  $2^{16} = 65.536$  combinazioni ovvero 64KB
- Caso pratico:
  - Se si utilizza un chip di memoria da 8 KB (8K word), sono necessari 13 bit di indirizzo per coprire l'intero spazio ( $2^{13} = 8.192 \text{ word} = 8\text{KB}$ ).
  - I 3 bit rimanenti vengono usati per selezionare quale tra 8 chip attivare, tramite un segnale di **Chip Enable** generato da una funzione combinatoria degli indirizzi.
- **Esclusione di zone di memoria:** Si può configurare il microcontrollore per ignorare certe aree di memoria, lui quindi ignorerà le letture/scritture in quell'area.

# Modalità di Interfaccia Periferica

Nei sistemi embedded, la comunicazione tra il microprocessore o microcontrollore e le periferiche può avvenire tramite tre principali modalità di interfaccia: **Polling**, **Interrupt** e **DMA (Direct Memory Access)**. Ciascuna modalità presenta vantaggi e svantaggi, adattandosi a specifiche esigenze applicative.

## Polling

Il Polling è una tecnica ciclica in cui il processore verifica continuamente lo stato di ciascuna periferica per verificare se ci sono nuovi dati da elaborare e se una trasmissione è completata.

- **Vantaggi:** La sua semplicità e nessuna necessità di hardware complesso.
- **Svantaggi:** Inefficienza, poiché La CPU rimane costantemente impegnata nel controllo delle periferiche, riduce le risorse disponibili per altre operazioni. Incompatibilità con il multitasking, dato che impedisce alla CPU di svolgere più compiti in parallelo. Dipendenza dal numero di periferiche, più periferiche più costo in tempo ed consumi.

## Interrupt

L'interrupt è un meccanismo hardware che consente alla CPU di reagire ad eventi asincroni, sospendendo il programma corrente, eseguendo la routine di interrupt, per poi riprendere l'esecuzione del programma interrotto.

Ci sono diversi tipi: **Mascherabili** che possono essere disattivati temporaneamente, ad esempio durante l'esecuzione di una routine critica; **Non mascherabili (NMI)** per processi prioritari che non possono essere disattivati, come surriscaldamento della CPU o caduta di alimentazione; **Preemption (Nesting):** Un interrupt ad alta priorità può interrompere una ISR a bassa priorità già in esecuzione, creando una struttura a "pila" di interruzioni annidate.

- Vantaggi: Ottimizza l'uso della CPU, che non è costretta a controllare ciclicamente le periferiche ed è ideale per periferiche ad alta velocità.
- Svantaggi: Richiede un sistema di gestione degli interrupt e se la frequenza degli interrupt è eccessiva rallenta il sistema.

## DMA - Direct Memory Access

Il **DMA** è un modulo hardware specializzato incaricato di gestire il trasferimento dati ad alta velocità tra memoria e periferiche/memoria senza l'intervento attivo della CPU. Fa ciò con la CPU che avvia il trasferimento configurando il **controller DMA**, il quale gestisce autonomamente il trasferimento di blocchi di dati e la CPU interviene solo per monitorare il processo.

- Vantaggi: è molto efficiente, riduce il carico di interrupt sulla CPU, permettendo il trasferimento rapido di grandi quantità di dati tra memoria e periferiche.
- Svantaggi: richiede hardware dedicato (il controller DMA) ed è più complesso rispetto alle altre modalità.

## ES7

# Digital Signal Processors

## Introduction

I Digital Signal Processors (DSP) sono microprocessori progettati specificamente per l'elaborazione numerica dei segnali. Il loro sviluppo è nato dalla necessità di tradurre le operazioni sui segnali da sistemi analogici a digitali, sfruttando la maggiore *flessibilità, prevedibilità e stabilità* offerte dal software digitale rispetto agli equivalenti sistemi hardware analogici. I DSP consentono di modificare parametri e algoritmi in modo dinamico senza la necessità di cambiare l'hardware, rendendo queste architetture ideali per una vasta gamma di applicazioni.

I DSP sono ampiamente utilizzati in ambiti embedded, tra cui telecomunicazioni, controllo di processo, radar, automotive e gestione di controllori. La loro capacità di elaborare segnali in modo continuo e in finestre temporali stringenti è cruciale per evitare la perdita di informazioni, che potrebbero compromettere le applicazioni finali.

Le architetture DSP si differenziano significativamente dai processori general purpose grazie a ottimizzazioni hardware e software per specifici carichi di lavoro, come l'elaborazione dei segnali digitali. Questa specificità li ha resi uno standard in molti settori.

## The Rationale

Il successo dei DSP si basa su quattro pilastri fondamentali che superano i limiti dell'elettronica analogica:

1. **Prevedibilità (Predictability):** L'elaborazione matematica pura. A parità di input, l'output è garantito, senza le tolleranze dei componenti fisici.
2. **Stabilità (Stability):** I sistemi digitali non soffrono di deriva termica né di invecchiamento (aging), garantendo prestazioni costanti nel tempo.
3. **Flessibilità (Flexibility):** Modificare il comportamento del sistema (es. cambiare la frequenza di taglio di un filtro) richiede solo un aggiornamento software, non la sostituzione fisica di hardware.
4. **Compattezza (Integration):** Grazie alla tecnologia VLSI, è possibile integrare funzioni complesse in un singolo chip minuscolo.

## DSP Peculiarities

1. Hardware dedicato per operazioni **MAC** (Multiply and Accumulate): Essenziale per operazioni matematiche ripetitive ad alta velocità, come la convoluzione e i filtri.
2. **Accessi multipli alla memoria:** Consentono al DSP di accedere simultaneamente a più aree della memoria, migliorando l'efficienza, che con l'Architettura Harvard garantisce un flusso continuo senza attese.
3. **Modalità di indirizzamento dedicate:** Facilitano il trattamento di flussi di dati continui e sequenze numeriche.
4. **Strutture di controllo dedicate:** Ottimizzano la gestione dei segnali e delle operazioni. L'hardware gestisce i cicli, il processore non spreca cicli di clock per decrementare contatori e gestire i salti.
5. **Periferiche dedicate on-chip:** ADC (Analog-to-Digital Converter), DAC (Digital-to-Analog Converter), e altre periferiche specifiche sono integrate direttamente nel chip per ridurre la latenza ingresso/uscita.
6. **Alta frequenza di funzionamento:** La velocità dei DSP è estremamente elevata, rendendoli adatti ad applicazioni ad alta intensità computazionale come radar e telecomunicazioni.

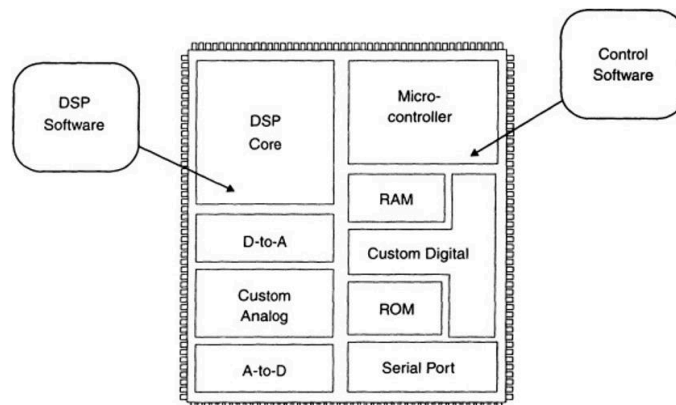
## DSP on System on Chip (SoC)

Invece di avere chip separati sulla scheda madre, con i DSP SoC si integra tutto su un unico pezzo di silicio. Si va a combinare un **DSP Core** (dedicato all'elaborazione matematica dei segnali tramite proprio software dedicato) con un **Microcontrollore**

(che gestisce la logica di controllo). Inoltre, il chip include direttamente: **Convertitori** A-to-D (Analogico-Digitale) e D-to-A (Digitale-Analogico) per interfacciarsi con l'esterno, **Memoria** tipo RAM e ROM condivise o dedicate. **Custom Logic** cioè blocchi digitali o analogici personalizzati per l'applicazione specifica e porte seriali per l'I/O.

I 3 Vantaggi Principali sono:

1. **Efficienza** (Core RISC): L'uso di core con architettura RISC (Reduced Instruction Set Computer) per il microcontrollore garantisce un'esecuzione rapidissima delle istruzioni elementari di controllo.
2. **Flessibilità** (Supporto CISC): La possibilità di integrare istruzioni complesse (tipo CISC) permette di gestire algoritmi avanzati e specifici senza dover scrivere codice eccessivamente lungo.
3. **Compattezza ed Economicità**: Mettere tutto su un solo chip riduce drasticamente: i costi di produzione, le dimensioni del dispositivo e il consumo energetico.



## Hardware

I **Digital Signal Processors (DSP)** sono processori altamente specializzati progettati specificamente per l'elaborazione di segnali digitali in **tempo reale**. Grazie a particolari caratteristiche hardware e architetture, questi dispositivi offrono prestazioni ottimizzate per compiti intensivi che richiedono rapidità e precisione, rendendoli fondamentali in settori come le telecomunicazioni, l'elaborazione audio/video e la grafica tridimensionale.

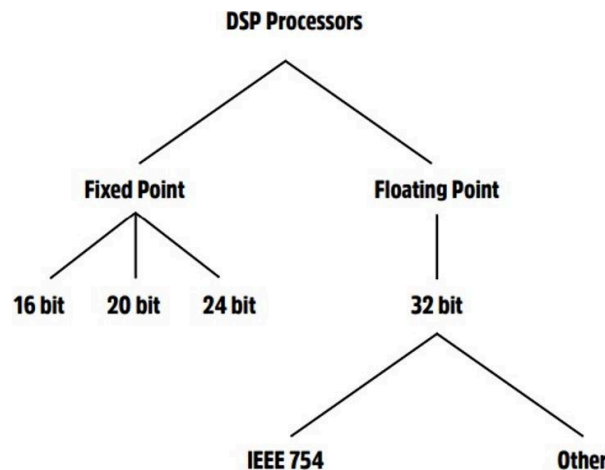
## Arithmetics

L'aritmetica dei processori DSP si divide in due grandi famiglie, ciascuna adatta a specifici compromessi ingegneristici:

- **Fixed Point (Virgola Fissa)**: Il processore tratta i numeri come se fossero interi puri. La posizione della virgola è gestita dal programmatore. Le dimensioni

tipiche sono **16, 20 o 24 bit**. È la scelta per applicazioni come telefonia dove la dinamica del segnale non è estrema ma il risparmio energetico è cruciale.

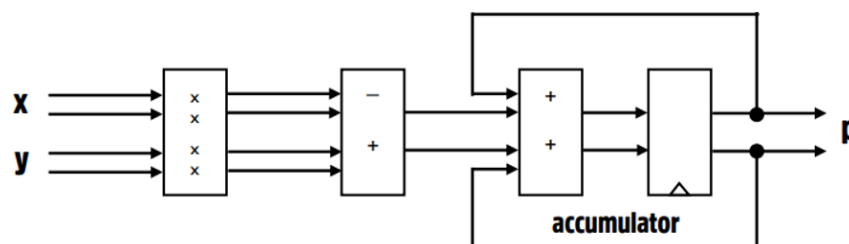
- **Floating Point (Virgola Mobile):** Il processore gestisce automaticamente mantissa ed esponente (solitamente a **32 bit** standard **IEEE 754**, ma esistono formati proprietari). Offre una **gamma dinamica** enormemente superiore, rendendolo ideale per audio hi-fi, grafica 3D e radar, dove i numeri possono variare da piccolissimi a grandissimi senza perdere precisione.



## MAC - Multiply and Accumulator

Il **MAC (Multiply and Accumulate)** è una componente hardware essenziale nei DSP, progettata per eseguire operazioni di moltiplicazione e somma in un singolo ciclo di clock. Questa unità è fondamentale per l'elaborazione del segnale digitale, dove le operazioni di somma e prodotto costituiscono la base di molti algoritmi.

Esempio: Il MAC riceve in input due bus  $x$  e  $y$ , che contengono parte reale e parte immaginaria, andando a moltiplicarli creando  $ac, bd, ad, bc$ . Allora si passa allo stadio di somma dove si esegue  $ac - bd$  e  $ad + bc$ . A questo punto due accumulatori separati con feedback sommano i risultati parziali per le parti reali e immaginarie, completando tutto in un unico ciclo di clock.

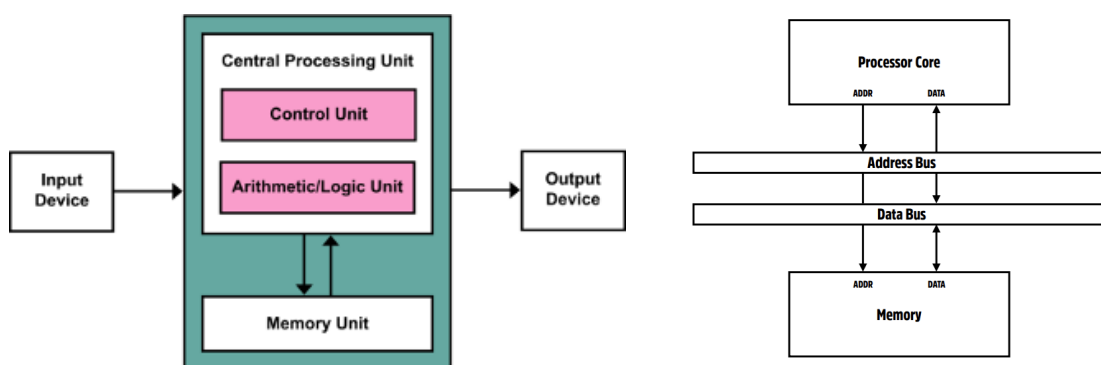


## Von Neumann Architecture

L'architettura di Von Neumann è caratterizzata dall'utilizzo di una singola memoria condivisa per dati e istruzioni. Questa memoria è connessa al processore tramite un unico bus, che viene utilizzato alternativamente per accedere alle istruzioni e ai dati.

- **Vantaggi:** È semplice da progettare, dato che si riduce la complessità e costo condividendo un solo bus
- **Svantaggi:** L'accesso condiviso crea un **collo di bottiglia** in quanto istruzioni e dati competono per la stessa risorsa. Il processore non può leggere un'istruzione e caricare un dato nello stesso istante: deve fare un fetch dell'istruzione e al ciclo successivo leggere il dato. Inoltre questa architettura è inefficiente nei DSP dato che l'elaborazione in tempo reale richiede una velocità di accesso che questa architettura non può garantire.

È il modello architetturale standard per i processori general-purpose, con i componenti principali che comprendono la CPU (dentro alla quale c'è la Control Unit e la ALU) per elaborare i dati e coordinare i trasferimenti, la Memoria che è unica contendendo sia le istruzioni sia i dati, i Bus uno per verso per i collegamenti.



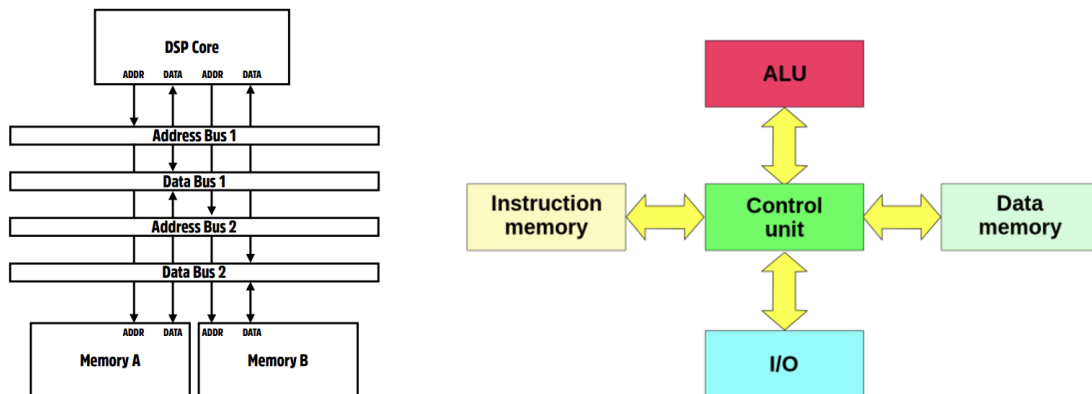
## Harvard Architecture

L'architettura Harvard separa fisicamente le memorie per dati e istruzioni, ognuna con il proprio **bus dedicato**. Questo approccio elimina il collo di bottiglia dell'architettura di Von Neumann, permettendo accessi paralleli.

- **Vantaggi:** L'accesso simultaneo permette al processore di leggere istruzioni e dati contemporaneamente, aumentando la velocità di esecuzione. Migliora l'efficienza diventando adatta per applicazioni in tempo reale.
- **Svantaggi:** Abbiamo un **costo maggiore** dell'Hardware aggiungendo bus e memorie separate.

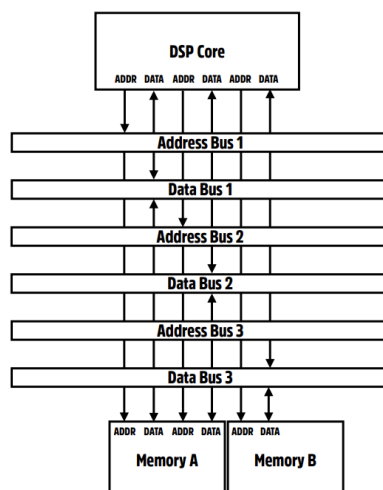
L'architettura si basa sulla separazione fisica delle risorse di memoria per massimizzare il parallelismo. A guidare il sistema è la control unit che si occupa di gestire i segnali di controllo doppi (dice all'Inst. Mem. di leggere la prossima istruzione e contemporaneamente dire alla Data Mem. di leggere/scrivere un dato). Infatti, la

Instruction Memory contiene la sequenza di istruzioni per l'unità di controllo, mentre la Data Memory, memorizza i dati necessari per l'elaborazione. La ALU esegue queste operazioni aritmetiche/logiche e l'I/O è il canale di Input/Output.



## Harvard Architecture - Triple Data Bus/Dual-port

Questa variante avanzata prevede l'uso di **memorie dual- port**, che consentono accessi simultanei alla stessa memoria, aumentando significativamente la velocità. Questa configurazione è particolarmente utile nei DSP, dove è frequente dover operare su più dati contemporaneamente. Utilizziamo quindi **un bus per le istruzioni** (per la lettura delle istruzioni) e **due bus per i dati** (Per la lettura simultanea di due operandi e la scrittura del risultato).



## Addressing Modes

Il termine *indirizzamento* si riferisce alla modalità con cui un microprocessore accede alla memoria, utilizzando indirizzi specifici per localizzare i dati. Nei Digital Signal

Processors (DSP), sono presenti modalità di indirizzamento che non si trovano comunemente nei microprocessori convenzionali. Questa caratteristica è dovuta ai requisiti software di alto livello richiesti dagli algoritmi di elaborazione del segnale digitale, che necessitano di modalità di accesso alla memoria altamente ottimizzate per garantire efficienza e velocità.

## DSP Addressing Examples

- **INDIRIZZAMENTO IMMEDIATO:** *Utilizza una costante specificata direttamente all'interno dell'istruzione. Questo metodo consente di accedere immediatamente a un valore specifico senza dover interagire con la memoria esterna o i registri. È Una modalità comune anche nei microprocessori convenzionali.*
- **INDIRIZZAMENTO INDIRETTO:** *L'indirizzo di memoria desiderato è contenuto in un registro. L'operazione specifica quale registro leggere per ottenere il puntatore ai dati. Questa modalità è utile per accedere a dati memorizzati in modo dinamico e è anch'essa presente nei microprocessori standard.*
- **INDIRIZZAMENTO PRE-POST INCREMENTO:** *Progettato per facilitare l'accesso a sequenze di dati, come accade in algoritmi che elaborano array o stream di valori consecutivi. Con il pre-incremento, il registro dell'indirizzo viene incrementato prima dell'accesso alla memoria, mentre con il post-incremento l'incremento avviene dopo l'accesso. Questa funzionalità automatizza il passaggio alla variabile successiva o precedente, evitando di doverlo specificare esplicitamente in ogni istruzione.*
- **INDIRIZZAMENTO CIRCOLARE:** *Particolarmente utile per la gestione di buffer circolari, comunemente utilizzati negli algoritmi di elaborazione del segnale, come le code FIFO (First-In-First-Out). Un registro puntatore, mantiene il puntatore all'interno di un intervallo definito, così quando il buffer raggiunge il suo limite, il puntatore ritorna automaticamente all'inizio, consentendo un utilizzo continuo senza sovrascrivere dati non ancora elaborati o fare costosi controlli di cicli ("if(sono alla fine) → torna all'inizio").*
- **INDIRIZZAMENTO CON INVERSIONE DI BIT:** *Progettato per ottimizzare l'implementazione della Trasformata Rapida di Fourier (FFT). I dati vengono riorganizzati in base all'inversione dei bit del loro indirizzo( es. l'indirizzo **001** diventa **100**), facilitando l'accesso ai dati nella sequenza richiesta dall'algoritmo FFT. Questo approccio migliora l'efficienza evitando operazioni di riordino dispendiose durante l'elaborazione.*

## Instruction Set

Il set di istruzioni dei processori DSP (Digital Signal Processor) include sia istruzioni standard, comuni ai microprocessori convenzionali, sia istruzioni non standard progettate per ottimizzare l'elaborazione di segnali e algoritmi complessi. Quest'ultime sono macro-operazioni hardware, non semplici comandi matematici:

**MAC** (Multiply and Accumulate): Esegue la moltiplicazione di due numeri e accumula il risultato in un registro, rendendo possibile la somma di prodotti in un unico ciclo. Tale funzionalità è essenziale per algoritmi DSP come filtri digitali e trasformate rapide di Fourier (FFT).

**Block Floating Point:** Permette di gestire blocchi di memoria con una maggiore precisione, con una versione ibrida tra fixed point e float, invece di avere un esponente per ogni numero (Floating Point IEEE 754), si assegna un unico esponente comune a un intero blocco di dati. Questo approccio è utile per operazioni che richiedono una manipolazione accurata dei numeri in virgola mobile.

**Hardware Loops:** Consentono l'esecuzione di cicli direttamente in hardware, senza necessità di istruzioni software per l'incremento del contatore o il controllo della condizione di fine ciclo. Questa caratteristica accelera significativamente le operazioni iterative tramite i registri speciali dei DSP che contano automaticamente i giri in parallelo.

**Nested Hardware Loops:** Implementano cicli annidati direttamente in hardware. La coalescenza hardware consente di eseguire cicli all'interno di altri cicli senza il supporto esplicito di software, migliorando l'efficienza.

**Data Block Movement:** Facilita il trasferimento rapido di blocchi di dati da una posizione di memoria a un'altra, riducendo il tempo richiesto per spostamenti di grandi volumi di dati.

## Altre Features Hardware

Oltre al set di istruzioni, i DSP includono caratteristiche hardware aggiuntive che ampliano le loro funzionalità e li rendono adatti a un'ampia varietà di applicazioni:

**Porte di comunicazione:** le porte seriali e parallele consentono l'integrazione diretta con altri dispositivi o sistemi per lo scambio di dati.

**Timer e Contatori:** Scandiscono la frequenza di campionamento, utilizzati per gestire eventi temporali e sincronizzazioni.

**Convertitori A/D e D/A:** Spesso integrati direttamente nel chip. Permettono la conversione tra segnali analogici e digitali, eliminando la necessità di componenti esterni.

**Gestione degli Interrupt:** È il sistema di "priorità" del processore. Se arriva un dato urgente, l'hardware ferma momentaneamente quello che sta facendo per gestire l'evento immediato, riducendo i tempi di latenza nelle risposte.

**DMA (Direct Memory Access):** Consente il trasferimento dei dati tra memoria e periferiche senza coinvolgere la CPU, liberando risorse per altre operazioni.

**Power Management:** Il DSP può andare in Sleep Mode (spegnendosi quando non ha dati) Idle Frequency Control (spegnendo solo le parti dell'hardware non necessarie) oppure Low Voltage Operation (operando a bassa tensione per ridurre il consumo energetico).

## ES8

# MIPS Architecture

## The Micro Processor Quantitative Design

Il Micro Processor Quantitative Design è un approccio sistematico all'analisi, progettazione e ottimizzazione dei microprocessori basato su principi quantitativi. Questo metodo si concentra sulla misurazione e valutazione delle prestazioni, dell'efficienza e dei compromessi di progettazione nei microprocessori, utilizzando metriche come il numero di cicli di clock per istruzione, la latenza e il throughput.

L'obiettivo è quello di massimizzare l'efficienza del processore attraverso l'analisi dettagliata delle istruzioni, della pipeline e delle unità funzionali. Viene posta particolare attenzione al design del datapath e delle linee di controllo, considerando configurazioni monolitiche, pipeline o multi-ciclo. Questo consente di bilanciare le prestazioni con i costi di implementazione in termini di risorse hardware e consumo energetico.

## Speedup

Lo speedup quantifica il miglioramento delle prestazioni di un sistema grazie a una nuova soluzione hardware o architetturale. È definito come  $S$  il rapporto tra il tempo di esecuzione (di uno specifico codice) senza la nuova soluzione ( $t_{without}$ ) e il tempo di esecuzione con la nuova soluzione ( $t_{with}$ )

$$S = t_{without} / t_{with}$$

Affinché la nuova soluzione sia vantaggiosa, deve risultare:

$$t_{with} < t_{without} \Rightarrow S > 1$$

## High Level Requirements

Sono i requirements del software che servono a capire se una scelta architetturale è vantaggiosa o meno, questi requirements sono chiamati bench-marks. Le scelte devono essere qualificate in base all'impatto della loro adozione in termini di impatto benefico (possibile) e in base al rapporto tra costo e prestazioni. La qualità della progettazione è determinata mettendo alla prova l'architettura sull'esecuzione di software benchmark standard.

## The Amdahl Law

La **legge di Amdahl** definisce il limite teorico massimo di miglioramento di un sistema, anche quando viene ottimizzata solo una parte del processo, infatti il miglioramento totale è vincolato dalla parte di programma che *non* può usare quella componente. L'equazione è:

$$S = 1 / [(1 - \text{frac\_enhanced}) + (\text{frac\_enhanced}/S\_enhanced)]$$

Dove:

- *frac\_enhanced*: è la frazione di codice ottimizzata,
- *S\_enhancement*: è lo speed-up della parte ottimizzata.

Nel caso in cui la *frac\_enhanced* = 1, cioè massima e il termine di somma "(1 – *frac\_enhanced*)" va a 0, rimane comunque la parte fratta, dove rimane  $S = 1 / (1/S\_enh)$ , quindi:  $S = S\_enh$ .

## Examples

### ESEMPIO 1

Una CPU spende 40% del suo tempo in elaborazione dei numeri e il 60% nell'aspettare i dati di I/O. Quanto si migliora la situazione adottando una CPU che è 10 volte più veloce?

Dati:

Frac\_enhanced = 0,4

S\_enhanced = 10.0

Allora:  $S = 1 / [(1 - 0.4) + (0.4/10)] \approx 1.56$

## ESEMPIO 2

Il calcolo della radice quadrata di un numero in virgola mobile (FPSR) rappresenta il 20% dell'esecuzione di un benchmark grafico. Un possibile miglioramento è l'introduzione di un acceleratore hardware, che migliora la situazione di un fattore 10. La seconda opzione è migliorare tutte le altre istruzioni in virgola mobile (FP), che rappresentano il 50% dell'esecuzione, di un fattore 1,6. Quale soluzione è preferibile?

Questo esempio mostra come i limiti del miglioramento dipendano dalla frazione non ottimizzata. Una CPU spende il 20% del tempo eseguendo radici quadrate ( $f_{enanced} = 0, 2$ ) e il 50% del tempo in altre istruzioni a virgola mobile ( $f_{enanced} = 0, 5$ ).

- Miglioramento radici quadrate ( $S_{enanced} = 10$ ) :

$$S = 1 / [(1 - 0.2) + (0.2/10)] \approx 1.22$$

- Miglioramento altre soluzioni ( $S_{enanced} = 1, 6$ ) :

$$S = 1 / [(1 - 0.5) + (0.5/1.6)] \approx 1.23$$

In questo caso, migliorare il blocco più grande è leggermente più vantaggioso.

## The Performance Equation

**L'Equazione delle Prestazioni (Iron Law)** Il tempo di CPU è l'unica metrica affidabile per valutare le prestazioni. Questa equazione misura il tempo totale di esecuzione della CPU:

$$CPU\_time = IC * CPI * T_{cycle}$$

Viene calcolato come il prodotto di tre fattori:

- **IC:** il numero di istruzioni, rappresenta il volume di lavoro software da svolgere
- **CPI:** il numero medio di cicli di clock per istruzione, misura l'efficienza architetturale, indicando quanti cicli la CPU "spende" mediamente per completare un'istruzione (CPI ideale = 1, ma reale > 1 a causa di stalli di memoria o conflitti)
- **Clock Cycle Time:** la durata del ciclo di clock

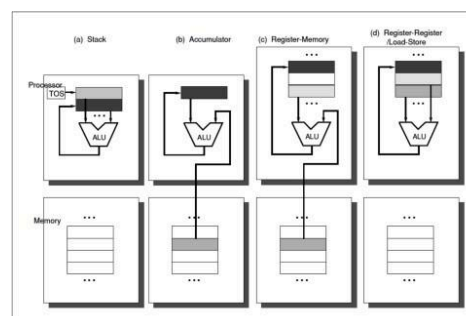
**Matrice di Influenza (Who is Who).** Ogni fattore dell'equazione è influenzato da specifici aspetti del design:

- **IC:** Dipende dall'**ISA** (vocabolario dei comandi della CPU) e dalla capacità del **Compilatore** di tradurre il codice in modo conciso. Riducendo IC, si migliora il compilatore.
- **CPI:** Dipende dall'**Organizzazione** interna (Pipeline, Cache) e dall'ISA.
- **Clock Time:** Dipende dalla **Tecnologia Hardware** e dall'Organizzazione. Diminuire il  $T_{cycle}$  vuol dire migliorare la tecnologia hardware.

## Architectural Types Evolution

L'evoluzione delle architetture dei microprocessori ha seguito un percorso scandito dai progressi tecnologici e dalla ricerca di prestazioni sempre maggiori. Negli anni '80, le architetture predominanti erano quelle basate sull'accumulatore. Ciò era dovuto al fatto che le tecnologie integrate erano agli albori e non permettevano soluzioni più complesse. Con il progredire della tecnologia, emersero architetture più sofisticate, note come **CISC (Complex Instruction Set Computer)**, progettate per eseguire operazioni complesse in un minor numero di istruzioni.

Tuttavia, agli inizi degli anni '90 si dimostrò che mantenere l'hardware semplice e veloce, approccio noto come **RISC (Reduced Instruction Set Computer)**, garantiva migliori prestazioni. Fu in questo contesto che iniziarono a diffondersi le cosiddette **architetture register-to-register**. Queste architetture sfruttavano appieno la località dei dati, introducendo l'uso delle memorie cache. Tale innovazione ridusse drasticamente gli accessi alla memoria centrale, incrementando significativamente le prestazioni dei processori.



| Stack  | Accumulator | Register<br>(register-memory) | Register<br>(load-store) |
|--------|-------------|-------------------------------|--------------------------|
| Push A | Load A      | Load R1, A                    | Load R1, A               |
| Push B | Add B       | Add R3, R1, B                 | Load R2, B               |
| Add    | Store C     | Store R3, C                   | Add R3, R1, R2           |
| Pop C  |             |                               | Store R3, C              |

Le architetture di microprocessore si dividono in diversi tipi:

- **Stack:** i dati vengono memorizzati in una pila. Operazioni come "push" o "pop" accedono alla cima della pila.
- **Accumulator:** un operando è contenuto in un registro dedicato, mentre l'altro è memorizzato in memoria.
- **Register-Memory:** gli operandi possono essere registri o locazioni di memoria.
- **Register-Register:** entrambi gli operandi risiedono nei registri, riducendo l'accesso alla memoria e aumentando la velocità.

Le architetture **RISC** (come MIPS) favoriscono soluzioni "Register-Register" o "Stack" per la loro semplicità e velocità

## Architectural Choices

Si può definire l'ordine con cui i byte che compongono una "word" vengono disposti nella memoria lineare. Le due filosofie opposte sono:

- **Big-Endian:** Il byte più significativo (MSB, Most Significant Bit) va all'indirizzo di memoria più basso. (da sinistra a destra).
- **Little-Endian:** Il byte meno significativo (LSB, Least Significant Bit) va all'indirizzo di memoria più basso.

## Memory Modes and Alignment

La velocità di accesso alla memoria dipende dalla dimensione dei dati e dall'allineamento della memoria. Un allineamento corretto garantisce un accesso rapido ai dati, evitando sprechi di risorse hardware.

| Width of object:      | Value of 3 low order bits of byte address: |         |            |         |            |         |            |         |
|-----------------------|--------------------------------------------|---------|------------|---------|------------|---------|------------|---------|
|                       | 0                                          | 1       | 2          | 3       | 4          | 5       | 6          | 7       |
| 1 Byte (Byte)         | Aligned                                    | Aligned | Aligned    | Aligned | Aligned    | Aligned | Aligned    | Aligned |
| 2 Bytes (Half word)   | Aligned                                    |         | Aligned    |         | Aligned    |         | Aligned    |         |
| 2 Bytes (Half word)   | Misaligned                                 |         | Misaligned |         | Misaligned |         | Misaligned |         |
| 4 Bytes (Word)        | Aligned                                    |         |            |         | Aligned    |         |            |         |
| 4 Bytes (Word)        | Misaligned                                 |         |            |         | Misaligned |         |            |         |
| 4 Bytes (Word)        | Misaligned                                 |         | Misaligned |         | Misaligned |         | Misaligned |         |
| 4 Bytes (Word)        | Misaligned                                 |         | Misaligned |         | Misaligned |         | Misaligned |         |
| 8 bytes (Double word) | Aligned                                    |         |            |         | Aligned    |         |            |         |
| 8 bytes (Double word) | Misaligned                                 |         |            |         | Misaligned |         |            |         |
| 8 bytes (Double word) | Misaligned                                 |         | Misaligned |         | Misaligned |         | Misaligned |         |
| 8 bytes (Double word) | Misaligned                                 |         | Misaligned |         | Misaligned |         | Misaligned |         |
| 8 bytes (Double word) | Misaligned                                 |         | Misaligned |         | Misaligned |         | Misaligned |         |
| 8 bytes (Double word) | Misaligned                                 |         | Misaligned |         | Misaligned |         | Misaligned |         |
| 8 bytes (Double word) | Misaligned                                 |         | Misaligned |         | Misaligned |         | Misaligned |         |
| 8 bytes (Double word) | Misaligned                                 |         | Misaligned |         | Misaligned |         | Misaligned |         |

Come mostra la tabella nell'immagine, un dato è "allineato" se il suo indirizzo di memoria è un multiplo della sua dimensione:

- **Byte (1 byte):** È sempre allineato (1 è multiplo di qualsiasi intero).
- **Half Word (2 byte):** È allineata solo se l'indirizzo è pari (termina con bit 0).
- **Word (4 byte):** È allineata solo se l'indirizzo è divisibile per 4 (termina con bit 00).
- **Double Word (8 byte):** È allineata solo se l'indirizzo è divisibile per 8 (termina con bit 000).

Se un dato da 4 byte viene messo all'indirizzo `0x01` (Misaligned), il processore deve fare uno sforzo extra per recuperarlo. In alcuni casi, è possibile eliminare l'allineamento per risparmiare risorse, ma ciò potrebbe influire negativamente sulle prestazioni complessive.

## MIPS Addressing Modes

**1. Register (Indirizzamento a Registro):** Usato quando il valore su cui lavorare è già stato caricato in un registro della CPU. È la modalità più veloce perché non richiede accesso alla memoria RAM.

- *Esempio:* `Add R4, R3` (Somma il contenuto di R3 a R4).

**2. Immediate (Indirizzamento Immediato):** L'operando è una costante numerica inclusa direttamente nell'istruzione stessa. Usato per definire costanti o inizializzare variabili (es. contatori o flag).

- *Esempio:* `Add R4, #3` (Aggiunge il numero 3 a R4).

**3. Displacement (Spiazzamento o Base-Offset):** Si calcola l'indirizzo di memoria sommando il contenuto di un registro con un valore costante (offset). Fondamentale per accedere a variabili locali nello stack o struttura dati, dove `R1` punta all'inizio della struttura e `100` è la posizione del campo specifico.

- *Esempio:* `Add R4, 100(R1)` (Accede alla memoria all'indirizzo `R1 + 100`).

**4. Register Indirect (Indirizzamento Indiretto a Registro):** Il registro contiene non il dato, ma l'indirizzo di memoria dove si trova il dato. Usato per l'accesso tramite puntatori.

- *Esempio:* `Add R4, (R1)` (Vai all'indirizzo contenuto in R1, prendi il dato e sommalo a R4).

**5. Indexed (Indirizzamento Indicizzato):** L'indirizzo finale è la somma del contenuto di due registri: uno funge da base (base array) e l'altro da indice. Utile

per l'accesso agli array, dove **R1** è l'indirizzo di partenza e **R2** è l'indice variabile dell'elemento.

- *Esempio:* **Add R3, (R1 + R2)** (Indirizzo = contenuto di R1 + contenuto di R2).

**6. Direct or Absolute (Indirizzamento Diretto o Assoluto):** L'istruzione contiene direttamente l'indirizzo di memoria completo dove trovare il dato. Usato per accedere a variabili statiche o globali con posizione fissa.

- *Esempio:* **Add R1, (1001)** (Vai alla cella di memoria 1001).

**7. Memory Indirect (Indirizzamento Indiretto a Memoria):** Il registro punta a una cella di memoria che contiene l'indirizzo del dato finale. Concetto di **puntatore a puntatore**.

- *Esempio:* **Add R1, @(R3)** (R3 punta a **Indirizzo A**, che contiene **Indirizzo B**. Il dato è in **Indirizzo B**).

**8. Autoincrement (Auto-incremento):** Simile al *Register Indirect*, ma dopo aver acceduto alla memoria, il registro viene automaticamente incrementato della dimensione. Utile nei loop per scorrere array. Dopo ogni lettura, il puntatore si sposta all'elemento successivo.

- *Esempio:* **Add R1, (R2)+**.

**9. Autodecrement (Auto-decremento):** Il registro viene decrementato della dimensione dell'elemento *prima* di accedere alla memoria.

- *Esempio:* **Add R1, -(R2)**.

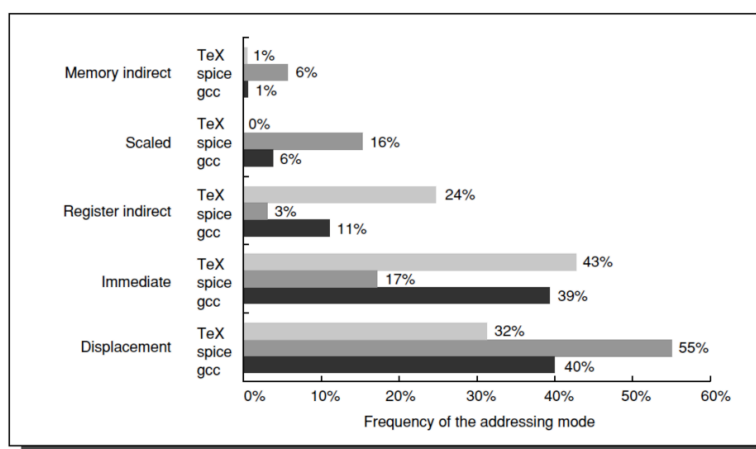
**10. Scaled (Indirizzamento Scalato)**

- **Definizione:** L'indirizzo è calcolato come: **Base + Offset + (Indice \* Scala)**. La "scala" è la dimensione del dato.
  - *Esempio:* **Add R1, 100(R2)[R3]**.

| Addressing mode    | Example instruction  | Meaning                                                                              | When used                                                                                                                             |
|--------------------|----------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Register           | Add R4, R3           | $Regs[R4] \leftarrow Regs[R4] + Regs[R3]$                                            | When a value is in a register.                                                                                                        |
| Immediate          | Add R4, #3           | $Regs[R4] \leftarrow Regs[R4] + 3$                                                   | For constants.                                                                                                                        |
| Displacement       | Add R4, 100(R1)      | $Regs[R4] \leftarrow Regs[R4] + Mem[100 + Regs[R1]]$                                 | Accessing local variables (+ simulates register indirect, direct addressing modes)                                                    |
| Register indirect  | Add R4, (R1)         | $Regs[R4] \leftarrow Regs[R4] + Mem[Regs[R1]]$                                       | Accessing using a pointer or a computed address.                                                                                      |
| Indexed            | Add R3, (R1 + R2)    | $Regs[R3] \leftarrow Regs[R3] + Mem[Regs[R1] + Regs[R2]]$                            | Sometimes useful in array addressing: R1 = base of array; R2 = index amount.                                                          |
| Direct or absolute | Add R1, 1001         | $Regs[R1] \leftarrow Regs[R1] + Mem[1001]$                                           | Sometimes useful for accessing static data; address constant may need to be large.                                                    |
| Memory indirect    | Add R1, @ (R3)       | $Regs[R1] \leftarrow Regs[R1] + Mem[Mem[Regs[R3]]]$                                  | If R3 is the address of a pointer p, then mode yields *p.                                                                             |
| Autoincrement      | Add R1, (R2) +       | $Regs[R1] \leftarrow Regs[R1] + Mem[Regs[R2]]$<br>$Regs[R2] \leftarrow Regs[R2] + d$ | Useful for stepping through arrays within a loop. R2 points to start of array; each reference increments R2 by size of an element, d. |
| Autodecrement      | Add R1, -(R2)        | $Regs[R2] \leftarrow Regs[R2] - d$<br>$Regs[R1] \leftarrow Regs[R1] + Mem[Regs[R2]]$ | Same use as autoincrement. Autodecrement/increment can also act as push/pop to implement a stack.                                     |
| Scaled             | Add R1, 100(R2) [R3] | $Regs[R1] \leftarrow Regs[R1] + Mem[100 + Regs[R2] + Regs[R3] * d]$                  | Used to index arrays. May be applied to any indexed addressing mode in some computers.                                                |

## Quantitative Design of Addressing Modes

L'approccio quantitativo misura la frequenza d'uso dei vari modi di indirizzamento. I dati evidenziano che **Displacement** (accesso a variabili locali/strutture) e **Immediate** (uso di costanti) rappresentando oltre l'80% degli accessi. Di conseguenza, le architetture moderne (RISC) sono ottimizzate per eseguire queste operazioni nel minor tempo possibile, spesso codificandole direttamente nel set di istruzioni base. Modi complessi come **Memory Indirect** o **Scaled**, pur essendo potenti, hanno frequenze di utilizzo trascurabili (spesso <1%). Secondo la filosofia RISC, supportarle è uno spreco di risorse è quindi preferibile eliminarli e lasciare al compilatore il compito di emularli tramite sequenze di istruzioni più semplici, senza impattare le prestazioni globali.



Nella maggior parte delle funzioni il **96%** del carico di lavoro totale è svolto da sole **10 istruzioni**. Le operazioni più frequenti sono il trasferimento dati (*Load* 22%, *Store* 12%) e il controllo del flusso (*Conditional Branch* 20%, *Compare* 16%), mentre le operazioni aritmetiche complesse sono rare.

| Rank  | 80x86 instruction      | Integer average<br>(% total executed) |
|-------|------------------------|---------------------------------------|
| 1     | load                   | 22%                                   |
| 2     | conditional branch     | 20%                                   |
| 3     | compare                | 16%                                   |
| 4     | store                  | 12%                                   |
| 5     | add                    | 8%                                    |
| 6     | and                    | 6%                                    |
| 7     | sub                    | 5%                                    |
| 8     | move register-register | 4%                                    |
| 9     | call                   | 1%                                    |
| 10    | return                 | 1%                                    |
| Total |                        | 96%                                   |

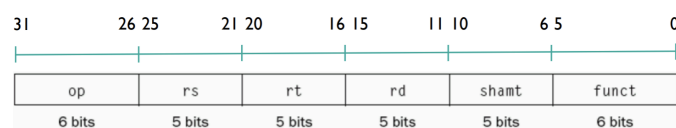
## The Sw/Hw Interface

L'interfaccia tra software e hardware, definita come **Instruction Set Architecture (ISA)**, specifica come il software interagisce con i componenti hardware. Un'architettura come MIPS adotta una struttura fissa delle istruzioni per semplificare la progettazione, evitando la complessità di istruzioni di lunghezza variabile.

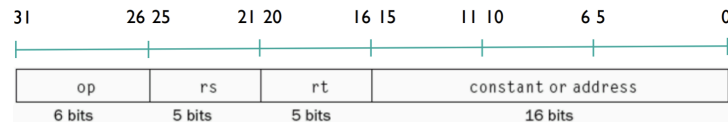
## MIPS Instruction

Il set di istruzioni MIPS, essendo un'architettura a lunghezza fissa (32 bit), suddivide le operazioni in tre formati principali per gestire diverse esigenze:

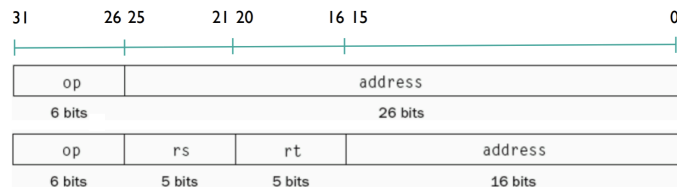
- **Tipo R (Register):** Usato per operazioni puramente aritmetico-logiche che lavorano sui dati presenti nei registri. La sua struttura divide i 32 bit in campi specifici: l'opcode (che è 0 per R-type), i registri sorgente (rs,rt), il registro destinazione (rd), lo shamt (shift amount, per gli shift bit a bit) e il funct (che specifica l'operazione esatta, es. addizione o sottrazione).



- **Tipo I (Immediate):** Usato quando uno degli operandi è un numero costante (immediato) o per calcolare indirizzi di memoria (offset). Sostituisce il terzo registro con un campo a 16 bit per il valore. Questo velocizza l'esecuzione evitando un accesso in memoria o un caricamento extra da registro.



- **Tipo J (Jump):** Dedicato ai salti incondizionati a indirizzi lontani (o salti condizionati tramite Branch). Dedicò la maggior parte dei bit (26) all'indirizzo di destinazione per permettere di spostarsi in punti distanti del programma.

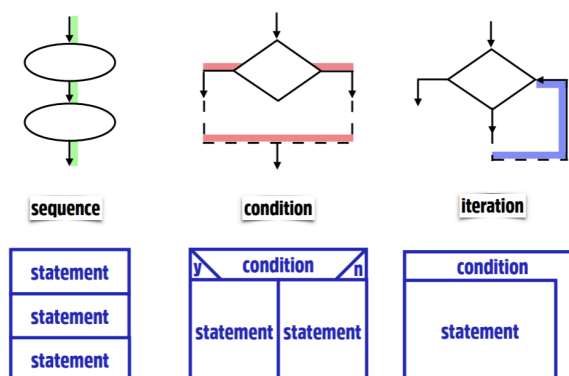


## Bohm- Jacopoi Theorem

Il **Teorema di Böhm-Jacopini** dimostra che qualsiasi algoritmo può essere implementato combinando solo tre strutture fondamentali:

- **Sequenza:** Esecuzione atomica delle istruzioni in ordine lineare.
- **Selezione (Selection):** Permette di eseguire blocchi di codice alternativi basati su una condizione booleana (struttura **IF-THEN-ELSE**).
- **Iterazione (Iteration):** Permette la ripetizione di un blocco di istruzioni (loop) finché una condizione rimane vera.

Questi costrutti sono fondamentali per garantire flessibilità e potenza di calcolo.



## MIPS Instruction Summary

Le istruzioni dell'architettura MIPS si dividono in cinque classi funzionali, ciascuna con un ruolo specifico nella gestione del processore:

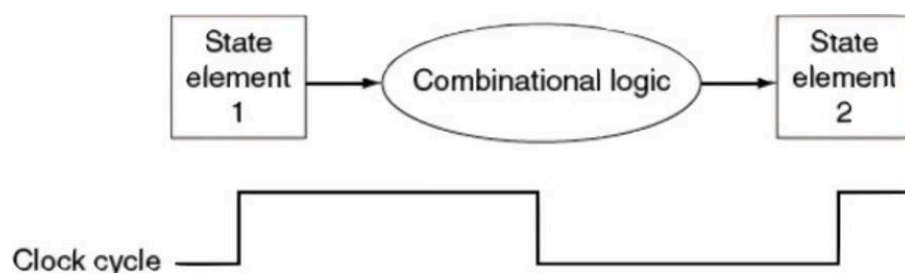
1. **Arithmetic:** Gestisce i calcoli interni, Include operazioni di somma e sottrazione (add, sub) e le versioni con costanti immediate (addi).
2. **Logical:** Opera sui singoli bit (and, or, nor). Include
  - Usa and/andi per isolare bit specifici, mentre or/ori per settare bit a 1. *Nota:* MIPS non ha NOT; quindi si usa una nor con il registro zero (A NOR 0 = NOT A).
  - Shift logici (sll, srl) utili per manipolazioni rapide e moltiplicazioni per potenze di 2.
3. **Data Transfer:** Poiché MIPS è un'architettura *Load/Store*, queste sono le uniche istruzioni che accedono alla RAM. Possiamo caricare/salvare intere word (32-bit: lw/sw), half word (16-bit: lh/sh) o singoli byte (8-bit: lb/sb).
4. **Conditional Branch (I-Type):** Usati per i salti con confronto. Il salto è relativo al Program Counter (PC-Relative Addressing), imposta un registro a 1 o 0 in base a un confronto (<, >) e valuta. Le istruzioni condizionali (beq, bne), cioè i branch, sono di Tipo I (Immediate).
5. **Unconditional Jump (J-Type):** Salta senza condizione, con diversi utilizzi:
  - j: Salto diretto a un indirizzo (etichetta).
  - jal (Jump And Link): Usato per le chiamate a funzione.
  - jr (Jump Register): Salta all'indirizzo contenuto in un registro. Usato per ritornare da una funzione (return) o per implementare switch-case.

| Category           | Instruction             | Example            | Meaning                                           | Comments                            |
|--------------------|-------------------------|--------------------|---------------------------------------------------|-------------------------------------|
| Arithmetic         | add                     | add \$s1,\$s2,\$s3 | $\$s1 = \$s2 + \$s3$                              | Three register operands             |
|                    | subtract                | sub \$s1,\$s2,\$s3 | $\$s1 = \$s2 - \$s3$                              | Three register operands             |
|                    | add immediate           | addi \$s1,\$s2,100 | $\$s1 = \$s2 + 100$                               | Used to add constants               |
| Data transfer      | load word               | lw \$s1,100(\$s2)  | $\$s1 = \text{Memory}[\$s2 + 100]$                | Word from memory to register        |
|                    | store word              | sw \$s1,100(\$s2)  | $\text{Memory}[\$s2 + 100] = \$s1$                | Word from register to memory        |
|                    | load half               | lh \$s1,100(\$s2)  | $\$s1 = \text{Memory}[\$s2 + 100]$                | Halfword memory to register         |
|                    | store half              | sh \$s1,100(\$s2)  | $\text{Memory}[\$s2 + 100] = \$s1$                | Halfword register to memory         |
|                    | load byte               | lb \$s1,100(\$s2)  | $\$s1 = \text{Memory}[\$s2 + 100]$                | Byte from memory to register        |
|                    | store byte              | sb \$s1,100(\$s2)  | $\text{Memory}[\$s2 + 100] = \$s1$                | Byte from register to memory        |
|                    | load upper immed.       | lui \$s1,100       | $\$s1 = 100 * 2^{16}$                             | Loads constant in upper 16 bits     |
| Logical            | and                     | and \$s1,\$s2,\$s3 | $\$s1 = \$s2 \& \$s3$                             | Three reg. operands; bit-by-bit AND |
|                    | or                      | or \$s1,\$s2,\$s3  | $\$s1 = \$s2   \$s3$                              | Three reg. operands; bit-by-bit OR  |
|                    | nor                     | nor \$s1,\$s2,\$s3 | $\$s1 = \sim (\$s2   \$s3)$                       | Three reg. operands; bit-by-bit NOR |
|                    | and immediate           | andi \$s1,\$s2,100 | $\$s1 = \$s2 \& 100$                              | Bit-by-bit AND reg with constant    |
|                    | or immediate            | ori \$s1,\$s2,100  | $\$s1 = \$s2   100$                               | Bit-by-bit OR reg with constant     |
|                    | shift left logical      | sll \$s1,\$s2,10   | $\$s1 = \$s2 \ll 10$                              | Shift left by constant              |
| Conditional branch | shift right logical     | srl \$s1,\$s2,10   | $\$s1 = \$s2 \gg 10$                              | Shift right by constant             |
|                    | branch on equal         | beq \$s1,\$s2,25   | if ( $\$s1 == \$s2$ ) go to PC + 4 + 100          | Equal test; PC-relative branch      |
|                    | branch on not equal     | bne \$s1,\$s2,25   | if ( $\$s1 != \$s2$ ) go to PC + 4 + 100          | Not equal test; PC-relative         |
|                    | set on less than        | slt \$s1,\$s2,\$s3 | if ( $\$s2 < \$s3$ ) $\$s1 = 1$ ; else $\$s1 = 0$ | Compare less than; for beq, bne     |
| Unconditional jump | set less than immediate | slti \$s1,\$s2,100 | if ( $\$s2 < 100$ ) $\$s1 = 1$ ; else $\$s1 = 0$  | Compare less than constant          |
|                    | jump                    | j 2500             | go to 10000                                       | Jump to target address              |
|                    | jump register           | jr \$ra            | go to \$ra                                        | For switch, procedure return        |
|                    | jump and link           | jal 2500           | $\$ra = PC + 4$ ; go to 10000                     | For procedure call                  |

# The MIPS Single-Cycle Architecture

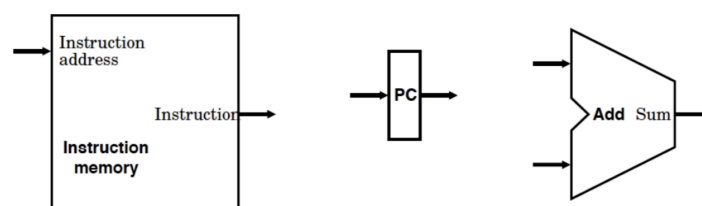
## Definition of Single Cycle Architecture

L'architettura a ciclo singolo MIPS esegue ogni istruzione in un singolo ciclo di clock (il che è veloce o lento dipendentemente dalla durata di tempo del clock). La durata del ciclo è determinata dal percorso critico, ovvero l'istruzione con la latenza più alta, infatti  $T_{clk} \leq T_{max\_instruction}$ . Nonostante l'architettura sia sincrona con il tempo di clock, le azioni eseguite al suo interno sono asincrone. Questo modello garantisce semplicità ma penalizza le istruzioni con latenza inferiore, perché le vincola (bound) alla durata del caso peggiore.



## I Componenti Base

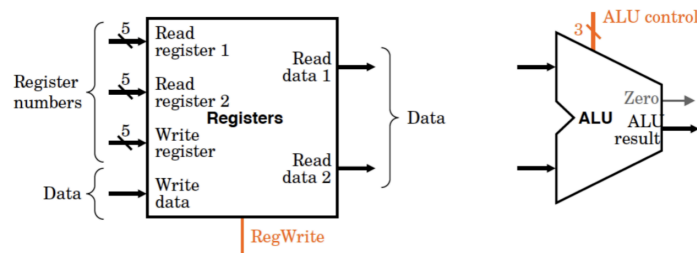
- **Instruction Memory:** È un'unità di memoria che contiene il codice macchina del programma da eseguire. Riceve in input un indirizzo a 32 bit (*Instruction address*) e fornisce in output l'istruzione a 32 bit memorizzata a quell'indirizzo.
- **Program Counter (PC):** È un registro a 32 bit che contiene l'indirizzo dell'istruzione che il processore sta attualmente eseguendo. All'inizio di ogni ciclo di clock, il valore del PC viene inviato alla Instruction Memory per recuperare l'istruzione. Il PC viene aggiornato alla fine di ogni ciclo per puntare all'istruzione successiva.
- **Adder (Sommatore)** È un circuito combinatorio aritmetico. Riceve in input un valore e una variabile o costante.



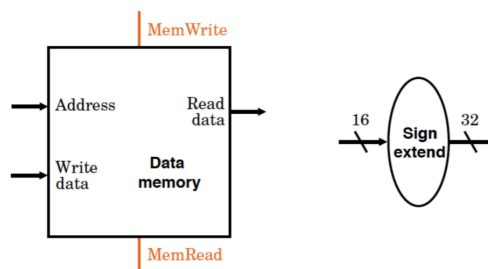
- **Registers (Register File):** È il banco dei registri del processore, contenente i 32 registri generali a 32 bit. L'unità è progettata per permettere la lettura simultanea di due registri (attraverso gli ingressi *Read register 1* e *2*) e la scrittura di uno

(attraverso *Write register* e *Write data*). L'operazione di scrittura è controllata dal segnale **RegWrite**.

- **Arithmetic Logic Unit (ALU):** È l'unità esecutiva responsabile delle operazioni di calcolo. Riceve due operandi a 32 bit e esegue l'operazione specificata dal segnale di controllo *ALU control* (es. somma, sottrazione, AND, OR). Produce due output: il risultato dell'operazione (*ALU result*) e un flag di stato chiamato *Zero*, che assume valore 1 logico se il risultato dell'operazione è zero (utilizzato per valutare le condizioni di salto **beq**).



- **Data Memory Unit:** È l'unità di memoria che contiene i dati del programma (variabili, strutture dati). Viene indirizzata dal risultato della ALU e supporta due operazioni: la lettura di un dato (*Read data*, attivata dal segnale **MemRead**) o la scrittura di un dato (*Write data*, attivata dal segnale **MemWrite**).
- **Sign-extension Unit:** È un'unità hardware che converte un valore immediato a 16 bit (presente in istruzioni come **addi**, **lw**, **sw**) in un valore a 32 bit. L'estensione è necessaria per rendere il dato compatibile con la ALU e registri che operano a 32 bit. L'operazione preserva il valore numerico in complemento a due, replicando il bit del segno (bit 15) nei 16 bit più significativi.

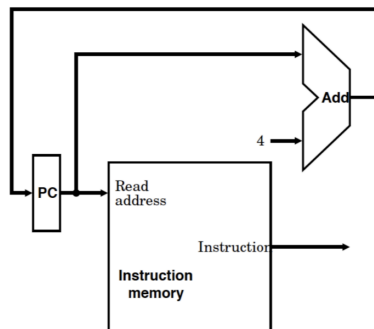


## Le 4 Sezioni

### ➤ Fetch Section:

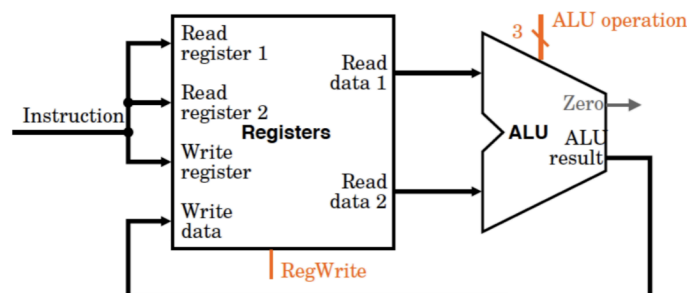
- E' la parte del datapath del processore che si occupa di **prelevare (fetch)**, dalla memoria, l'istruzione da eseguire ad ogni ciclo di clock e di determinare l'indirizzo della prossima istruzione.
- Comprende Program Counter (PC), Instruction Memory e un Adder.
- Funzionamento: Il PC contiene l'indirizzo dell'istruzione corrente. Tale indirizzo viene fornito alla Instruction Memory, che restituisce l'istruzione corrispondente. In parallelo, un adder calcola **PC + 4** per determinare

l'indirizzo della prossima istruzione. Tutti i blocchi, ad eccezione del PC, sono combinatori: in questo modo il prossimo valore del PC viene calcolato nello stesso ciclo di clock e caricato nel PC solo al fronte di clock successivo.



### ➤ Arithmetic Section:

- E' la parte del datapath che si occupa di **eseguire le operazioni aritmetiche e logiche** richieste dalle istruzioni del programma.
- Comprende i blocchi di Registers e di ALU.
- Funzionamento: Il "**Registers**" per la lettura riceve in input dei registri (*Read register 1* e *2*) e fornisce in output i dati contenuti (*Read data 1* e *2*).. Per la scrittura riceve l'indice del registro di destinazione (*Write register*) e il dato da salvare (*Write data*). La scrittura avviene solo se il segnale di controllo **RegWrite** è attivo (1 logicamente). La "**ALU**" prende i due valori a 32 bit forniti dai registri, esegue l'operazione selezionata dal segnale *ALU operation* e genera il *risultato* del calcolo e il flag **Zero** (usato per i salti condizionali).

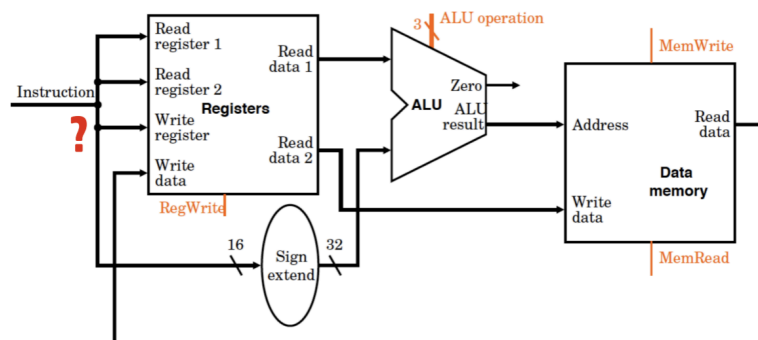


### ➤ Data Memory Access Section: ...

- La **Data Memory Access Section** è la parte del datapath che si occupa di **accedere alla memoria dati**, cioè di leggere o scrivere dati durante l'esecuzione delle istruzioni di tipo load e store.
- Comprende i blocchi della Arithmetic Section (Registers e ALU) e in più aggiunge i blocchi di Sign Extend e Data Memory.
- Funzionamento:
  - Seguendo ciò che abbiamo visto nella Arithmetic Section, in parallelo il Sign-extension Unit riceve i 16 bit meno significativi dell'istruzione (l'offset o costante) e li converte in un numero a 32 bit preservando il segno, rendendoli compatibili per essere sommati dalla ALU.
  - La ALU riceve il "Base Address" dal *Read data 1* dei Registri e l'offset esteso a 32 bit dalla Sign-extension unit, andando ad

eseguire una somma (**Base + Offset**) per determinare l'indirizzo effettivo di memoria a cui accedere.

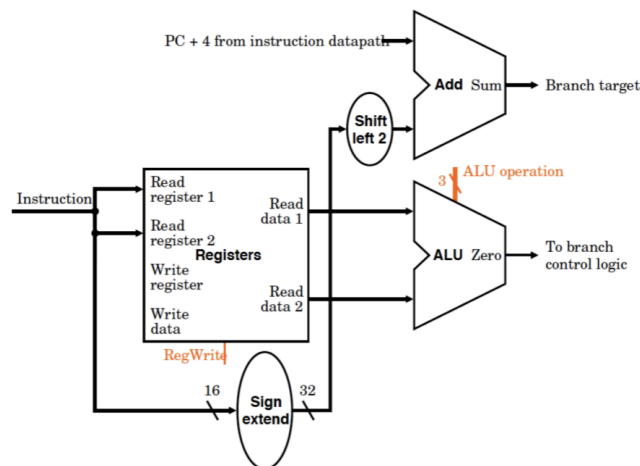
- A questo punto la Data Memory che riceve il risultato della ALU (*ALU result* = che indica *dove* leggere o scrivere) e il valore da scrivere (*Write data*) direttamente dal *Read data 2* del banco registri, esegue un controllo:
  - Se **MemRead** è attivo (istruzione **lw**), il dato viene letto e messo in output.
  - Se **MemWrite** è attivo (istruzione **sw**), il dato presente all'ingresso viene scritto nella cella di memoria.



#### ➤ Conditional Branch Section:

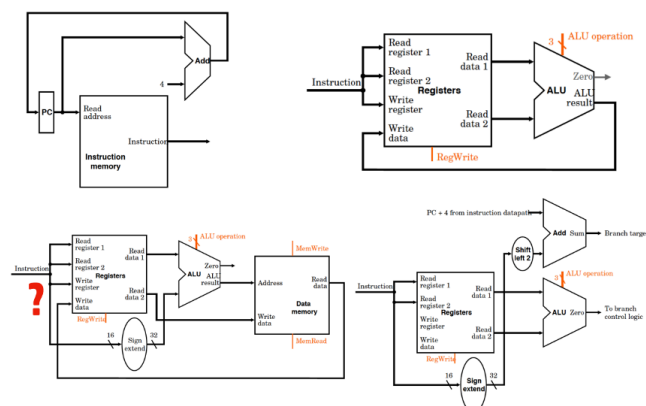
- La **Conditional Branch Section** è la parte del datapath che si occupa di **valutare le condizioni di salto** e di decidere il prossimo valore del Program Counter (PC) nel caso di istruzioni di tipo branch.
- Comprende i blocchi della Arithmetic Section (Registers e ALU) e in più aggiunge una seconda ALU, un blocco di Sign Extend e uno di Shift Left di 2.
- Funzionamento:
  - Dai Registers vengono letti i due registri da confrontare (*Read data 1* e 2). A differenza delle istruzioni aritmetiche, qui non c'è scrittura (RegWrite è 0), servono solo i valori per il confronto.
  - A questo punto la ALU esegue una sottrazione tra i due operandi. Ci interessa solo il flag Zero, non il risultato numerico. Se **(Registro 1 - Registro 2) = 0**, significa che i numeri sono uguali. Il flag Zero diventa 1 e segnala alla logica di controllo che il salto deve essere effettuato ("Taken").
  - Il Sign-extension Unit trasforma l'istruzione contenente un offset di salto a 16 bit a 32 bit (mantenendo il segno) per poter essere sommato all'indirizzo del Program Counter (che è a 32 bit).
  - Allora lo Shift Left 2, prende l'offset esteso a 32 bit e lo "shifta" a sinistra di 2 posizioni. (Perché 1 istruzione = 4 byte, quindi bisogna moltiplicare l'offset per 4).
  - Infine, l'Adder (Sommatore del Branch Target) calcola l'indirizzo di destinazione finale con la formula: **Branch Target = (PC + 4) + (Offset \* 4)**. Somma il **PC + 4** (l'indirizzo della prossima istruzione, già calcolato nella fase di fetch) con l'offset "shiftato". Il

risultato è l'indirizzo dove il programma andrà se il confronto nell'ALU risulta vero.



## How to Integrate Them?

L'integrazione delle quattro sezioni (Fetch, Arithmetic, Memory, Branch) in un unico Datapath avviene attraverso il principio della condivisione delle risorse (Resource Sharing). Come hai notato, sarebbe inefficiente avere tre ALU separate per ogni tipo di operazione. Invece, si utilizza **una singola ALU principale** per gestire sia i calcoli aritmetici (R-Type), sia il calcolo degli indirizzi di memoria (Load/Store), sia i confronti per i salti (Branch). Per gestire questo traffico condiviso, si introducono i **Multiplexer (MUX)**: dispositivi di selezione che, guidati dai segnali di controllo, decidono istante per istante quale dato inviare alla ALU o scrivere nei registri.



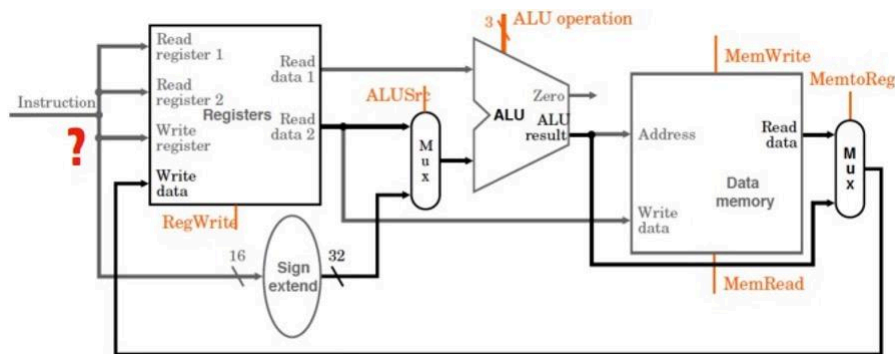
## FIRST INTEGRATION - ARITHMETIC + DMA

In questa fase uniamo la capacità di calcolo (ALU) con l'accesso ai dati (Load/Store) e lo facciamo aggiungendo 3 strutture:

- **MUX RegDst** (Register destination) a sostituire il punto interrogativo. Dato che il processore deve sempre scrivere il risultato finale in un registro l'integrazione

richiede l'uso di multiplexer per gestire le differenze tra i formati di istruzione R-Type e I-Type. Poiché nelle **R-Type**, il registro di destinazione (rd) è nei bit **15-11**, mentre nelle **I-Type**, il registro di destinazione (rt) è nei bit **20-16**. Il MUX seleziona i bit corretti da inviare all'ingresso *Write Register* del Register File in base al tipo di istruzione.

- **MUX ALUSrc** (Source): Seleziona il secondo operando della ALU. Se 0 (R-Type): Sceglie *Read Data 2* (valore registro rt). Quando 1 (Load/Store): Sceglie l'uscita del *Sign Extend* (l'offset costante a 16 bit esteso a 32).
- **MUX MemtoReg**: Seleziona la fonte del dato da scrivere nel registro finale (Write Back). Se 0 (R-Type): Il dato proviene dall'uscita della ALU (*ALU Result*). Se 1 (Load): Il dato proviene dalla Memoria Dati (*Read Data*).

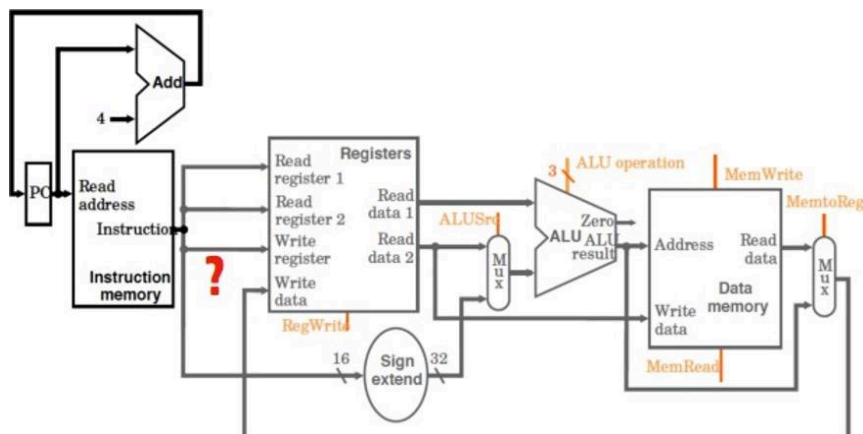


## SECOND INTEGRATION - FETCH + ARITHMETIC + DMA

Collegiamo la Fetch Section al datapath. L'istruzione a 32 bit, appena letta dalla Instruction Memory all'indirizzo puntato dal PC, viene **divisa in fasci di cavi** (bit slicing). Questi cavi portano parti diverse dell'istruzione direttamente agli ingressi del Register File e del Sign Extend:

- **Instr[25-21] (rs)**: Va sempre all'ingresso *Read Register 1*.
- **Instr[20-16] (rt)**: Va all'ingresso *Read Register 2* (e potenzialmente all'ingresso del MUX RegDst).
- **Instr[15-0] (Immediate)**: Va all'unità *Sign Extend* per diventare un numero a 32 bit (usato in *addi*, *lw*, *sw*).

Il "?" è ancora lì per ricordarci l'implementazione del MUX *RegDst*, che dobbiamo fare, per poter distribuire le istruzioni. I due Adder sono in parallelo.

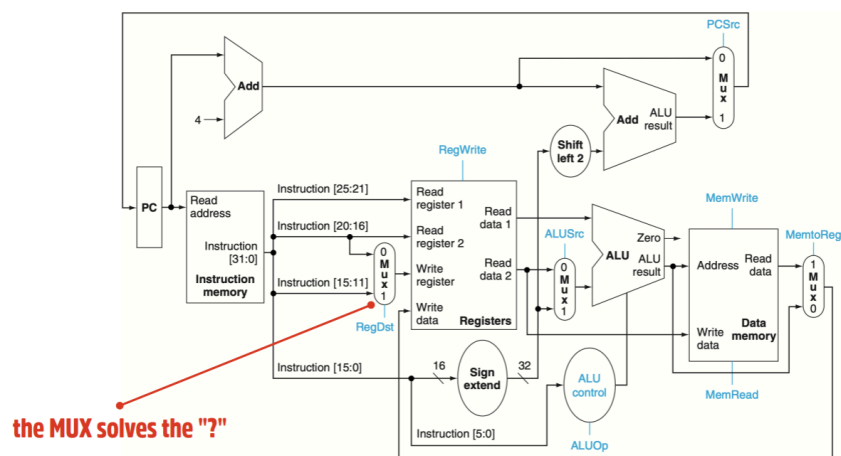


### THIRD INTEGRATION - FETCH + ARITHMETIC + DMA + BRANCH

Collegiamo la Conditional Branch Section al datapath. Risolviamo definitivamente il problema del “?” con un multiplexer che se 0 seleziona i bit 20-16 (rt), se 1 seleziona i bit 15-11 (rd). Aggiungiamo poi lo Shift Left 2 serve nello schema del Branch perché l'offset nell'istruzione conta word (da 4 byte), ma il PC lavora con indirizzi di byte. Per convertire, dobbiamo moltiplicare per 4 (che in binario equivale a spostare i bit a sinistra di 2 posizioni). Un MUX pilotato da PCSrc permette di decidere se saltare o meno. C'è una porta AND che combina il segnale *Branch* (la Control Unit dice "questa è un'istruzione di salto") e il segnale *Zero* della ALU (il calcolo dice "i numeri sono uguali"). Solo se entrambi sono 1, il MUX sceglie il nuovo indirizzo di salto.

Serve la ALU Control perché per le istruzioni di Type-R (es: Add, sub, and, ...) l'Opcode è sempre 0, quindi va a guardare il campo *funct* (ultimi 6 bit, Instruction [5-0]) per decidere l'operazione esatta della ALU.

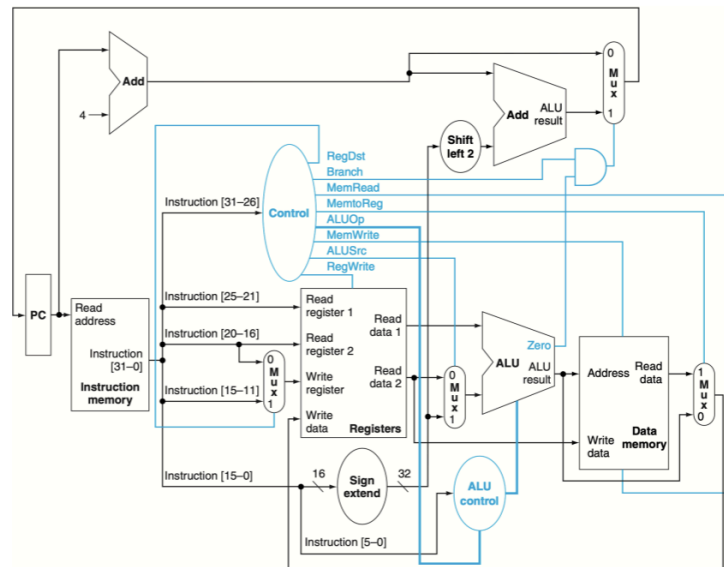
La logica è puramente combinatoria, quindi i due Adder lavorano in parallelo. solo il Program Counter è noto a priori



### FOURTH INTEGRATION - ADDING THE GLOBAL FSM

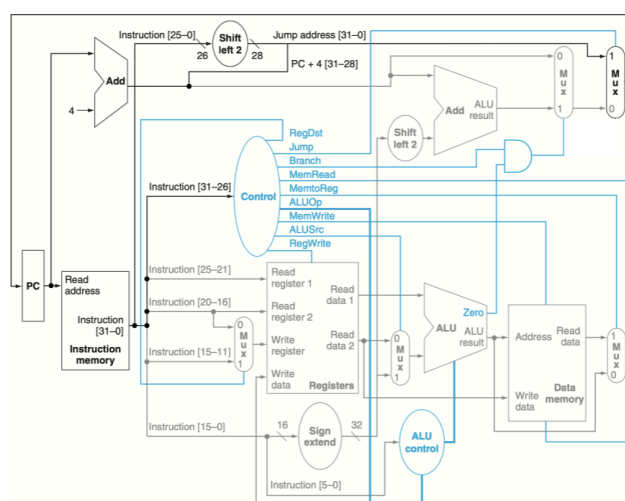
(LA GLOBAL FINITE STATE MACHINE E' UNA SEMPLICE TABELLA DI VERIA' O NO?? PERCHE' DI SOLITO SI USA SOLO NELLE ARCHITETTURE MULTI-CYCLE - CHIEDERE AL PROF)

A questo punto aggiungiamo il “cervello” (Finite State Machine) sul “corpo” (Datapath). Finora avevamo i segnali di controllo, ma non sapevamo da dove venivano, aggiungendo una Control Unit, essa prende i primi 6 bit dell'istruzione (Opcode, bit 31-26) e genera tutti i segnali di controllo (come RegDst, Memread, RegWrite) che abilitano le scritture o pilotano i MUX. Aggiungiamo una Porta AND che combina il segnale *Branch* (dalla Control Unit) con il segnale *Zeron*(dalla ALU) per decidere se attivare effettivamente il salto.



## FOURTH INTEGRATION - ADDING THE JUMP SECTION

Così andiamo a completare definitivamente il Datapath integrando l'istruzione di salto incondizionato. L'istruzione Jump (Type-J) non usa l'aritmetica dei puntatori (come il Branch), quindi aggiunge solo uno Shift Left 2 (in alto) che prende i 26 bit del campo *address* dell'istruzione (Instr[25-0]) e li shifta a sinistra di 2 posizioni. Come sempre, per convertire l'indirizzo da *word* a *byte* (x4), ottenendo 28 bit (a cui poi possiamo sommare 4 (con PC+4) per arrivare a 32). MUX (a destra) controllato dal segnale di Jump generato dalla Control Unit, seleziona tra l'Ingresso 0 che propaga il valore dalla logica precedente e l'ingresso 1: L'indirizzo di salto calcolato (**Jump Address**).



## ES09

# Multi-Cycle MIPS

## The Didactical Multi-Cycle MIPS Architecture

Abbiamo analizzato l'architettura Single-Cycle, semplice ma inefficiente. Ora entriamo nel **Multi-Cycle**, che è molto più vicino a come funzionano i processori reali (prima dell'avvento del Pipelining avanzato). L'architettura **Multi-Cycle**, invece di eseguire un'istruzione in un unico, lungo, ciclo di clock, esso ora è molto più breve, calibrato sulla durata dei singoli step e l'esecuzione viene spezzata in una serie di passi elementari e sequenziali. Poiché l'esecuzione si spalma nel tempo, il sistema non è più puramente combinatorio ma diventa una **Macchina a Stati Finiti (FSM)**. Il processore quindi deve "ricordarsi" a che punto è arrivato dell'esecuzione. Per far ciò, vengono introdotti registri temporanei posti tra le unità funzionali al fine di memorizzare i risultati parziali affinché siano disponibili come input per il ciclo successivo.

## The Complete Datapath

Nel Datapath Multi-Ciclo Completo, l'architettura presenta un'unificazione delle Risorse (Resource Sharing):

- Non esistono più *Instruction Memory* e *Data Memory* utilizziamo solo il blocco Memory per istruzioni e dati. L'accesso è gestito dal multiplexer **lorD** (*Instruction or Data*), che seleziona l'indirizzo proveniente dal PC (Fetch) o dall'ALUOut (Load/Store).
- Riduciamo il numero di ALU ad una che esegue tutte le operazioni: incremento del PC ( $PC+4$ ), calcolo degli indirizzi ( $Base + Offset$ ), confronti per i branch e operazioni aritmetiche. Con i multiplexer **ALUSrcA** e **ALUSrcB** selezionano gli ingressi corretti ciclo per ciclo.

Per "parcheggiare" i dati intermedi in registri aggiungiamo **Registri Temporanei** non visibili al programmatore (non architetturali):

1. **Instruction Register:** Mantiene l'istruzione corrente per tutta la durata dell'esecuzione (altrimenti andrebbe persa leggendo altri dati dalla memoria).
2. **Memory Data Register:** Salva il dato letto dalla memoria prima di scriverlo nel Register File.
3. **A & B:** Buffer per gli operandi letti dal Register File.
4. **ALUOut:** Buffer per il risultato della ALU, usato poi come indirizzo di memoria o dato da scrivere.



- (c)  $\frac{d}{dt} \left( \frac{\partial L}{\partial v^j} \right) = \frac{\partial L}{\partial x^j}$

- **R-Type:** La ALU esegue l'operazione ( $A \text{ op } B$ ) e salva in ALUOut.
- **Branch:** La ALU esegue la sottrazione ( $A - B$ ). Se il risultato è Zero, il PC viene aggiornato con il valore calcolato nella fase precedente (Target). ( $\text{if}(A == B) \text{ PC} \leftarrow \text{ALUOut};$ )
- **Jump:** Il PC viene sovrascritto concatenando i bit del PC corrente con l'indirizzo nell'istruzione. ( $\text{PC} \leftarrow \text{PC}[31:28] \& (\text{IR}[25:0] \& 0b"00");$ )

#### Fase 4: **Memory Access** (Load/Store)

- **Load (lw):** Legge il dato dalla memoria all'indirizzo calcolato (in ALUOut) e lo salva nel registro temporaneo *Memory Data Register*. È necessario questo passaggio intermedio perché la lettura della memoria occupa l'intero ciclo. ( $\text{MDR} \leftarrow \text{Memory}[\text{ALUOut}]$ )
- **Store (sw):** Scrive il valore del registro B in memoria e basta. ( $\text{Memory}[\text{ALUOut}] = B$ )
- **R-Type (Write Back):** Scrive il valore contenuto in ALUOut nel registro di destinazione rd. ( $\text{Reg}[\text{IR}[15:11]] \leftarrow \text{ALUOut}$ )

Fase 5: **Write Back** (Solo Load). Questa fase esiste solo per l'istruzione **lw**. Sposta il dato "parcheggiato" nel *Memory Data Register* verso il registro di destinazione finale rt nel Register File.

| Step name                                              | Action for R-type instructions                                                                                                                                                   | Action for memory-reference instructions                                                                               | Action for branches                                     | Action for jumps                                                      |
|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|-----------------------------------------------------------------------|
| Instruction fetch                                      | $\text{IR} \leftarrow \text{Memory}[\text{PC}]$<br>$\text{PC} \leftarrow \text{PC} + 4$                                                                                          |                                                                                                                        |                                                         |                                                                       |
| Instruction decode/register fetch                      | $A \leftarrow \text{Reg}[\text{IR}[25:21]]$<br>$B \leftarrow \text{Reg}[\text{IR}[20:16]]$<br>$\text{ALUOut} \leftarrow \text{PC} + (\text{sign-extend}(\text{IR}[15:0]) \ll 2)$ |                                                                                                                        |                                                         |                                                                       |
| Execution, address computation, branch/jump completion | $\text{ALUOut} \leftarrow A \text{ op } B$                                                                                                                                       | $\text{ALUOut} \leftarrow A + \text{sign-extend}(\text{IR}[15:0])$                                                     | if ( $A == B$ )<br>$\text{PC} \leftarrow \text{ALUOut}$ | $\text{PC} \leftarrow \{\text{PC}[31:28], (\text{IR}[25:0], 2'b00)\}$ |
| Memory access or R-type completion                     | $\text{Reg}[\text{IR}[15:11]] \leftarrow \text{ALUOut}$                                                                                                                          | Load: $\text{MDR} \leftarrow \text{Memory}[\text{ALUOut}]$<br>or<br>Store: $\text{Memory}[\text{ALUOut}] \leftarrow B$ |                                                         |                                                                       |
| Memory read completion                                 |                                                                                                                                                                                  | Load: $\text{Reg}[\text{IR}[20:16]] \leftarrow \text{MDR}$                                                             |                                                         |                                                                       |

## ES10

# Cache Principles

## Introduzione

La memoria cache è un componente essenziale dell'architettura dei sistemi di calcolo, progettata per migliorare le prestazioni della CPU riducendo i tempi di accesso alla memoria.

L'idea di base è semplice: tenere a portata di mano i dati usati di frequente, sfruttando le metriche di **località spaziale** e **località temporale**.

## The Rationale

La cache si basa sui principi di località per garantire che le operazioni più comuni siano eseguite con accesso rapido. In particolare:

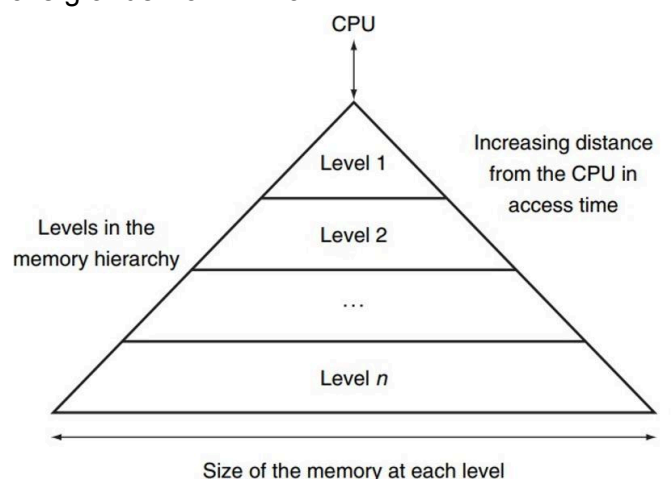
- **Località temporale:** i dati recentemente utilizzati hanno un'alta probabilità di essere riutilizzati, quindi li teniamo a disposizione.
- **Località spaziale:** i dati vicini a quelli recentemente utilizzati saranno probabilmente richiesti in breve tempo. Andiamo a prelevare così interi blocchi di memoria che contengono sia il dato richiesto sia quelli adiacenti

## The Memory Hierarchy

La gerarchia di memoria è una struttura a piramide progettata per bilanciare tre fattori contrastanti: **velocità**, **capacità** e **costo**. Poiché non esiste una tecnologia di memoria che sia contemporaneamente velocissima, enorme ed economica, si utilizzano più livelli:

- **Livello 1 (L1):** Integrata nel core della CPU, offre il tempo di accesso minimo ma ha una capacità molto ridotta.
- **Livello 2 (L2):** Un livello intermedio che funge da cuscinetto tra la cache ultra-rapida e la memoria principale.
- **Livello N** (ad esempio RAM e memoria secondaria): più capienti ma più drasticamente lente.

Al variare dei livelli, aumenta la distanza dalla CPU. Man mano che aumenta la distanza dalla CPU aumentano i tempi di accesso (access time) e la dimensione delle memorie. Primo livello, tempo minimo, ma anche grandezza minima.



## The Cache Figure of Merit

Le **Figure di Merito** sono i parametri quantitativi che permettono di giudicare l'efficacia della cache:

- **Capacità/Dimensione:** La dimensione fisica della memoria (strettamente legata al livello gerarchico: più è vicina alla CPU, più è piccola).
- **Hit Rate:** La percentuale di successo nel trovare i dati al livello attuale.
- **Miss Rate:** La percentuale di fallimento ( $1 - \text{Hit Rate}$ ). Un Miss Rate alto indica una cache inefficiente per quel particolare carico di lavoro.
- **Hit Time:** La latenza minima di accesso quando il dato è presente (tempo necessario per accedere ai dati in cache).
- **Miss Penalty:** Il costo temporale aggiuntivo richiesto per scendere nella gerarchia e recuperare il dato dal livello inferiore.

## Cache Architecture

### Single Block, Direct Mapped, Cache Architecture

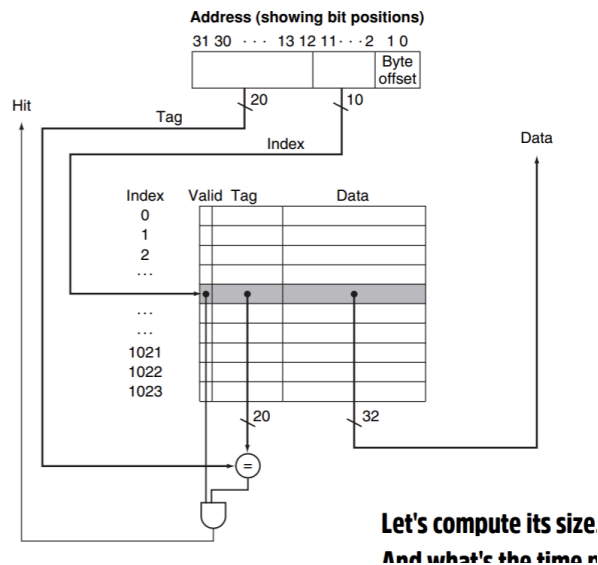
**Architettura Direct Mapped:** In una cache a mappatura diretta, ogni linea della cache può memorizzare un solo blocco di dati. Quando si ha bisogno di un dato A, si controlla se si ha nella Cache e se non c'è portiamo il dato dalla memoria, avendo così una copia del contenuto in memoria. In questa architettura, ogni blocco di memoria principale è assegnato a una specifica riga della cache tramite l'**Index** dell'indirizzo. Questa struttura è veloce ed economica da implementare, poiché richiede un solo confronto di Tag per ogni accesso, ma è soggetta a conflitti se più dati necessari al programma competono per la stessa riga.

Struttura dell'Indirizzo (32-bit):

- **Tag (31-12):** Utilizzato per identificare il blocco di memoria.
- **Index (11-2):** Indica quale linea della cache contiene il dato richiesto.
- **Byte Offset (1-0):** Per la selezione del byte specifico all'interno del blocco.

Funzionamento:

1. L'**Index** individua la linea della cache.
2. Il **Tag** dell'indirizzo viene confrontato (mediante un comparatore) con il **Tag** memorizzato nella linea selezionata.
3. Se il confronto è positivo e la linea è valida, si ha un **hit**; altrimenti, un **miss**.
4. Il **Byte Offset** seleziona il byte specifico all'interno del blocco (utilizzando un multiplexer).



## Multi-block, Cache Architecture

L'architettura **Multi-block (o Multi-word Block)**, è l'evoluzione della "Direct Mapped", mantiene la logica della mappatura diretta (una posizione fissa per ogni indirizzo), ma espande la dimensione della riga di cache. Ogni riga non contiene più una singola parola (32 bit), ma un blocco più grande (**512 bit** totali di dati per riga).

L'indirizzo a 32 bit viene quindi reinterpretato per gestire questa "larghezza" extra:

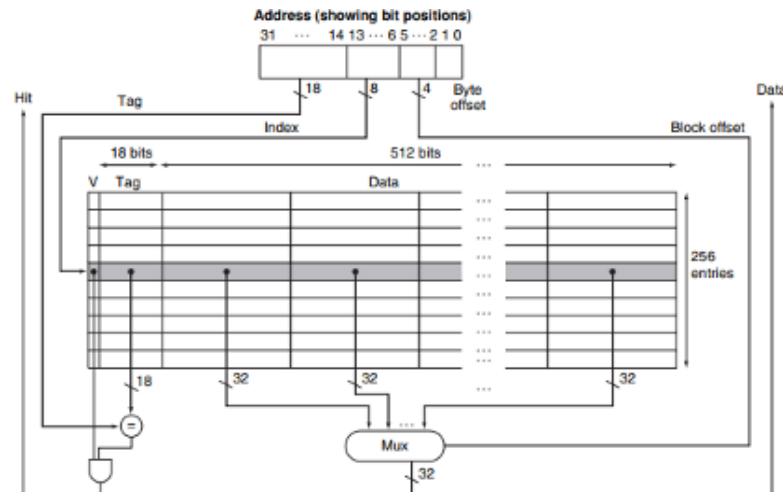
- **Tag (18 bit):** Identifica se il blocco in memoria appartiene all'indirizzo richiesto. Nota che il Tag è più piccolo rispetto all'esempio precedente (era 20 bit) perché il blocco dati è più grande.
- **Index (8 bit):** Seleziona una delle **256 righe** (entries) disponibili ( $2^8 = 256$ ).
- **Block Offset (4 bit):** Individua il blocco specifico all'interno della linea. Poiché la riga contiene 512 bit (ovvero 16 parole da 32 bit), servono 4 bit ( $2^4 = 16$ ) per decidere quale di queste 16 parole mandare alla CPU tramite il MUX.
- **Byte Offset (2 bit):** Seleziona il byte all'interno della parola (standard per architetture a 32 bit).

Pro e Contro:

- **Vantaggio: Località Spaziale.** Questa architettura è progettata per massimizzare la località spaziale, abbiamo un solo Tag per 512 bit di dati. Se leggi l'elemento *Array[0]* la cache carica automaticamente anche da *Array[1]* a *Array[15]* nella stessa linea. I successivi accessi saranno *Hit* immediati senza dover interrogare la RAM.
- **Svantaggio: Miss Penalty più alta.** Caricare 512 bit dalla RAM è molto più lento che caricarne 32. Se c'è un Miss, la CPU deve aspettare che *tutto* il blocco venga copiato, aumentando la latenza del singolo miss.

Logica di selezione e verifica:

- La verifica del **Tag** avviene tramite un confronto tra il **Tag** dell'indirizzo e quello memorizzato nella linea selezionata.
- Un **multiplexer** controllato dal **Block Offset** seleziona il blocco corretto e un ulteriore **multiplexer** utilizza il **Byte Offset** per selezionare il byte richiesto.



## Set Associative Cache Architecture

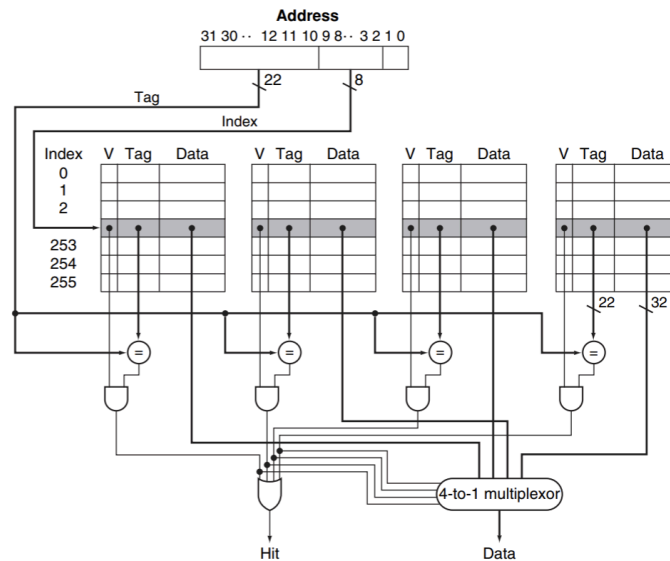
Questa architettura organizza la cache in S insiemi (Sets), dove ogni insieme contiene N blocchi (Ways). Combina le caratteristiche della *Direct Mapped* e della *Fully Associative*.

Suddivisione dell'indirizzo (32 bit):

- **Tag (31-9)**: Utilizzato per identificare in quale dei 4 blocchi di memoria risiede il dato.
- **Index (8-2)**: Indica quale "Set" (la riga) contiene il blocco richiesto. Rendendo accessibili simultaneamente tutti i blocchi contenuti in quel set.
- **Byte Offset (1-0)**: Seleziona il byte specifico all'interno del blocco.

Funzionamento:

1. L'**Index** seleziona il set della cache in cui cercare il dato.
2. Il **Tag** dell'indirizzo viene confrontato in parallelo con i **Tag** memorizzati all'interno di tutte le linee del set selezionato.
3. Se uno dei confronti è positivo e la linea è valida, si ha un **hit**; altrimenti, un miss. Un multiplexer pilotato dal risultato del confronto seleziona il blocco corretto.
4. Il **Byte Offset** seleziona il byte richiesto all'interno del blocco tramite l'utilizzo di un multiplexer.



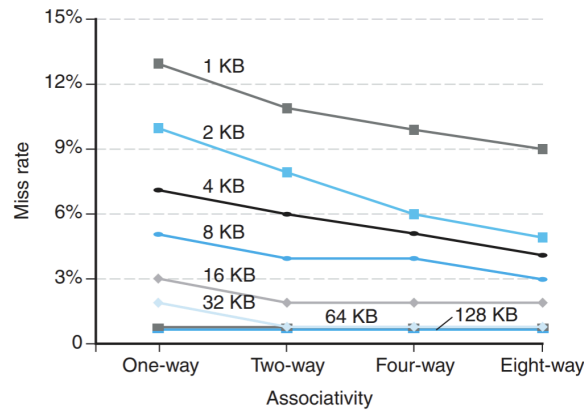
## Cache Organization

Confronto delle Organizzazioni: Esiste una relazione inversa tra flessibilità e velocità. La **Direct Mapped** è la più veloce (minor Hit Time) ma soffre di alti Miss di conflitto. La **Fully Associative** elimina completamente i conflitti (ogni blocco può andare ovunque), ma ha l'Hit Time più alto e consuma più energia dovendo confrontare tutti i tag simultaneamente. La **Set Associative** bilancia questi estremi.



Impatto sull'Hardware (Comparatori): La complessità circuitale scala con l'associatività. Mentre la mappatura diretta richiede un solo comparatore digitale, la Fully Associative richiede un comparatore per ogni linea della cache (o l'uso di memorie CAM), rendendola impraticabile per cache di grandi dimensioni. L'aumento dei comparatori attivi in parallelo incrementa il consumo dinamico di potenza.

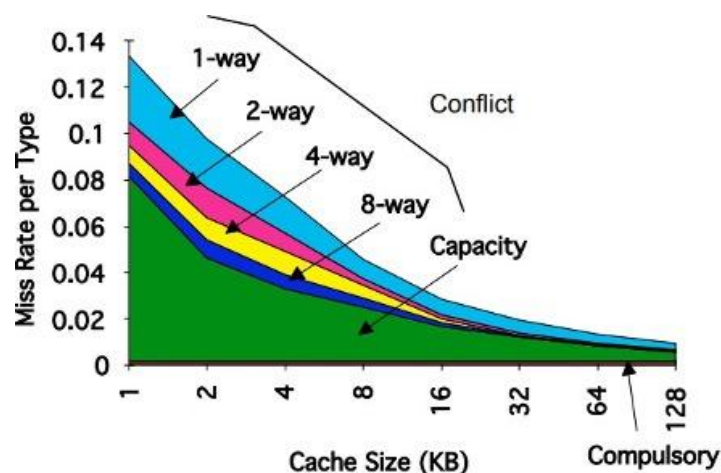
Evoluzione dell'Indirizzamento: All'aumentare dell'associatività (N), il numero di Set diminuisce (Totale Linee/N). Di conseguenza, i bit dedicati all'**Index** diminuiscono e vengono assorbiti dal **Tag**. Nel caso limite della Fully Associative, l'Index scompare del tutto e il processore utilizza l'intero indirizzo (escluso l'offset) come Tag per la ricerca associativa globale.



## Reducing Misses

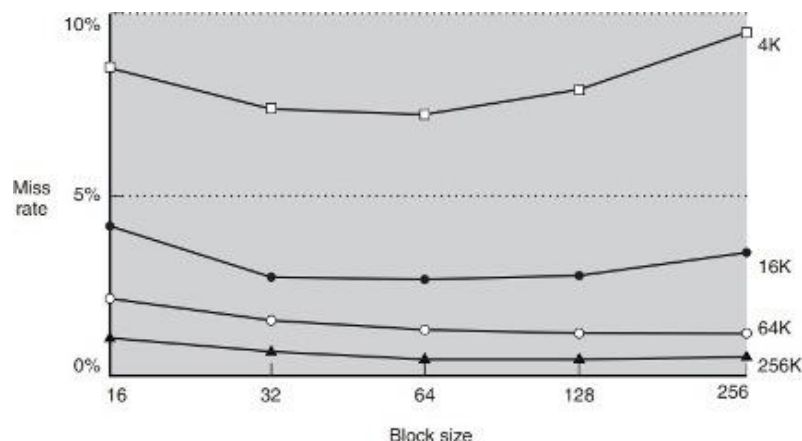
### Misses Classification: the 3Cs

- **Compulsory:** Sono i Miss fisiologici. Sono i Miss che avvengono al primo accesso assoluto ad un blocco. Non possiamo avere il dato se non l'abbiamo mai chiesto prima.
- **Capacity:** Si verificano quando la cache non è sufficientemente grande per contenere tutti i blocchi necessari all'esecuzione corrente del programma (il *Working Set*).
- **Conflict:** Questi miss accadono quando più blocchi competono per lo stesso *Set* o riga, anche se ci sarebbe spazio libero in altre parti della cache. Sono tipici delle architetture *Direct Mapped*. (Soluzione è aumentare l'associatività (da 1-way a N-way))



### Miss Rate vs. Block Size

Il tasso di Miss varia con l'aumentare delle dimensioni del blocco. Inizialmente riduce i Miss obbligatori e di capacità sfruttando la **località spaziale**, fino a raggiungere una dimensione ottimale del blocco. Oltre questo punto le prestazioni degradano. Infatti, blocchi troppo grandi riducono drasticamente il numero totale di linee nella cache, aumentando i **Miss di Conflitto**. Avere un blocco troppo grande influenza anche la latenza del recupero dati, aumentando la **Miss Penalty**, ovvero il tempo necessario per trasferire il blocco dalla memoria principale alla cache.



## Coherence Strategies (when writing)

La scrittura in cache pone un problema fondamentale di **Coerenza**: abbiamo due copie dello stesso dato (una in cache L1 e una in RAM). Quando la CPU modifica la copia in cache, quella in RAM diventa obsoleta ("stale").

Le strategie per gestire questo disallineamento sono due approcci opposti:

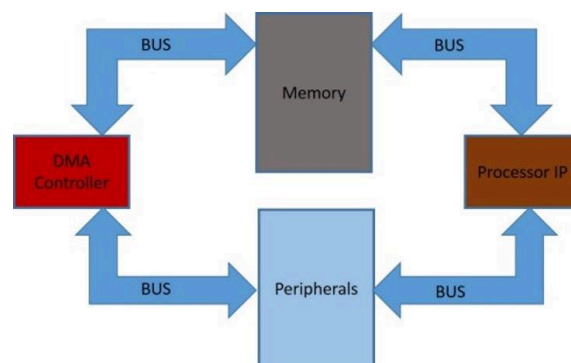
- **Write-Through (Scrittura Passante):** È l'approccio "prudente". Ogni volta che la CPU scrive in cache, il controller della memoria scrive *immediatamente* anche in RAM.
  - *Risultato:* Cache e RAM sono sempre identiche (coerenti).
  - *Prezzo:* Lentezza. La CPU deve aspettare i tempi della RAM per ogni singola scrittura.
- **Write-Back (Scrittura Posticipata):** È l'approccio "ottimista". La CPU scrive solo nella cache. L'aggiornamento del dato nella RAM avviene solo quando quel blocco di cache sta per essere cancellato per far posto a qualcos'altro.
  - *Risultato:* Velocissimo, perché si scrive a velocità di cache.
  - *Prezzo:* Complessità. Serve hardware extra per ricordarsi quali dati sono stati modificati (con utilizzo di "Dirty bit") e rischio di incoerenza se salta la corrente o se un'altra periferica (DMA) legge la RAM vecchia.

# Virtual Memory

La memoria virtuale simula una memoria di grandi dimensioni utilizzando lo spazio su disco. Disaccoppiando gli indirizzi usati dal software (indirizzi virtuali) dagli indirizzi fisici della RAM. Questo meccanismo illude ogni processo di avere a disposizione una memoria contigua e molto estesa (fino alla capacità del disco fisso), indipendentemente dalla quantità di RAM fisica installata. Si crea così una gerarchia dove la RAM agisce come cache ad alta velocità per il disco.

Caratteristiche:

- **Paging:** suddivide la memoria in blocchi, chiamati "Pagine" a dimensione fissa, per una gestione efficiente.
- **Swapping (Disk-Memory Swap):** La RAM funge da "cache" per il disco rigido. Le pagine usate stanno in RAM; quelle non usate vengono parcheggiate su disco (nell'area di Swap).
- **Address Translation:** Meccanismo che traduce in tempo reale l'indirizzo virtuale nell'indirizzo fisico.



## ES22

# Multiply and Accumulate (MAC)

## Introduzione

Il Multiply and Accumulate (MAC) è un acceleratore hardware fondamentale per velocizzare le operazioni di elaborazione nei DSP (Digital Signal Processors).

Consente di combinare, in un unico ciclo di clock (single-cycle), operazioni di moltiplicazione, somma, differenza e accumulazione. Questa capacità riduce significativamente i tempi di calcolo e migliora l'efficienza computazionale, rendendolo ideale per applicazioni che richiedono elaborazioni complesse.

## Digital Signal Processing/Processor - DSP

I processori DSP sono progettati per eseguire algoritmi complessi quali filtraggio, trasformate di Fourier e convoluzioni. Questi algoritmi si basano su operazioni esprimibili come somme di prodotti. Ad esempio, per calcolare una convoluzione discreta tra due sequenze, i DSP utilizzano intensivamente moltiplicazioni e somme, rendendo il MAC uno strumento indispensabile.

## The Complex Product

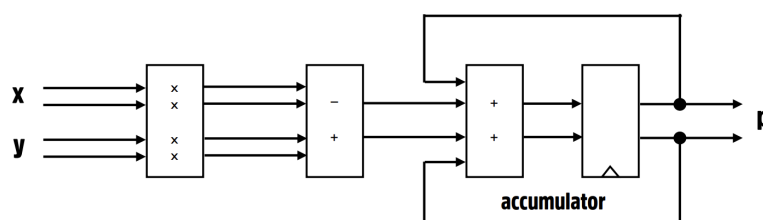
Il prodotto complesso viene calcolato utilizzando la moltiplicazione binaria. Questa tecnica divide l'operazione in prodotti parziali tra i bit di due numeri, che vengono poi sommati insieme propagando i riporti necessari, come mostrato sotto:

$$S = x + y = (x_r + y_r) + i(x_{im} + y_{im})$$

$$P = x \times y = (x_r y_r - x_{im} y_{im}) + i(x_r y_{im} - x_{im} y_r)$$

## The Single Cycle MAC

Un MAC a ciclo singolo esegue somme e prodotti accumulati in un solo ciclo di clock. Ogni ciclo consente di memorizzare i risultati parziali, pronti per essere elaborati successivamente. Questo design è particolarmente efficace in applicazioni a bassa latenza, dove l'efficienza di calcolo è prioritaria. Infatti l'unità è composta da tre stadi in cascata (Moltiplicatore - ALU - Accumulatore) privi di registri intermedi, al fine di minimizzare la latenza e ottenendo un **throughput elevato** (Grazie a questa architettura, un DSP può calcolare un "tap" di un filtro FIR per ogni ciclo di clock. Se il processore gira a 100 MHz, può eseguire 100 Milioni di MAC al secondo).

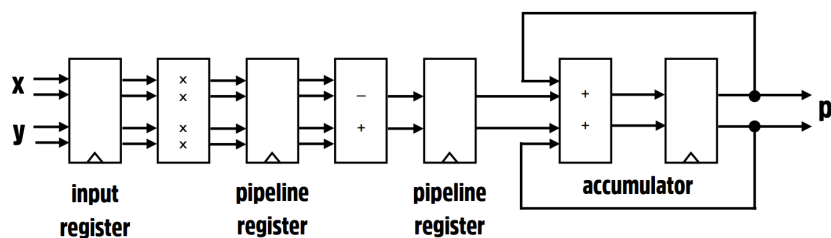


## The Pipelined MAC

Il MAC pipelined suddivide la catena di elaborazione in sotto-catene sincrone, consentendo l'esecuzione parallela di operazioni. Rispetto al MAC Single Cycle, si aggiungono registri di pipeline (che consistono in banchi di flip-flop) tra le unità funzionali, in questo modo il clock non deve più coprire i ritardi del Sommatore + il Moltiplicatore, ma solo il ritardo del singolo stadio più lento.

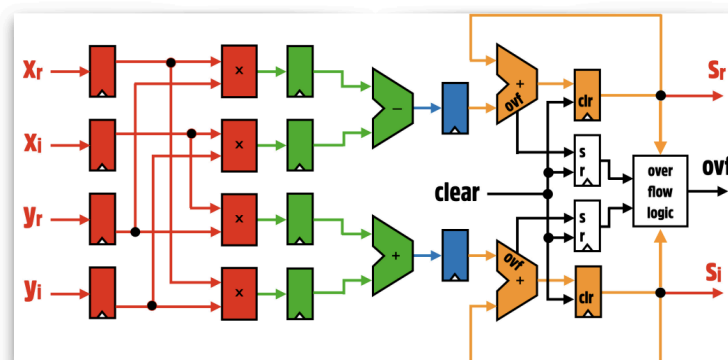
Sebbene la latenza aumenti (un singolo dato impiega più cicli di clock per attraversare tutta la catena), il throughput cresce significativamente, grazie alla frequenza più alta e al parallelismo, garantendo un flusso continuo di dati.

Ad esempio, in un MAC pipelined con quattro stadi, mentre il primo stadio elabora un nuovo dato, i successivi processano i dati precedenti. Ciò aumenta il tasso di uscita dei dati, migliorando le prestazioni complessive.



## RTL Structural View

La struttura del **Complex MAC Pipelined** è progettata per elaborare numeri complessi (come nella FFT), il DSP utilizza una struttura hardware parallela composta da 4 moltiplicatori (blocchi rossi) e 2 sommatori (blocchi verdi) per calcolare simultaneamente parte reale ( $Re = ArBr - AiBi$ ) e immaginaria ( $Im = ArBi + AiBr$ ) in un singolo passaggio della pipeline.



Dopo l'operazione di somma troviamo i Pipeline Registers (blocchi blu) che si occupano di salvare temporaneamente il risultato della moltiplicazione complessa. A valle di essi, ci sono gli Accumulatori (blocchi arancioni), qui i nuovi prodotti vengono sommati al valore precedente che torna indietro tramite un feedback loop. Infine troviamo la Overflow Logic (blocchi in bianco) che si occupa di controllare se la somma supera la capacità massima dei bit disponibili.

La **gestione del "Bit Growth"** consiste nell'aumentare la larghezza in bit dei dati man mano che attraversano il circuito, poiché dobbiamo aumentare la precisione per non perdere dati.

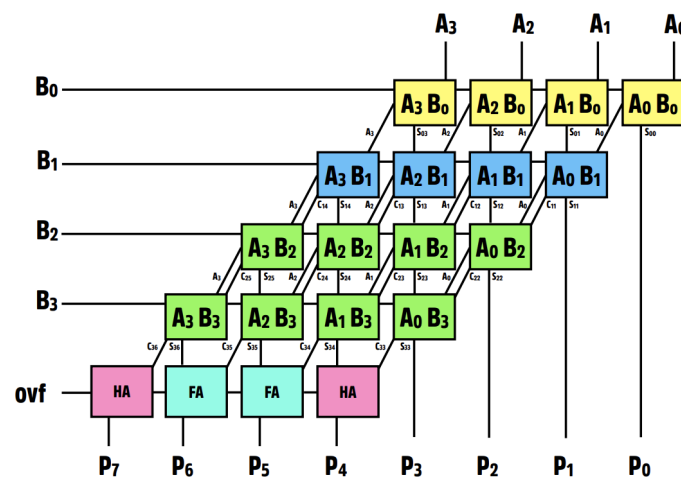
- **WI** (Input - **16** bit): Si parte con dati a 16 bit (15-0), range  $[-1, +1]$ , zero bit davanti al punto.
- **WPP** (Partial Products - **32** bit): Quando moltiplichiamo due numeri a 16 bit, il risultato ne richiede 32. Il sistema espande la larghezza. Range  $[-1, +1]$ , un digit davanti al punto.
- **WPC** (Complex Products - **33** bit): Sommando due numeri a 32 bit (nello stadio verde), serve un bit in più per il riporto, per rappresentare il range esteso  $[-2, +2]$ , due digit davanti al punto.
- **WT** (Truncation - **20** bit): Rappresenta uno stadio intermedio in cui il risultato del prodotto complesso viene ridotto di precisione prima di essere accumulato. Range  $[-2, +2]$ , due digit davanti al punto.
- **WA** (Accumulator - **22** bit): È il risultato della somma di tutti i risultati parziali, corrisponde ai segnali che escono dai sommatore arancioni. Range  $[-16, +16]$ , si chiamano **Guard Bits**, che permettono al valore della somma di crescere durante l'accumulazione (finché il totale non supera 16). Quattro digit davanti al punto.
- **WO** (Output - **16** bit): Alla fine di tutto, il risultato deve tornare in memoria o andare al DAC. Si prendono i 22 bit dell'accumulatore e si "tagliano" (troncamento o arrotondamento) per tornare a 16 bit, un range di  $[-1, 1)$  e zero digit davanti al punto.

La **Logica di Overflow e Saturazione** viene gestita prima di scrivere il risultato in memoria (ritornando a 16 bit - WO), controllando i Guard Bits.

- Se il valore finale supera il range rappresentabile  $[-1, +1]$ , il circuito applica la **Saturazione**: invece di troncare i bit (che causerebbe un wrap-around e inversione di segno), l'uscita viene forzata al massimo valore positivo o negativo rappresentabile.
- Infine, il segnale **CLEAR** resetta l'accumulatore ogni N cicli per iniziare una nuova operazione.

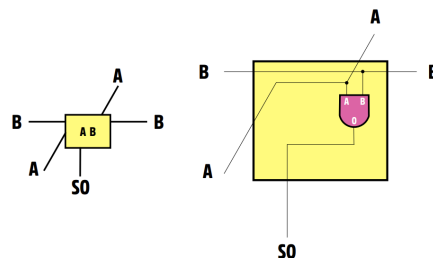
# Binary Multiplication

**Version #1:** È il classico algoritmo di **moltiplicazione "in colonna"** applicato al sistema binario: ogni bit del moltiplicatore (B) genera un **prodotto parziale** che corrisponde a una copia del moltiplicando (A) se il bit è 1, o a una serie di zeri se il bit è 0. Ogni riga successiva viene fatta scorrere a sinistra (shift) per rispettare il peso posizionale. Infine, si sommano tutte le colonne verticalmente per ottenere il risultato S. NB! Il prodotto di due numeri a 4 bit richiede fino a **8 bit** per essere rappresentato.



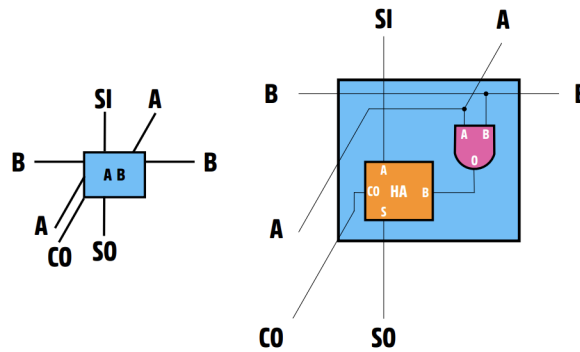
Definizione dei vari blocchi:

- **Blocco #1** (Generazione LSB): Cella base per il calcolo del primo bit del risultato ( $S_0$ ). Al suo interno contiene una **porta logica AND**, che esegue la moltiplicazione booleana tra i bit in ingresso. I segnali A e B attraversano il blocco ("feed-through") per essere disponibili agli stadi successivi, mentre l'uscita della porta AND fornisce il prodotto parziale.

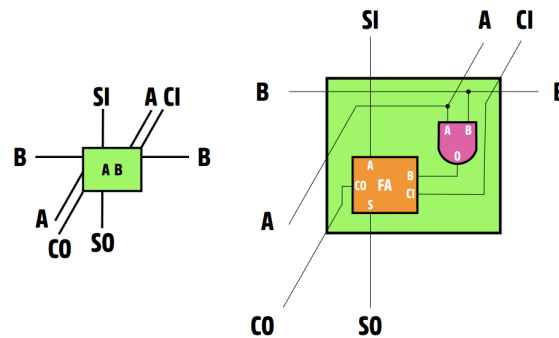


- **Blocco #2** (Half Adder Cell): Utilizzato per sommare il prodotto locale a una somma parziale precedente, ma senza gestire un riporto in ingresso. Combina una porta **AND** (che calcola  $A \cdot B$ ) con un Half-Adder (**HA**), che somma il risultato della AND con la Somma in Ingresso (**SI** - Sum In). In uscita produce un nuovo bit di

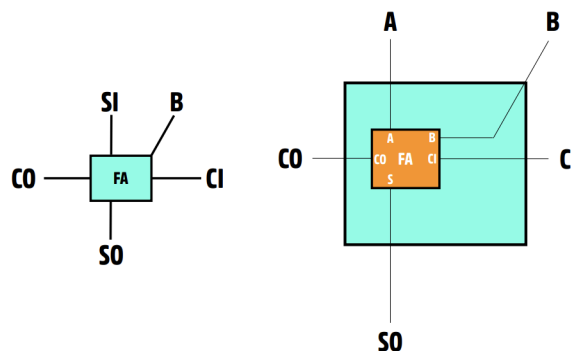
somma (**SO** - Sum Out) e un bit di riporto (**CO** - Carry Out) che viene inviato alla colonna successiva a sinistra.



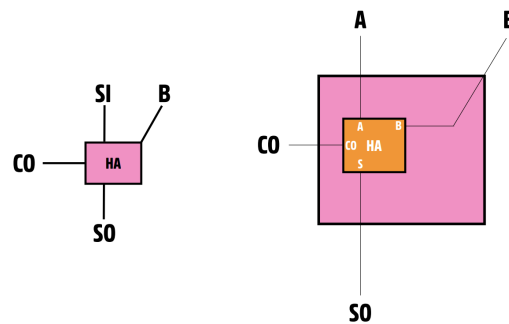
- **Blocco #3** (Full Adder Cell): Combina una porta **AND** con un Full-Adder (**FA**) che somma il prodotto locale generato dalla porta AND ( $A \cdot B$ ), il bit di somma parziale (**SI** - Sum In) proveniente dall'alto, il riporto in ingresso (**CI** - Carry In) proveniente dalla cella sopra a destra. In uscita genera un nuovo bit di somma (**SO**) e un nuovo riporto (**CO**) che si propaga a sinistra.



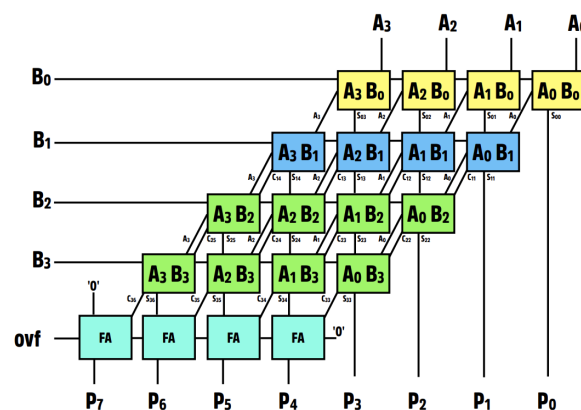
- **Blocco #4** (Pure Full Adder): Contiene solo un **FA**, che somma tra loro il bit di somma proveniente dal blocco sopra ( $=A$ ), il bit Carry Out ( $=B$ ) proveniente da il blocco sopra a destra e il bit di Carry In ( $=CI$ ) proveniente dal blocco alla sua destra sulla stessa riga. In uscita genera il risultato binario finale e un Carry Out.



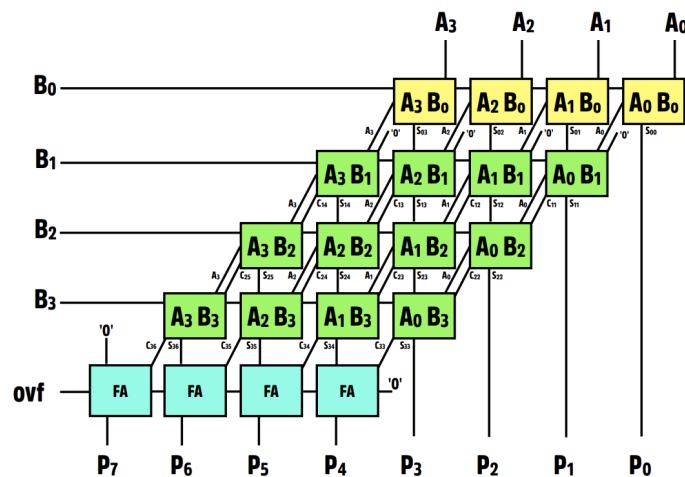
- **Blocco #5** (Pure Half Adder): Somma il bit della Somma Out ( $= SI$ ) ottenuto dal blocco superiore e il bit di Carry Out ( $=B$ ) del blocco a fianco sulla stessa riga. In uscita genera il risultato binario finale e un Carry Out.



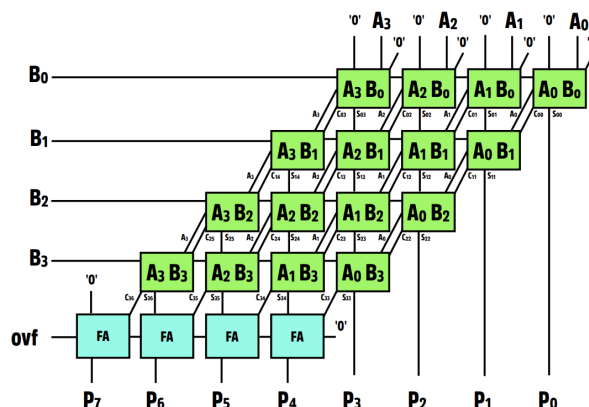
**Version #2:** In questa variante, nella riga finale (Vector Merging Adder) gli Half-Adder (HA) sono stati sostituiti con dei **Full-Adder (FA)**, con i loro ingressi extra collegati logicamente a '0' per mantenere la correttezza dell'operazione binaria ( $A+B+0=A+B$ ). Questo ci permette di semplificare il layout del chip usando celle identiche e rende il modulo scalabile data la presenza di un ingresso di riporto (Carry-in) anche nel bit meno significativo, facilitando la connessione in cascata di più moltiplicatori per gestire numeri a bitaggio superiore (es. da 4 bit a 8, 16, 32 bit).



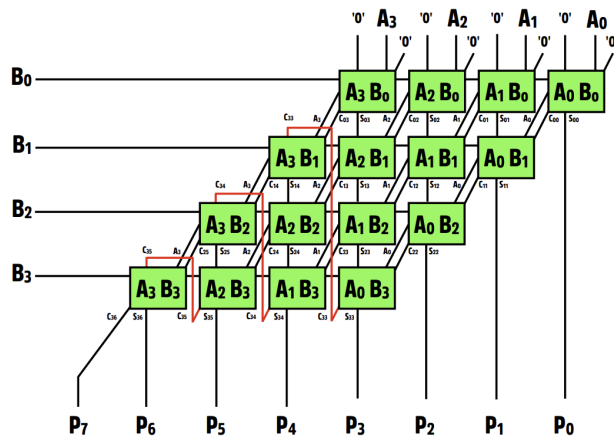
**Versione #3:** In questa è la versione eliminiamo i blocchi semplificati (AND e Half-Adder), estendendo l'uso del **Blocco #3 (FA + AND)**. Nella seconda riga, dove i riporti o le somme parziali non sono necessari, gli ingressi extra del Full Adder vengono forzati a '0'. Nonostante questo comporti un leggero aumento di area e latenza (poiché un FA è più complesso di un HA), garantisce la **massima regolarità strutturale**, rendendo il chip composto dalla ripetizione di una singola cella base identica.



**Versione #4:** In questa versione andiamo a sostituire la prima riga della matrice con i Blocchi #3 (FA + AND) e dove i riporti o le somme parziali non ci sono, gli ingressi extra del Full Adder vengono forzati a '0'. Facciamo questo per migliorare la scalabilità dato che i collegamenti a '0' permettono di collegare più moduli in cascata se necessario creare moltiplicatori più grandi (es. 8x8, 16x16) senza aggiungere logica esterna ("glue logic").



**Versione 5:** In questa versione usiamo la tecnica del "Folding". Infatti, sfruttando gli ingressi liberi (precedentemente collegati a '0') nei blocchi superiori della matrice, è possibile reindirizzare i riporti. Questo permette di **eliminare l'intera riga finale** di Full-Adders (Vector Merging Adder), riducendo drasticamente il numero di transistor necessari, accorciando il percorso critico e quindi rendendo il sistema più veloce.



## ES11

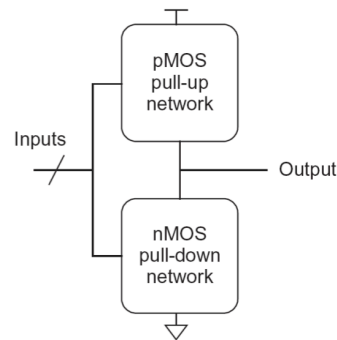
# Digital Circuits and Layout

## Combinational Logic

### CMOS Devices

Sviluppati per il principio di avere dispositivo che permettesse di avere un segnale di acceso ( $1 = V_{DD}$ ) e spento ( $0 = GND$ ), andando a sostituire le resistenze di pull-up, quando fu possibile raggiungere un blocco p, ai livelli tecnologici dei blocchi n. Questi dispositivi, permettono di avere un consumo di potenza molto minore dei suoi predecessori, nell'ipotesi di un transistor ideale, questo non consuma potenza nelle transizioni, sappiamo però che il consumo parassito di potenza è un fattore da tener conto.

|               | Pull-up OFF  | Pull-up ON     |
|---------------|--------------|----------------|
| Pull-down OFF | Z (floating) | 1              |
| Pull-down ON  | 0            | X (contention) |



## Complementary Conduction

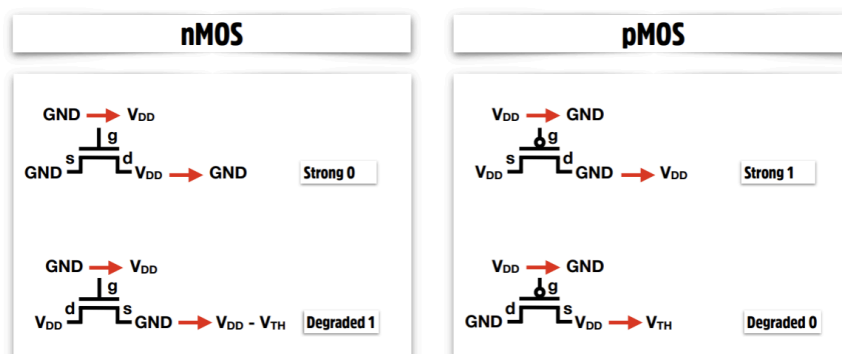
I gate CMOS producono sempre o un 1 o uno 0, mai un valore indefinito. Per garantire ciò, le reti di Pull-Up (PUN) e Pull-Down (PDN) devono essere topologicamente duali. L'operazione **AND** ( $A \cdot B$ ) si realizza con nMOS in **Serie** + pMOS in **Parallelo**, per la **OR** ( $A+B$ ), invece, si usano nMOS in **Parallelo** + pMOS in **Serie**. Questa struttura garantisce che quando la rete di Pull-Down è accesa, quella di Pull-Up è spenta, e viceversa.

La tecnologia CMOS permette di realizzare **Compound Gates**, cioè funzioni logiche complesse (come  $Y=AB+CD$ ) in un singolo stadio logico, risparmiando area e ritardo rispetto all'uso di porte base separate.

## Signal Strength

Il "signal strength" identifica quanto vicino un segnale si avvicina al valore ideale,  $V_{DD}$  e GND sono rispettivamente 1 e 0. Per i blocchi n- e p-:

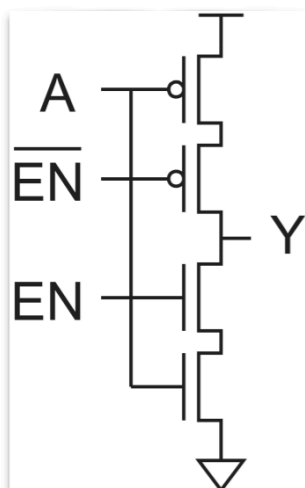
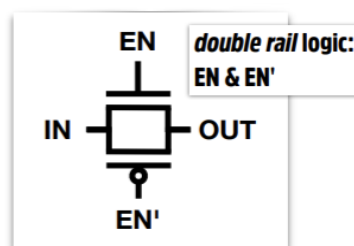
- n-MOS: trasmette uno 0 forte e un 1 debole, rendendoli i migliori per i pull-down
- p-MOS: trasmette un 1 forte e uno 0 debole, rendendoli i migliori per i pull-up



Per i blocchi a “Transmission Gates”, combinando un blocco n con un p, in parallelo, otteniamo un sistema che a seconda del segnale sui gate dei MOSFET, fa passare il segnale da una parte o l'altra del circuito.



Per evitare conflitti, tra più dispositivi, sulla stessa linea, aggiungiamo un tri-state buffer composto da un pMOS verso Vdd e un nMOS verso GND controllati da un segnale di Enable (EN). Si trovano entrambi disabilitati quando EN=0, portando l'uscita in alta impedenza, e li abilita in modo complementare quando EN=1 per pilotare correttamente 0 o 1 senza conflitti.



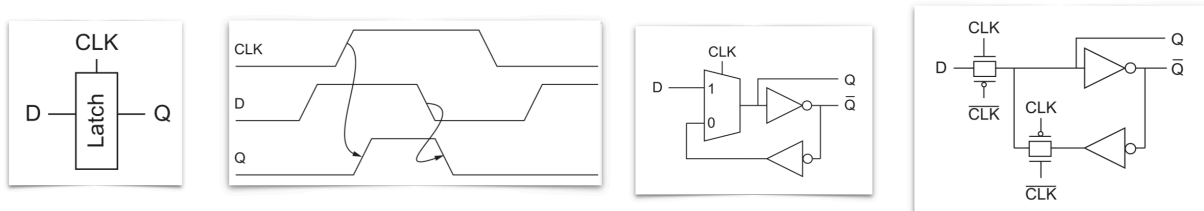
Il problema con questa configurazione è che non presentano una logica di ristorazione del segnale, di conseguenza il rumore in ingresso, che sporca il segnale, si ripercuote su quello in uscita. Per garantire l'integrità del segnale (Noise Immunity), si utilizza il **Restoring Tristate Inverter**, con i due transistor centrali fungono da interruttori di abilitazione (EN) e quando accesi, i due transistor esterni (A) fungono da normale inverter CMOS, collegando l'uscita a VDD o GND. Infine, poiché l'uscita è pilotata direttamente dalle linee di alimentazione, il livello logico viene rigenerato (ripulito dal rumore) e ha una forte capacità di pilotaggio.

# Sequential Logic

## D (Transparent) Latch

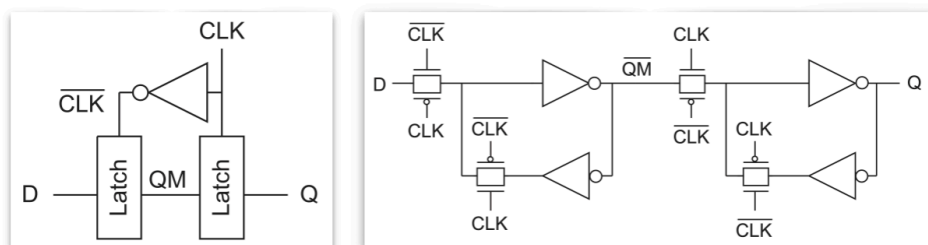
Sono blocchi usati per memorizzare l'ultimo dato che lo attraversa. Quando il clock (CLK) è 1, il latch è trasparente e propaga il dato da D a Q, mentre quando il clock è uguale a 0, il latch trattiene in Q l'ultimo valore visto in D.

Logicamente, un Latch è modellabile come un **Multiplexer** controllato dal Clock che chiude un anello di retroazione (Feedback Loop). Il Mux seleziona tra l'ingresso esterno D (aggiorna) e l'uscita attuale Q (memorizza), risolvendo eventuali conflitti. Quando il loop è chiuso (CLK=0), due inverter in cascata mantengono il dato rigenerandosi continuamente (Feedback Positivo).



## D Edge-Triggered Flip Flop Design

È l'elemento base dei circuiti sequenziali sincroni (come le CPU), sensibile solo alle transizioni del clock (Fronte di Salita/Discesa). La sua architettura è costruita da due D-Latch, in serie, controllati da fasi di clock opposte. Quando il clock è basso il Master (trasparente) campiona l'ingresso D, ma lo Slave (bloccato) isola l'uscita Q. Quando il clock è alto il Master si blocca (memorizzando il valore) e lo Slave diventa trasparente, trasferendo il dato all'uscita Q.



In un chip reale, dove il segnale di clock non arriva istantaneamente, può succedere che in due latch in cascata il dato arrivi, prima del clock, al secondo latch (Clock Skew) che è ancora aperto. Presentandosi quella condizione, il latch campiona l'ultimo dato, invece del vecchio, corrompendo così la memoria. Il design Master-Slave con il suo meccanismo "a

camera stagna" impedisce questi eventi, chiamati **Race Conditions**, nei circuiti con feedback.

## Layout

### Gate Layout

Disegnare chip transistor per transistor ("Custom Layout") è troppo lento per processori moderni con miliardi di transistor. Si usa una libreria di mattoncini pre-fatti (AND, OR, FLIP-FLOP) chiamati **Standard Cells**. Ogni cella ha dei contatti (quadratini neri) per polarizzare il substrato, altrimenti il chip non funziona.

Il Layout segue delle linee guida precise:

- **Dimensioni:** Ogni porta logica è contenuta in una cella rettangolare con altezza fisse, ma larghezza variabile in base alla complessità della funzione.
- **Power Rails:** Le linee di alimentazione (VDD e GND) devono combaciare. Se si mettono due celle vicine, i loro fili di alimentazione si toccano creando un binario continuo.
- **Struttura CMOS:** In alto si disegnano i **pMOS** (che vanno a VDD), in basso gli **nMOS** (che vanno a GND).

### Design Rules

**Calcolo del "Pitch"** (Passo): Si basa sull'area occupata, dove il passo (ingombro totale) di una traccia è definito dalla somma della larghezza del conduttore e dello spazio di isolamento necessario verso il conduttore adiacente.

- Internal Spacing:
  - $1\lambda$  tra Metal - Diffusion Contact
  - $2\lambda$  tra Metal - Polysilicon Contact
  - $4\lambda$  larghezza Metal e Diffusion
  - $2\lambda$  larghezza Polysilicon
- Intra Components Spacing:
  - $4\lambda$  tra i Metal e tra i Diffusion
  - $3\lambda$  tra i Polysilicon
  - $3\lambda$  tra i contatti sui Metal

- $12\lambda = 6\lambda + 6\lambda$  tra i Pull-Up e Pull-Down

**Wiring Tracks:** Rappresentano lo spazio fisico minimo richiesto per stendere un singolo conduttore metallico rispettando le regole di design. È l'unità base per definire la griglia di layout.

## Examples: Inverter & NAND3

- Inverter: Composto da 2 MOS, vediamo la divisione tra rete di pull-down e pull-up, con quest'ultima racchiusa nel quadrante grigio (**N-Well** - "vasca N" necessaria per creare il pMOS su un substrato di tipo P). In alto e in basso troviamo due Metal1 che rappresentano rispettivamente  $V_{DD}$  e GND. La barretta grigia, il polisilicio, rappresenta A, il contatto con i gate dei transistor, mentre parallela ad essa troviamo la connessione del *Drain* del pMOS con il *Drain* del nMOS, che rappresenta l'uscita Y.

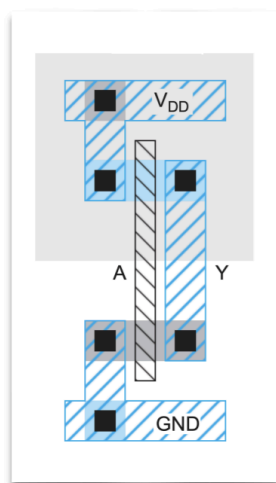


Figura 1: Inverter

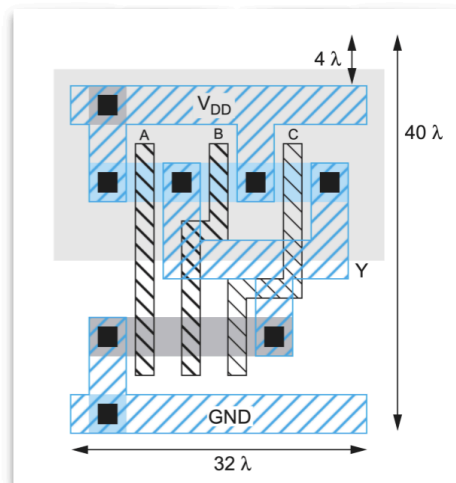


Figura 2: NAND3

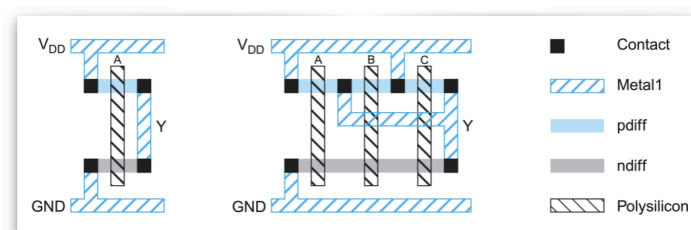
- NAND3: Composto da un Pull-Down, di 3 nMOS in serie nella parte inferiore, e un Pull-Up di 3 pMOS in parallelo nella parte superiore. Quest'ultima racchiusa nel quadrante grigio (**N-Well** - "vasca N" necessaria per creare il pMOS su un substrato di tipo P). In alto e in basso troviamo due Metal1 che rappresentano rispettivamente  $V_{DD}$  e GND. Le barrette grigie, i polisilicio, rappresentano gli ingressi A,B,C, che arrivano ai gate di ciascun transistor, mentre a collegare il parallelo con la serie, troviamo la connessione dei *Drain* dei pMOS con il *Drain* del nMOS più in alto, che rappresenta l'uscita Y.

## Stick Diagrams

Rappresentano una visione topologica (non in scala) del circuito, permettendo al progettista di pianificare il "Floorplan" della cella, ottimizzando il posizionamento dei componenti e il routing dei segnali prima del disegno dettagliato.

- Per il routing le linee di **Metallo** possono passare sopra Poly e Diffusione senza connettersi. La connessione elettrica avviene solo in presenza di un **Contatto**.
- Quando una linea di **Polisilicio** incrocia una di **Diffusione**, lì esiste un transistor.

Add on: Per ottimizzare la topologia della rete, andiamo ad usare il Cammino di Eulero comune alla rete di Pull-Up e Pull-Down, andando così a minimizzare l'area del chip e le capacità parassite di giunzione.



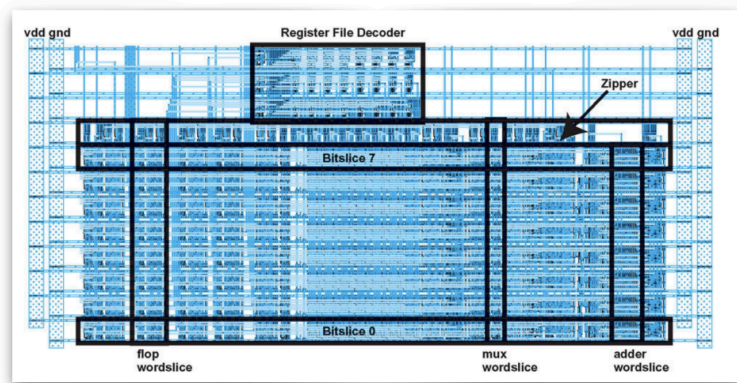
## Physical Design

### The MIPS process

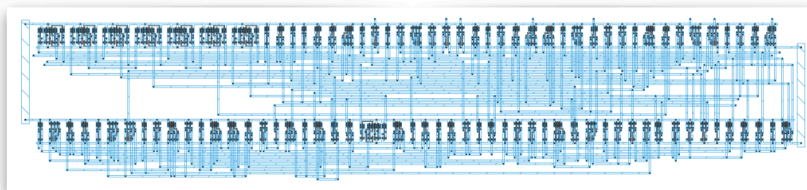
Il **Floorplanning** è il primo passo del physical design, dove si decide come disporre fisicamente i blocchi funzionali di un sistema all'interno di un'area (chip/scheda).

L'obiettivo è stimare l'area e la posizione dei blocchi principali per valutare lunghezza e congestione dei cablaggi.

Il **Datapath** (ALU, Registri) ha una struttura altamente regolare e ripetitiva, quindi possiamo usare il **Bit-Slice**, che è una tecnica di progettazione hardware in cui un'unità logica viene costruita replicando più volte lo stesso blocco elementare, ognuno responsabile di un singolo bit. (Si progetta il layout per un singolo bit e lo si replica)

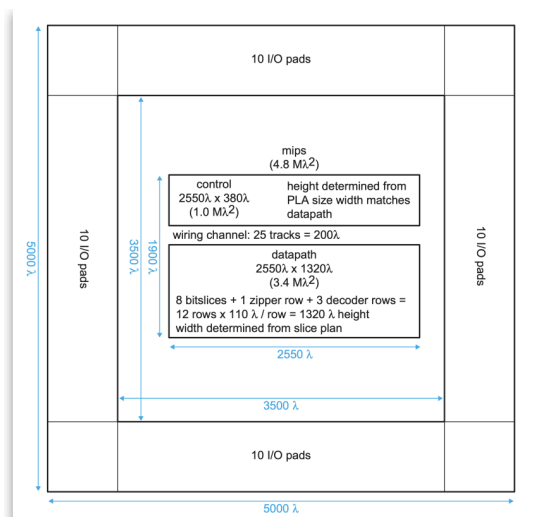


Il layout per il **Controller** è irregolare e spesso implementato con Standard Cells. Per collegare il Controller al Datapath, si usa il **Pitch Matching**, che consiste nell'allineare dimensioni e spaziatura di segnali/celle/blocchi adiacenti (in questo caso blocco Datapath e Controller), affinché le connessioni combacino, minimizzando l'area di interconnessione.



|   |   |   |   |   |
|---|---|---|---|---|
| A | A | A | A | B |
| A | A | A | A | B |
| A | A | A | A | B |
| A | A | A | A | B |
| C |   | C |   | D |

Example: Analizzando il layout del processore MIPS: Il Pad Frame consiste nell'anello esterno contiene i 40 Pad di I/O (29 segnali, 11 alimentazione ( $V_{DD}$  e GND)). Il design è **Pad-Limited**, in quanto la dimensione del chip ( $5000\lambda \times 5000\lambda$ ) è dettata dalla necessità di posizionare i 40 pad sul perimetro, non dall'area della logica interna (Core), che risulterebbe più piccola. Se la logica fosse stata più grande e avesse dettato le dimensioni, si parlerebbe di design **Core-Limited**.



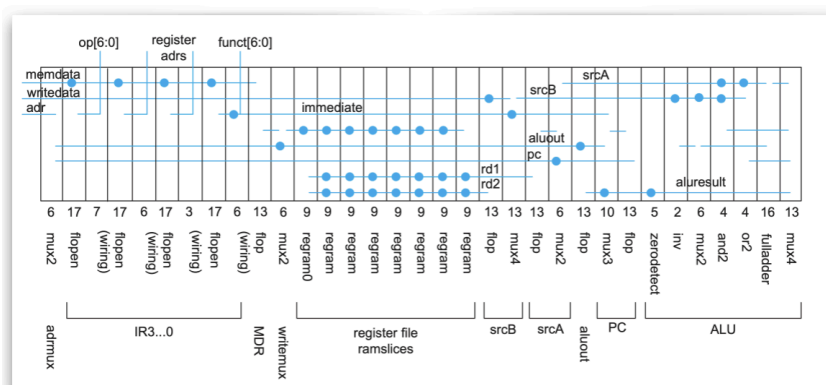
## Datapath Slice Plan

Layout per capire a quanti componenti/elementi che compongono il circuito, vengono connessi dalle varie data lines (interconnessioni). Questo ci permette di capire l'addensamento nella scheda, la complessità del circuito e una visione dei vari collegamenti.

È un diagramma strutturale utilizzato per stimare l'ingombro dei collegamenti all'interno di una singola Bit-Slice (la striscia di logica che elabora 1 bit).

A differenza dei circuiti su scheda, nel chip i fili di interconnessione passano fisicamente **sopra** le celle logiche (ALU, Registri) utilizzando layer metallici superiori, con la tecnica di **Routing Over-the-Cell**. Dato che l'altezza della cella è influenzata principalmente dal numero di fili che devono passarci sopra, il grafico identifica le aree a maggiore densità, con

- **Asse Orizzontale:** Rappresenta i componenti presenti
- **Linee Blu:** Rappresentano le tracce metalliche (Wiring Tracks) necessarie per portare i segnali, mentre i pallini indicano i punti di contatto (Vias).



(Programmable Logic Array) che mappa direttamente le equazioni booleane in un layout a griglia.

La PLA implementa qualsiasi funzione combinatoria come "Somma di Prodotti" attraverso due matrici adiacenti:

- **AND Plane** (Generazione Mintermini): Riceve gli ingressi (e i loro complementi). Le linee orizzontali (Product Lines) eseguono l'AND logico degli ingressi connessi.
- **OR Plane** (Generazione Uscite): Raccoglie le linee di prodotto e le combina tramite OR logico per generare i segnali di controllo in uscita (Outputs).

Le dimensioni dell'area occupata dalla PLA sono deterministiche: la **larghezza** è proporzionale alla somma di  $2 \times N_{\text{ingressi}} + N_{\text{uscite}}$  e l'**altezza** è proporzionale al numero di **Product Terms** (i diversi stati o transizioni della FSM). Questa regolarità permette di generare automaticamente il layout del controller e di incastrarlo ("Pitch Matching") con il Datapath.

## ES12

# Digital Circuits Pitfalls

## Process Variations

Mentre la litografia stampa il design delle maschere, quindi è un'applicazione diretta, mantenere il processo costante sarebbe la cosa migliore, dato che il processo sul wafer è un'attività tecnologica che ha un risultato statistico.

Il processo viene eseguito su tutti i wafer, come garantire che venga eseguito uniformemente su tutta la superficie del wafer?

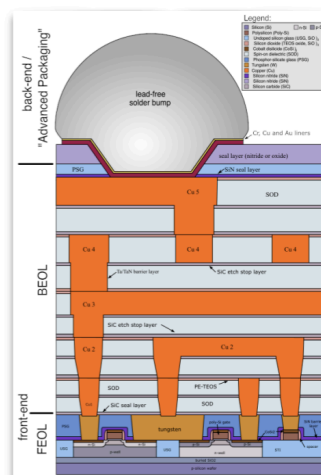
Process Statistics:

Il processo è attività tecnologica che produce risultati con una distribuzione statistica dipendente da tecnologia, ambiente, invecchiamento dei device etc.

## Process Corners

Sono le condizioni limite entro le quali il progetto/design deve funzionare, non mi basta progettare un circuito digitale che risponda ai valori nominali del programma, ma devo essere in grado di progettare un design entro dei parametri commerciali entro cui il prodotto deve funzionare. È ragionevole pensare che se il circuito funziona bene sui corner, è ragionevole pensare che funzioni bene anche al suo interno.

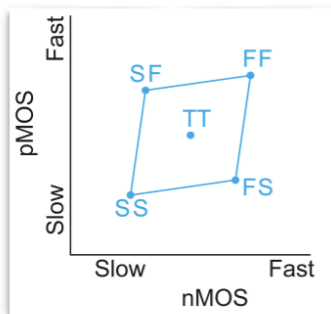
Ci sono due tipi di corner: Front-End, che riguarda i contatti e la parte di realizzazione fisica alla base, i dispositivi in quanto circuito (MOS, ecc.); poi abbiamo il Back-End, che è quello che sta sopra il Front-End, e si occupa della gestione dei dispositivi.



## FEOL Corners

- La convenzione di denominazione per i **process corners** è composta da due lettere (**T**: Typical/Tipico, **F**: Fast/Veloce; **S**: Slow/Lento); la prima si riferisce agli **nMOSFET**, la seconda ai **pMOSFET**. In sostanza, sono legate alla **mobilità dei portatori** di carica.
- Esistono cinque combinazioni: **TT** (sebbene tecnicamente non sia un "corner", viene comunque chiamato così), **FF**, **SS**, **FS**, **SF**.
- **TT**, **FF**, **SS** sono chiamati **even corners** (corner bilanciati o pari) poiché influenzano uniformemente entrambi i tipi di MOS; i circuiti risultanti sono in grado di funzionare normalmente, più velocemente o più lentamente.
- **FS** e **SF** sono chiamati **skewed corners** (corner sbilanciati o asimmetrici) poiché descrivono circuiti con prestazioni n-pMOS sbilanciate: una circostanza che desta preoccupazione a causa della commutazione asimmetrica e, di conseguenza, degli *slew* (pendenze) di transizione sbilanciati, che possono causare un'errata memorizzazione dei dati nei **latch**. (I misti sono detti Skewed, perché hanno uno sbilanciamento tra n e p)

## Esempio di Corners FEOL:



| Corner |      |      |          |      | Purpose                                                                                      |
|--------|------|------|----------|------|----------------------------------------------------------------------------------------------|
| nMOS   | pMOS | Wire | $V_{DD}$ | Temp |                                                                                              |
| T      | T    | T    | S        | S    | Timing specifications (binned parts)                                                         |
| S      | S    | S    | S        | S    | Timing specifications (conservative)                                                         |
| F      | F    | F    | F        | F    | Race conditions, hold time constraints, pulse collapse, noise                                |
| S      | S    | ?    | F        | S    | Dynamic power                                                                                |
| F      | F    | F    | F        | S    | Subthreshold leakage noise and power, overall noise analysis                                 |
| S      | S    | F    | S        | S    | Races of gates against wires                                                                 |
| F      | F    | S    | F        | F    | Races of wires against gates                                                                 |
| S      | F    | T    | F        | F    | Pseudo-nMOS and ratioed circuits noise margins, memory read/write, race of pMOS against nMOS |
| F      | S    | T    | F        | F    | Ratioed circuits, memory read/write, race of nMOS against pMOS                               |

## BEOL Corners

Per quanto riguarda il design, il BEOL è meno interessante, perché ci interessa essere compliant con i FEOL in primis. Di BEOL possiamo citare P (Process), V (Voltage), T (Temperature)

In questo caso la caratterizzazione definisce i circuiti come **nominali** (riflettendo la sezione media del processo), mentre **cbest** e **cworst** indicano le sezioni estreme del processo che sono ancora considerate accettabili

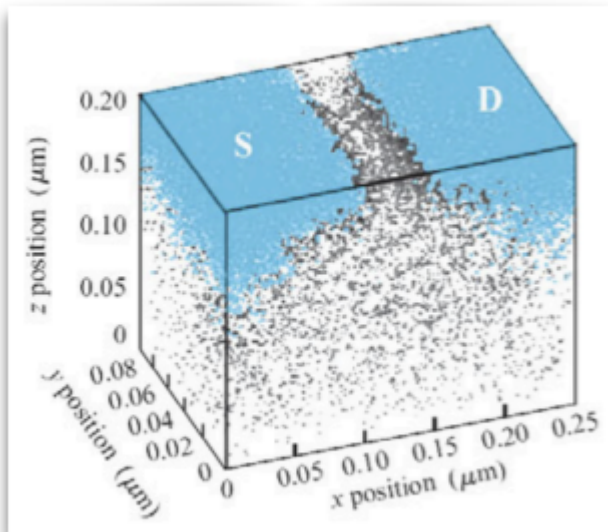
## Simulations

Ovviamente esistono tecnologie di simulazione che tramite metodi statistici ci permette di simulare le variazioni attese nel design su un certo processo (come l'impatto della variazione di  $V_T$  su  $I_{ON}$  o  $I_{OFF}$ ). Dei simulatori sfruttano la tecnica di simulazione Monte Carlo (spesso usata nel trasporto elettrico, dove si assume che il movimento delle cariche sia dato dalla distribuzione di Laplace, con una similitudine alle leggi dei Gas e le formule di Boltzmann), con una variazione di parametri random, sequenze lunghe ma con periodicità, per la generazione di sequenze random (semi-random in quanto hanno lo stesso seed) per analizzare se sto dentro o meno i corners.

(Processo ergodico: è un tipo di processo stocastico (casuale) in cui la media calcolata su una singola realizzazione nel tempo è uguale alla media statistica (o di insieme) calcolata su molteplici istanze del processo, permettendo di studiare l'intero sistema attraverso un'unica osservazione prolungata, sostituendo l'analisi temporale con quella collettiva) → da qui esce la Fermi Golden Rule: che stabilisce una necessaria interazione tra due elettroni vicini.

## Process-induced Variations

Il processo non è uniforme, quindi le distribuzioni di drogaggio non sono uniformi. Dato che lavoriamo con materiali reali non ideali, ci sono non idealità che provocano una variazione di tensione; l'informazione scritta sul Silicio produce effetti spuri.



## Spacial Variations

Ci possono essere variazioni tra lotti dei wafer, tra wafer stessi, tra chip o nel chip stesso. La variazione è particolarmente legata alla variabilità del processo. Si è dimostrato che transistori vicini hanno un matching maggiore.

## Environmental Variations

Con ambiente non intendiamo solo l'ambiente fisico dove opera, ma anche l'ambiente elettrico, come la sensibilità dell'alimentazione, che può essere variabile, infatti aggiungiamo una sensibilità del 10%

Ci sono diversi standard: Commercial Industrial Military

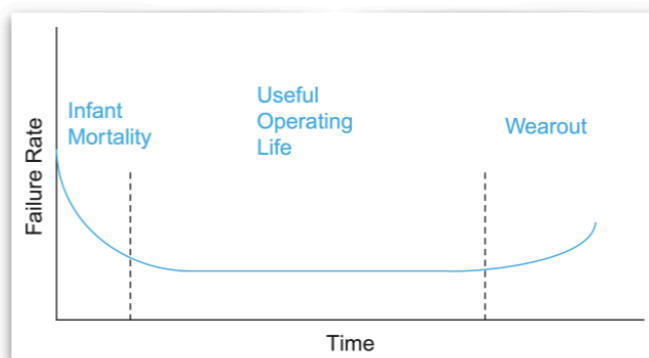
# Noise and Reliability

## Noise

Le fonti di rumore hanno diverse sorgenti: dall'alimentazione, dalla massa, dalla carica scambiata in gate pass transistor dinamici, per fenomeni di leakage (correnti di leakage perché i transistori non sono ideali), per motivi di feedthrough per input vicini ai limiti legali (passo attraverso linee di input). questi causano delay e valutazioni errate

## Reliability

Ci interessa quanto sia lungo il “Useful Operating Life”, nella curva della reliability bathtub, poiché più è lunga più è affidabile un prodotto. La reliability è caratterizzata da il Mean Time Between Failures (MTBF), tempo medio tra due componenti difettosi, e Failures in Time. Si vede che la probabilità di “morte” del componente nel suo periodo di vita iniziale e finale aumenta rispetto alla parte centrale della sua vita.



## Accelerated Lifetime testing

Per testare la vita dei prodotti, dato che non possiamo aspettare la sua vita intera (immaginiamo se abbiamo un prodotto con vita assegnata di 10 anni, non possiamo aspettare 10 anni per verificarne la verità dell'affermazione), quindi andiamo a testare il prodotto in situazioni estreme di temperatura e altre caratteristiche, tramite test particolari.

## Hot Carriers

Gli Hot Carriers sono elettroni che acquisiscono dal campo elettrico un'energia cinetica molto superiore a quella di equilibrio termico, che normalmente ha una media di  $\frac{3}{2}K_B T$ . Questo fenomeno si verifica quando il campo elettrico è sufficientemente intenso da accelerare gli elettroni lungo il loro libero cammino medio, permettendo loro di accumulare un'energia che non riescono a dissipare attraverso gli urti con il reticolo cristallino. La problematica principale emerge quando questi elettroni ad alta energia interagiscono con l'ossido di gate, penetrandovi per effetto tunneling o iniezione diretta e rimanendo intrappolati in stati spuri o difetti del dielettrico. Poiché la carica intrappolata non viene rilasciata, essa si oppone al campo di gate causando una variazione progressiva della tensione di soglia ( $V_T$ ) del transistor. Sebbene in passato questo meccanismo sia stato sfruttato intenzionalmente per la programmazione delle memorie floating gate, nei circuiti logici rappresenta un serio problema di affidabilità che rende gli Hot Carriers un parametro vivo da monitorare, costringendo i progettisti a scegliere accuratamente la tensione di alimentazione ( $V_{DD}$ ) per limitare il campo elettrico. Per gestire queste inevitabili variabilità e assicurare il funzionamento del dispositivo, la caratterizzazione del processo definisce tre scenari: il caso "Nominal", che riflette la sezione media del processo, e gli estremi "Cbest" e "Cworst", che rappresentano rispettivamente le condizioni migliori e peggiori, ma comunque accettabili, entro cui il circuito deve operare.

## Breakdown dell'Ossido

Il **breakdown dell'ossido** è un processo di degradazione progressiva indotto dallo stress dei campi elettrici applicati. Per garantire la **reliability a lungo termine** (evitando il *Time-Dependent Dielectric Breakdown* o TDDb), è fondamentale limitare il campo elettrico nell'ossido ( $E_{ox}$ ): valori troppo elevati accelerano infatti la generazione di trappole e difetti nel dielettrico, portando alla rottura definitiva anche a temperature di funzionamento standard.

## Negative Bias Temperature Instability (NBTI)

Il **Campo elettrico** può generare trappole in presenza di legami non strutturati (dangling bonds); questi stati per gli elettroni, se si riempiono, cambiano il valore di soglia. È un fenomeno molto più diffuso nei p-MOSFET:  $V_T \propto \frac{E_{ox}}{E_o} + 0.25$

### Time Depended Dielectric Breakdown (TDDB)

Il **Time Dependent Dielectric Breakdown (TDDB)** descrive il progressivo degrado dell'ossido del transistor causato dalla continua esposizione ai campi elettrici, che porta a un graduale aumento della corrente di perdita (*gate leakage*).

Per garantire un'affidabilità a lungo termine (ad esempio 10 anni a 125°C), è necessario limitare il campo elettrico attraverso l'ossido ( $E_{ox}$ ) al di sotto di circa **0.7 V/nm**.

### Electromigration

L'elettromigrazione è un fenomeno di usura dei conduttori causato dal passaggio di una forte corrente elettrica, dove il flusso degli elettroni colpisce gli atomi del metallo con una forza tale da spostarli fisicamente dalla loro posizione (effetto meccanico), un effetto noto come vento elettronico. Questo spostamento di materia è pericoloso perché può svuotare alcune zone creando interruzioni nel circuito, dette open circuit, oppure accumulare materiale altrove causando cortocircuiti con le linee vicine. Il problema è particolarmente grave in regime di corrente continua (DC), poiché gli elettroni spingono gli atomi sempre nella stessa direzione. La vita utile del conduttore (indicata come **MTTF** o *Mean Time To Failure*) è descritta dall'**Equazione di Black**, che evidenzia come il guasto dipenda dalla densità di corrente e, in modo esponenziale, dalla **temperatura** di esercizio.

### Self Heating

Il fenomeno del **Self Heating** si verifica quando la corrente che attraversa le interconnessioni genera calore per effetto Joule a causa della resistenza del materiale. Poiché il circuito è ricoperto da uno strato di ossido di passivazione che funge da isolante termico, questo agisce come una sorta di "coperta" che ostacola la dissipazione, portando a un accumulo di calore locale. L'aumento della temperatura causa un incremento della resistenza dei fili metallici, il che si traduce in un rallentamento della propagazione dei segnali e quindi delle prestazioni complessive dei componenti. Per garantire un'adeguata affidabilità, è fondamentale limitare la densità di corrente in regime AC.

### Overvoltage Failure

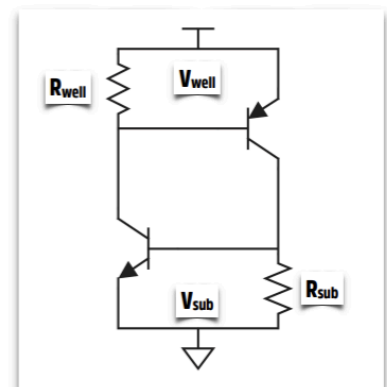
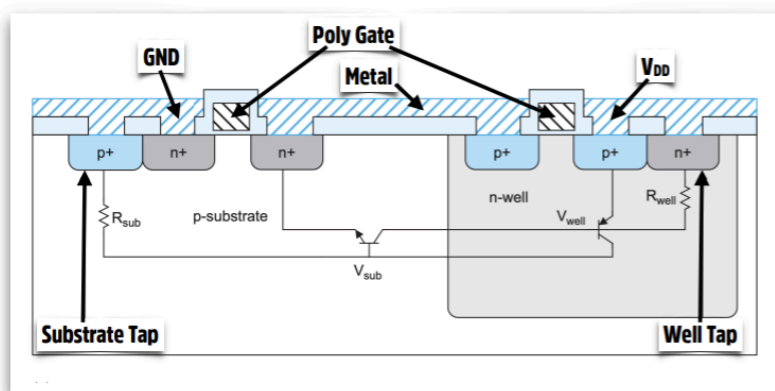
Il guasto da sovratensione, o **Overvoltage Failure**, si verifica quando una qualsiasi tensione eccessiva porta alla distruzione dei transistor, spesso a causa di scariche

elettrostatiche (ESD) ad alto voltaggio trasferite durante la manipolazione del circuito, il che rende necessario l'uso di diodi di protezione sui pin e braccialetti di messa a terra per gli operatori.

Altre cause interne di danno includono la rottura dell'ossido, dove una tensione di gate eccessiva crea un arco elettrico attraverso lo strato sottile, e il fenomeno del **punchthrough**, in cui un'elevata tensione tra drain e source ( $V_{DS}$ ) fa toccare le rispettive regioni di svuotamento, generando correnti incontrollate e un surriscaldamento distruttivo.

### Latch up

Noi siamo abituati a vedere i blocchi nMOS e pMOS, ma sono collegati da una giunzione parassita. Se scorre corrente attraverso la  $R_{SUB}$  porta a un aumento di tensione e conseguente accensione del transistor parassita. Questo transistor agisce sulla corrente e tensione (abbassa  $V_{WELL}$ ). La retroazione positiva indesiderata porta alla fusione l'intero circuito; sono state introdotte delle soluzioni per questo problema: le trench o guard ring diffusion attorno ai transistor



### Soft Errors

I **Soft Errors** sono malfunzionamenti casuali, riscontrati frequentemente nelle memorie dinamiche (DRAM), causati dall'interazione del silicio con particelle ad alta energia come le **particelle Alpha** o i **neutroni dei raggi cosmici**. L'impatto di queste particelle genera per collisione coppie elettrone-lacuna nel substrato; queste cariche viaggiano attraverso le giunzioni verso le zone polarizzate e disturbano la tensione interna, portando all'inversione involontaria del bit (bit flip) e quindi alla modifica del contenuto della cella di memoria.

## Radiation Hardening

Per mitigare l'impatto dei **Soft Errors** si possono adottare diverse strategie. Una delle più efficaci a livello circuitale è l'uso della **ridondanza**, implementata ad esempio attraverso celle con **Dual Interlocked Feedback** (DICE), che sfruttano nodi duplicati e meccanismi di retroazione per impedire che il singolo evento di disturbo modifichi lo stato logico salvato.

Oltre a questa soluzione topologica, è possibile intervenire fisicamente aumentando la **capacità del nodo** (per renderlo meno sensibile alla carica iniettata dalle particelle) oppure, a livello di sistema, adottare **codici di correzione degli errori** (ECC) che permettono di identificare e recuperare i dati corrotti.

## ES16

# IC Testing and Design for Testability

## The Rationale

I circuiti devono essere progettati, tra le altre cose, per essere testati facilmente, perché a livello di spesa il costo dell'intervento della nella soluzione aumenta drasticamente di un ordine di grandezza di  $10^4$ :

|                       |                  |                     |
|-----------------------|------------------|---------------------|
| ■ wafer level         |                  | ■ \$ 0.01 - \$ 0.10 |
| ■ packaged chip level |                  | ■ \$ 0.10 - \$ 1    |
| ■ board level         | <b>with cost</b> | ■ \$ 1 - \$ 10      |
| ■ system level        |                  | ■ \$ 10 - \$ 100    |
| ■ field level         |                  | ■ \$ 100 - \$ 1000  |

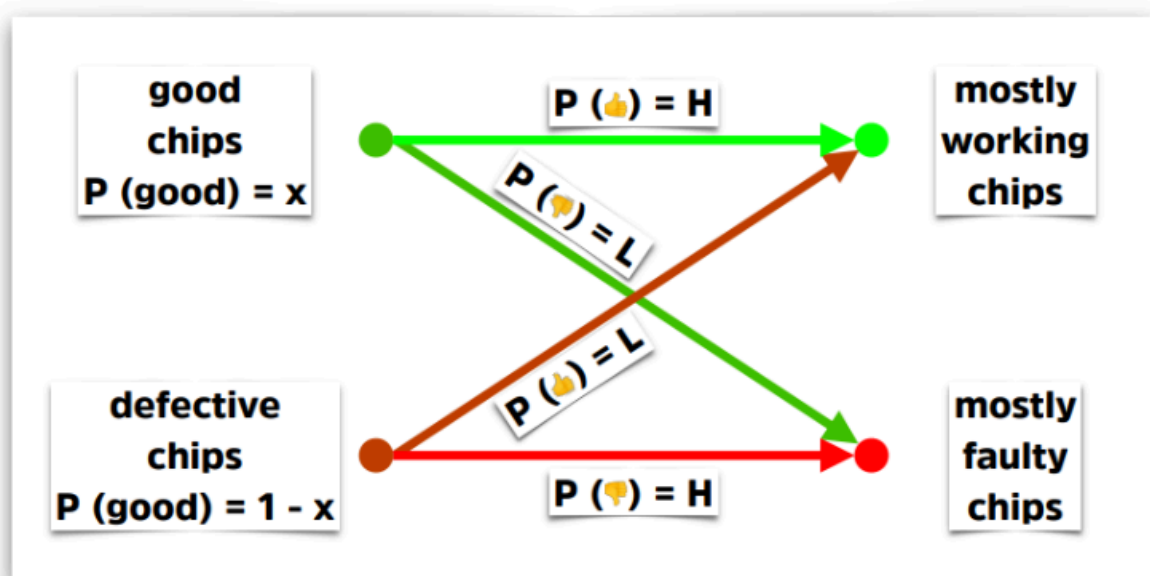
## Test Mindset

|                                                     |                                                |
|-----------------------------------------------------|------------------------------------------------|
| ■ <i>Understand the system .....</i>                | Knowledge on the design is a prerequisite      |
| ■ <i>Make it fail .....</i>                         | Find a repeatable way to elicit the bug        |
| ■ <i>Quit thinking and look .....</i>               | Propose a test and investigate                 |
| ■ <i>Divide and conquer .....</i>                   | Use hierarchy to eliminate parts of the design |
| ■ <i>Change one thing at a time .....</i>           | Decouple causes and effects                    |
| ■ <i>Keep an audit trail .....</i>                  | Write down a documentation                     |
| ■ <i>Check the plug .....</i>                       | Check the complete test structure              |
| ■ <i>Get a fresh view .....</i>                     | Involve a coworker, take a break, sleep on it  |
| ■ <i>If you didn't fix it, it ain't fixing.....</i> | Black magic does not exist!                    |

## Test circuiti integrati

Per una validazione efficace è necessario avere una profonda conoscenza del sistema e sottoporlo a **stress test** in condizioni limite (**corner cases**), con l'obiettivo di far emergere eventuali difetti latenti. Per gestire la complessità si applica l'approccio **Divide and Conquer**: il sistema viene scomposto in blocchi funzionali isolati, permettendo di testare separatamente ogni comportamento. Durante questo processo, è fondamentale garantire la tracciabilità dei segnali (**observability**) per monitorare l'evoluzione interna del circuito e individuare esattamente dove si genera l'errore.

## Test come filtro



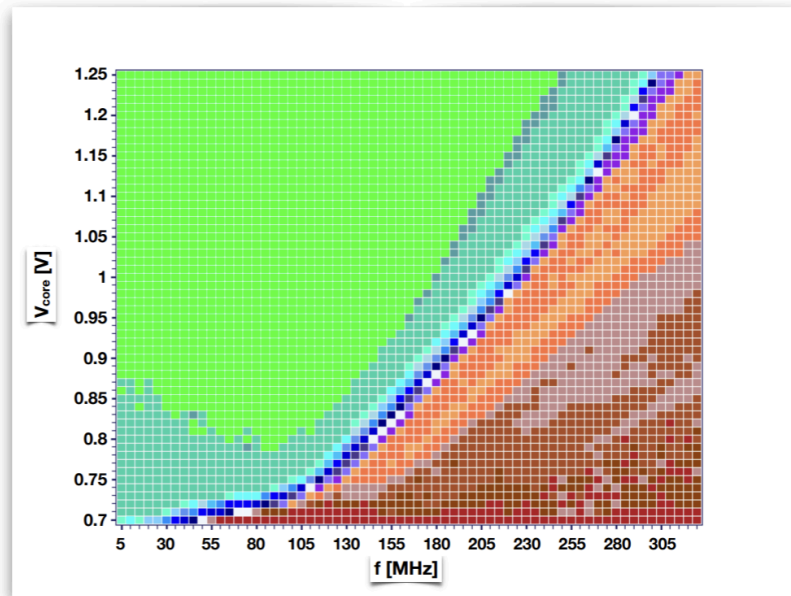
Ci sono probabilità non nulle di riconoscere i circuiti funzionanti per buoni, per non funzionanti (correttamente) o il caso duale.

## Testing

- Detection: determinare se il dispositivo sotto test è difettoso o meno
- Diagnosi: determinare cos'è difettoso nel dispositivo. Procedura molto costosa e si fa solo se necessaria la correzione
- Caratterizzazione sulla popolazione: prevede la selezione di campioni significativi e la loro analisi in diverse condizioni ambientali per diagnosticare e correggere i difetti di progettazione, producendo come risultato grafico fondamentale lo **Schmoo Plot**.
- Analisi di Fault Mode: determinare delle lacune nel processo di produzione

## Schmoo Plot

Gli Schmoo Plot consentono di visualizzare in maniera completa, le diverse sfumature dei dispositivi testati, è una caratterizzazione di dove si posizionano gli N dispositivi testati, rispetto a frequenza sulle ascisse e volt sulle ordinate.



## Manufacturing Test

L'obiettivo del manufacturing test determina quanto il circuito integrato (IC) rispetta le specifiche. Si cerca di trovare il maggior numero di errori possibili in quanto i circuiti sono così complessi e sviluppati che trovarli tutti è pressoché impossibile. Non ci si pone il problema di capire da dove viene il problema e viene eseguito su tutti i circuiti prodotti.

## Stress Test & Incoming Inspection

Lo **Stress Test** sottopone campioni di circuiti integrati a condizioni estreme di temperatura e tensione per far emergere la cosiddetta "mortalità infantile" (guasti precoci), solitamente entro circa due giorni.

L'**Incoming Inspection**, invece, avviene dal lato del cliente: è una verifica più esaustiva e orientata all'applicazione specifica, eseguita su lotti casuali per bloccare componenti difettosi prima che vengano montati nei sistemi finali.

## Production Yield (Fatto molto veloce)

$$Production\ Yield = \frac{\text{numero chip buoni}}{\text{numero chip totali}} = Y$$

$$Cost\ chip = \frac{\text{costo fabbricazione} + \text{costo test}}{Y \cdot \text{Numero chip sul wafer}}$$

Yield Models (Non importante/trascurabile)

Sono stati creati molti modelli di yield come Poisson, Murphy, Seeds, Moore

Coverage

Probabilità che il test identifichi il difetto, ad oggi si può fare in modo statistico.

## Fault Modeling

Controllabilità: Capacità di controllare lo stato del sistema e porlo in una condizione specifica

Osservabilità: Capacità di propagare all'esterno il risultato di un nodo e di farlo vedere

Ripetibilità: Poter riprodurre le condizioni e si ottiene lo stesso output con lo stesso input

Survivability: Il test deve continuare anche dopo aver riscontrato un difetto

IC Real Faults:

- Process faults
- Material faults
- Time Dependent faults
- Packaging

IC Faults Models

Nella misura in cui la fase di test ha un costo in funzione del tempo, sottoponiamo il circuito solo ai test necessari e non a tutti i disponibili. Se alcuni risultati sono stati coperti da altri vettori di test, è inutile ripeterli. (ES AND  $\rightarrow$  1,1; 1,0; 0,1 è inutile testare 0,0 (risparmio un test))

Single Stuck-at

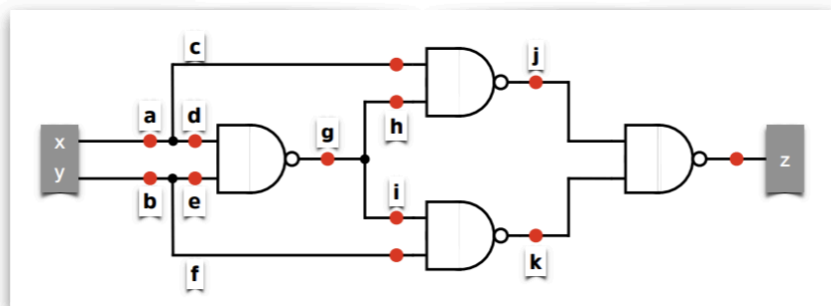
Un certo nodo del circuito che dovrebbe essere in grado di muoversi liberamente (a seconda di come viene stimolato) da 0 a  $V_{DD}$  rimane fisso a un valore dovuto, per esempio, a una goccia di stagno caduta durante la fabbricazione. Tale difetto mantiene "ancorato" il valore del nodo a 0 (stuck at 0 (sa0)) o a 1 (stuck at 1 (sa1))

Si definisce single perché si assume in questo modello che è presente al massimo una deformità del circuito di questo tipo ed è dovuta alla presenza di un corto gate-ossido indesiderato

### Esempio

Prendiamo un semplice circuito: Facendo le tabelle di verità otteniamo quella di una XOR (00→0, 01→1, 10→1, 11→0). Gli stuck at sono quelli che possono forzare il circuito ad avere un risultato che non vogliamo e sono rappresentati dai pallini rossi, che rappresentano anche gli fault sites.

Se “h” è uguale a 0 (stuck-at-0), g non può essere 1, mi forza su j un 1 indipendentemente dal valore sull’altro ingresso della NAND, avendo j=1 (elemento neutro) devo considerare l’altro ingresso sulla NAND finale, con z che dipende da k; per essere certi che z dipenda da k, devo tentare di far andare k a 0. Se x o y sono a 0, h=0, che mi fa andare j=1 come dicevamo, con poi y=1, su f=1 e su i ho 1, per avere k=0, quindi su i deve esserci 1, quindi anche g=1 di conseguenze z=0 con x=0 e y=1



### Fault Equivalence

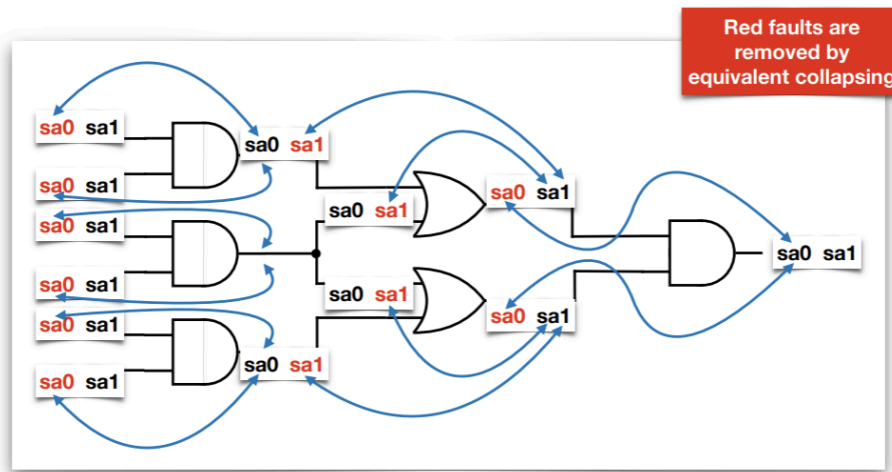
Il numero di fault site (fs) è uguale al numero di pin in ingresso(ip) sommato al numero di gate (g) e al fan out (fo)

$$\#fs = \#ip + \#g + \#fo$$

### Fault Collapsing

Tutte le fault si possono dividere in sottoclassi precise a seconda del tipo. Esistono delle equivalenze che permettono di scorrere il circuito in avanti e indietro andando a evidenziare le dipendenze di fault futuri da punti precedenti. Si possono quindi rimuovere dei fault site andando a snellire il numero di vettori di test.

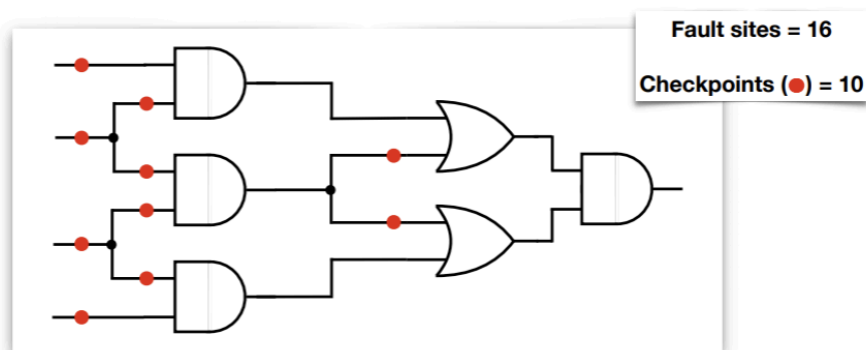
Posso andare a eliminare i controlli sulle detection degli stack-at per i segnali rossi dato che avere un stack-at-0/1 implica una conseguenza in uscita, quindi posso considerare direttamente quello.



### The Checkpoint Theorem

Si dimostra che è sufficiente rivelare gli stack-at nei punti rossi per capire il risultato della logica del circuito

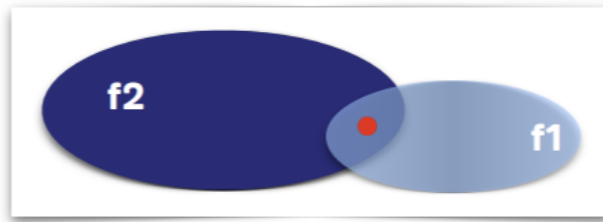
I checkpoint sono dati dai fan in + i rami dei gate con fan out > 1



### Side Effects

Alcuni test, possono rilevare più fault di classi diverse, si trova quindi che il numero di test, in questi casi, è minore del numero dato dalla somma dei test necessari a ogni classe.

$$N_{\text{TOTALE DI TEST}} \leq N_{\text{TEST F1}} + N_{\text{TEST F2}}$$



### Bridging Faults

Fault dovuto al cortocircuito di nodi non adiacenti, pure in questo caso sono necessari dei vettori di test “adatti” che verificano la potenziale presenza di questi errori. Si verificano per errori di produzione

### Open Faults

Si può considerare l'opposto dei Bridging, sono Faults dovuti alla mancanza di una connessione. Ad esempio una traccia è rovinata o non si è creata bene la connessione di due componenti nella fabbricazione del chip

### IDD Faults

Dato da un cortocircuito a  $V_{DD}$  che porta a correnti in eccesso rispetto alle nominali di funzionamento. È un problema di tipo analogico e non logico, devo fare dei test tramite vettori di test per triggerare il problema e misurare le correnti presenti nel circuito.

## Design For Testability

Si spende a livello di area per introdurre dei circuiti di testing.

I criteri appartengono a 3 categorie principali: Ad Hoc Testing, Scan-based approaches (Spesso riprodotti dai software di test, con “scan-path”), BIST (Built-In-Self-Test, si autocontrollano da soli)

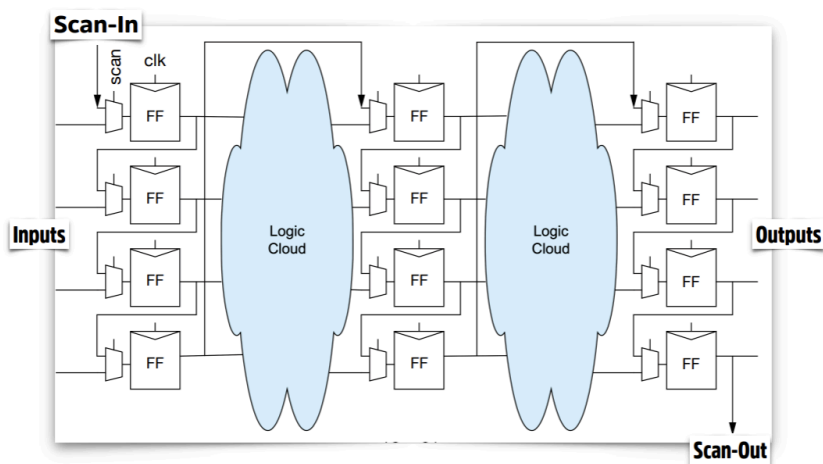
### Ad Hoc Testing

Sono dei circuiti non standard e difficili da implementare. Vado ad aggiungere dei test point per facilitare la procedura dei test; si introducono tramite dei pad interni, ovvero connessioni intermedie a nodi inaccessibili direttamente.

## Scan design

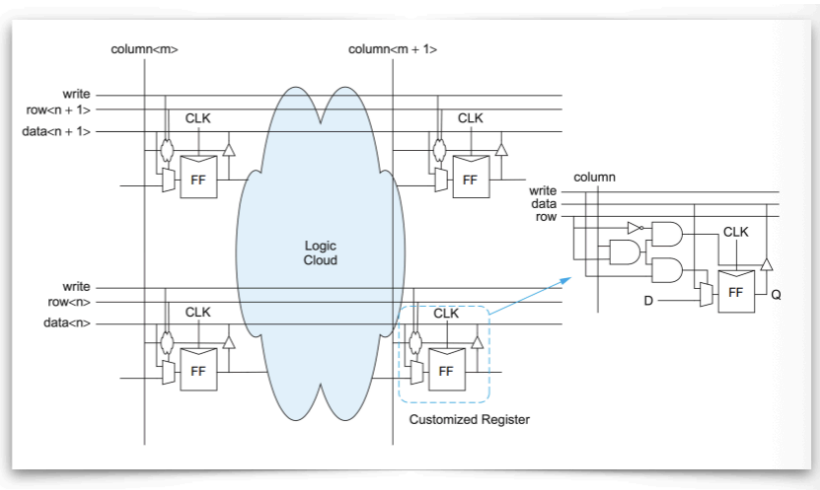
Per risolvere la complessità degli Ad Hoc testing sono state sviluppate le Scan-Based creando uno “scan-path” con il vantaggio che possono essere piazzati in modo automatico dai software di sintesi, riducendo i tempi di test, ma comporta un grosso consumo d’area.

Si creano dei percorsi alternativi realizzati con dei registri a scorrimento per propagare gli stati interni del circuito in un uscita per poterli leggere. Questa tecnica permette anche di ridurre i tempi di test oltre l’inserimento automatico.



La versione parallela dello Scan Design, è la Parallel Scan, la lunga catena di scansione (*scan chain*) viene divisa in segmenti multipli e più corti che vengono caricati in parallelo.

Portando questo concetto al limite si ottiene il **random access scan**, un approccio simile al caricamento della memoria di configurazione nelle FPGA, che permette di indirizzare e accedere quasi singolarmente agli elementi di memoria.



## BIST

i più usati sono i LFSR, registri generatori di sequenze pseudo random seguendo il campo di Galois, l'idea è di avere un certo numero di registri, con una retroazione positiva, con la retroazione si ottengono sequenze randomiche con una periodicità anche molto lunga.

Lo stato che otteniamo facendo passare tutti i dati, otteniamo una hash (codice distintivo) per tutti i dati che passano. Se ora prendiamo un circuito, questo passato ottiene una firma digitale e la probabilità di avere due circuiti randomici uguali decresce con la radice quadrata del numero di registri.

Se ho un circuito che non funziona, ho una probabilità del  $\frac{1}{\sqrt{2^{N_{REGISTRI}}}}$  che abbia la stessa firma.

Un processore moderno ha un numero di stati interni così grande che è troppo complesso implementare questa logica.

Otteniamo un payoff del 99.7% con un consumo del 10% di area.

## High-Speed Digital, RF and Analog Circuit Testing

Il problema si presenta quando andiamo ad alta frequenza, vengono inseriti degli oscilloscopi e degli analizzatori di logica interni che evitano di doverne usare di esterni per i test.

## ADC and DAC Testing

Si verifica il corretto funzionamento tramite dei test molto specifici in input

IDDQ Testing

La possibilità di analizzare la corrente di corto circuito del sistema, con l'obiettivo di trovare le sorgenti di problemi di cc.

## ES13

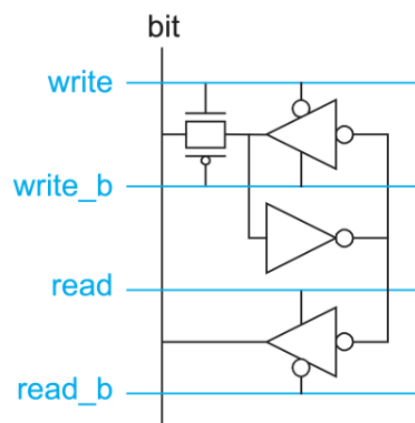
# Memories & Arrays

## Static RAMs (Ha fatto giusto un riassunto)

L'organizzazione della struttura è regolare, facile da disegnare e ad alta densità.

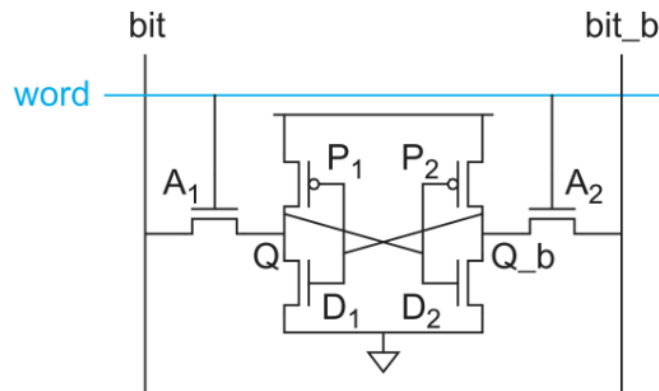
Layout tipici:

- 12T Static Ram:



La cella di memoria 12T mostrata in figura è progettata per operare a bassa tensione (spesso in regime sotto-soglia) separando fisicamente il percorso di **scrittura** da quello di **lettura** per evitare disturbi accidentali al dato memorizzato

- 6T Static Ram:



## Memory Read and Write

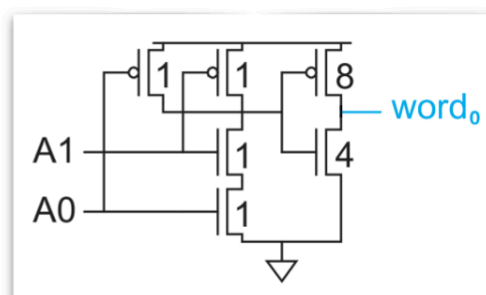
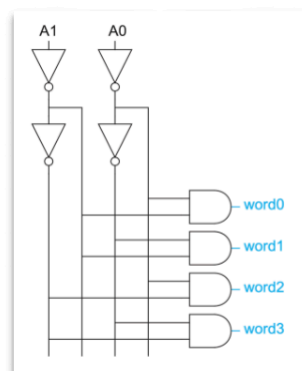
La lettura si fa precaricando le BL, e affinché sia stabile, il dimensionamento tra  $D1/D2$  deve essere molto maggiore di  $A1/A2$ . Mentre la Scrittura  $A1/A2 \gg P1/P2$

Questa differenza di dimensionamento è per evitare delle tensioni troppo alte sui nodi intermedi che potrebbero accendere/spegnere dei transistori nel processo.

Sono presenti due bitline che portano in uscita il dato in forma vera e in forma negata, questa doppia opzione è fondamentale per gli amplificatori di sense. Essendo le capacità della BL molto elevate, per scrivere uno zero o un uno si dovrebbero caricare/scaricare completamente. Tramite l'uso degli amplificatori, si possono sbilanciare di un 10% della capacità massima per riuscire a leggere uno zero o un uno.

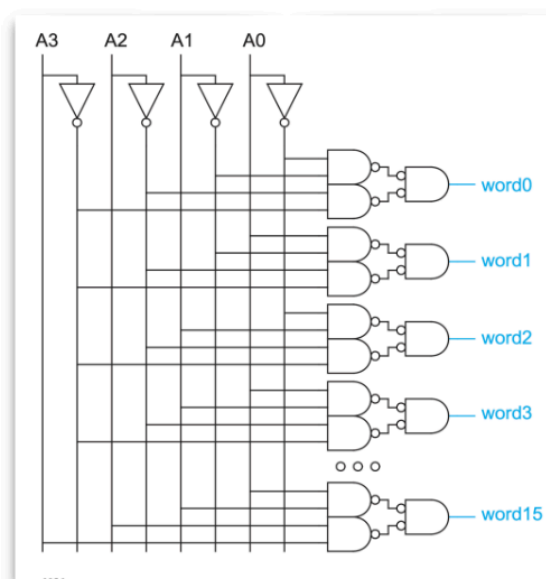
## Decoders

Questa configurazione permette di avere un decoder di  $2^N$  con  $N_{\max} = 4$ . È una configurazione molto semplice e intuitiva ma, appunto, ha un numero limitato di word decodificabili.



## Large Decoders

Per decoder con  $N > 4$  si usa un **Large Decoder** come quello in figura. Esso è realizzato con una struttura gerarchica o di **pre-decoding**, che utilizza stadi multipli di porte logiche (NAND) invece di una singola porta complessa per ogni uscita. Questa architettura riduce il numero di ingressi per singola porta (*fan-in*), migliorando notevolmente la velocità di commutazione e ottimizzando l'area occupata rispetto a un decoder piatto tradizionale. Si deve notare come in questi casi il numero maggiore di porte porta un vantaggio rispetto ad averne un numero inferiore con fan-in più elevato.



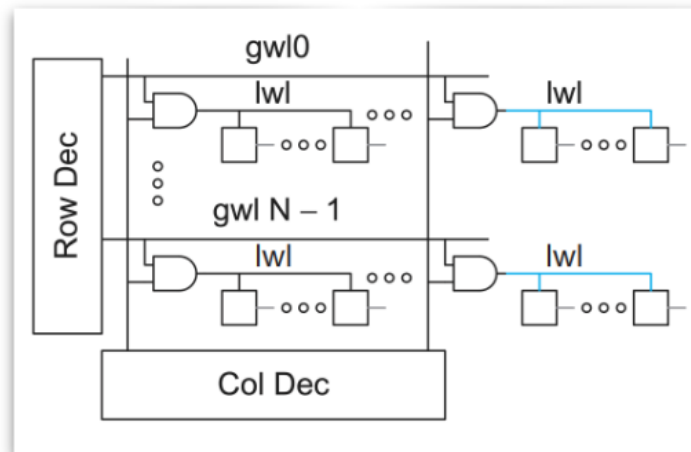
## Pre Decoding

Il **Predecoding** è una tecnica architetturale che suddivide i bit di indirizzo in gruppi più piccoli per generare segnali intermedi prima dello stadio finale. Riduce l'occupazione di area e lo stesso path effort rispetto a un circuito che non ne fa uso.

## Hierarchical wordlines

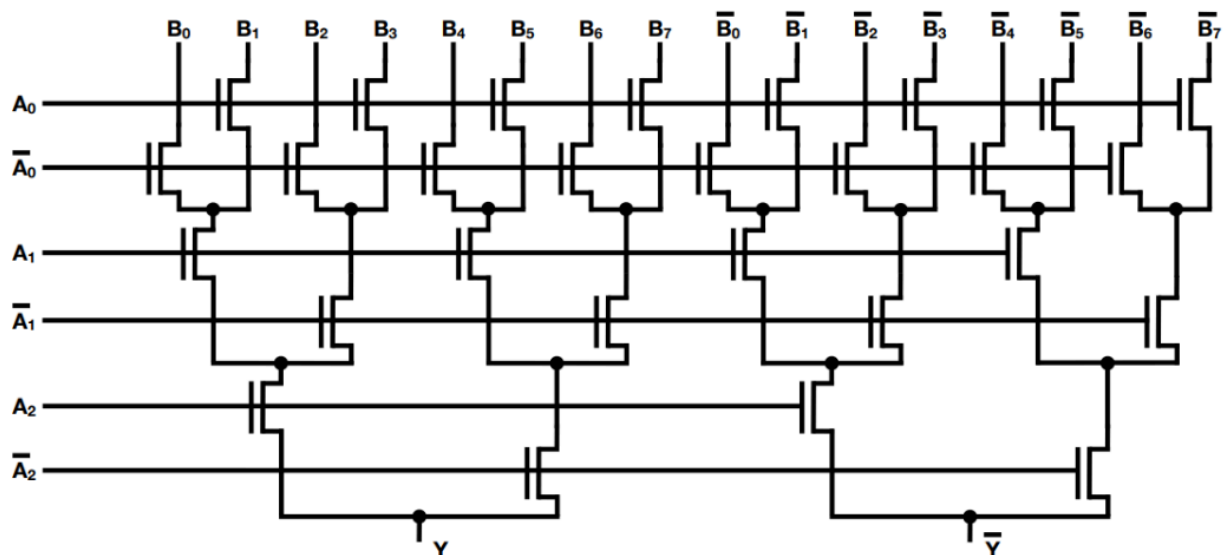
L'architettura **Hierarchical Wordlines** risolve i problemi di alta resistenza e capacità parassita delle lunghe interconnessioni suddividendo la decodifica in due livelli: **Global Wordlines (gwl)**, che corrono su strati metallici superiori più larghi (bassa resistenza), e **Local Wordlines (lwl)**, segmenti molto corti che pilotano direttamente le celle.

Questa suddivisione migliora drasticamente le prestazioni di velocità riducendo il ritardo RC complessivo, poiché il segnale principale viaggia su linee veloci e attiva solo la porzione locale necessaria della memoria.



## Multiplexers

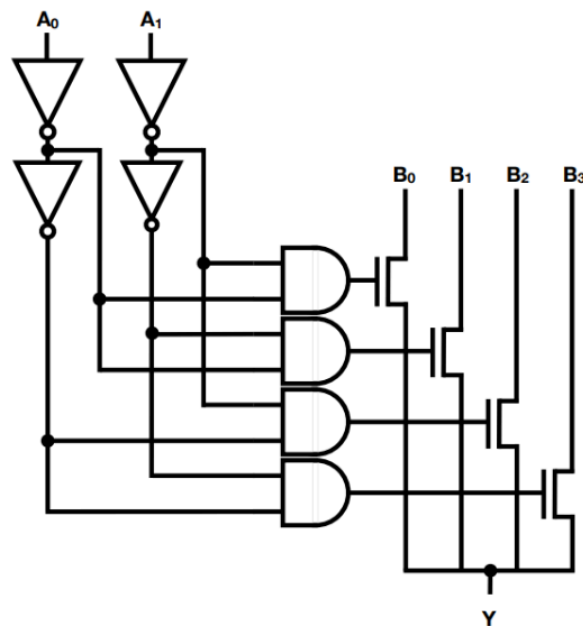
### Column Tree Decoder MUX



Il **Column Tree Mux** è un circuito di multiplexing realizzato con una struttura "ad albero" di pass-transistors, utilizzato per selezionare una specifica coppia di bitline tra le molte disponibili nell'array di memoria. Utilizzando segnali di indirizzo gerarchici ( $A_0, A_1, A_2$ ), il circuito attiva un unico percorso elettrico che collega la colonna desiderata all'uscita (Y), offrendo una soluzione compatta che riduce la capacità parassita rispetto ai decoder tradizionali.

In base a quali segnali delle linee A sono attivi, si crea un percorso che porta una data linea B in uscita.

## Single Pass Transistor MUX



Il **Single Pass Transistor Mux** è la versione più semplice ed essenziale di un multiplexer: utilizza un **singolo transistor** per ogni linea di ingresso che agisce come un semplice interruttore on/off.

Quando il segnale di controllo (Gate) è attivo, il transistor si chiude e lascia passare fisicamente il segnale dall'ingresso all'uscita senza rigenerarlo (a differenza delle porte logiche classiche), rendendo il circuito piccolissimo ma soggetto a una lieve perdita di segnale.

Non avendo transistori in serie come nel Tree Decoder, non ho perdite a livello di corrente e velocità legate ad essa.

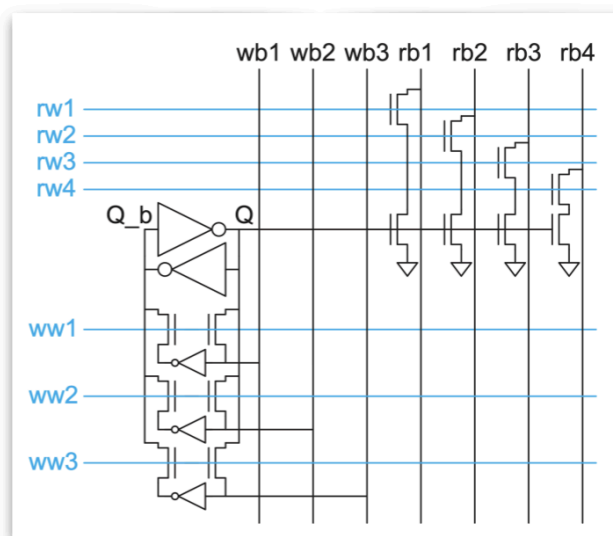
## Lard SRAMs

Memorie Grandi vengono partizionate in sotto-array per aumentare la velocità. Più la memoria è piccola, più i decoder sono piccoli, più l'accesso risulta veloce

## Multiport RAM

Ci permette di avere maggiore flessibilità, tramite maggior numero di porte, che forniscono più valori alla cella

Questa sotto ha 4 porte di lettura e 3 porte di scrittura: Questo comporta una maggior numero di righe per selezionare cosa fare e che riga usare



## ROM Memories

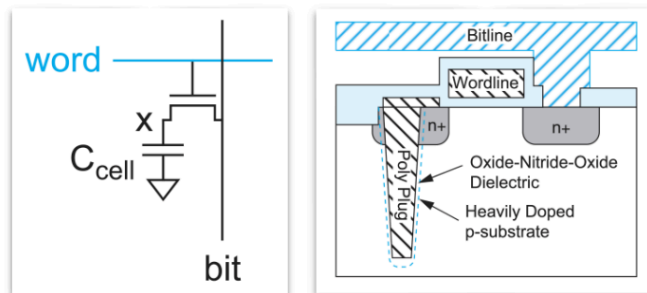
Nelle memorie ROM, l'interpretazione del valore logico dipende strettamente dalla configurazione circuitale dei transistor rispetto alla linea dei dati (bitline). Nell'architettura NOR, dove le celle sono disposte in parallelo, la linea viene mantenuta normalmente a un livello di tensione alto; di conseguenza, la presenza di un transistor attivo crea un percorso di scarica verso terra portando il valore a zero logico, mentre l'assenza di connessione o di conduzione lascia la linea al livello alto, corrispondente all'uno logico. Nella configurazione NAND, invece, le celle sono collegate in serie e la lettura richiede che tutti i transistor della catena, tranne quello selezionato, siano accesi per fungere da passaggi; in questo contesto, se il transistor interrogato è programmato per condurre sempre (ad esempio un tipo "depletion"), la corrente fluisce verso terra e si legge uno zero, mentre se si comporta come un transistor normale che si spegne sotto la tensione di test, interrompe il circuito facendo leggere un uno logico. In sintesi, in entrambe le logiche è il passaggio di corrente verso massa a determinare lo stato basso (0), ma cambia il modo in cui il circuito realizza o impedisce questo collegamento.

## DRAM Memories

Le memorie **DRAM** (Dynamic RAM) immagazzinano l'informazione sotto forma di carica elettrica all'interno di un condensatore (Ccell), accessibile tramite un singolo transistor di pass-gate controllato dalla wordline. Questa architettura essenziale (1T-1C) permette di ottenere densità di integrazione elevatissime, spesso realizzando il condensatore in profondità nel substrato (trench capacitor) per risparmiare spazio superficiale, ma

costringe a un continuo "refresh" dei dati poiché la carica nel condensatore tende naturalmente a disperdersi nel tempo.

La lettura, inoltre, è distruttiva e c'è la necessità di ricaricare le wordline in modo tale da sovrascrivere il valore letto nella cella stessa tramite l'uso degli amplificatori a valle. La tecnologia attuale permette l'utilizzo di un condensatore realizzato come un transistor senza terminazioni.



## Serial Memories

### Shift Register

Uno Shift Register (registro a scorrimento) è un circuito sequenziale composto da una catena di flip-flop connessi in cascata e sincronizzati dallo stesso segnale di clock, dove l'uscita di ogni stadio è collegata all'ingresso del successivo permettendo al dato binario di traslare di una posizione a ogni ciclo.

### Queues

Le code sono una struttura dati fondamentale anche a livello di programmazione che permette un accesso circolare alla memoria. Si basa su logica FIFO (contraria a quella dello stack). Dal punto di vista fisico viene realizzata con una SRAM come immagazzina i dati, due linee distinte per lettura e scrittura e altre due per coda piena e vuota. Viene sostanzialmente gestita tramite puntatori alla memoria per tenere traccia di dove si è arrivati a scrivere e leggere. Esiste anche un'architettura LIFO (Stack) che

usa un unico puntatore per lettura e scrittura che punta alla stessa cella.

